

Java 是第一大编程语言和开发平台。它有助于企业降低成本、缩短开发周期、推动创新以及改善应用服务。如今全球有数百万开发人员运行着超过 51 亿个 Java 虚拟机，Java 仍是企业和开发人员的首选开发平台

课程内容的介绍

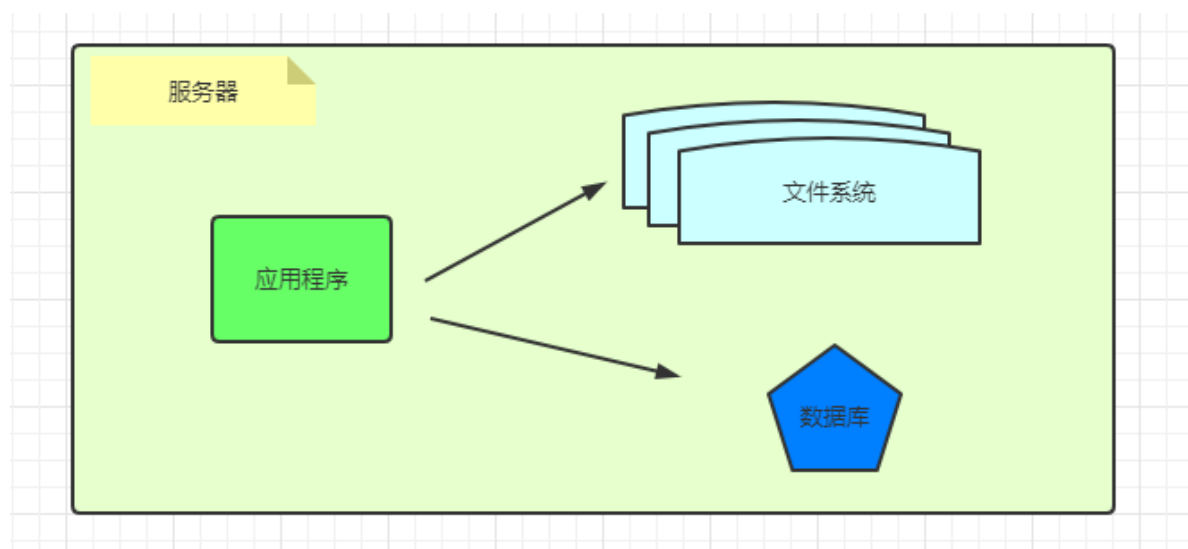
1. 微服务项目介绍
2. Eureka
3. Ribbon
4. Feign
5. Hystrix
6. Zuul
7. Config
8. Bus
9. Stream

一、微服务项目介绍

1. 系统架构演变

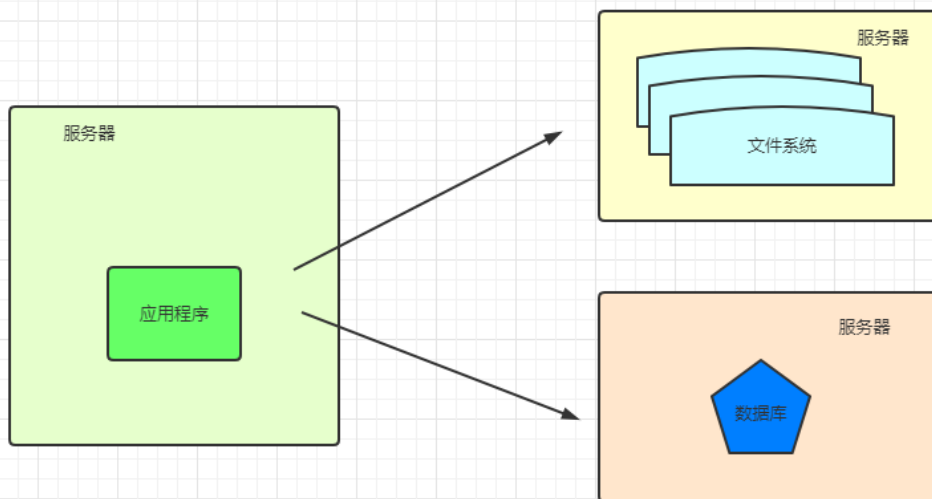
1.1 单体架构

后端服务的所有的功能集中在一个项目中。



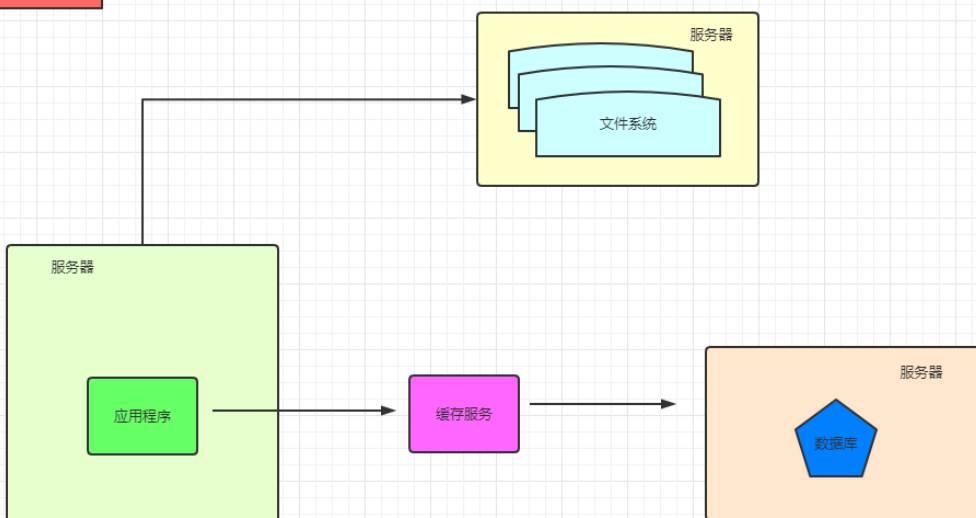
应用服务和数据服务分离

应用数据分离

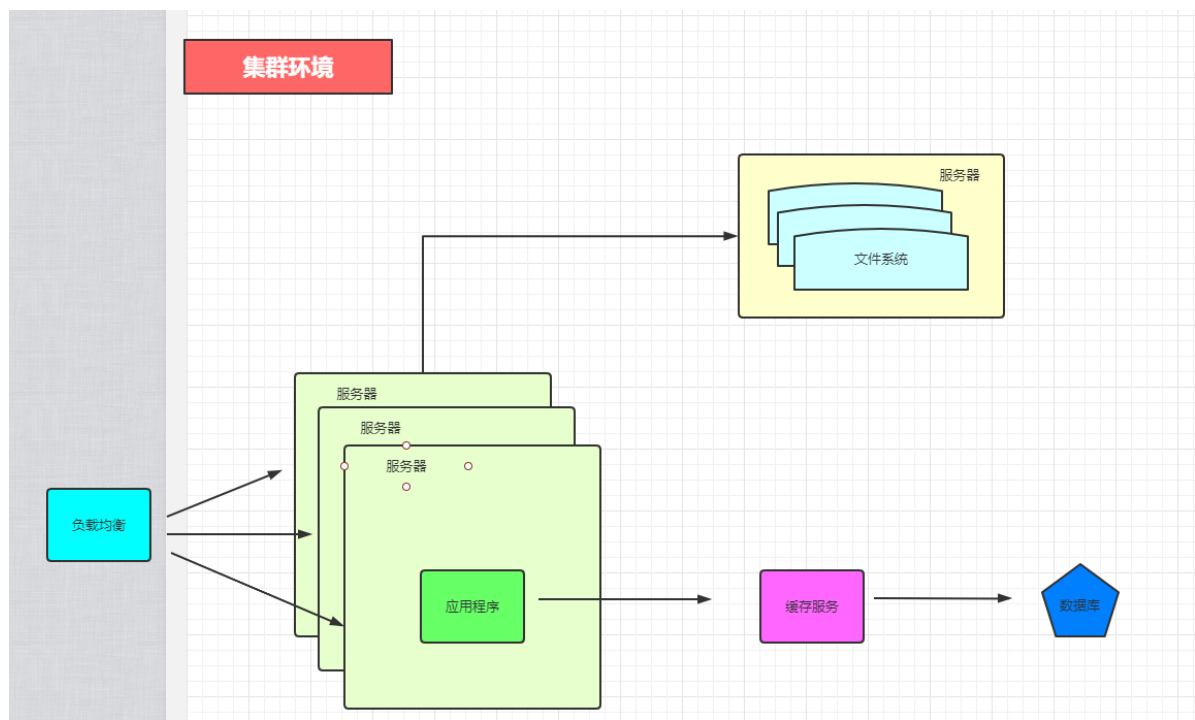


缓存使用

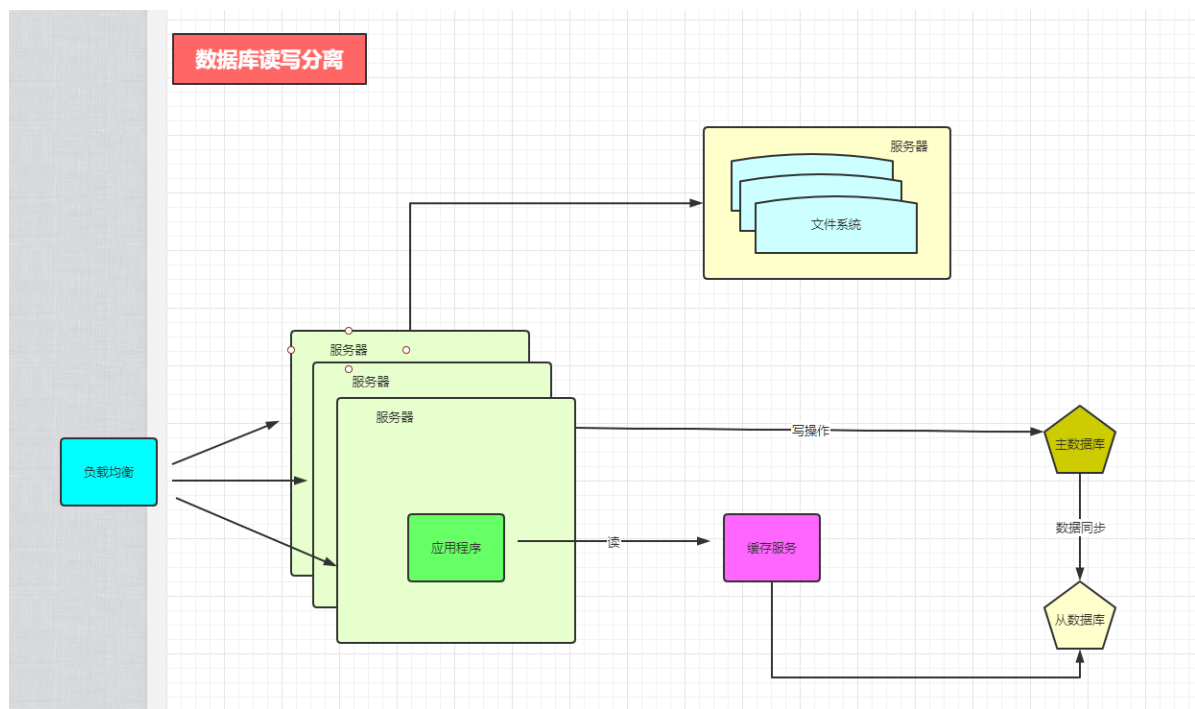
缓存使用



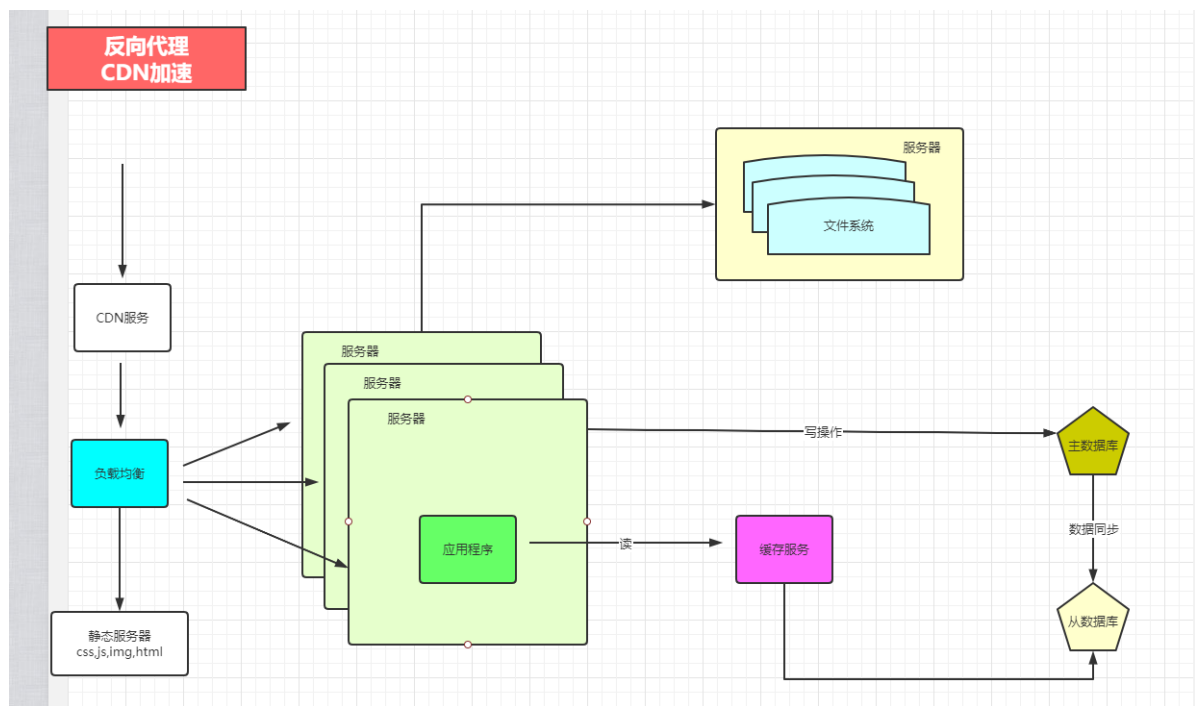
集群处理



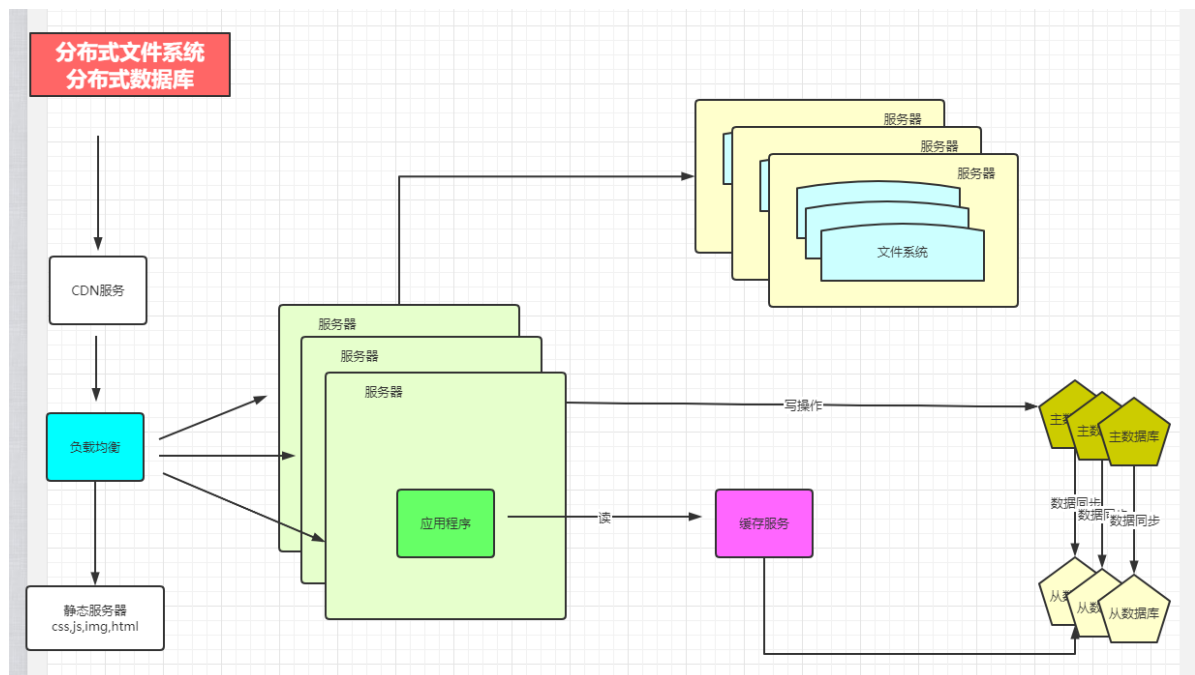
数据库的读写分离



反向代理和CDN加速



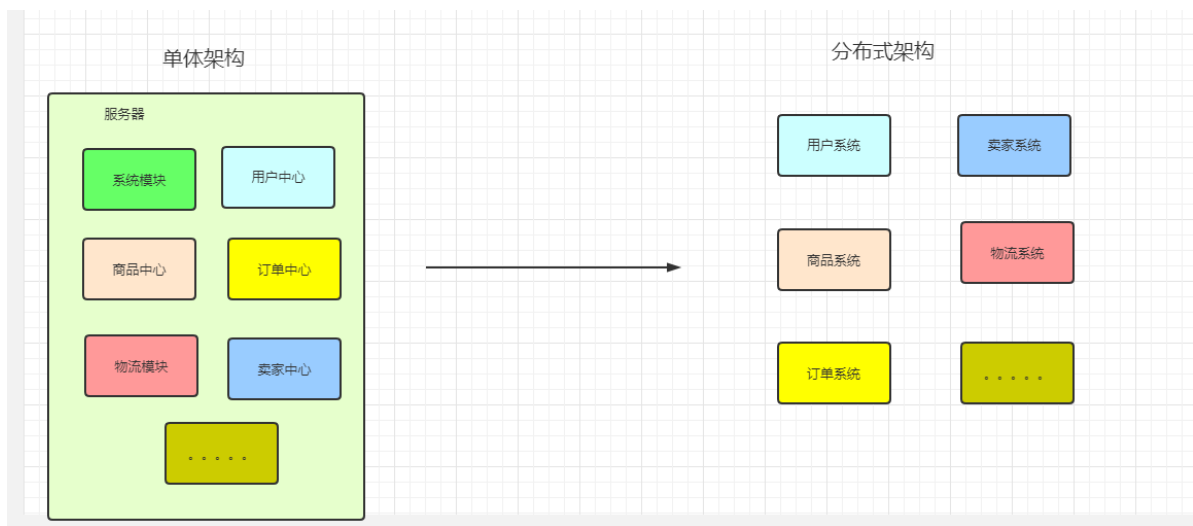
分布式文件系统和分布式数据库



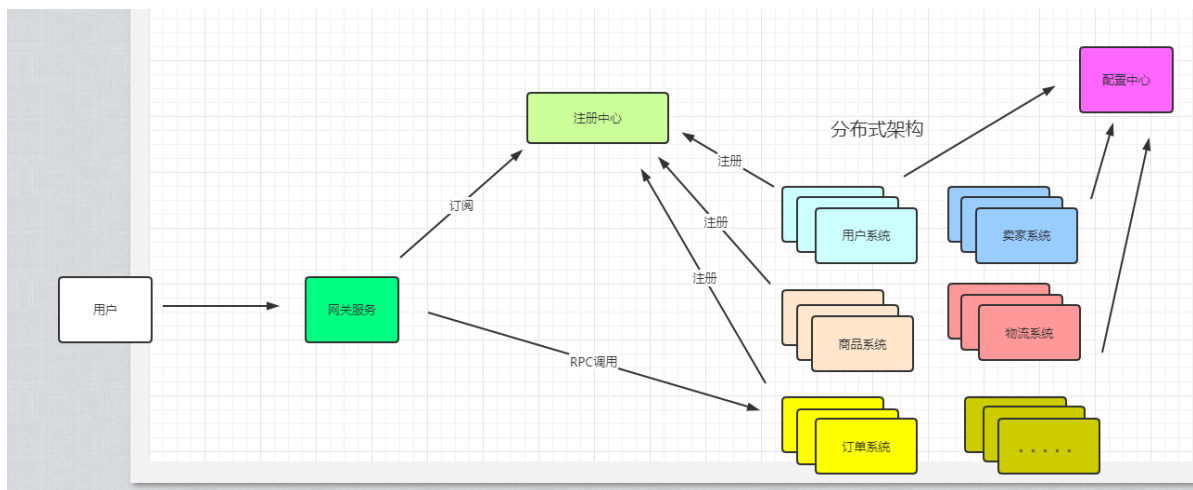
还可以通过NoSQL数据和搜索引擎等来提供系统的处理能力

1.2 分布式架构

在上面所介绍的单体架构的基础上演变出来的。也就是将单体架构中相对独立的模块抽取出来建立成独立的系统，降低了各个模块之间的耦合性，提高了整体系统的性能和稳定性。



相比较在单体架构下的场景，在分布式环境下又会多出很多要处理的问题，比如服务的发现，服务的治理，服务调用，配置中心等等问题



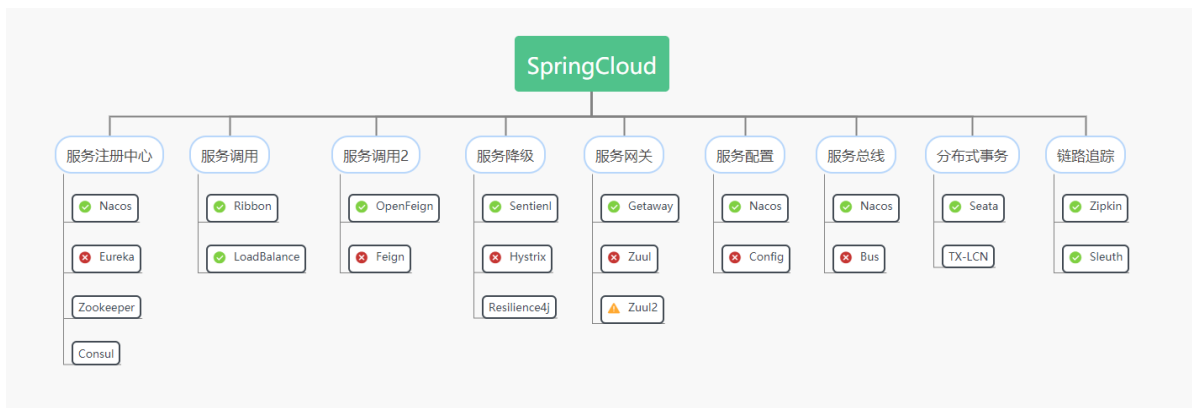
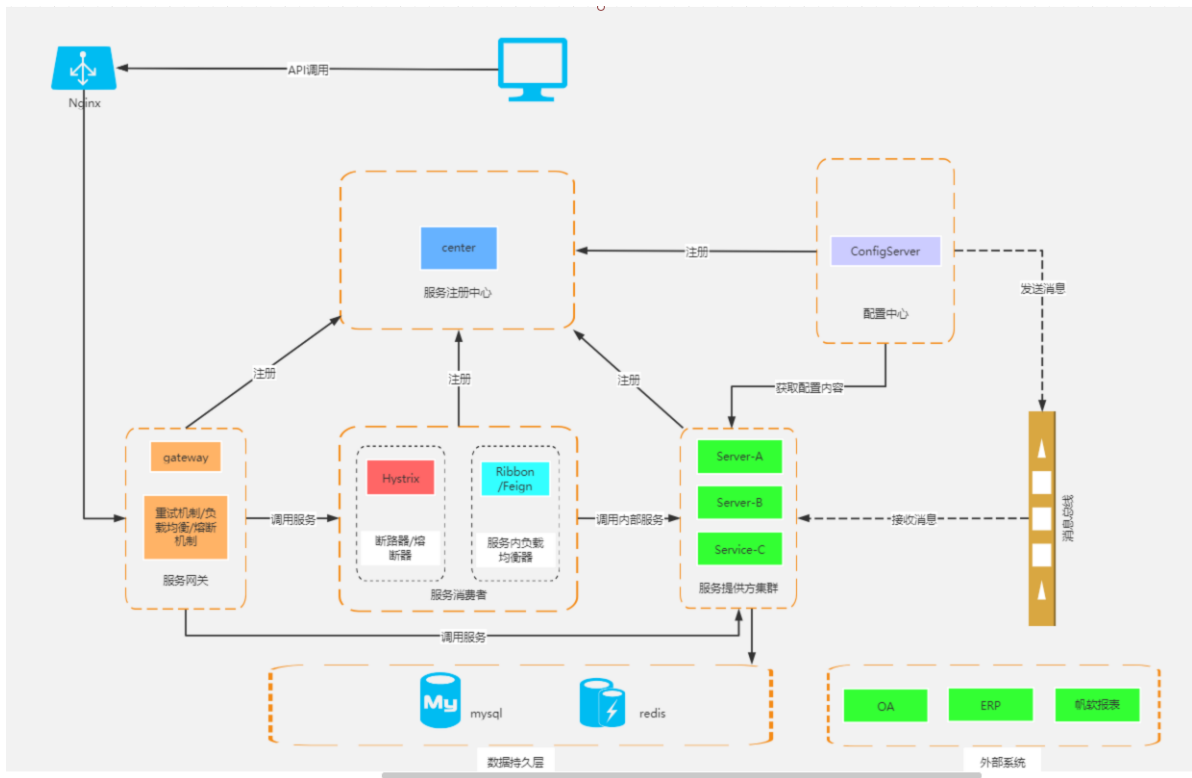
微服务其实也就是分布式！微服务是在分布式的基础上抽取了公共模块，增加的复用性。让整个系统架构更加的合理科学。而我们要学习的SpringCloud是分布式微服务系统的一套很好的解决方案，在SpringCloud中提供了我们在微服务中会遇到的各种问题的解决方案。

2. SpringCloud

官网: <https://spring.io/projects/spring-cloud>

中文网: <https://www.springcloud.cc/>

SpringCloud是一系列框架的有序集合，微服务架构的集大成者，将一系列优秀的组件进行了整合。基于SpringBoot来构建。



3. 五大组件

在SpringCloud中提供的子系统非常多，而且每个都有各自特殊的功能，有些是不可或缺的，有些则是可有可无的。那么中SpringCloud中有五大核心组件，分别是：

3.1 服务发现

注册中心：Netflix Eureka

3.2 负载均衡

Ribbon, Feign

3.3 断路器

Netflix Hystrix

3.4 服务网关

Netflix Zuul

3.5 分布式配置

SpringCloud Config

二、Eureka

1. 什么是服务注册中心

服务注册中心是服务实现服务化管理的核心组件,类似于目录服务的作用,主要用来存储服务信息,譬如提供者 url 串、路由信息等。服务注册中心是 SOA 架构中最基础的设施之一。

1.1 服务注册中心的作用

1. 服务的注册
2. 服务的发现

1.2 常见的注册中心有哪些

1. Dubbo 的注册中心 Zookeeper
2. SpringCloud 的注册中心 Eureka, Nacos

1.3 解决了什么问题

1. 服务管理
2. 服务的依赖关系管理

1.4 什么是 Eureka 注册中心

Eureka 是 Netflix 开发的服务发现组件,本身是一个基于 REST 的服务。Spring Cloud将它集成在其子项目 spring-cloud-netflix 中,以实现 Spring Cloud 的服务注册于发现,同时还提供了负载均衡、故障转移等能力。

1.5 Eureka 注册中心三种角色

Eureka Server

通过 Register、Get、Renew 等接口提供服务的注册和发现。

Application Service (Service Provider)

服务提供方把自身的服务实例注册到 Eureka Server 中

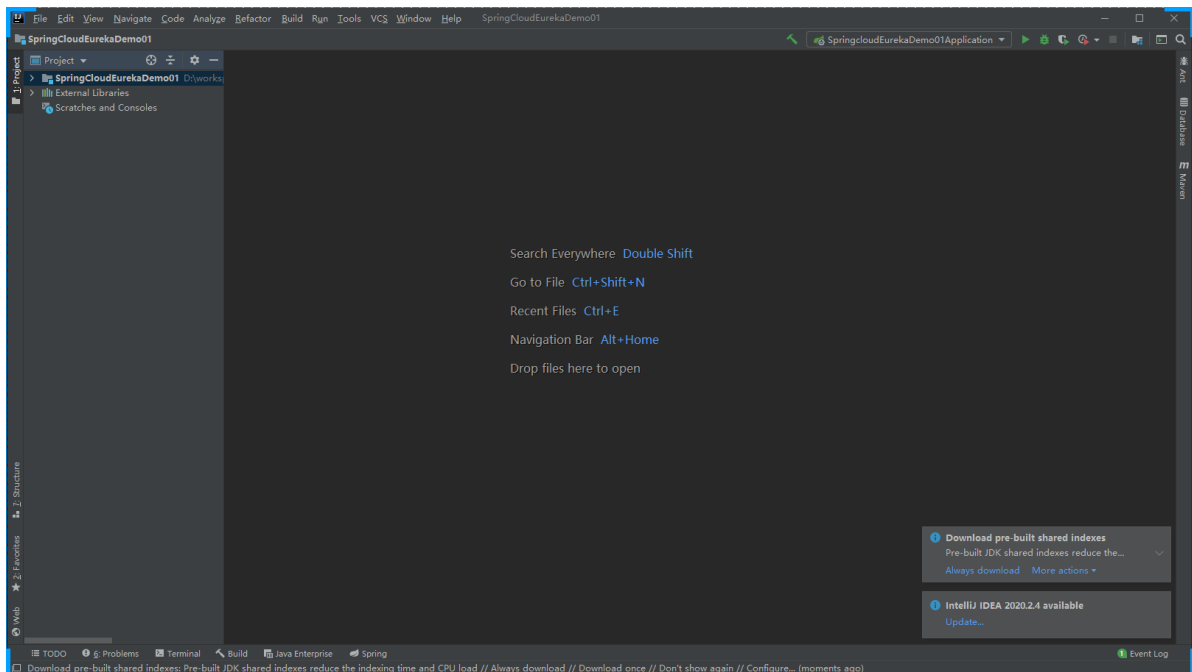
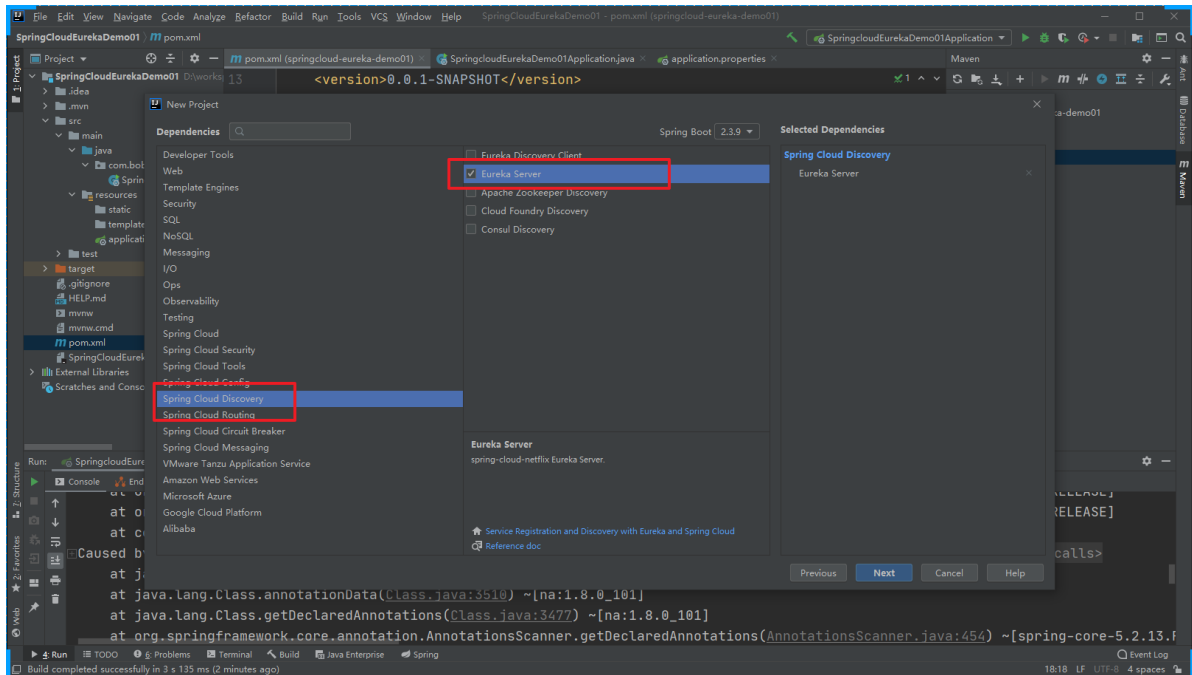
Application Client (Service Consumer)

服务调用方通过 Eureka Server 获取服务列表,消费服务

2.入门案例

2.1 创建项目

创建一个SpringBoot项目即可



2.2 添加相关的依赖

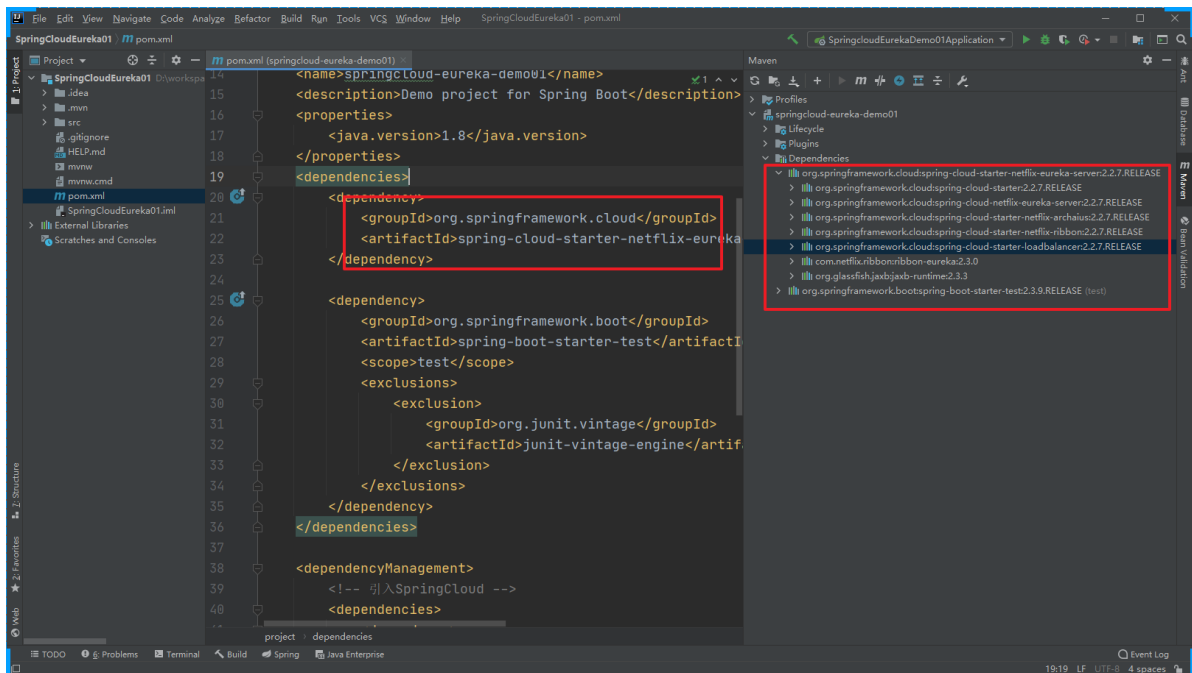
```
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
<exclusions>
<exclusion>
<groupId>org.junit.vintage</groupId>
<artifactId>junit-vintage-engine</artifactId>
```

```

        </exclusion>
    </exclusions>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
</dependencies>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>Dalston.SR5</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>

```



2.3 启动器

我们需要在启动器中添加@EnableEurekaServer,表明这时一个Eureka的服务端

```

package com.bobo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@EnableEurekaServer // 放开Eureka 服务端
@SpringBootApplication
public class SpringcloudEurekaDemo01Application {

    public static void main(String[] args) {

```

```
        SpringApplication.run(SpringcloudEurekaDemo01Application.class, args);
    }
}
```

2.4 配置文件

```
spring.application.name=eureka-server
server.port=8761

# 是否将自己注册到Eureka中，默认true 设置为false
eureka.client.register-with-eureka=false

# 是否从Eureka中获取注册的服务信息 默认为true 设置为false
eureka.client.fetch-registry=false
```

2.5 启动服务

启动SpringBoot服务，然后访问 <http://localhost:8761> 即可

The screenshot shows the Spring Eureka web interface. At the top, there's a navigation bar with the Spring Eureka logo and links for HOME and LAST 1000 SINCE STARTUP. The main content area is divided into several sections:

- System Status:** A table showing environment (test), data center (default), current time (2021-03-25T17:51:58+0800), uptime (00:00), lease expiration enabled (false), renew threshold (1), and renewals (last min) (0).
- DS Replicas:** A section showing the local host (localhost).
- Instances currently registered with Eureka:** A table with columns Application, AMIs, Availability Zones, and Status. It shows "No instances available".
- General Info:** A table with columns Name and Value. It shows various system metrics like total-avail-memory (589mb), environment (test), num-of-cpus (16), current-memory-usage (117mb (19%)), server-uptime (00:00), registered-replicas (http://localhost:8761/eureka/), and available-replicas (http://localhost:8761/eureka/).

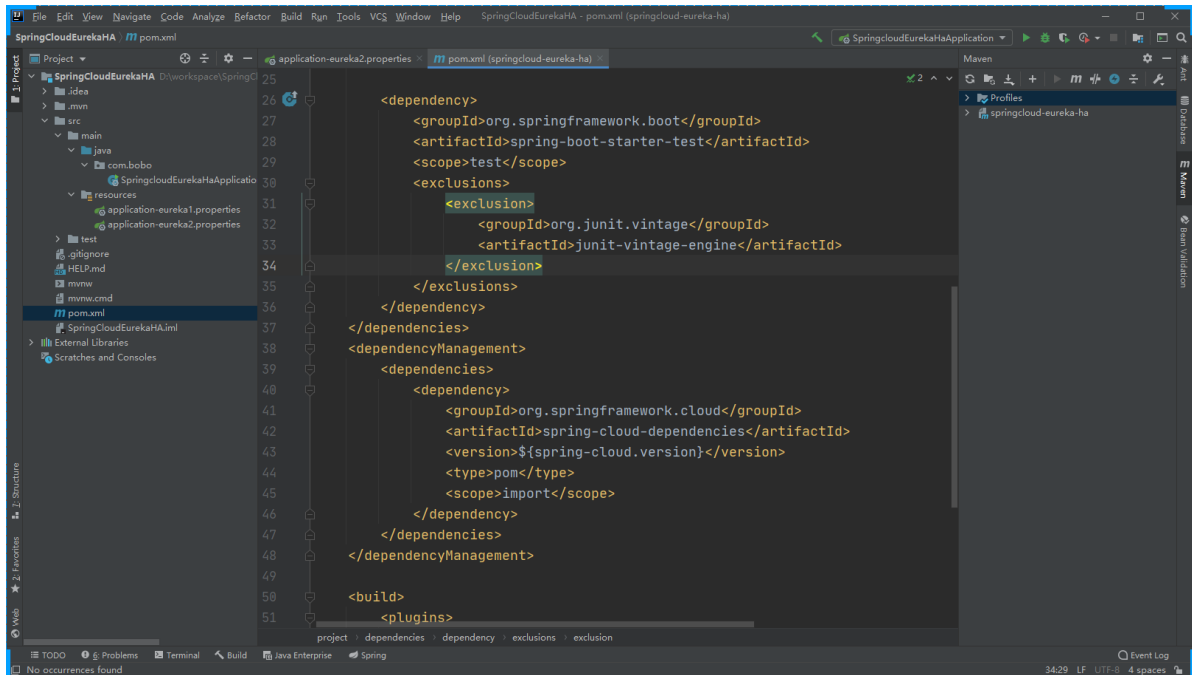
表示启动成功

3. Eureka集群

Eureka作为我们的注册中心，那么在整个系统环境中的作用是非常重要的，如果Eureka服务停掉的话，那么整个系统就不可用了，为了提高整个系统的稳定性，我们需要对Eureka做集群部署。

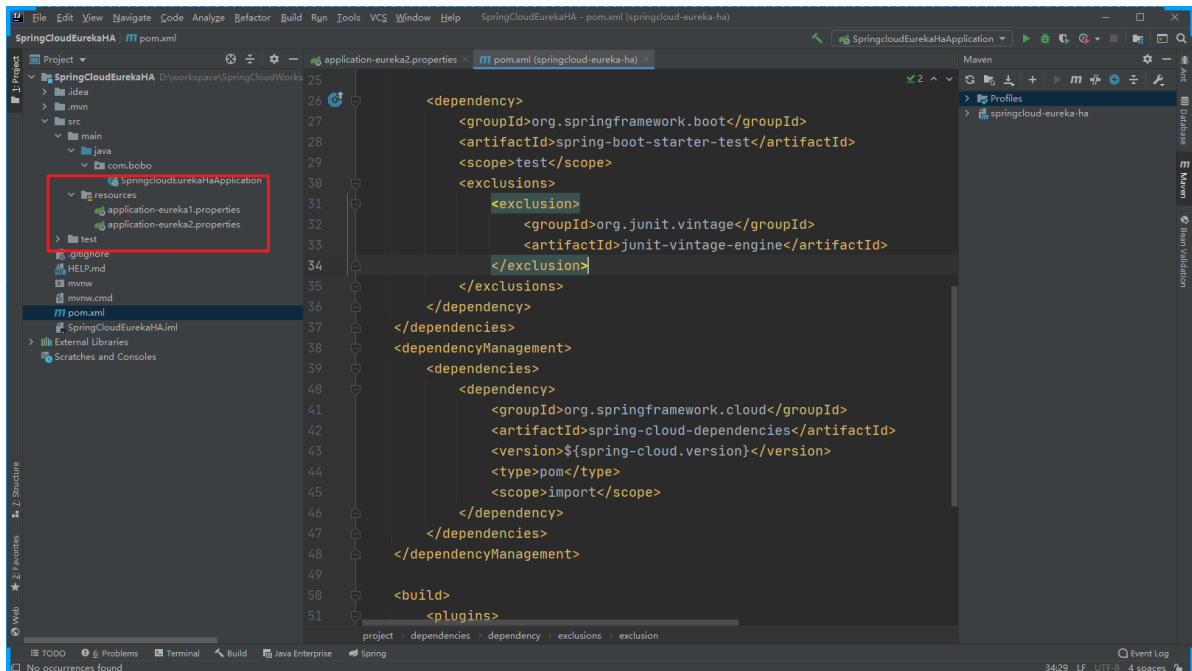
3.1 创建项目

创建一个基于Eureka的单机版的项目即可



3.2 配置文件

在搭建Eureka集群环境时，需要添加多个配置文件，并且使用SpringBoot的多环境配置，集群中需要多少个节点那么就添加多少的配置文件



eureka1的配置文件

```

spring.application.name=eureka-server
server.port=8761

# 设置eureka的实例名称 与配置文件中的变量为主
eureka.instance.hostname=eureka1
# 设置注册中心的地址，指向另一个注册中心 192.168.100.120
eureka.client.service-url.defaultZone=http://192.168.100.121:8761/eureka

```

eureka2的配置文件

```

spring.application.name=eureka-server
server.port=8761

# 设置eureka的实例名称 与配置文件中的变量为主
eureka.instance.hostname=eureka2
# 设置注册中心的地址，指向另一个注册中心 192.168.100.121
eureka.client.service-url.defaultZone=http://192.168.100.120:8761/eureka

```

3.3 logback

添加logback的配置文件

```

<?xml version="1.0" encoding="UTF-8" ?>
<configuration>
<!--定义日志文件的存储地址 勿在 LogBack 的配置中使用相对路径-->
    <property name="LOG_HOME" value="${catalina.base}/logs/" />
    <!-- 控制台输出 -->
    <appender name="Stdout" class="ch.qos.logback.core.ConsoleAppender">
        <!-- 日志输出编码 -->
        <layout class="ch.qos.logback.classic.PatternLayout">
            <!--格式化输出：%d表示日期，%thread表示线程名，%-5level：级别从左显示5个字符宽度%msg：日志消息，%n是换行符-->
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50}
- %msg%n
        </pattern>
        </layout>
    </appender>
    <!-- 按照每天生成日志文件 -->
    <appender name="RollingFile"
class="ch.qos.logback.core.rolling.RollingFileAppender">
        <rollingPolicy
class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
            <!--日志文件输出的文件名-->
            <FileNamePattern>${LOG_HOME}/server.%d{yyyy-MM-dd}.log</FileNamePattern>
            <MaxHistory>30</MaxHistory>
        </rollingPolicy>
        <layout class="ch.qos.logback.classic.PatternLayout">
            <!--格式化输出：%d表示日期，%thread表示线程名，%-5level：级别从左显示5个字符宽度%msg：日志消息，%n是换行符-->

```

```

        <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50}
- %msg%n
    </pattern>
</layout>
    <!--日志文件最大的大小-->
    <triggeringPolicy
class="ch.qos.logback.core.rolling.SizeBasedTriggeringPolicy">
        <MaxFileSize>10MB</MaxFileSize>
    </triggeringPolicy>
</appender>

    <!-- 日志输出级别 -->
    <root level="DEBUG">
        <appender-ref ref="Stdout" />
        <appender-ref ref="RollingFile" />
    </root>

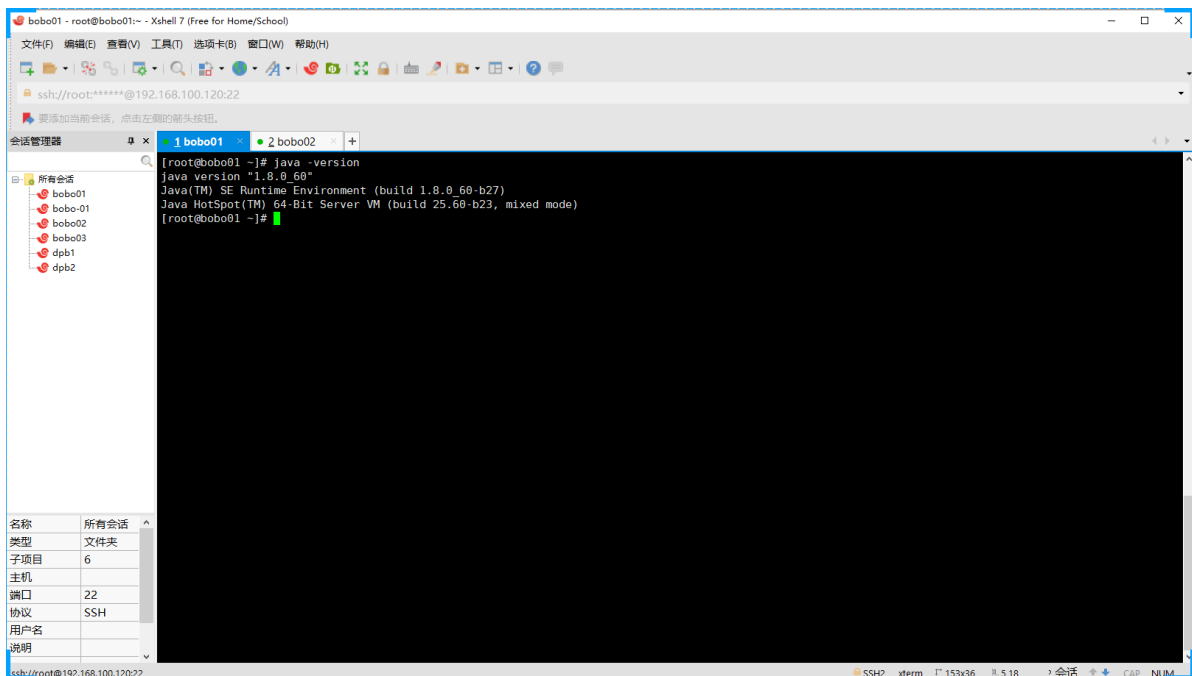
<!--日志异步到数据库 -->
<!--      <appender name="DB" class="ch.qos.logback.classic.db.DBAppender">
    日志异步到数据库
        <connectionSource
class="ch.qos.logback.core.db.DriverManagerConnectionSource">
    连接池
            <dataSource class="com.mchange.v2.c3p0.ComboPooledDataSource">
                <driverClass>com.mysql.jdbc.Driver</driverClass>
                <url>jdbc:mysql://127.0.0.1:3306/databaseName</url>
                <user>root</user>
                <password>root</password>
            </dataSource>
        </connectionSource>
    </appender> -->

</configuration>

```

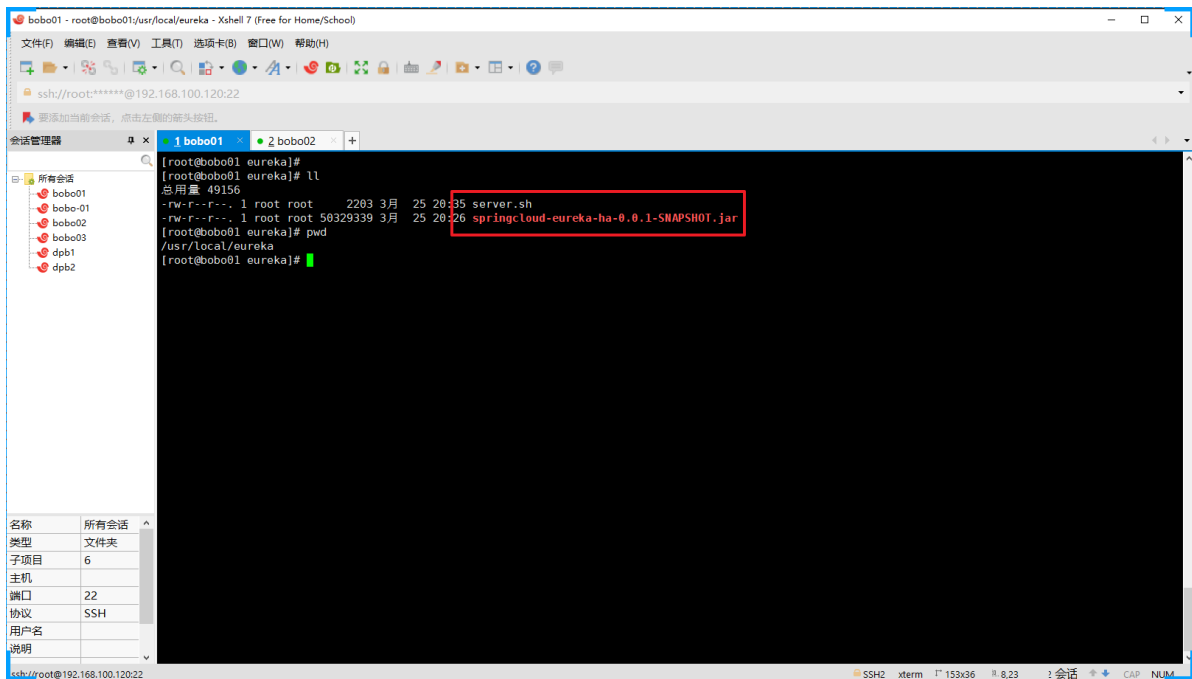
3.4 集群环境的搭建

部署环境：需要安装jdk1.8,正确配置对应的环境变量信息，注意关闭防火墙，或者放开8761端口



上传jar包

在/usr/local创建一个eureka目录，将项目的jar包拷贝到/usr/local/eureka中



然后创建一个启动脚本

```
#!/bin/bash

cd `dirname $0`

CUR_SHELL_DIR=`pwd`
CUR_SHELL_NAME=`basename ${BASH_SOURCE}`

JAR_NAME="项目名称"
JAR_PATH=$CUR_SHELL_DIR/$JAR_NAME
```

```

#JAVA_MEM_OPTS="-server -Xms1024m -Xmx1024m -XX:PermSize=128m"
JAVA_MEM_OPTS=""

SPRING_PROFILES_ACTIV="-Dspring.profiles.active=配置文件变量名称"
#SPRING_PROFILES_ACTIV=""
LOG_DIR=$CUR_SHELL_DIR/logs
LOG_PATH=$LOG_DIR/${JAR_NAME%..log}

echo_help()
{
    echo -e "syntax: sh $CUR_SHELL_NAME start|stop"
}

if [ -z $1 ];then
    echo_help
    exit 1
fi

if [ ! -d "$LOG_DIR" ];then
    mkdir "$LOG_DIR"
fi

if [ ! -f "$LOG_PATH" ];then
    touch "$LOG_PATH"
fi

if [ "$1" == "start" ];then

    # check server
    PIDS=`ps --no-heading -C java -f --width 1000 | grep $JAR_NAME | awk '{print $2}'`
    if [ -n "$PIDS" ]; then
        echo -e "ERROR: The $JAR_NAME already started and the PID is ${PIDS}."
        exit 1
    fi

    echo "Starting the $JAR_NAME..."

    # start
    nohup java $JAVA_MEM_OPTS -jar $SPRING_PROFILES_ACTIV $JAR_PATH >> $LOG_PATH
2>&1 &

    COUNT=0
    while [ $COUNT -lt 1 ]; do
        sleep 1
        COUNT=`ps --no-heading -C java -f --width 1000 | grep "$JAR_NAME" | awk
'{print $2}' | wc -l`
        if [ $COUNT -gt 0 ]; then
            break
        fi
    done
    PIDS=`ps --no-heading -C java -f --width 1000 | grep "$JAR_NAME" | awk
'{print $2}'`
    echo "${JAR_NAME} Started and the PID is ${PIDS}."
    echo "You can check the log file in ${LOG_PATH} for details."

elif [ "$1" == "stop" ];then

```

```

PIDS=`ps --no-heading -C java -f --width 1000 | grep $JAR_NAME | awk '{print $2}'`
if [ -z "$PIDS" ]; then
    echo "ERROR:The $JAR_NAME does not started!"
    exit 1
fi

echo -e "Stopping the $JAR_NAME..."

for PID in $PIDS; do
    kill $PID > /dev/null 2>&1
done

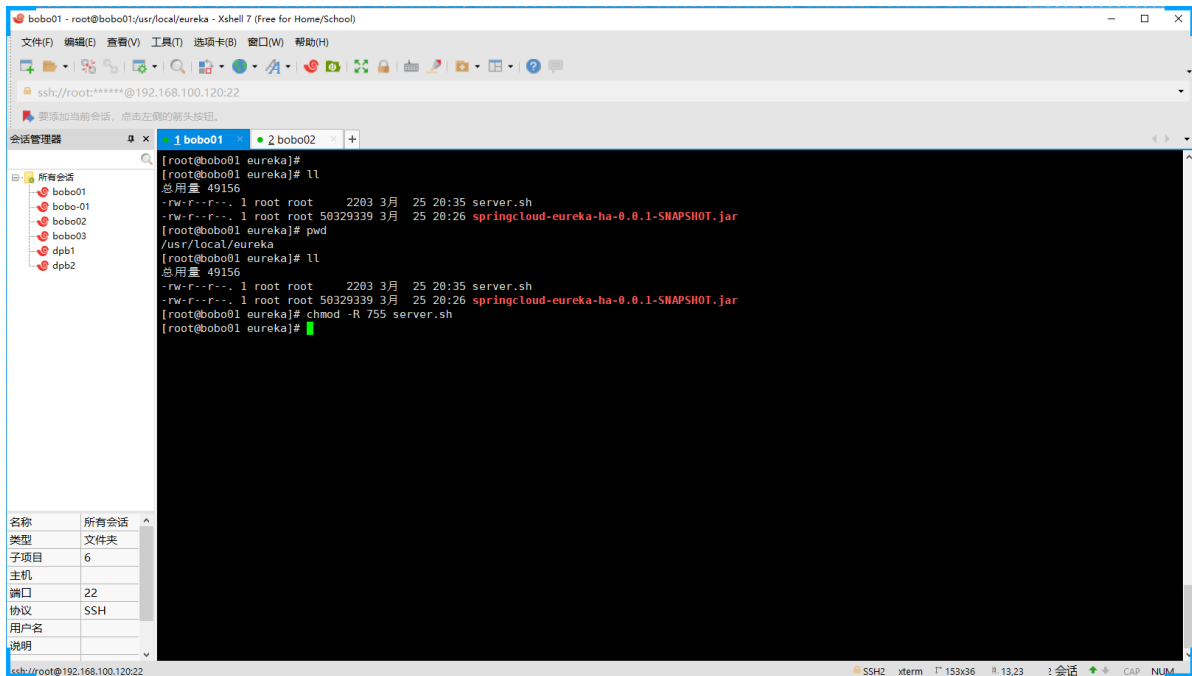
COUNT=0
while [ $COUNT -lt 1 ]; do
    sleep 1
    COUNT=1
    for PID in $PIDS ; do
        PID_EXIST=`ps --no-heading -p $PID`
        if [ -n "$PID_EXIST" ]; then
            COUNT=0
            break
        fi
    done
done

echo -e "${JAR_NAME} Stopped and the PID is ${PIDS}."
else
    echo_help
    exit 1
fi

```

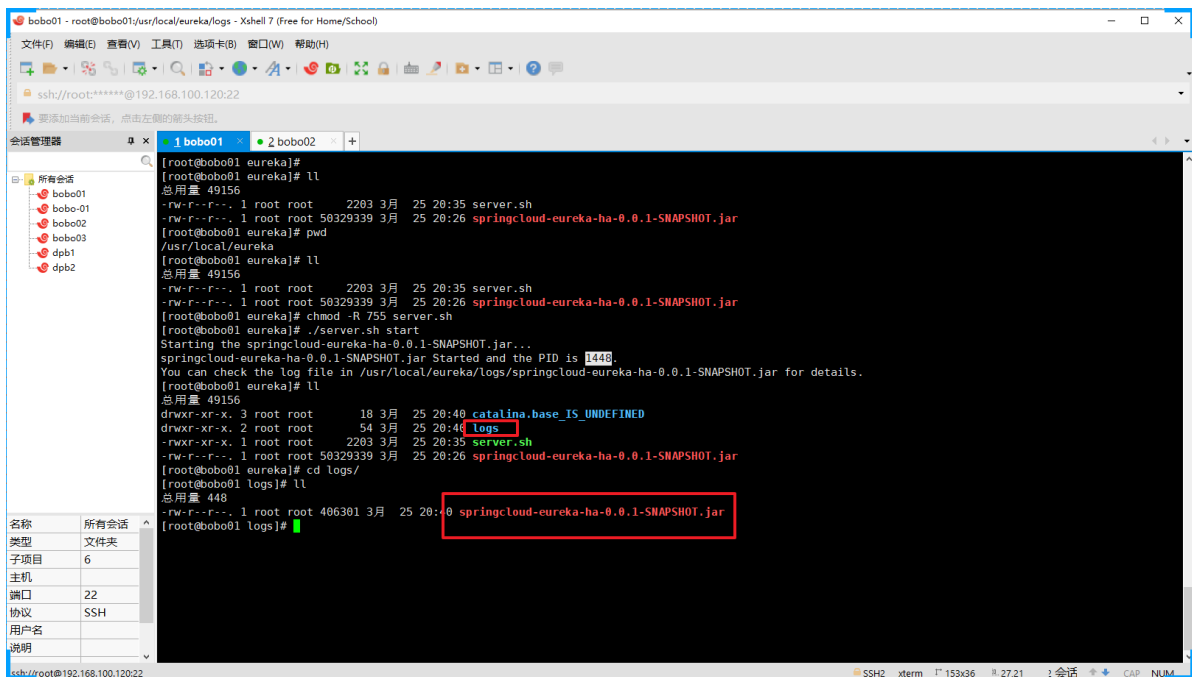
设置脚本的运行权限

```
chmod -R 755 server.sh
```

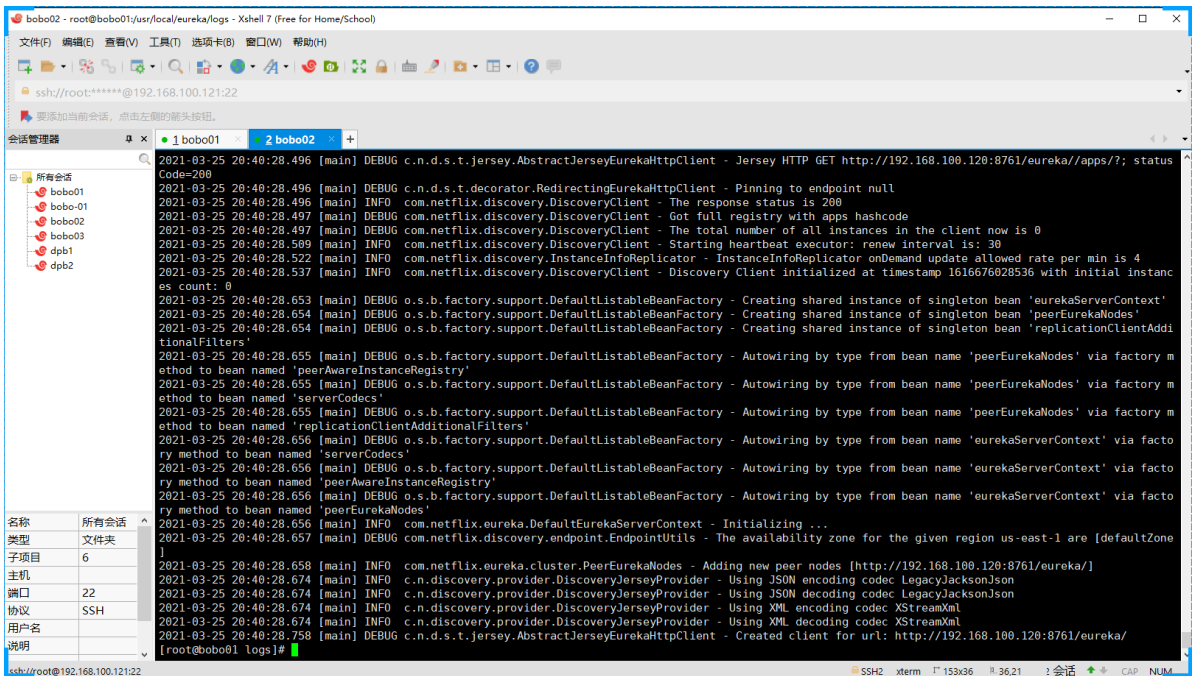


分别启动服务

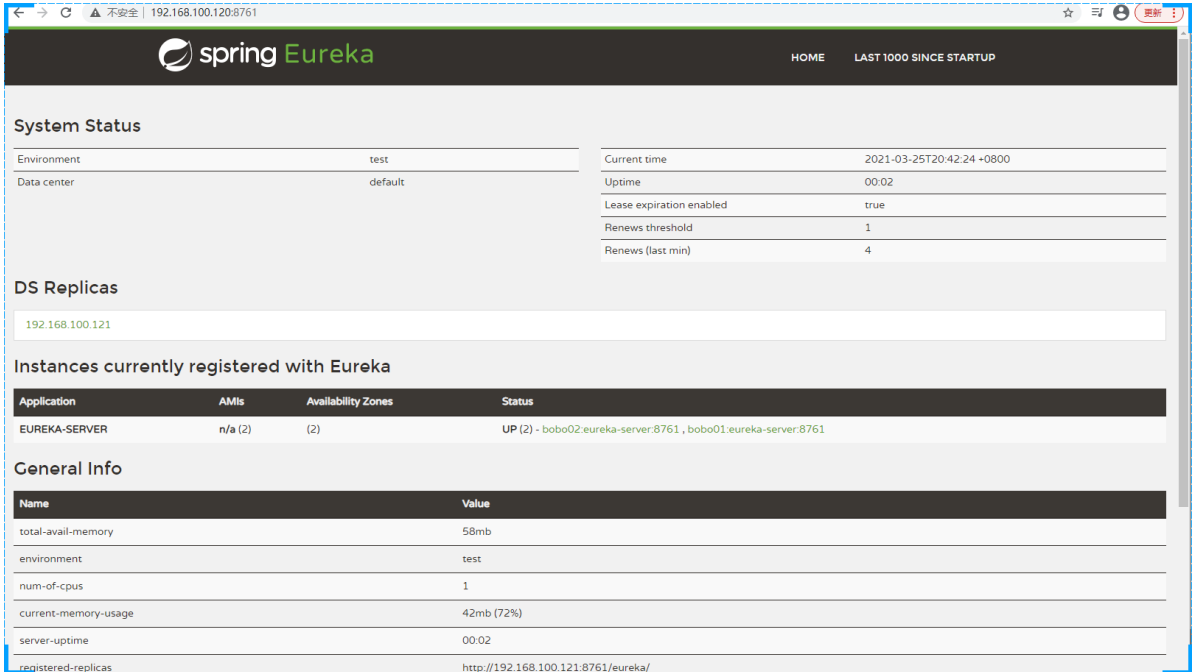
```
./server.sh start // 启动  
./server.sh stop // 关闭
```



日志文件信息

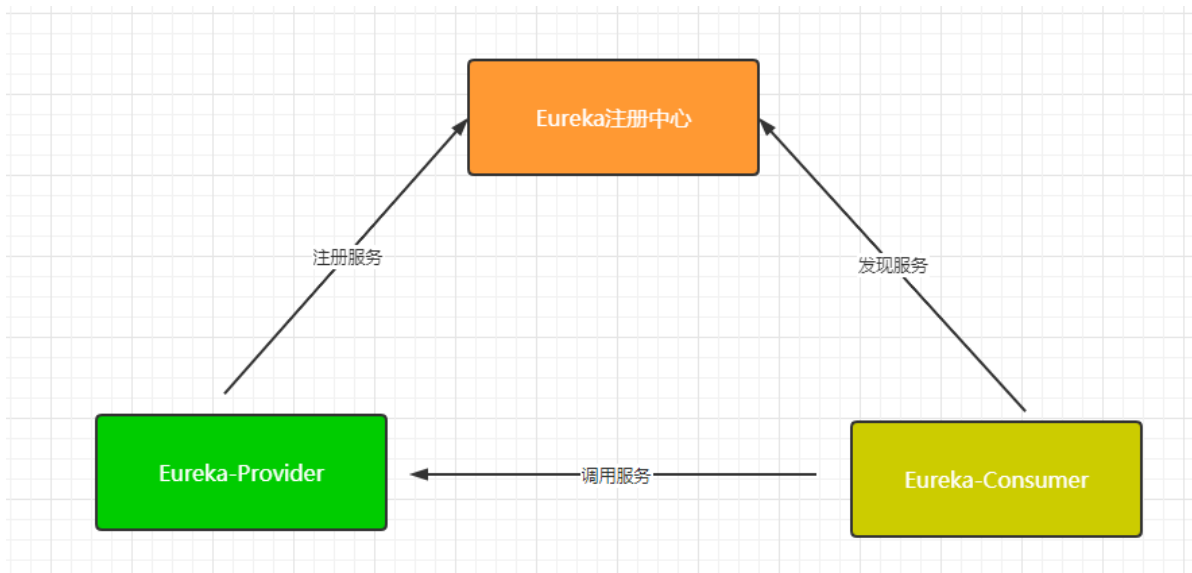


访问测试



4. 服务案例

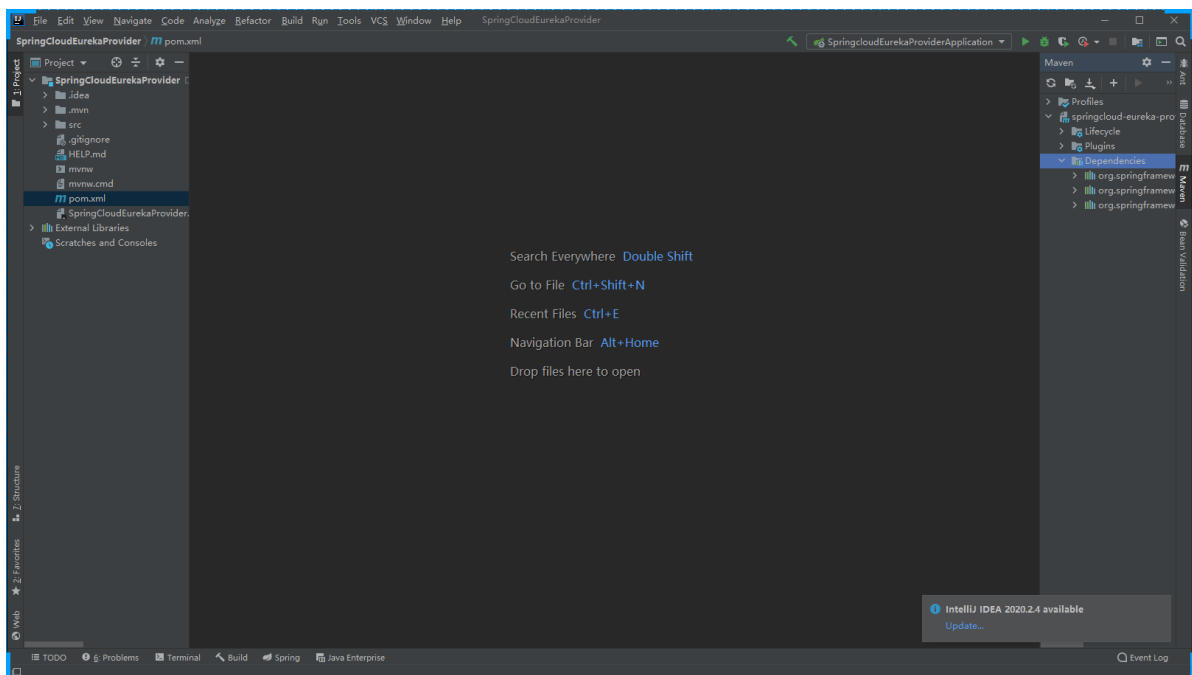
Eureka作为注册中心，要完成的功能是服务的注册和服务的发现，有如下的案例结构



4.1 服务提供者

对外提供功能的服务，需要到Eureka中注册服务

创建一个普通的SpringBoot项目



添加对应的配置文件

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

```

        <exclusions>
            <exclusion>
                <groupId>org.junit.vintage</groupId>
                <artifactId>junit-vintage-engine</artifactId>
            </exclusion>
        </exclusions>
    </dependency>
    <!-- 表示这是一个SpringCloud Eureka 的Client -->
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
    </dependency>
</dependencies>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>Hoxton.SR10</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>

```

客户端的启动类，在低版本中我们需要添加对应的注解(@EnableEurekaClient),但是在高版本中我们就不需要添加了，默认就是放开的

```

package com.bobo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;

/**
 * 在Eureka低版本中我们需要添加 @EnableEurekaClient @EnableDiscoveryClient
 */
// @EnableEurekaClient
@SpringBootApplication
public class SpringcloudEurekaProviderApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringcloudEurekaProviderApplication.class, args);
    }

}

```

修改配置信息

```
spring.application.name=eureka-provider
server.port=9090
```

设置服务注册中心地址 执行Eureka服务端 如果有多个注册地址 那么用逗号连接
eureka.client.service-
url.defaultZone=http://192.168.100.120:8761/eureka/,http://192.168.100.121:8761/
eureka/

启动程序后我们可以在Eureka服务中发现我们的provider

The screenshot shows the Spring Eureka web interface in a browser. The page has a dark header with the 'spring Eureka' logo and navigation links 'HOME' and 'LAST 1000 SINCE STARTUP'. Below the header, there's a 'System Status' section with two tables. The left table shows 'Environment: test' and 'Data center: default'. The right table shows 'Current time: 2021-03-25T21:47:31 +0800', 'Uptime: 01:07', 'Lease expiration enabled: false', 'Renews threshold: 5', and 'Renews (last min): 4'. Below this, a red warning message states: 'EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.' Under 'DS Replicas', there's a search bar containing '192.168.100.120'. The 'Instances currently registered with Eureka' section features a table with columns 'Application', 'AMIs', 'Availability Zones', and 'Status'. Two instances are listed: 'EUREKA-PROVIDER' with status 'UP (1) - LAPTOP-O17142BD:eureka-provider:9090' and 'EUREKA-SERVER' with status 'UP (2) - bobo02:eureka-server:8761, bobo01:eureka-server:8761'. The 'EUREKA-PROVIDER' row is highlighted with a red border. At the bottom, the 'General Info' section shows a table with 'Name' and 'Value' columns, listing 'total-avail-memory: 58mb', 'environment: test', and 'num-of-cpus: 1'.

Application	AMIs	Availability Zones	Status
EUREKA-PROVIDER	n/a (1)	(1)	UP (1) - LAPTOP-O17142BD:eureka-provider:9090
EUREKA-SERVER	n/a (2)	(2)	UP (2) - bobo02:eureka-server:8761, bobo01:eureka-server:8761

添加对外提供的服务

```
package com.bobo.pojo;

public class User {

    private Integer userId;

    private String userName;

    private Integer age;

    public Integer getUserId() {
        return userId;
    }

    public void setUserId(Integer userId) {
        this.userId = userId;
    }

    public String getUserName() {
        return userName;
    }

    public void setUserName(String userName) {
        this.userName = userName;
    }
}
```

```

    }

    public Integer getAge() {
        return age;
    }

    public void setAge(Integer age) {
        this.age = age;
    }

    public User(Integer userId, String userName, Integer age) {
        this.userId = userId;
        this.userName = userName;
        this.age = age;
    }

    public User() {
    }
}

```

添加一个处理请求的控制器

```

package com.bobo.controller;

import com.bobo.pojo.User;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;
import java.util.List;

@RestController
public class UserController {

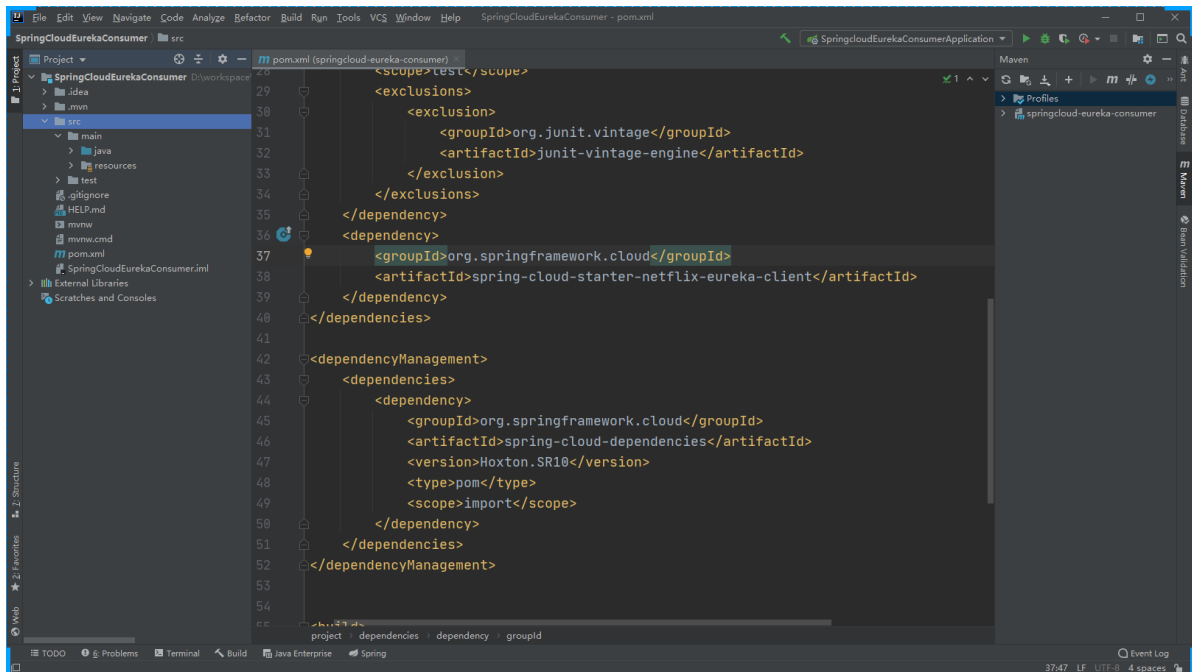
    @RequestMapping("/user")
    public List<User> getUsers(){
        List<User> list = new ArrayList<>();
        list.add(new User(1,"张三",18));
        list.add(new User(2,"李四",19));
        list.add(new User(3,"王五",20));
        return null;
    }
}

```

4.2 服务消费者

想要获取服务提供者中提供的用户信息，但是是消费者没法直接找到服务提供者，需要通过Eureka注册中心获取服务提供者的访问地址信息，然后再去访问

创建一个SpringBoot项目



然后添加相关的依赖

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
        <exclusions>
            <exclusion>
                <groupId>org.junit.vintage</groupId>
                <artifactId>junit-vintage-engine</artifactId>
            </exclusion>
        </exclusions>
    </dependency>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
    </dependency>
</dependencies>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>Hoxton.SR10</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

客户端的启动类，在低版本中我们需要添加对应的注解(@EnableEurekaClient),但是在高版本中我们就不需要添加了，默认就是放开的

配置文件

```
spring.application.name=eureka-consumer
server.port=9091

# 设置服务注册中心地址 执行Eureka服务端 如果有多个注册地址 那么用逗号连接
eureka.client.service-
url.defaultZone=http://192.168.100.120:8761/eureka/,http://192.168.100.121:8761/
eureka/
```

完成服务调用

```
package com.bobo.pojo;

public class User {

    private Integer userId;

    private String userName;

    private Integer age;

    public Integer getUserId() {
        return userId;
    }

    public void setUserId(Integer userId) {
        this.userId = userId;
    }

    public String getUserName() {
        return userName;
    }

    public void setUserName(String userName) {
        this.userName = userName;
    }

    public Integer getAge() {
        return age;
    }

    public void setAge(Integer age) {
        this.age = age;
    }

    public User(Integer userId, String userName, Integer age) {
        this.userId = userId;
        this.userName = userName;
        this.age = age;
    }
}
```

```

    }

    public User() {
    }
}

```

service

```

package com.bobo.service;

import com.bobo.pojo.User;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.client.loadbalancer.LoadBalancerClient;
import org.springframework.core.ParameterizedTypeReference;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

import java.util.List;

@Service
public class UserService {

    /**
     * Ribbon 实现的负载均衡
     * LoadBalancerClient 通过服务名称可以获取对应服务的相关信息
     * ip 端口 等
     */
    @Autowired
    private LoadBalancerClient loadBalancerClient;

    /**
     * 远程调用 服务提供者获取用户信息的方法
     * 1.发现服务
     * 2.调用服务
     * @return
     */
    public List<User> getUsers(){
        // 1. 服务发现
        // 获取服务提供者的信息 ServiceInstance封装的有相关的信息
        ServiceInstance instance = loadBalancerClient.choose("eureka-provider");
        StringBuilder sb = new StringBuilder();
        // http://localhost:9090/user
        sb.append("http://")
            .append(instance.getHost())
            .append(":")
            .append(instance.getPort())
            .append("/user");
        System.out.println(sb.toString());
        // 2. 服务调用 SpringMVC中提供的有 调用组件 RestTemplate
        RestTemplate rt = new RestTemplate();
        ParameterizedTypeReference<List<User>> type = new
        ParameterizedTypeReference<List<User>>() {};
    }
}

```

```

        ResponseEntity<List<User>> response = rt.exchange(sb.toString(),
HttpMethod.GET, null, type);
        List<User> list = response.getBody();
        return list;
    }
}

```

控制器

```

package com.bobo.controller;

import com.bobo.pojo.User;
import com.bobo.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

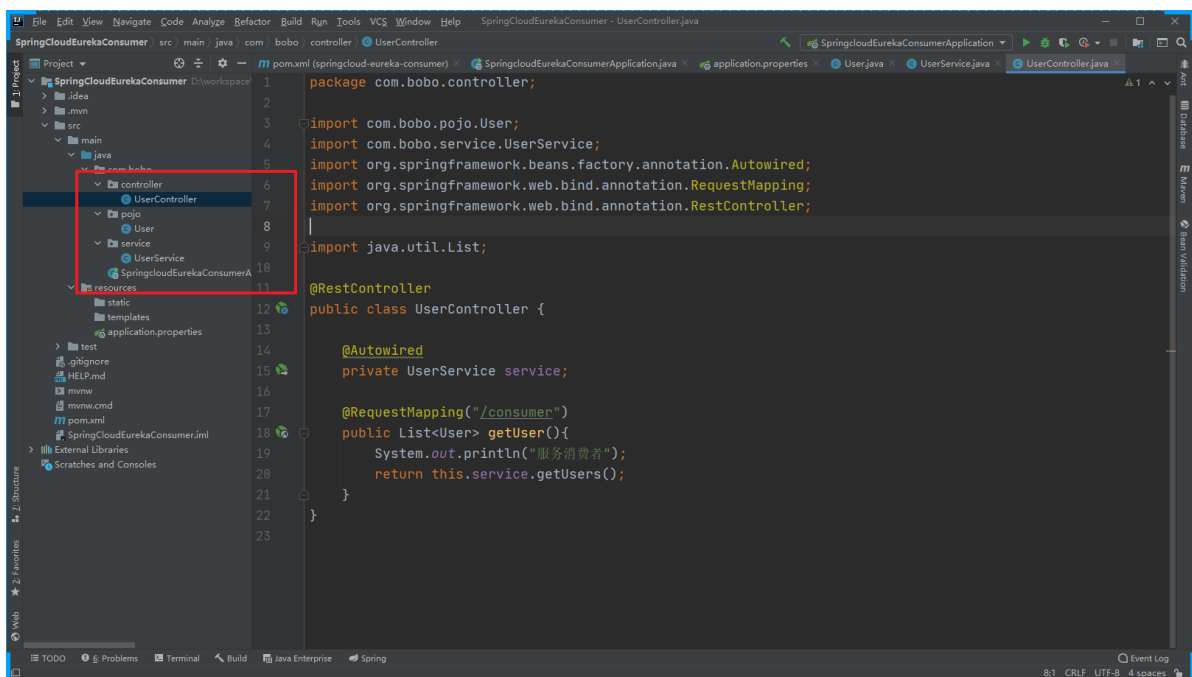
import java.util.List;

@RestController
public class UserController {

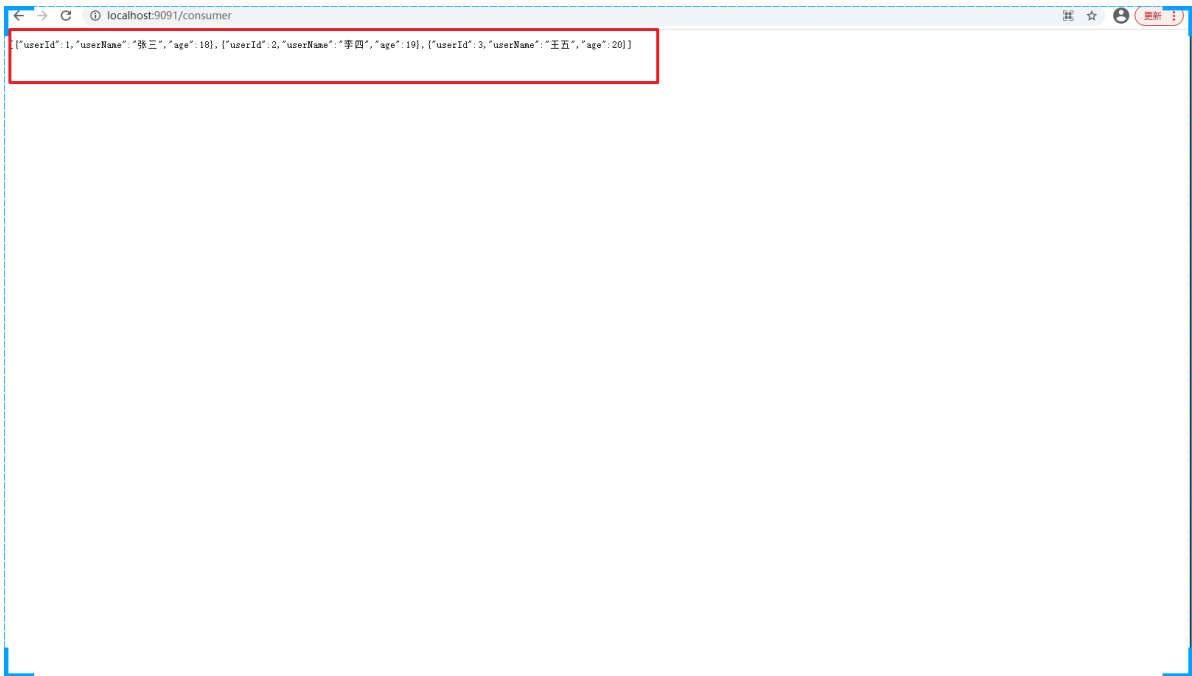
    @Autowired
    private UserService service;

    @RequestMapping("/consumer")
    public List<User> getUser(){
        system.out.println("服务消费者");
        return this.service.getUsers();
    }
}

```



测试访问



5. RestTemplate

SpringResetTemplate是Spring提供的用于访问Rest服务的客户端，RestTemplate提供了多种便捷访问远程http服务的方法，能够大大提高客户端编写的效率。

API	说明
getForEntity()	发送一个HTTP GET请求，返回的ResponseEntity包含了响应体所映射成的对象
getForObject()	发送一个HTTP GET请求，返回的请求体将映射为一个对象
postForEntity()	POST 数据到一个URL，返回包含一个对象的ResponseEntity，这个对象是从响应体中映射得到的
postForObject()	POST 数据到一个URL，返回根据响应体匹配形成的对象
headForHeaders()	发送HTTP HEAD请求，返回包含特定资源URL的HTTP头
optionsForAllow()	发送HTTP OPTIONS请求，返回对特定URL的Allow头信息
postForLocation()	POST 数据到一个URL，返回新创建资源的URL
put()	PUT 资源到特定的URL
delete()	在特定的URL上对资源执行HTTP DELETE操作
exchange()	在URL上执行特定的HTTP方法，返回包含对象的ResponseEntity，这个对象是从响应体中映射得到的
execute()	在URL上执行特定的HTTP方法，返回一个从响应体映射得到的对象

5.1 无参请求

getForEntity

服务端

```

/**
 * 无参的方法
 * @return
 */
@RequestMapping("/query1")
public String query1(){
    system.out.println("provider ... query1");
    return "success";
}

```

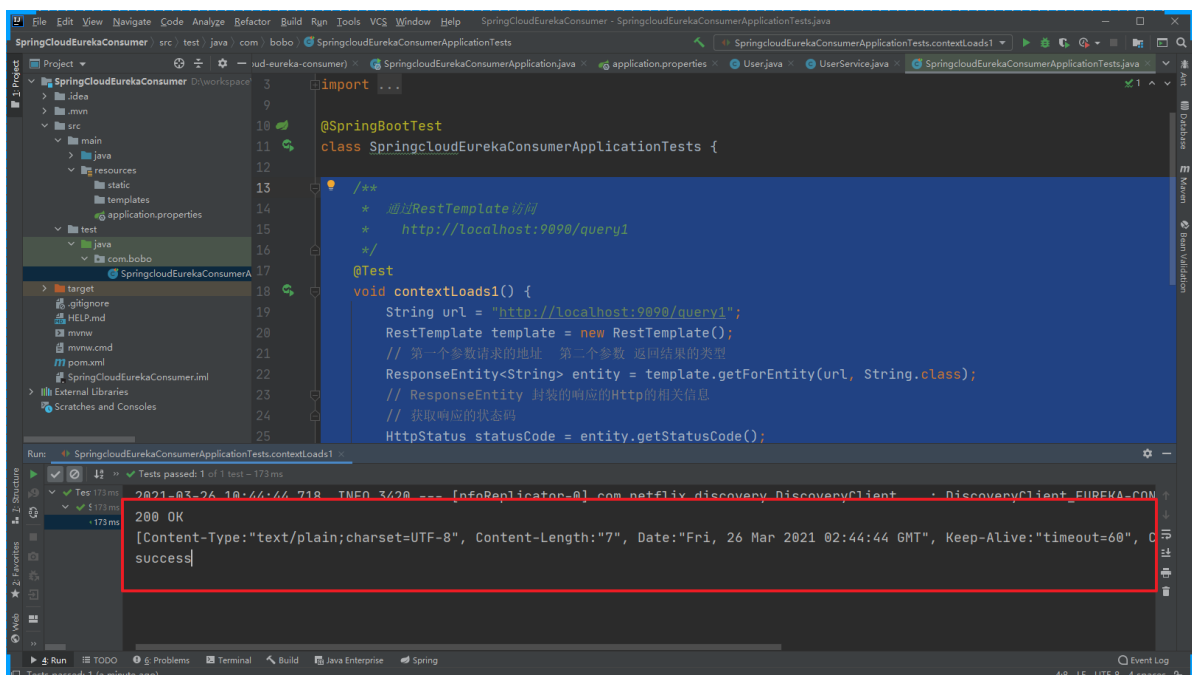
调用

```

/**
 * 通过RestTemplate访问
 * http://localhost:9090/query1
 */
@Test
void contextLoads1() {
    String url = "http://localhost:9090/query1";
    RestTemplate template = new RestTemplate();
    // 第一个参数请求的地址 第二个参数 返回结果的类型
    ResponseEntity<String> entity = template.getForEntity(url,
String.class);
    // ResponseEntity 封装的响应的Http的相关信息
    // 获取响应的状态码
    HttpStatus statusCode = entity.getStatusCode();
    // 获取响应的 head
    HttpHeaders headers = entity.getHeaders();
    // 获取响应体
    String msg = entity.getBody();
    system.out.println(statusCode);
    system.out.println(headers);
    system.out.println(msg);
}

```

响应结果

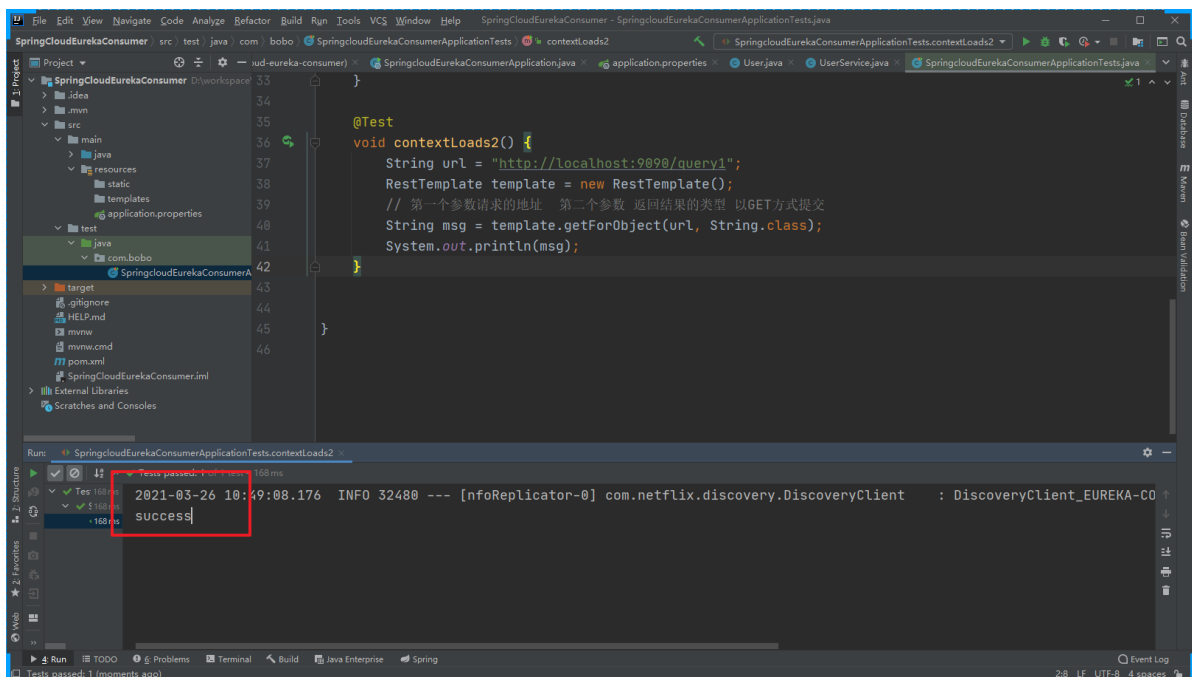


getForObject

getForObject函数实际上是对getForEntity函数的进一步封装，如果我们只关注返回的消息体的内容，对其他信息不关注，此时我们可以使用getForObject

```
@Test
void contextLoads2() {
    String url = "http://localhost:9090/query1";
    RestTemplate template = new RestTemplate();
    // 第一个参数请求的地址 第二个参数 返回结果的类型 以GET方式提交
    String msg = template.getForObject(url, String.class);
    System.out.println(msg);
}
```

返回结果



5.2 有参请求

服务端方法

```
/**
 * 有参的方法
 * 基本数据类型
 * @return
 */
@RequestMapping("/query2")
public String query2(Integer id, String userName) {
    System.out.println(id + " " + userName);
    return "success";
}

/**
 * 有参的方法
 * 自定义类型
```

```

    * @return
    */
@RequestMapping("/query3")
public String query3(@RequestBody User user){
    System.out.println(user);
    return "success";
}

```

getForEntity

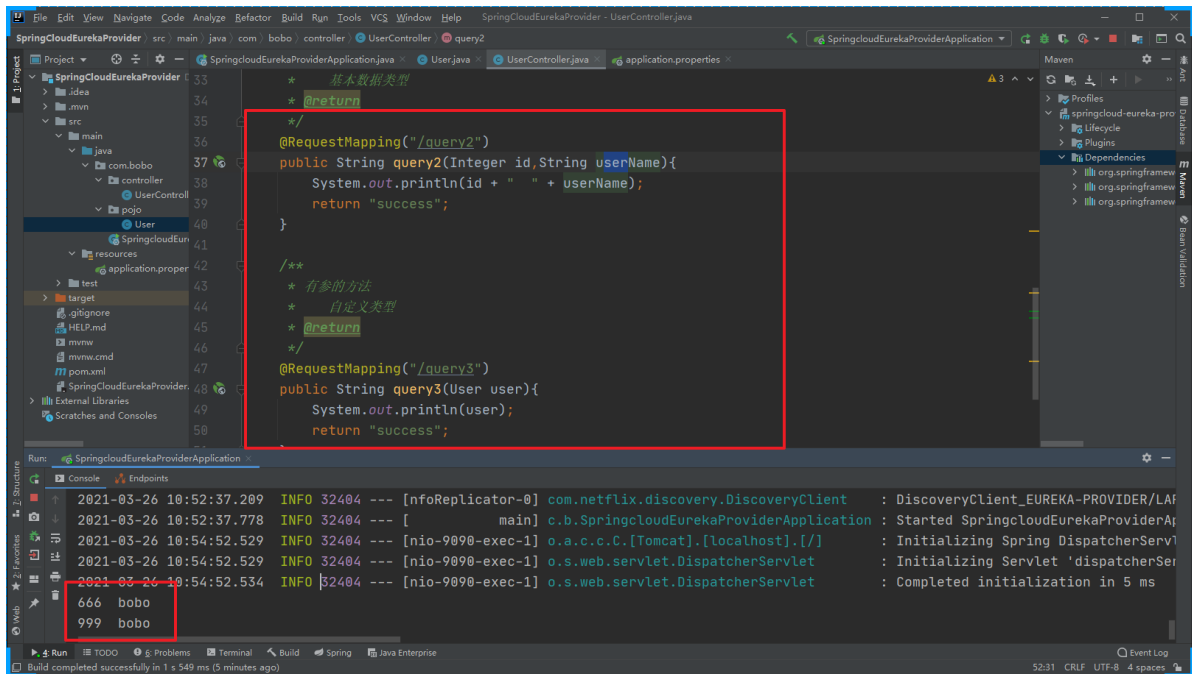
两种传值的方式：一种是数字占位符，一种是Map对象处理

```

/**
 * 传参两种方式
 *      数字占位符的方式
 */
@Test
void contextLoads3() {
    String url = "http://localhost:9090/query2?id={1}&userName={2}";
    RestTemplate template = new RestTemplate();
    ResponseEntity<String> entity = template.getForEntity(url, String.class,
666, "bobo");
    System.out.println(entity.getBody());
}

/**
 * 传参两种方式
 *      名称占位符的方式
 */
@Test
void contextLoads4() {
    String url = "http://localhost:9090/query2?id={id}&userName={userName}";
    RestTemplate template = new RestTemplate();
    // 将我们要传递的参数封装为一个Map对象
    Map<String, Object> map = new HashMap<>();
    map.put("id", 999);
    map.put("userName", "bobo");
    ResponseEntity<String> entity = template.getForEntity(url, String.class,
map);
    System.out.println(entity.getBody());
}

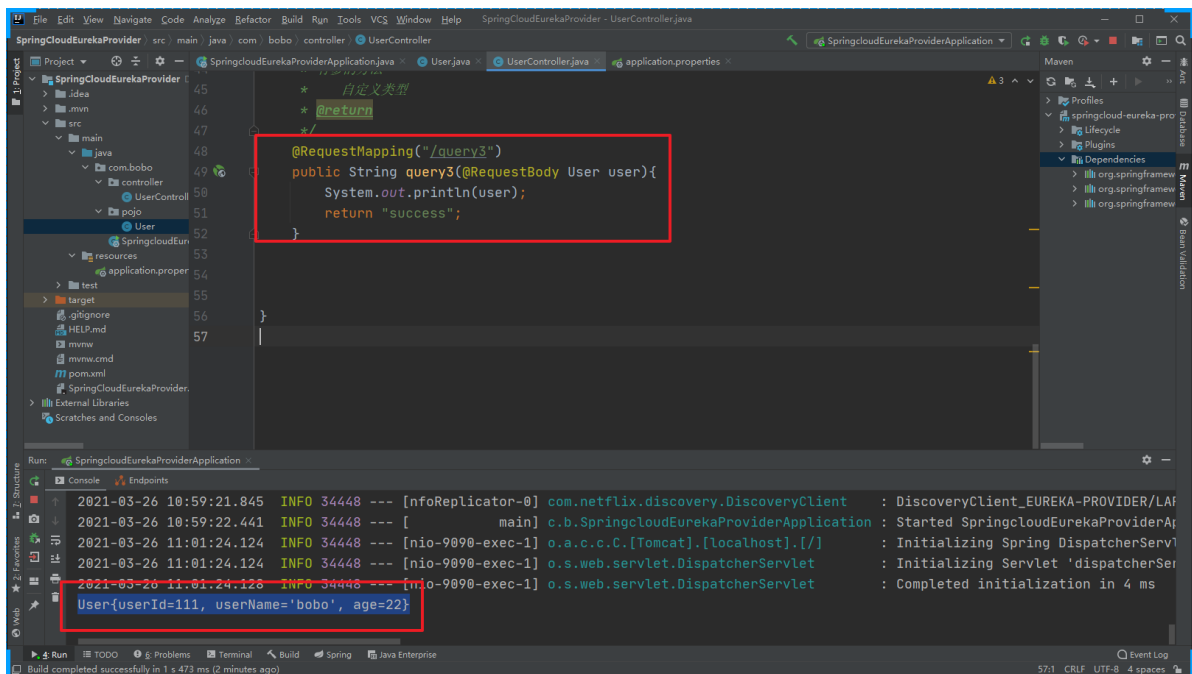
```



postForEntity

如果是post方式提交请求参数，我们可以通过postEntity来提交，注意接收参数的对象需要添加@RequestBody

```
/**
 * 传参两种方式
 *      postForEntity
 */
@Test
void contextLoads5() {
    String url = "http://localhost:9090/query3";
    RestTemplate template = new RestTemplate();
    User user = new User(111,"bobo",22);
    String msg = template.postForEntity(url, user, String.class).getBody();
    System.out.println(msg);
}
```

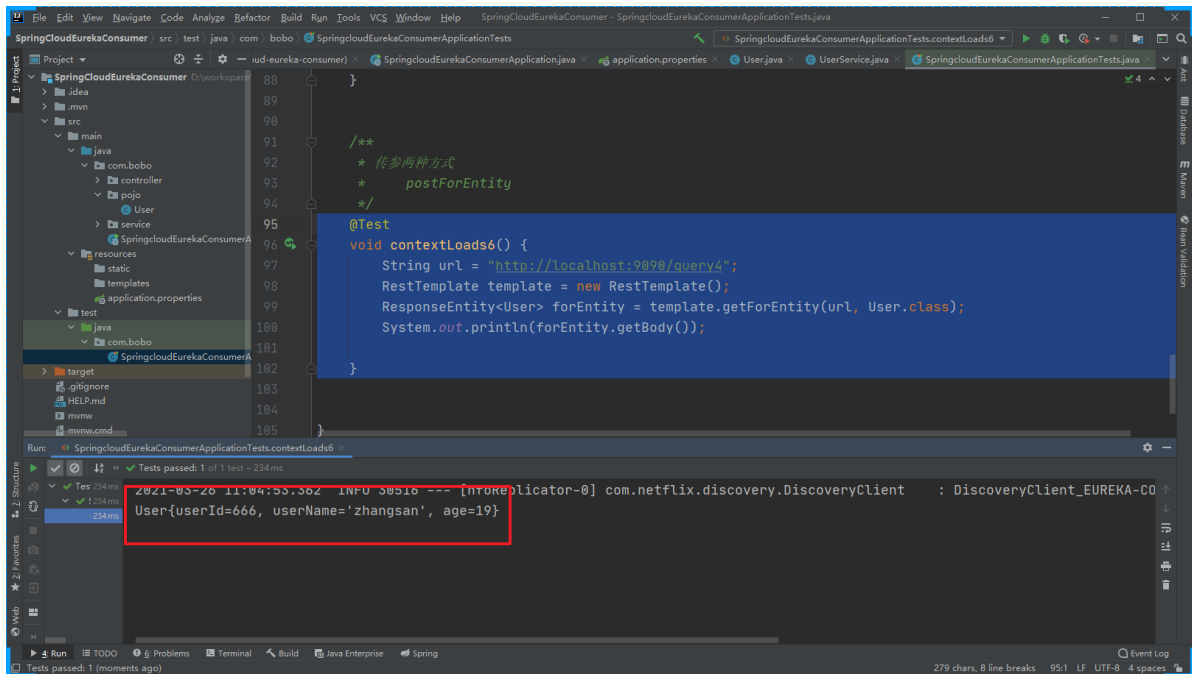


5.3 返回普通类型

前面的案例返回的类型都是String类型，我们再列举一个返回User类型的案例

```
@RequestMapping("/query4")
public User query4(){
    System.out.println("query4 ....");
    return new User(666,"zhangsang",19);
}
```

```
@Test
void contextLoads6() {
    String url = "http://localhost:9090/query4";
    RestTemplate template = new RestTemplate();
    ResponseEntity<User> forEntity = template.getForEntity(url, User.class);
    System.out.println(forEntity.getBody());
}
```

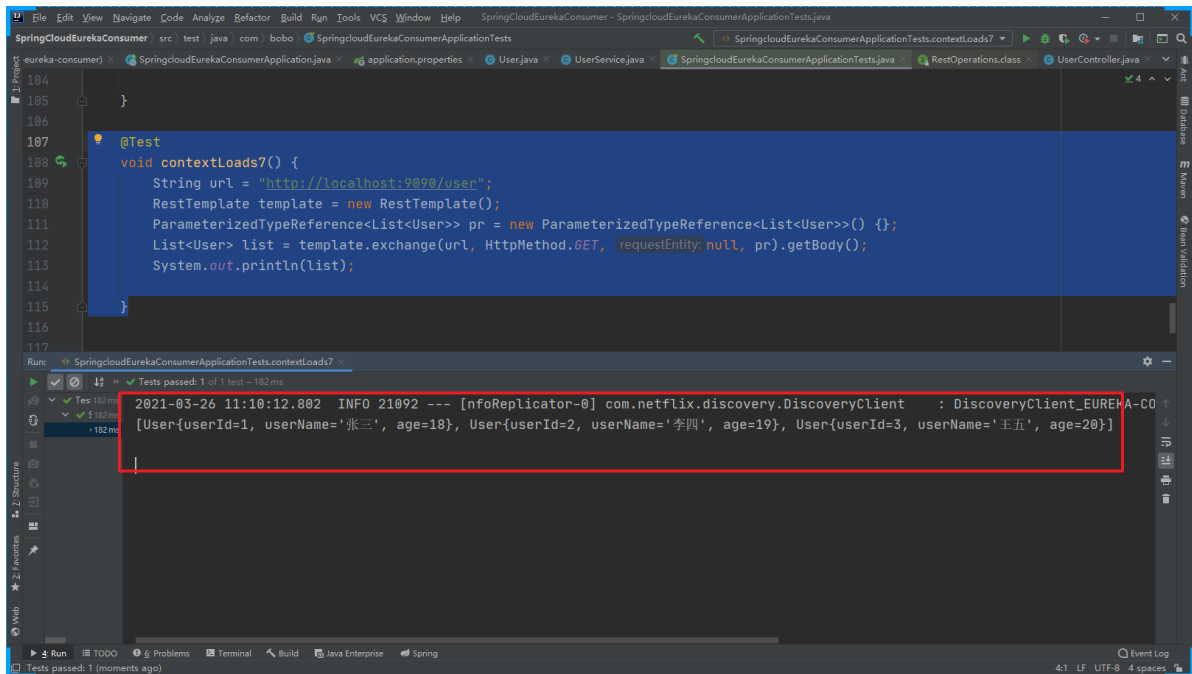


5.4 返回List类型

当前面介绍的方法不足以满足我们需求的时候，我们可以通过exchange方法来实现

1. 允许调用者指定Http请求的方法(GET,POST等)
2. 可以在请求中增加body以及head信息(HttpEntity封装)
3. exchange支持含参数的类型(泛型)，通过ParameterizedTypeReference来实现

```
@Test
void contextLoads7() {
    String url = "http://localhost:9090/user";
    RestTemplate template = new RestTemplate();
    ParameterizedTypeReference<List<User>> pr = new
ParameterizedTypeReference<List<User>>() {};
    List<User> list = template.exchange(url, HttpMethod.GET, null,
pr).getBody();
    System.out.println(list);
}
```



6. Actuator

SpringBoot Actuator可以帮助我们监控和管理SpringBoot应用，比如健康检查，审计，统计和Http追踪等，

6.1 创建SpringBoot项目

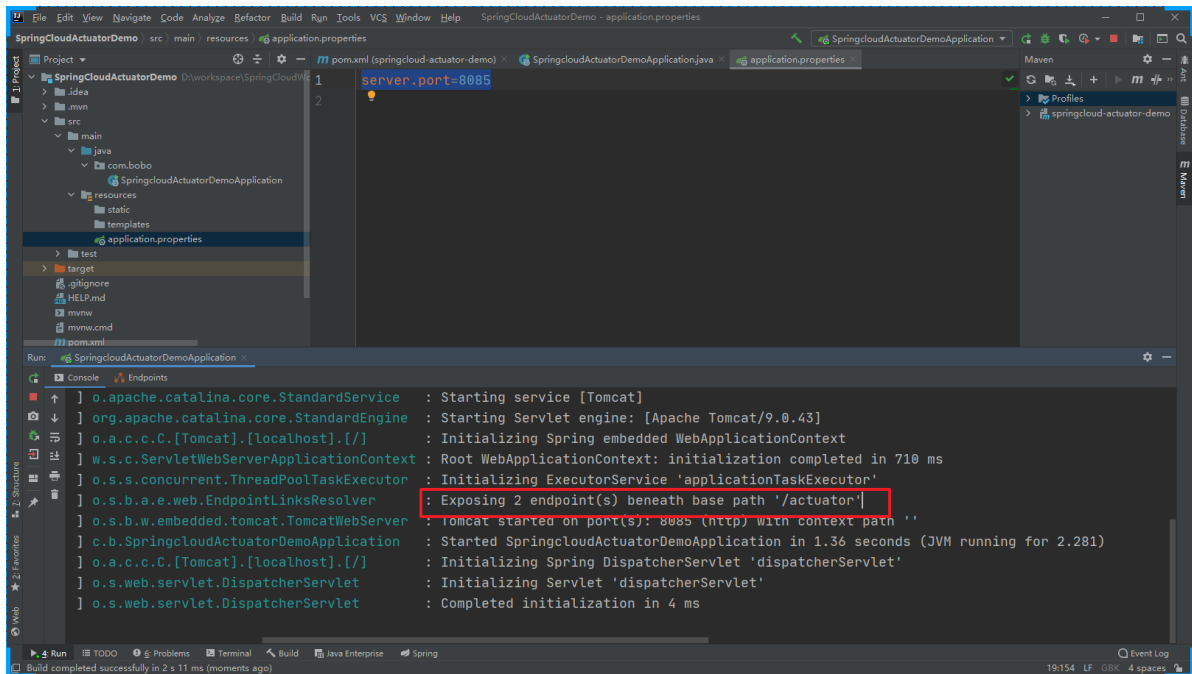
创建一个普通的SpringBoot项目添加如下依赖即可

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

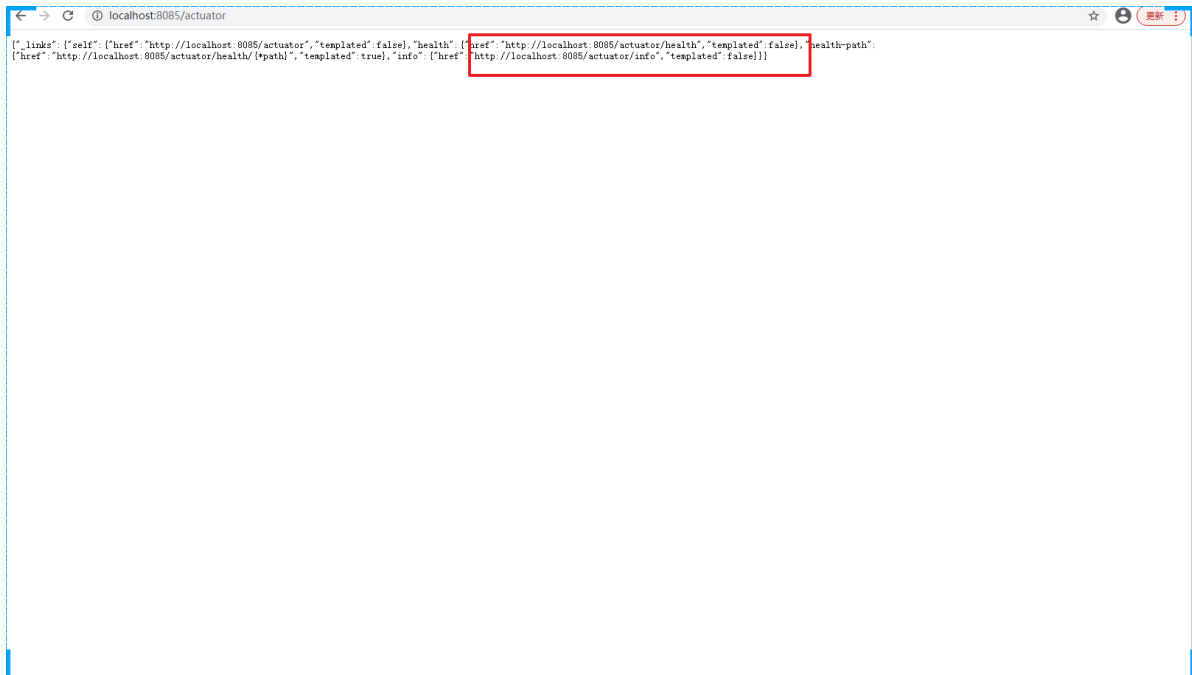
6.2 启动项目

修改端口并启动

```
server.port=8085
```



在Actuator中默认只给我们放开了两个接口 info和health



6.3 放开限制

修改属性文件

```
server.port=8085
# 在Actuator中默认只放开了 info和health 如果要放开所有*
management.endpoints.web.exposure.include=*
# 放开shutdown 接口
management.endpoints.enabled-by-default=true
```

```
localhost:8085/actuator
[{"links":[{"rel":"self","href":"http://localhost:8085/actuator","templated":false},"beans":{"rel":"http://localhost:8085/actuator/beans","templated":false},"cache-caches":{"rel":"http://localhost:8085/actuator/cache-caches","templated":true},"caches":{"rel":"http://localhost:8085/actuator/cache-caches","templated":false},"health":{"rel":"http://localhost:8085/actuator/health","templated":false},"health-path":{"rel":"http://localhost:8085/actuator/health/{path}","templated":true},"info":{"rel":"http://localhost:8085/actuator/info","templated":false},"conditions":{"rel":"http://localhost:8085/actuator/conditions","templated":false},"shutdown":{"rel":"http://localhost:8085/actuator/shutdown","templated":false},"configprops":{"rel":"http://localhost:8085/actuator/configprops","templated":false},"env":{"rel":"http://localhost:8085/actuator/env","templated":false},"env-toMatch":{"rel":"http://localhost:8085/actuator/env/{toMatch}","templated":true},"loggers":{"rel":"http://localhost:8085/actuator/loggers","templated":false},"loggers-name":{"rel":"http://localhost:8085/actuator/loggers/{name}","templated":true},"heapdump":{"rel":"http://localhost:8085/actuator/heapdump","templated":false},"threaddump":{"rel":"http://localhost:8085/actuator/threaddump","templated":false},"metrics-requiredMetricName":{"rel":"http://localhost:8085/actuator/metrics/{requiredMetricName}","templated":true},"metrics":{"rel":"http://localhost:8085/actuator/metrics","templated":false},"scheduledtasks":{"rel":"http://localhost:8085/actuator/scheduledtasks","templated":false},"mappings":{"rel":"http://localhost:8085/actuator/mappings","templated":false}]}
```

ID	描述	是否需要鉴权
actuator	为其他端点提供“发现页面”。要求 Spring HATEOAS 在 classpath 路径上。	需要
auditevents	陈列当前应用程序的审计事件信息。	需要
autoconfig	展示自动配置信息并且显示所有自动配置候选人以及他们“被不被”应用的原因。	需要
beans	显示应用程序中所有 Spring bean 的完整列表。	需要
configprops	显示所有配置信息。	需要
dump	所有线程。	需要
env	陈列所有环境变量。	需要
flyway	Shows any Flyway database migrations that have been applied.	需要
health	显示应用程序运行状况信息	不需要
info	显示应用信息。	不需要
loggers	显示和修改应用程序中的 loggers 配置。	需要
liquibase	显示已经应用的任何 Liquibase 数据库迁移。	需要
metrics	显示当前应用程序的“指标”信息。	需要
mappings	显示所有@RequestMapping 的 url 整理列表。	需要
shutdown	关闭应用（默认情况下不启用）。	需要
trace	显示跟踪信息（默认最后 100 个 HTTP 请求）。	需要

7. SpringBootAdmin

我们前面介绍的Actuator监控返回的是一堆JSON数据，查看起来非常的不方便，那么这时我们可以通过可视化的监控报表工具SpringBootAdmin来单独处理这些数据，给用户更好的体验

7.1 监控服务器

搭建一个SpringBootAdmin服务，获取Actuator中的json数据。

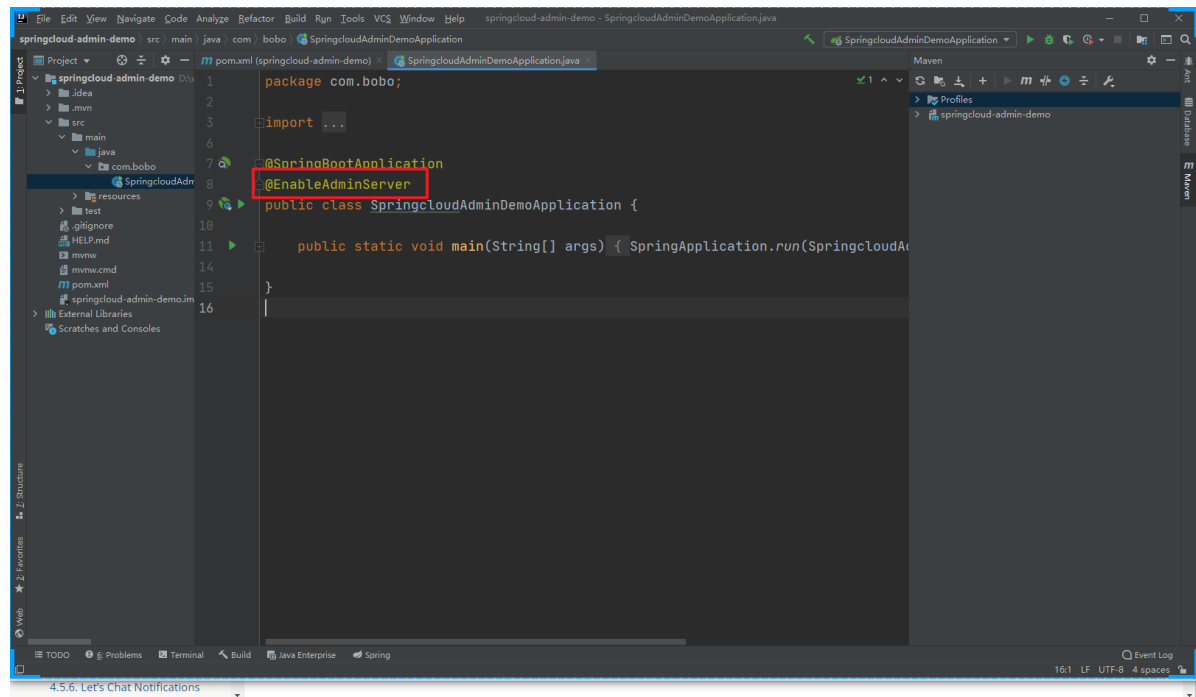
具体配置参考官网：<https://github.com/codecentric/spring-boot-admin>

创建一个SpringBoot项目，并添加相关的依赖

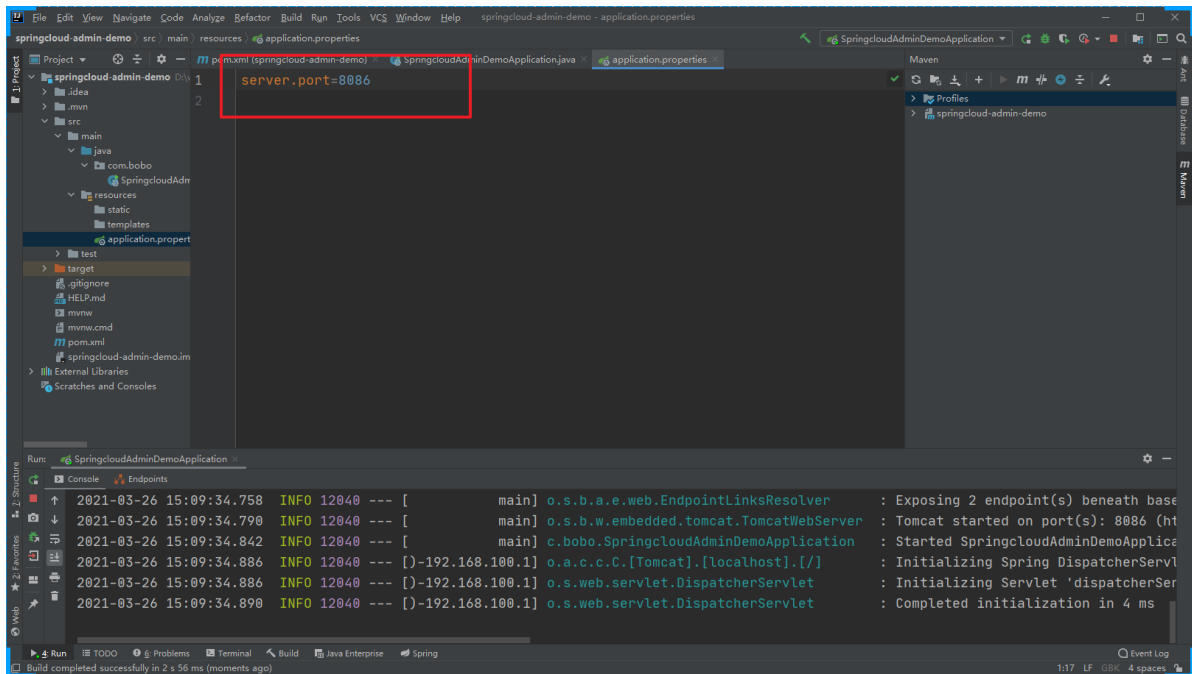
pom.xml

```
<dependency>
  <groupId>de.codecentric</groupId>
  <artifactId>spring-boot-admin-starter-server</artifactId>
  <version>2.3.1</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

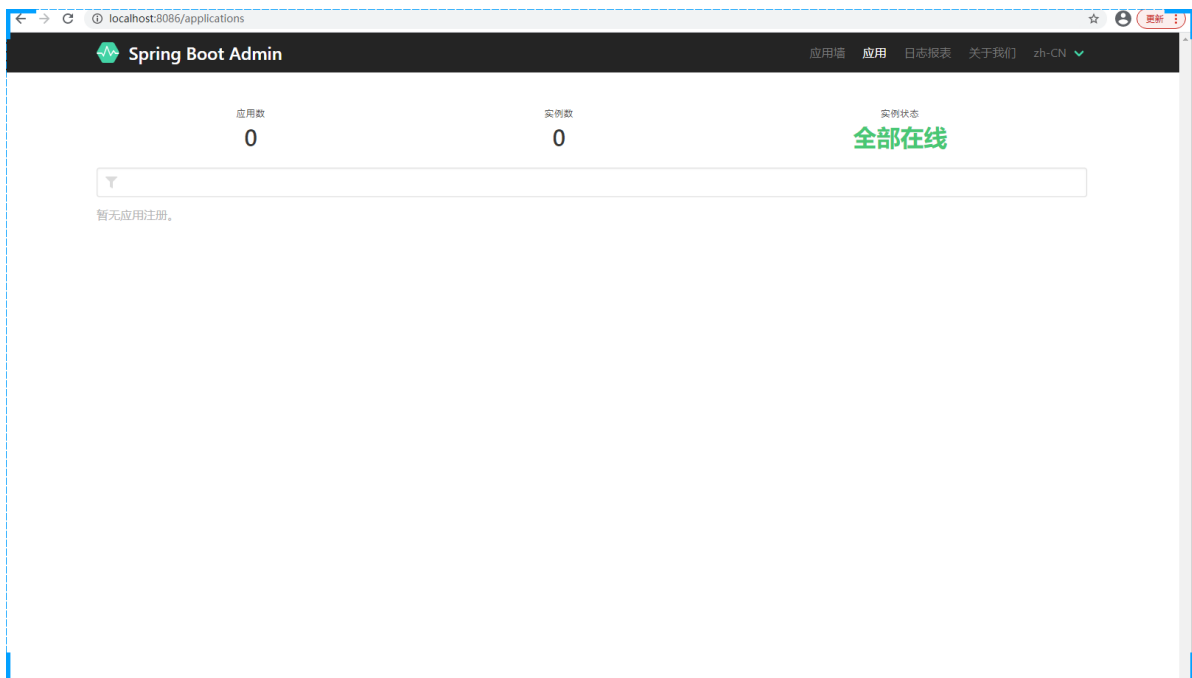
在启动类中添加@EnableAdminServer注解



修改端口



启动服务，访问即可



表示SpringBootAdmin服务端创建成功

7.2 客户端

将客户端注册到SpringBootAdmin服务器中

添加相关的依赖

```

<dependency>
  <groupId>de.codecentric</groupId>
  <artifactId>spring-boot-admin-starter-client</artifactId>
  <version>2.3.1</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

```

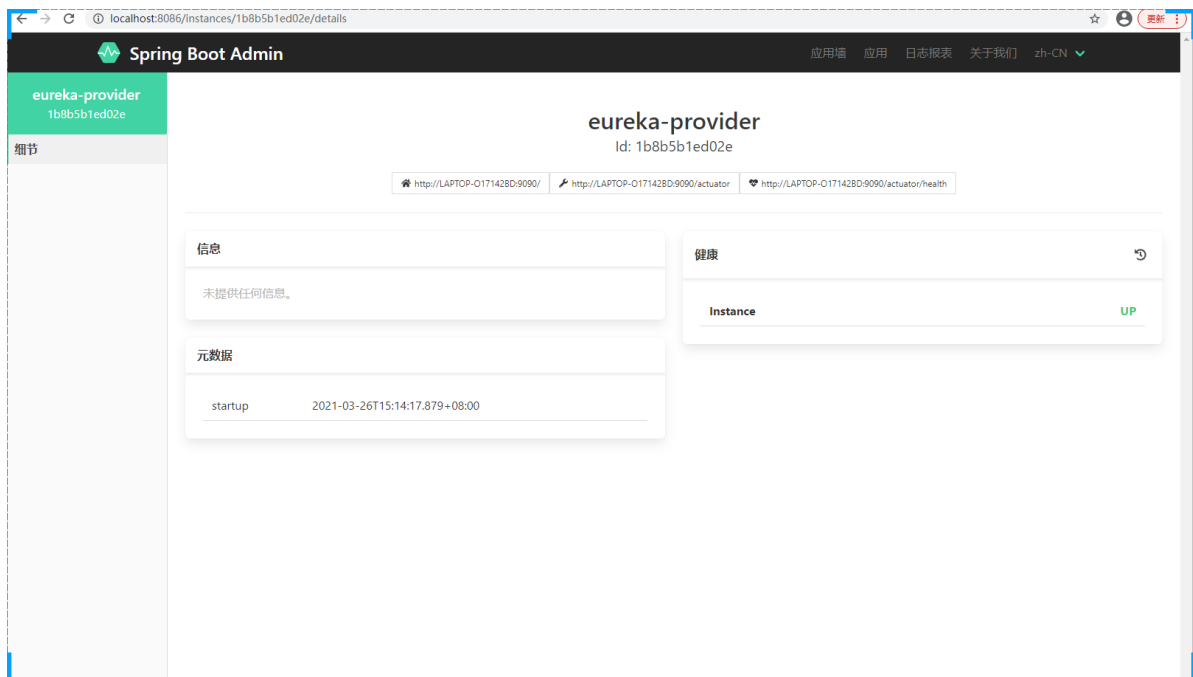
修改属性文件

```

# 配置SpringBootAdmin 服务端的地址
spring.boot.admin.client.url=http://localhost:8086
management.endpoints.web.exposure.include=*
management.endpoints.enabled-by-default=true

```

启动测试

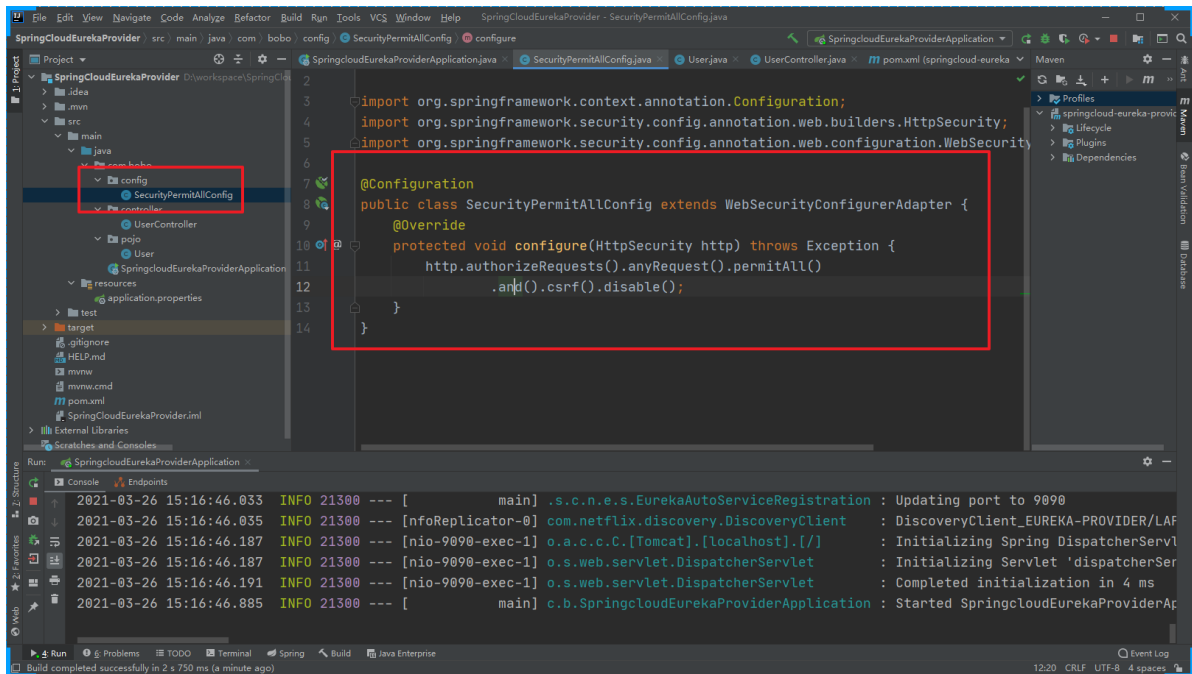


添加SpringSecurity的配置文件，放开所有的权限

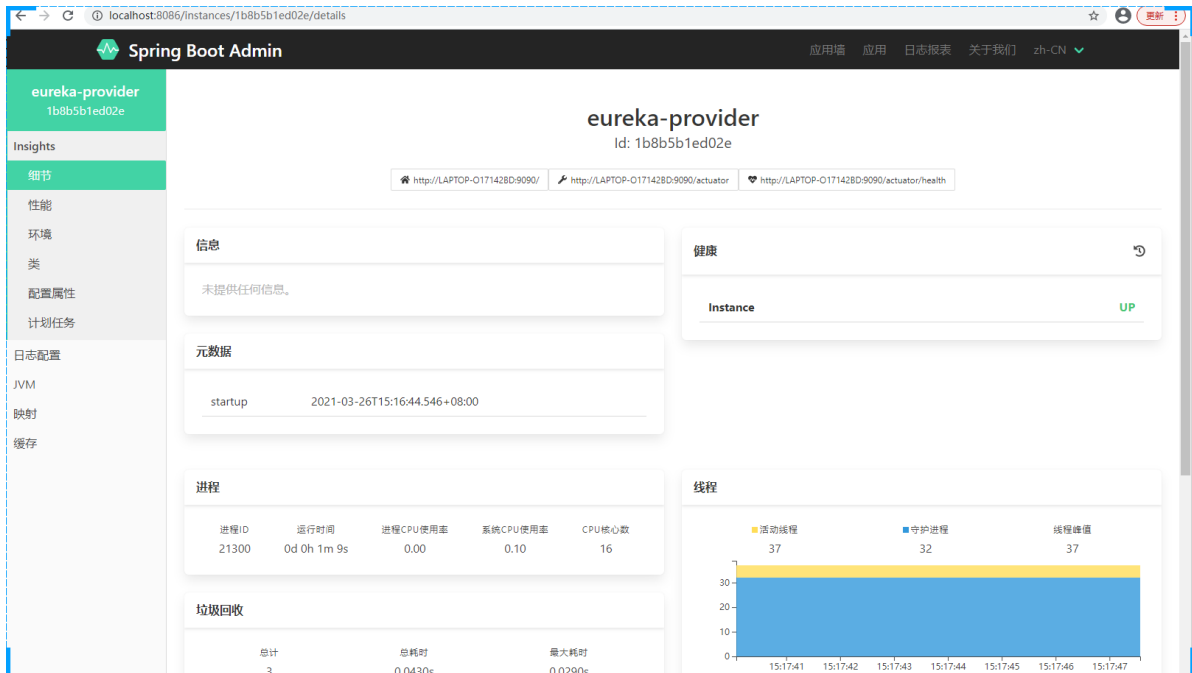
```

@Configuration
public class SecurityPermitAllConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.authorizeRequests().anyRequest().permitAll()
            .and().csrf().disable();
    }
}

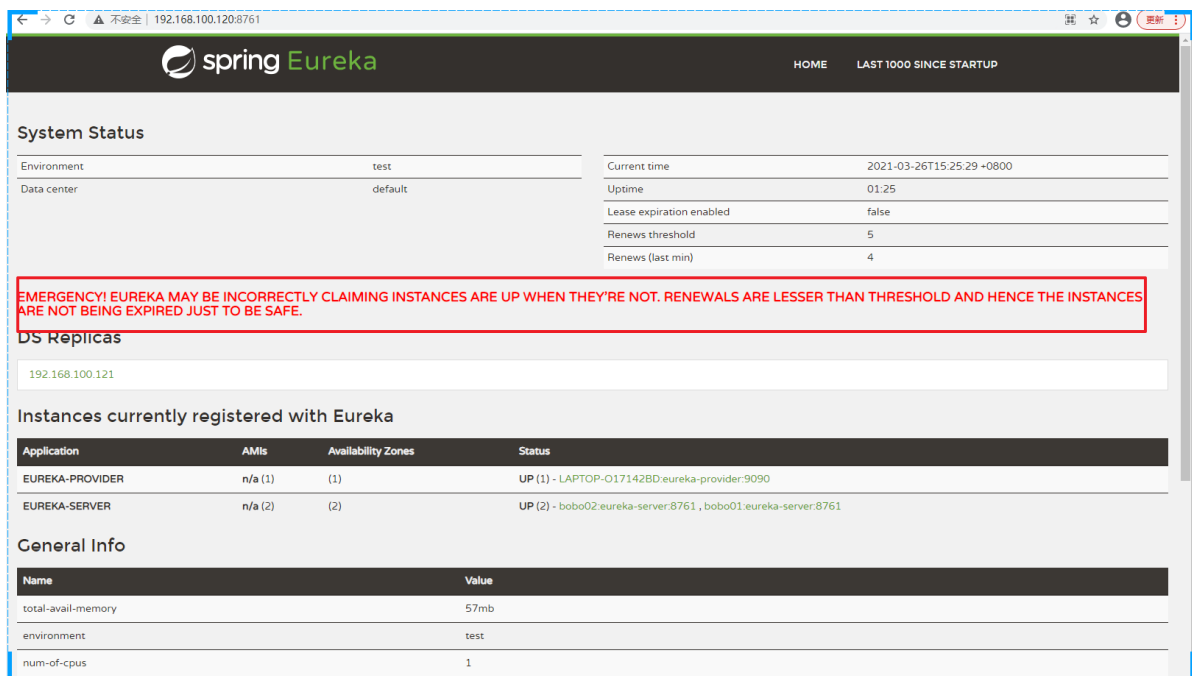
```



再访问



8.自我保护模式



什么是自我保护模式

1.自我保护的条件

一般情况下，微服务在 Eureka 上注册后，会每 30 秒发送心跳包，Eureka 通过心跳来判断服务是否健康，同时会定期删除超过 90 秒没有发送心跳服务。

2.有两种情况会导致 Eureka Server 收不到微服务的心跳

- 是微服务自身的原因
- 是微服务与 Eureka 之间的网络故障

通常(微服务的自身的故障关闭)只会导致个别服务出现故障，一般不会出现大面积故障，而(网络故障)通常会导致 Eureka Server 在短时间内无法收到大批心跳。考虑到这个区别，Eureka 设置了一个阈值，当判断挂掉的服务的数量超过阈值时，Eureka Server 认为很大程度上出现了网络故障，将不再删除心跳过期的服务。

3.那么这个阈值是多少呢？

15 分钟之内是否低于 85%；Eureka Server 在运行期间，会统计心跳失败的比例在 15 分钟内是否低于 85%，这种算法叫做 Eureka Server 的自我保护模式。

为什么要自我保护

- 因为同时保留"好数据"与"坏数据"总比丢掉任何数据要更好，当网络故障恢复后，这个 Eureka 节点会退出"自我保护模式"。
- Eureka 还有客户端缓存功能(也就是微服务的缓存功能)。即便 Eureka 集群中所有节点都宕机失效，微服务的 Provider 和 Consumer 都能正常通信。
- 微服务的负载均衡策略会自动剔除死亡的微服务节点。

如何关闭自我保护

修改 Eureka Server 配置文件

```
spring.application.name=eureka-provider
server.port=9090
```

设置服务注册中心地址 执行Eureka服务端 如果有多个注册地址 那么用逗号连接

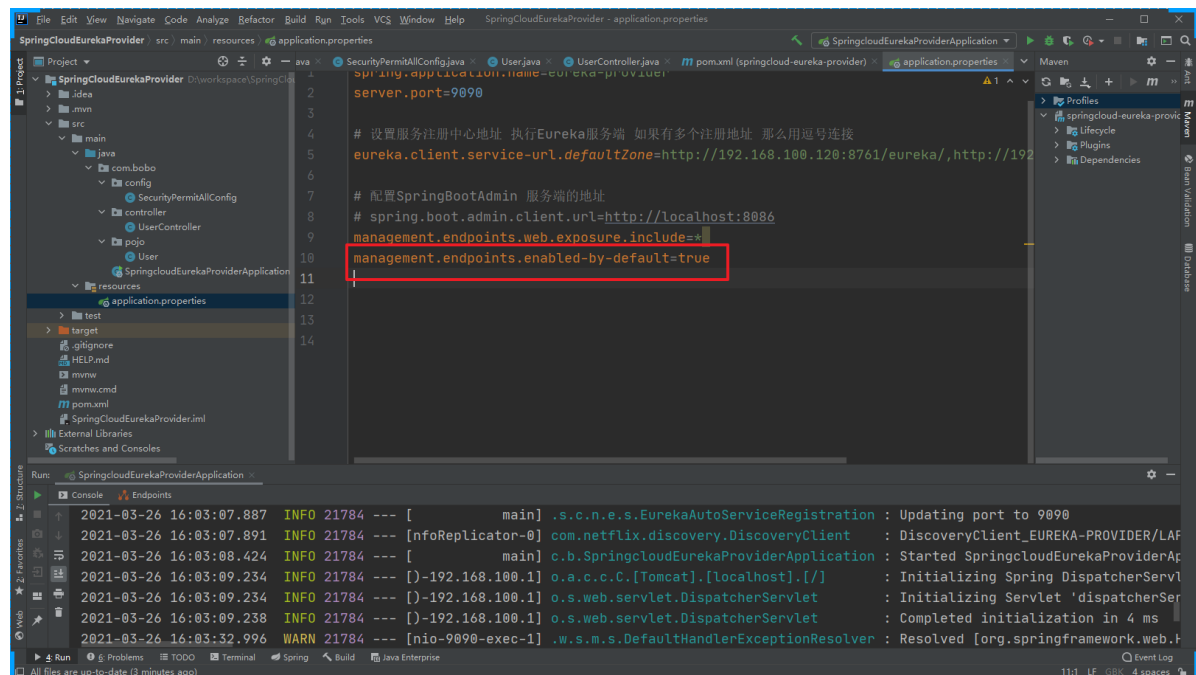
```
eureka.client.service-
url.defaultZone=http://192.168.100.120:8761/eureka/,http://192.168.100.121:8761/
eureka/

# 配置SpringBootAdmin 服务端的地址
spring.boot.admin.client.url=http://localhost:8086
management.endpoints.web.exposure.include=*
management.endpoints.enabled-by-default=true

# 关闭Eureka的自我保护
eureka.server.enable-self-preservation=false
# 设置删除无效服务的间隔时间 单位毫秒
eureka.server.eviction-interval-timer-in-ms=10000
```

9.优雅停机

服务提供者下线的时候如果更好的将自身在Eureka中的注册信息移除呢？，Eureka Server中的自我保护模式就不需要关闭了。需要添加Actuator的支持



spring Eureka

HOME LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2021-03-26T16:06:41 +0800
Data center	default	Uptime	02:06
		Lease expiration enabled	false
		Renews threshold	5
		Renews (last min)	5

EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.

DS Replicas

192.168.100.121

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
EUREKA-PROVIDER	n/a (1)	(1)	UP (1) - LAPTOP-017142BD:eureka-provider:9090
EUREKA-SERVER	n/a (2)	(2)	UP (2) - bobo02:eureka-server:8761, bobo01:eureka-server:8761

General Info

Name	Value
total-avail-memory	57mb
environment	test
num-of-cpus	1

Postman

My Workspace

POST http://127.0.0.1:9090/actuator/shutdown

Send Save

Params Authorization Headers Body Pre-request Script Tests

KEY	VALUE	DESCRIPTION
Key	Value	Description

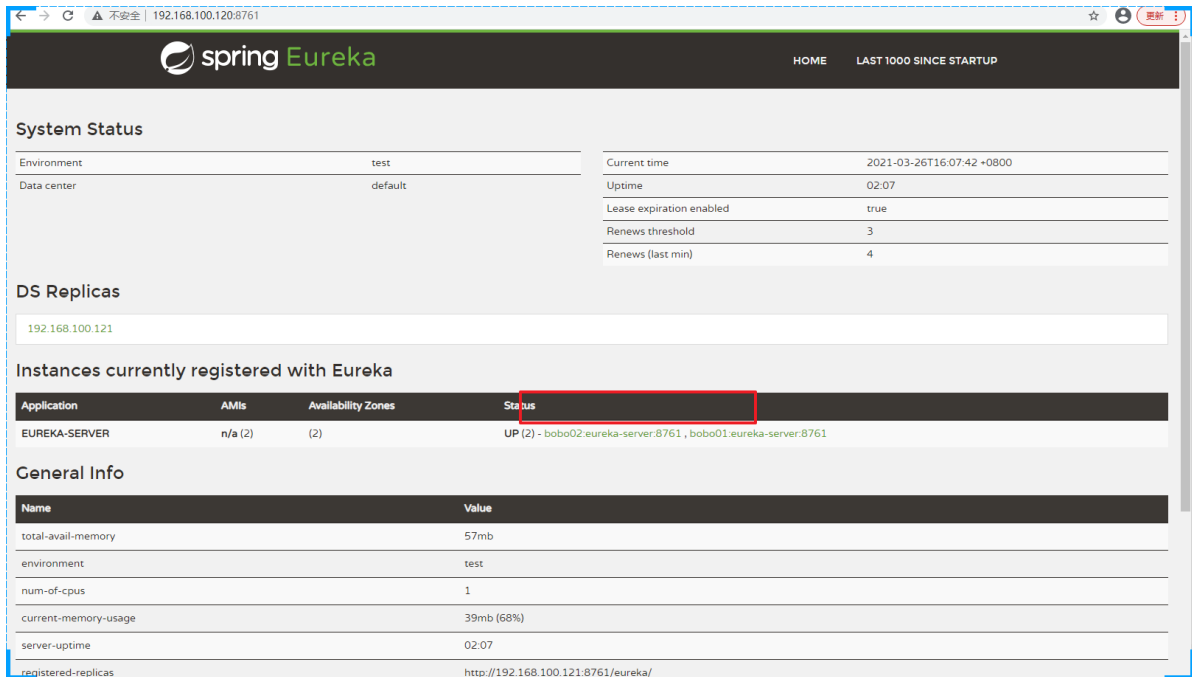
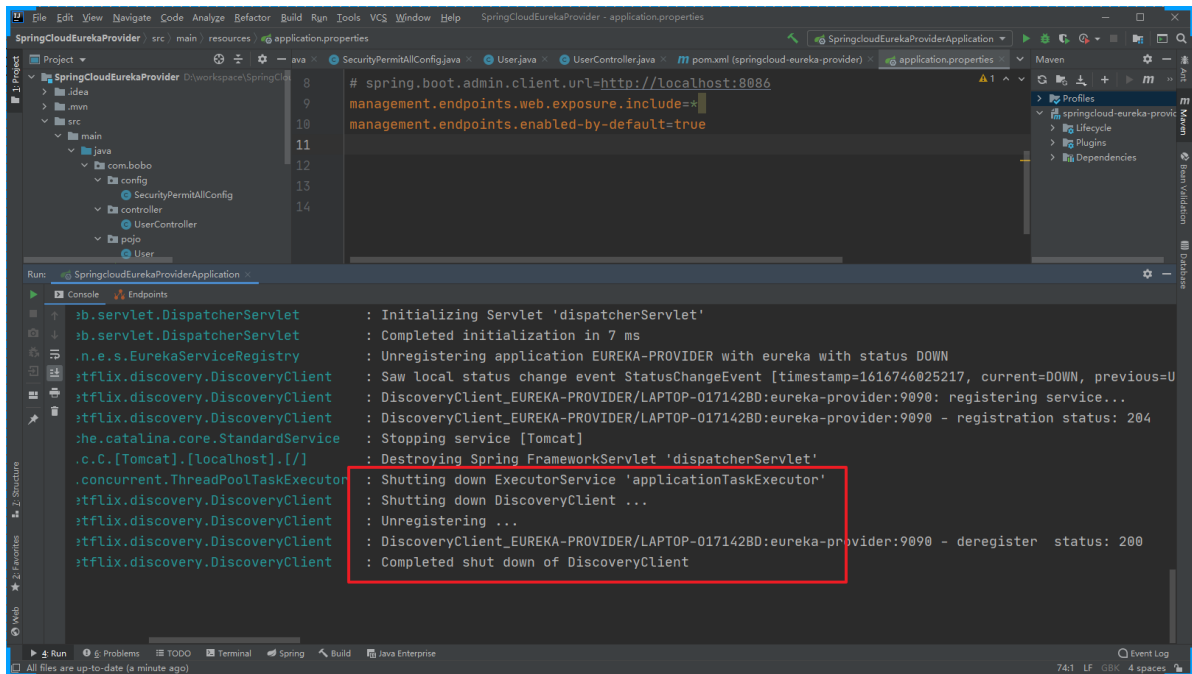
Body Cookies Headers (11) Test Results

Status: 200 OK Time: 27 ms Size: 409 B Download

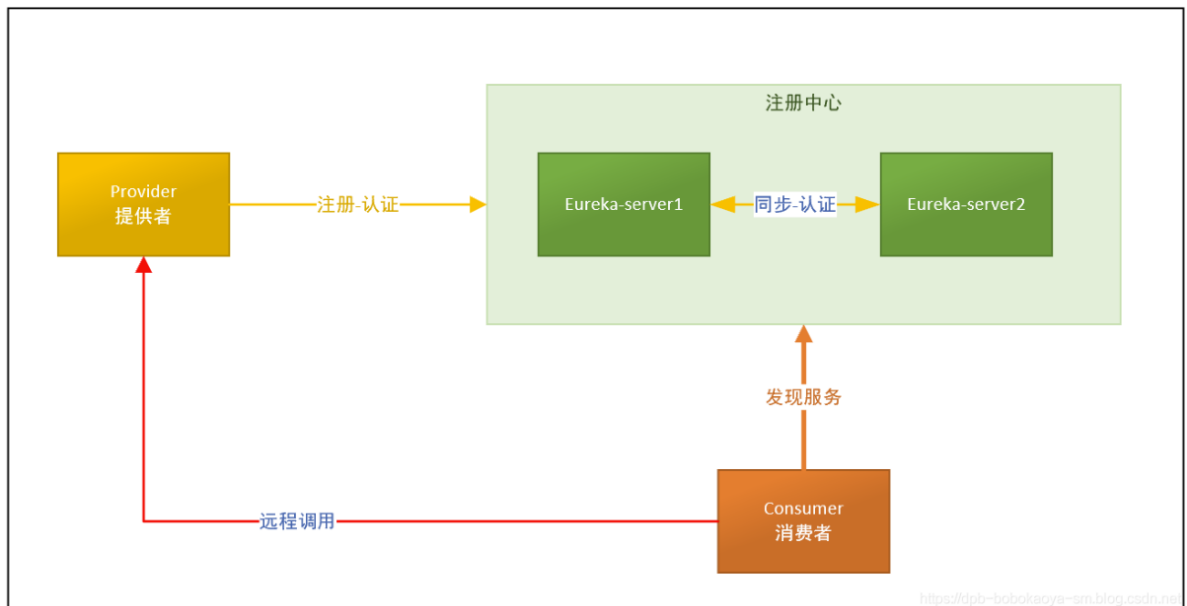
Pretty Raw Preview JSON

```
1 - [
2   "message": "Shutting down, bye..."
3 ]
```

provider服务被停止了



10.安全认证



10.1 注册中心配置

我们可以在Eureka的注册中心中添加SpringSecurity来增加安全认证

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

添加SpringSecurity的配置文件

```
@Configuration
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {

        http.authorizeRequests()
            .anyRequest()
            .authenticated()
            .and().httpBasic()
            .and().csrf().disable();
    }
}
```

然后两个属性文件

```
spring.application.name=eureka-server
server.port=8761

# 开启 http basic 的安全认证
spring.security.user.name=bobo
spring.security.user.password=123456

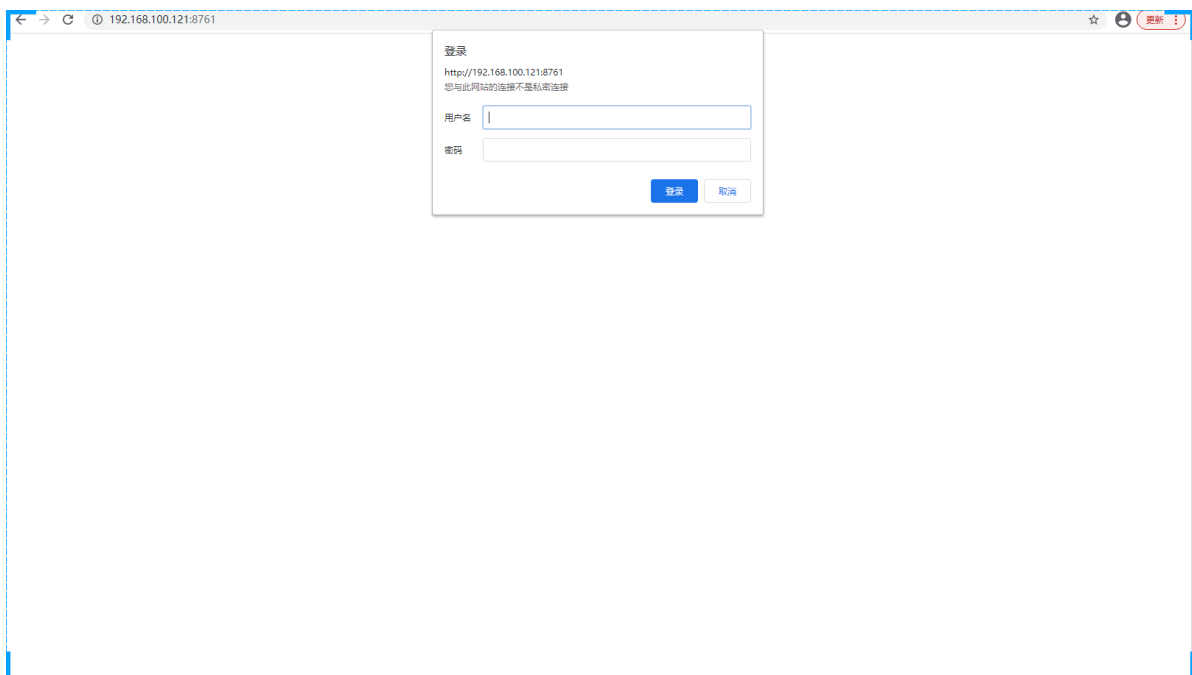
# 设置eureka的实例名称 与配置文件中的变量为主
eureka.instance.hostname=eureka1
# 设置注册中心的地址，指向另一个注册中心 192.168.100.120
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.121:8761/eureka
```

```
spring.application.name=eureka-server
server.port=8761

# 开启 http basic 的安全认证
spring.security.user.name=bobo
spring.security.user.password=123456

# 设置eureka的实例名称 与配置文件中的变量为主
eureka.instance.hostname=eureka2
# 设置注册中心的地址，指向另一个注册中心 192.168.100.121
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka
```

然后打包部署即可



The screenshot shows the Spring Eureka web interface. At the top, there's a navigation bar with the Spring Eureka logo and links for HOME and LAST 1000 SINCE STARTUP. The main content area is divided into several sections:

- System Status:** A table showing environment (test), data center (default), current time (2021-03-26T16:34:28 +0800), uptime (00:02), lease expiration enabled (true), renew threshold (1), and renews (last min) (5).
- DS Replicas:** A table showing the address 192.168.100.120.
- Instances currently registered with Eureka:** A table with columns Application, AMIs, Availability Zones, and Status. The first row shows EUREKA-SERVER with n/a (2) AMIs, (2) Availability Zones, and Status UP. A red box highlights the status text: "2) - bobo02.eureka-server:8761, bobo01.eureka-server:8761".
- General Info:** A table with columns Name and Value. It lists various system metrics like total-avail-memory (60mb), environment (test), num-of-cpus (1), current-memory-usage (38mb (63%)), server-uptime (00:02), and registered-replicas (http://192.168.100.120:8761/eureka/).

10.2 服务提供者和消费者

我们需要在属性文件中添加相关的账号信息

服务提供者

```
spring.application.name=eureka-provider
server.port=9090

# 设置服务注册中心地址 执行Eureka服务端 如果有多个注册地址 那么用逗号连接
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

# 配置SpringBootAdmin 服务端的地址
# spring.boot.admin.client.url=http://localhost:8086
management.endpoints.web.exposure.include=*
management.endpoints.enabled-by-default=true
```

服务消费者

```
spring.application.name=eureka-consumer
server.port=9091
```

设置服务注册中心地址 执行Eureka服务端 如果有多个注册地址 那么用逗号连接
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

The screenshot shows the Spring Eureka web interface. The top navigation bar includes 'HOME' and 'LAST 1000 SINCE STARTUP'. The 'System Status' section displays a table with system metrics. A red warning message is visible: 'EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.' Below this, the 'DS Replicas' section shows the address '192.168.100.120'. The 'Instances currently registered with Eureka' section contains a table with columns: Application, AMIs, Availability Zones, and Status. The 'Status' column is highlighted with a red box. The 'General Info' section at the bottom shows system details.

Application	AMIs	Availability Zones	Status
EUREKA-CONSUMER	n/a (1)	(1)	UP (1) - LAPTOP-O17142BD:eureka-consumer:9091
EUREKA-PROVIDER	n/a (1)	(1)	UP (1) - LAPTOP-O17142BD:eureka-provider:9090
EUREKA-SERVER	n/a (2)	(2)	UP (2) - bobo02:eureka-server:8761, bobo01:eureka-server:8761

Name	Value
total-avail-memory	60mb
environment	test

访问

The screenshot shows a web browser window with the address bar displaying 'localhost:9091/consumer'. The page content shows a JSON array of user objects:

```
[{"userId":1,"userName":"张三","age":18}, {"userId":2,"userName":"李四","age":19}, {"userId":3,"userName":"王五","age":20}]
```

三、Ribbon

1.Ribbon介绍

1.1 什么是 Ribbon

1. Ribbon 是一个基于 Http 和 TCP 的客户端负载均衡工具，它是基于 Netflix Ribbon 实现的。
2. 它不像 spring cloud 服务注册中心、配置中心、API 网关那样独立部署，但是它几乎存在于每个 Spring cloud 微服务中。包括 feign 提供的声明式服务调用也是基于该 Ribbon实现的。
3. Ribbon 默认提供很多种负载均衡算法，例如 轮询、随机 等等。甚至包含自定义的负载均衡算法。

1.2 Ribbon 解决了什么问题

他解决并提供了微服务的负载均衡的问题。

1.3 集中式与进程内负载均衡的区别

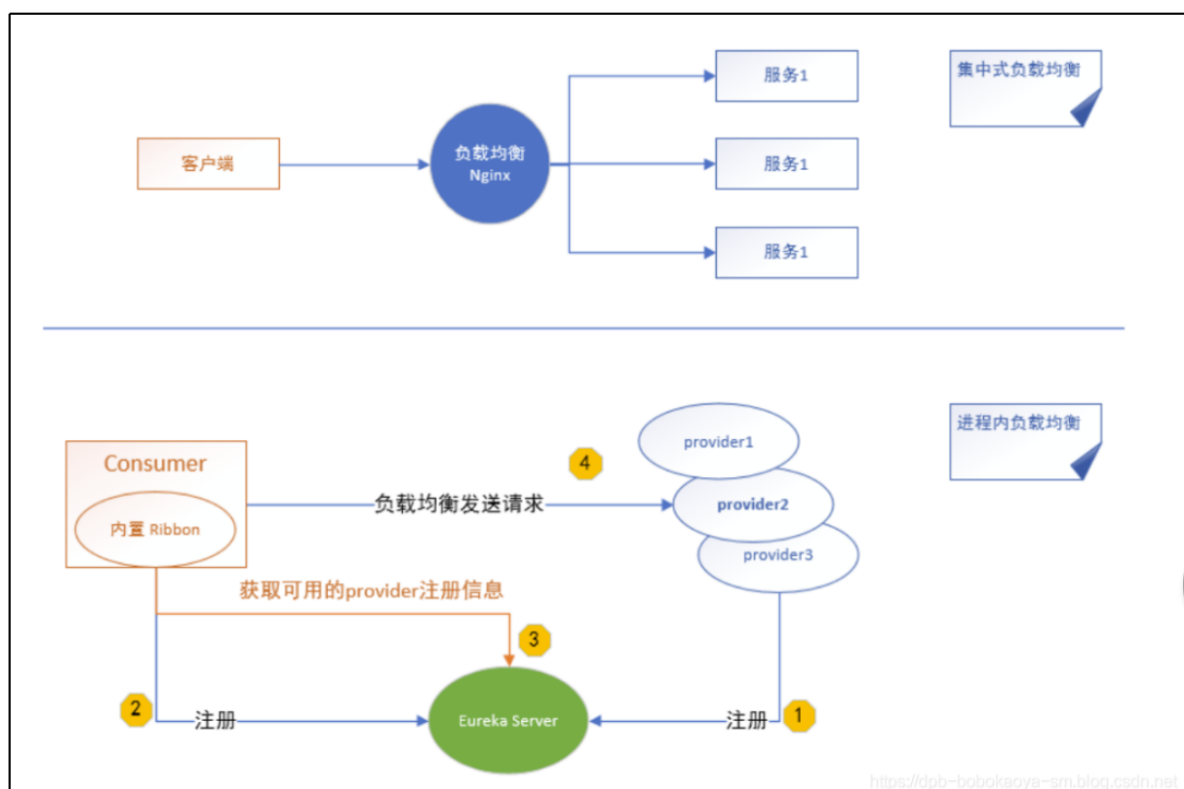
1.3.1 负载均衡分类

目前业界主流的负载均衡方案可分成两类：

第一类：集中式负载均衡, 即在 consumer 和 provider 之间使用独立的负载均衡设施(可以是硬件，如 F5, 也可以是软件，如 Nginx), 由该设施负责把 访问请求 通过某种策略转发至 provider；

第二类：进程内负载均衡，将负载均衡逻辑集成到 consumer，consumer 从服务注册中心获知有哪些地址可用，然后自己再从这些地址中选择出一个合适的 provider。Ribbon 就属于后者，它只是一个类库，集成于 consumer 进程，consumer 通过它来获取到 provider 的地址。

1.3.2 两种负载均衡结构图

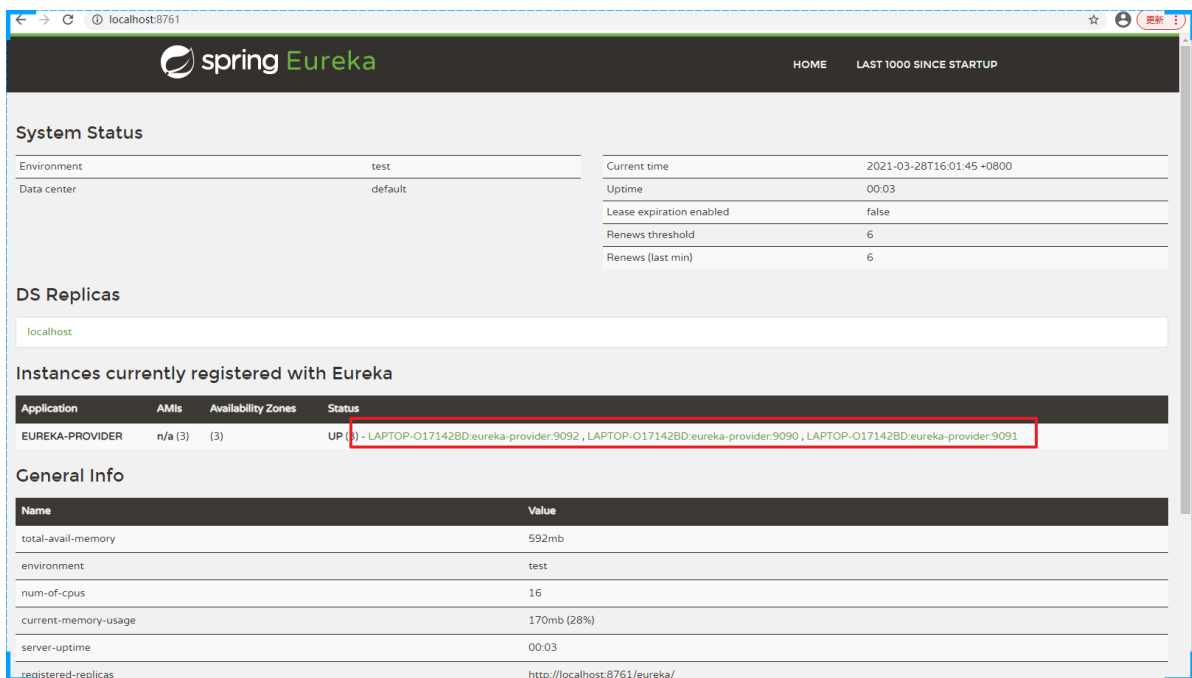
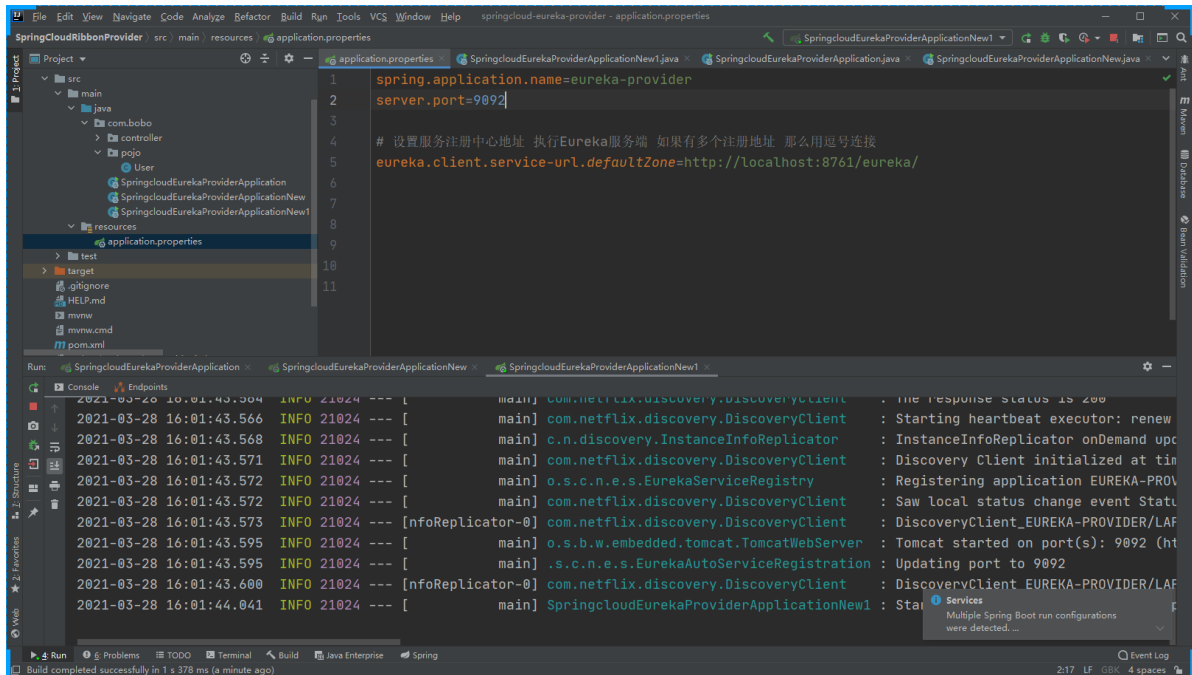


2. 案例讲解

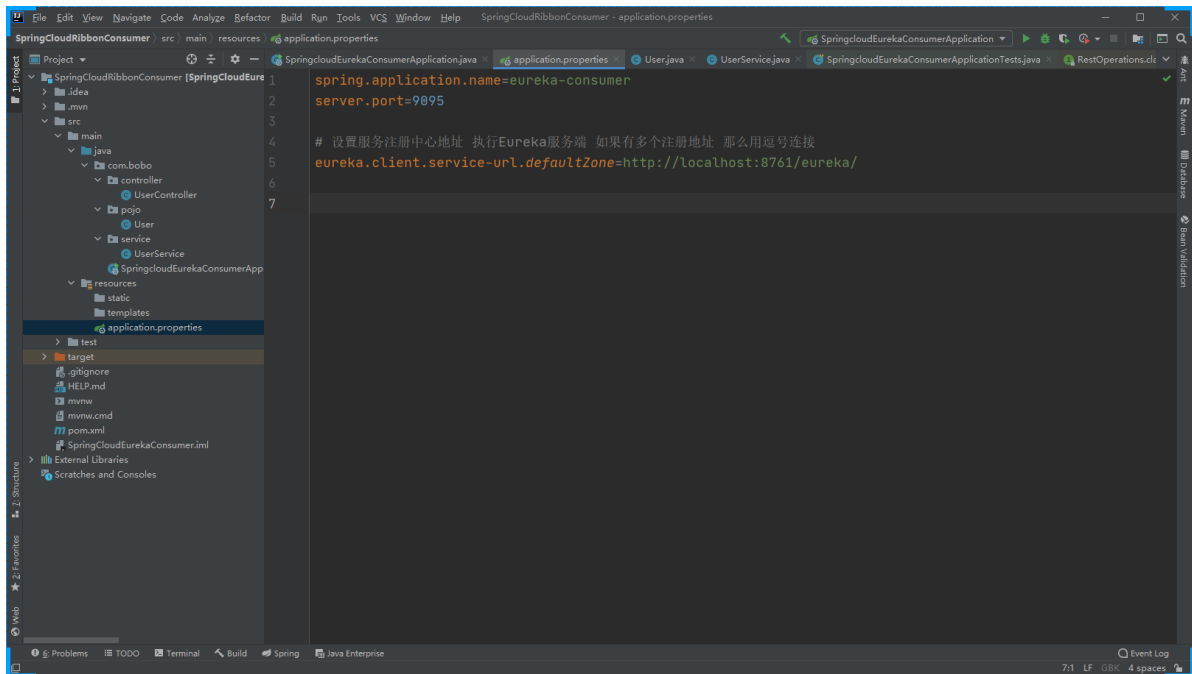
启动一个单机版的Eureka服务，然后创建一个服务提供者

2.1 创建服务提供者

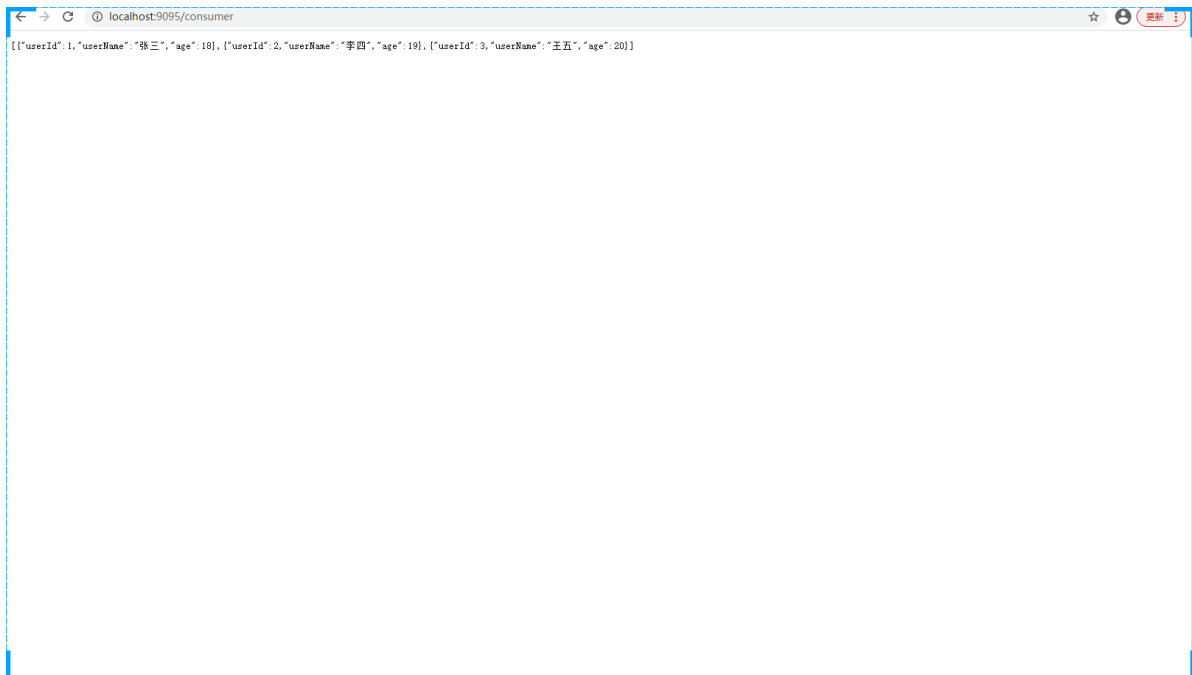
服务提供者程序，我们启动三个，端口号分别是9090,9091,9092

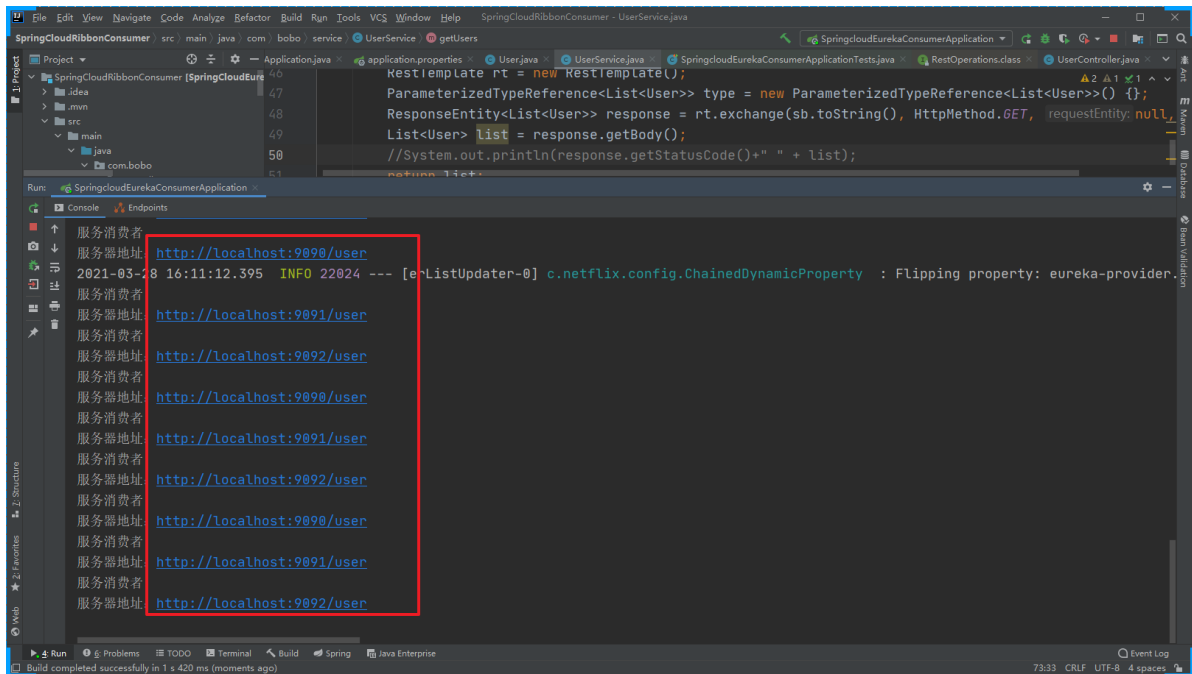


2.2 创建消费者



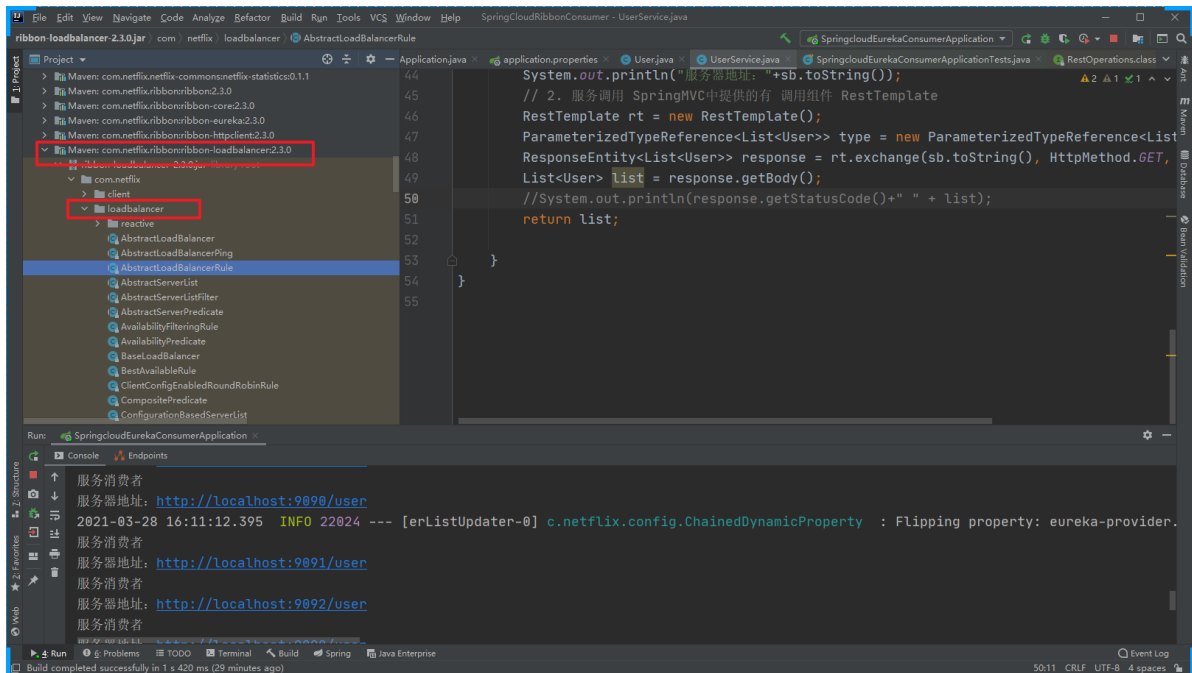
2.2 测试访问





通过验收案例我们可以发现Ribbon默认是通过 轮询的方式处理的

3.负载均衡策略



策略名称	策略对应的类名	实现原理
轮询策略（默认）	RoundRobinRule	轮询策略表示每次都顺序取下一个 provider，比如一共有 5 个 provider，第 1 次取第 1 个，第 2 次取第 2 个，第 3 次取第 3 个，以此类推
权重轮询策略	WeightedResponseTimeRule	1.根据每个 provider 的响应时间分配一个权重，响应时间越长，权重越小，被选中的可能性越低。 2.原理：一开始为轮询策略，并开启一个计时器，每 30 秒收集一次每个 provider 的平均响应时间，当信息足够时，给每个 provider 附上一个权重，并按权重随机选择 provider，高权重的 provider 会被高概率选中。
随机策略	RandomRule	从 provider 列表中随机选择一个 provider
最少并发数策略	BestAvailableRule	选择正在请求中的并发数最小的 provider，除非这个 provider 在熔断中。
在“选定的负载均衡策略”基础上进行重试机制	RetryRule	1.“选定的负载均衡策略”这个策略是轮询策略 RoundRobinRule 2.该重试策略先设定一个阈值时间段，如果在这个阈值时间段内当选择 provider 不成功，则一直尝试采用“选定的负载均衡策略：轮询策略”最后选择一个可用的 provider
可用性敏感策略	AvailabilityFilteringRule	过滤性能差的 provider，有 2 种： 第一种：过滤掉在 eureka 中处于一直连接失败 provider 第二种：过滤掉高并发的 provider
区域敏感性策略	ZoneAvoidanceRule	1.以一个区域为单位考察可用性，对于不可用的区域整个丢弃，从剩下区域中选可用的 provider 2.如果这个 ip 区域内有一个或多个实例不可达或响应变慢，都会降低该 ip 区域内其他 ip 被选中的权重。

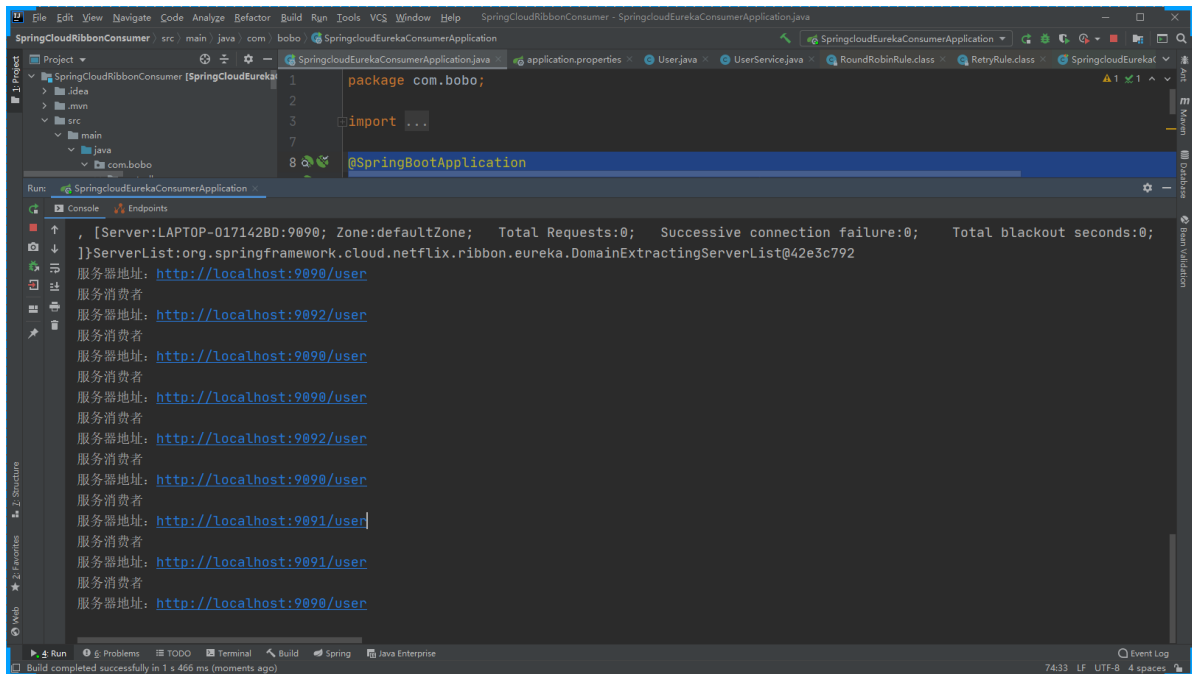
修改负载均衡的策略

直接在项目的配置类中显示的向Spring的IoC容器中注入 负载均衡策略的实现类即可。

```
@SpringBootApplication
public class SpringcloudEurekaConsumerApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringcloudEurekaConsumerApplication.class, args);
    }

    /**
     * 显示的实例化 负载均衡的策略 那么默认的轮询策略就会失效
     * @return
     */
    @Bean
    public RandomRule randomRule(){
        return new RandomRule();
    }
}
```



通过输出我们对比可以发现客户端处理请求的服务器是随机分配的，说明我们的配置是起作用的

四、Feign

1.什么是 Feign

Feign是一种声明式、模板化的HTTP客户端(仅在 consumer 中使用)。

2.什么是声明式

什么是声明式，有什么作用，解决什么问题？

声明式调用就像调用本地方法一样调用远程方法;无感知远程 http 请求。

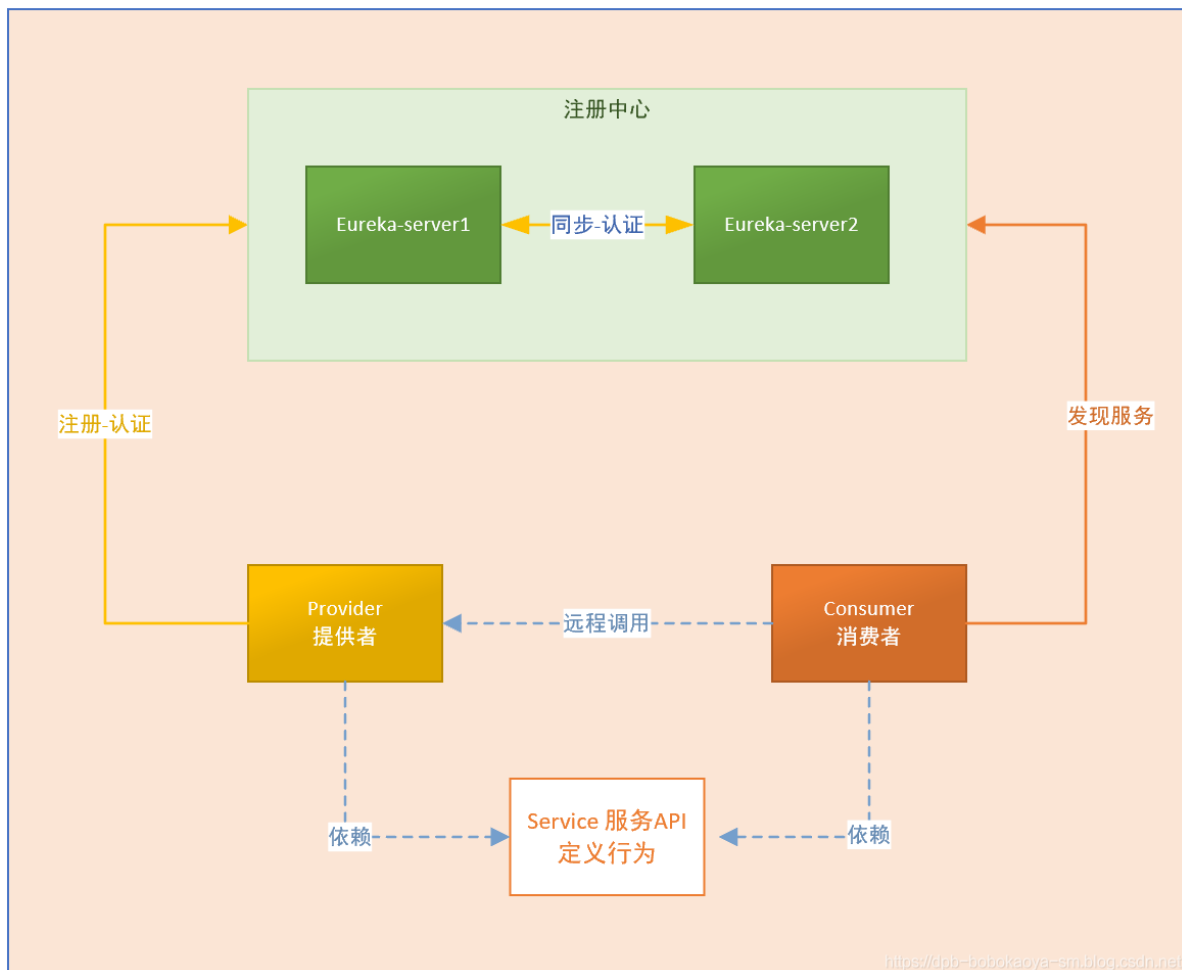
1. Spring Cloud 的声明式调用, 可以做到使用 HTTP 请求远程服务时能就像调用本地方法一样的体验，开发者完全感知不到这是远程方法，更感知不到这是个 HTTP 请求。
2. 它像 Dubbo 一样，consumer 直接调用接口方法调用 provider，而不需要通过常规的Http Client 构造请求再解析返回数据。
3. 它解决了让开发者调用远程接口就跟调用本地方法一样，无需关注与远程的交互细节，更无需关注分布式环境开发。

3.Feign入门案例

3.1.设计需求

我们模拟一个分布式电商系统中消费者通过注册中心(eureka)获取提供者提供的获取商品信息的服务的场景。

3.2.服务结构



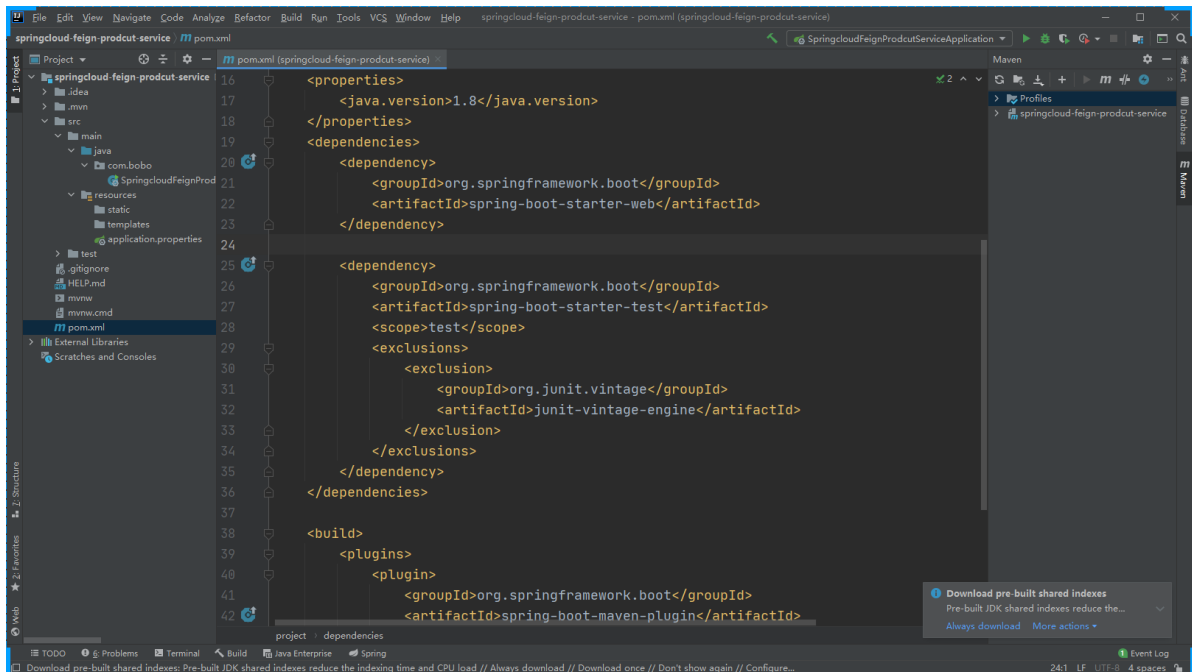
组件说明

服务	说明
eureka集群	注册中心，提供服务的注册和发现
service服务	声明service接口，定义方法，provider和consumer都需要依赖此服务而且接口中的方法需要使用到SpringMVC的组件
provider服务	提供商品信息查询服务，需要依赖service服务
consumer服务	服务消费者，需要依赖service服务，同时通过feign调用provider的服务获取商品信息

3.3 编写Service服务

3.3.1 创建项目

因为要在接口中使用SpringMVC组件，所有我们直接创建一个SpringBoot项目，添加Web依赖



3.3.2 添加依赖

因为在Service服务中我们需要使用到@RequestMapping注解，所以需要添加Web依赖

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
    <exclusions>
      <exclusion>
        <groupId>org.junit.vintage</groupId>
        <artifactId>junit-vintage-engine</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
</dependencies>
```

3.3.3 声明接口

我们需要在Service服务中声明Provider中提供的功能有哪些,同时声明Pojo对象。

```
package com.bobo.pojo;

/**
 * 商品的实体对象
 */
public class Product {

    private Integer id;
```

```

    private String name;

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Product(Integer id, String name) {
        this.id = id;
        this.name = name;
    }

    public Product() {
    }

    @Override
    public String toString() {
        return "Product{" +
            "id=" + id +
            ", name='" + name + '\'' +
            '}';
    }
}

```

服务接口

```

package com.bobo.service;

import com.bobo.pojo.Product;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;

import java.util.List;

/**
 * 公共服务接口
 */
@RequestMapping("/product")
public interface ProductService {

    /**
     * 查询所有商品的方法
     */
}

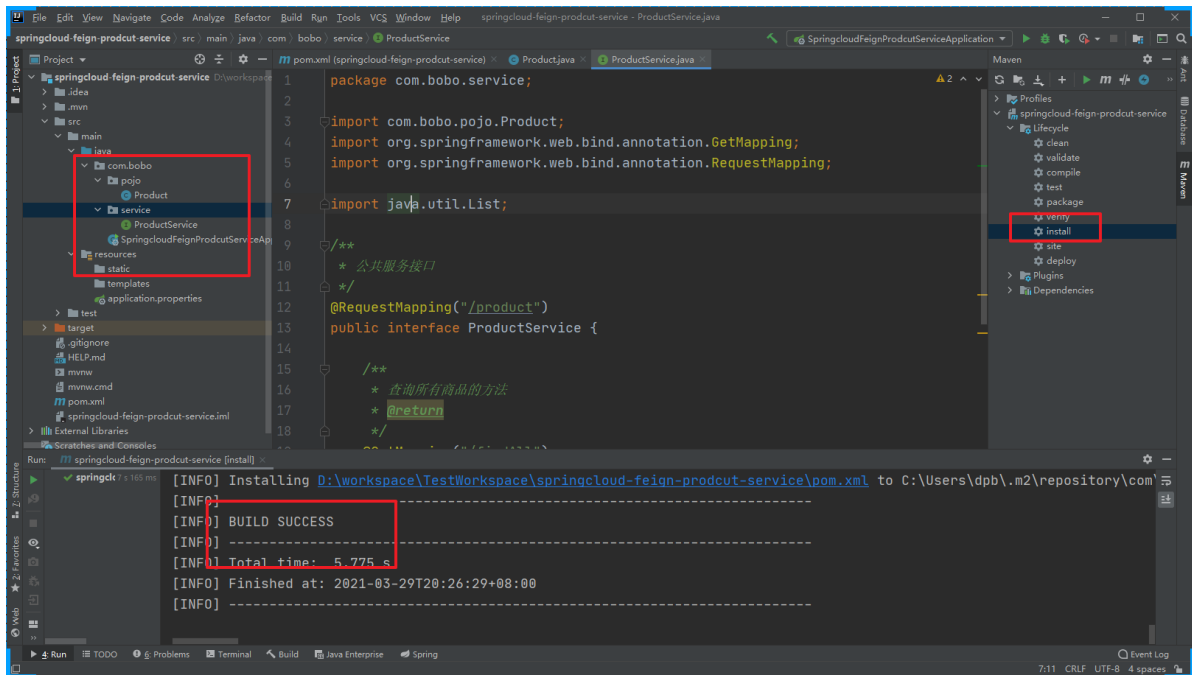
```

```

    * @return
    */
    @GetMapping("/findAll")
    public List<Product> findAll();
}

```

然后install即可

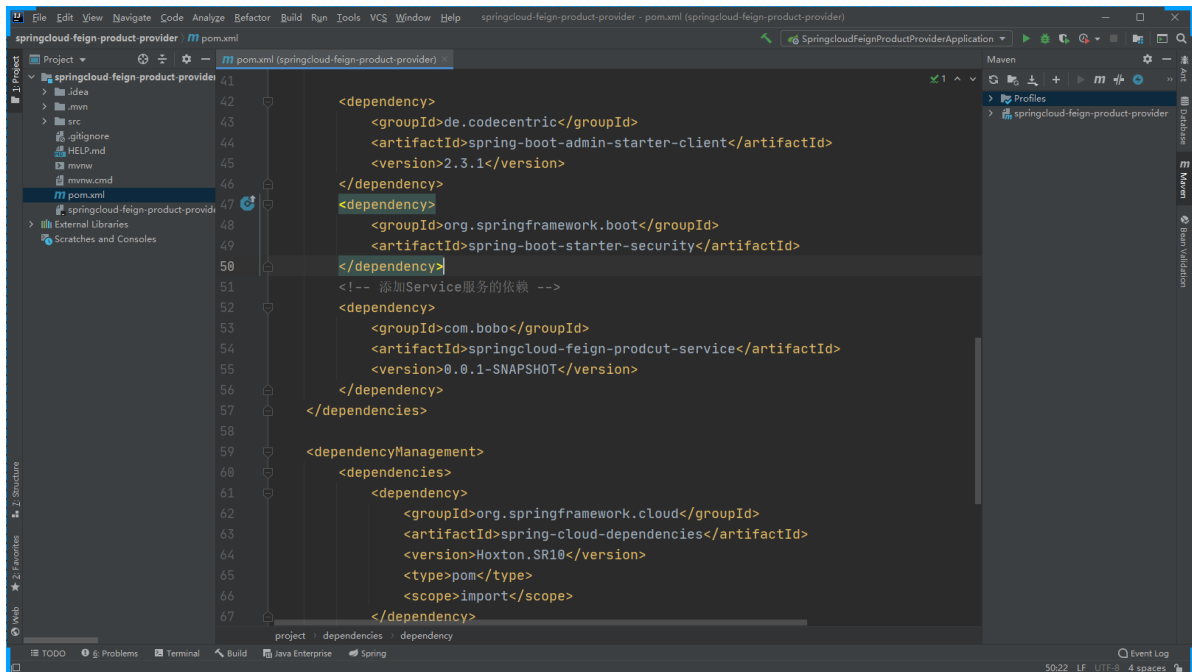


3.4 编写Provider服务

Provider服务提供的是商品信息的查询服务，同时需要在Eureka中注册

3.4.1 创建项目

创建一个SpringCloud项目即可



添加对应的依赖

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
    <exclusions>
      <exclusion>
        <groupId>org.junit.vintage</groupId>
        <artifactId>junit-vintage-engine</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
  <!-- 表示这是一个SpringCloud Eureka 的Client -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
  </dependency>
  <!-- 添加Service服务的依赖 -->
  <dependency>
    <groupId>com.bobo</groupId>
    <artifactId>springcloud-feign-prodcut-service</artifactId>
    <version>0.0.1-SNAPSHOT</version>
  </dependency>
</dependencies>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>Hoxton.SR10</version>
```

```

<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

```

3.4.1 配置文件设置

在配置文件中设计基本的配置信息即可

```

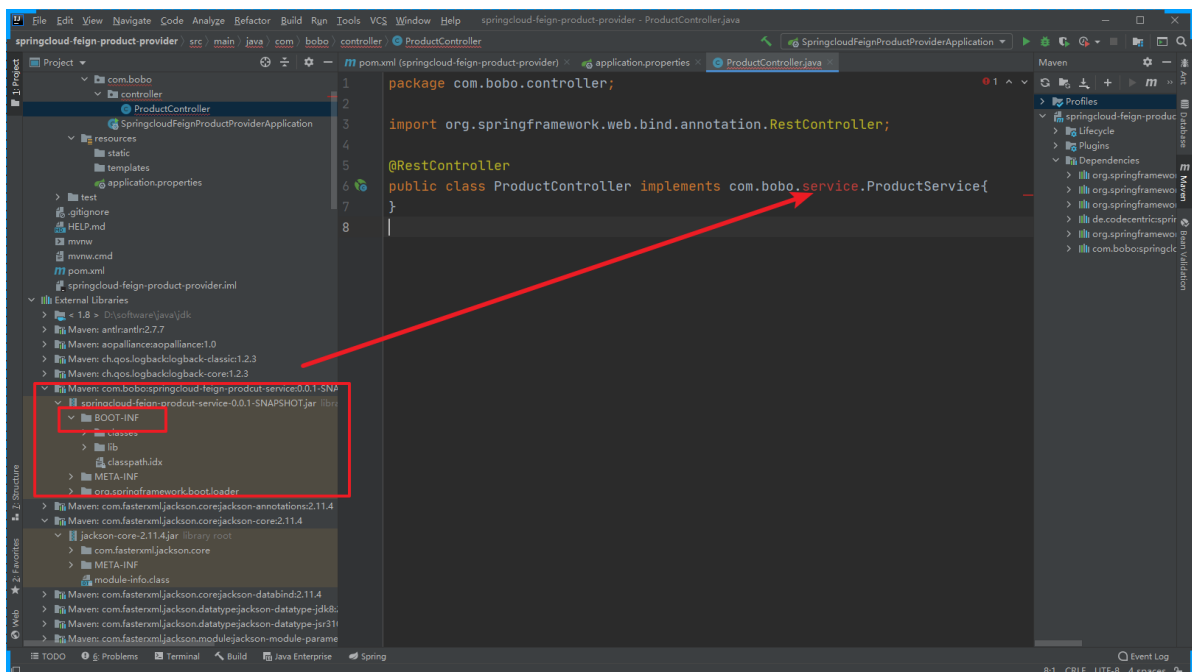
spring.application.name=shop-product-provider
server.port=9081

# 设置服务注册中心的地址
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

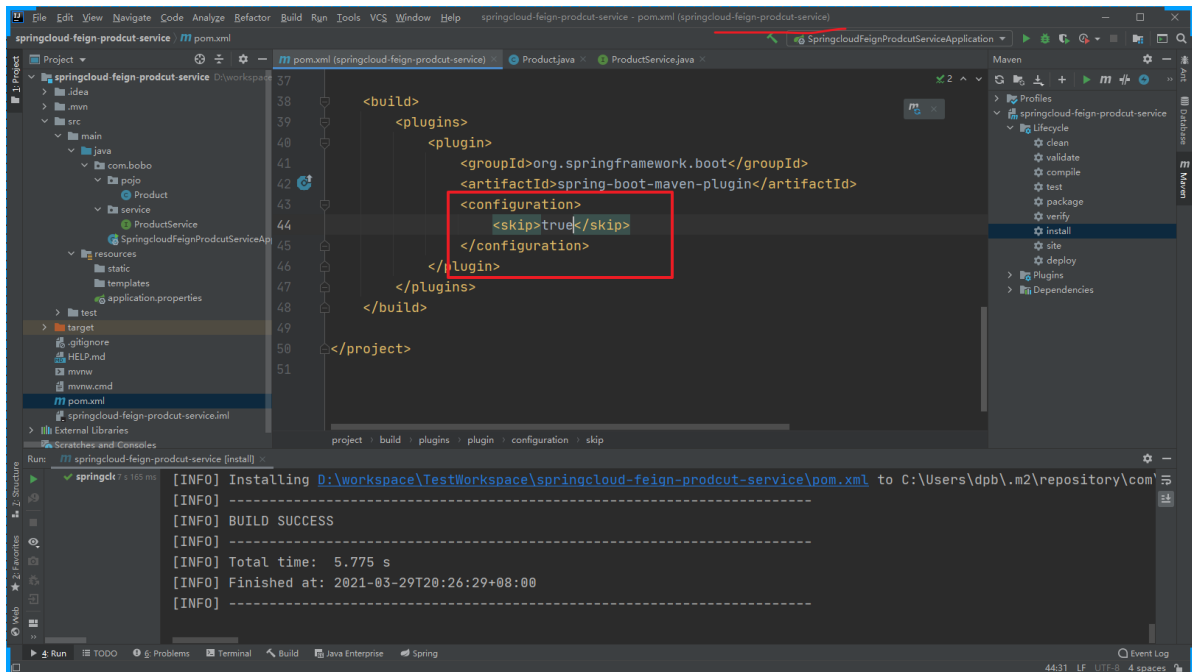
```

3.4.2 实现服务

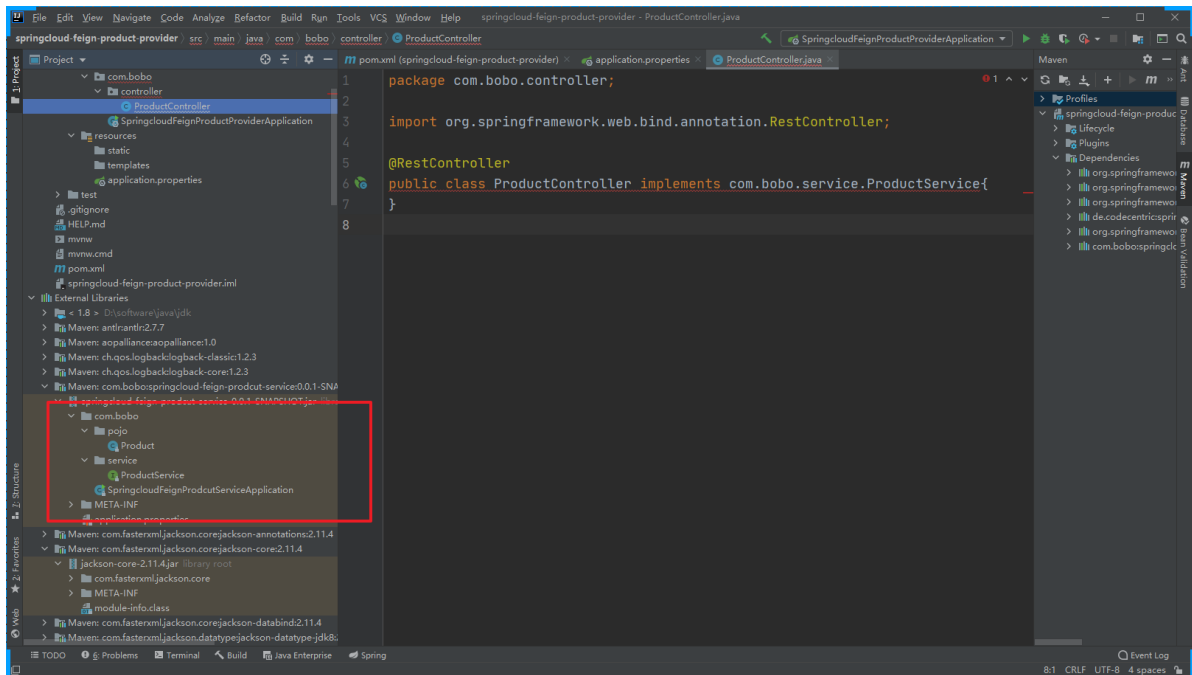
服务提供者最终是需要对外提供服务的，所以我们需要实现服务，SpringBoot项目install后的jar中的class文件出现在了BOOT-INF目录下，造成依赖的工程引用不到对应的源码



解决方式是在service的服务中我们添加如下的设置



解决了



```
package com.bobo.controller;

import com.bobo.pojo.Product;
import com.bobo.service.ProductService;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;
import java.util.List;

@RestController
public class ProductController implements ProductService {

    /**
     * 具体实现Service中声明的接口
     * 同时要注意我们需要在该方法的头部添加@RequestMapping等注解
     */
}
```

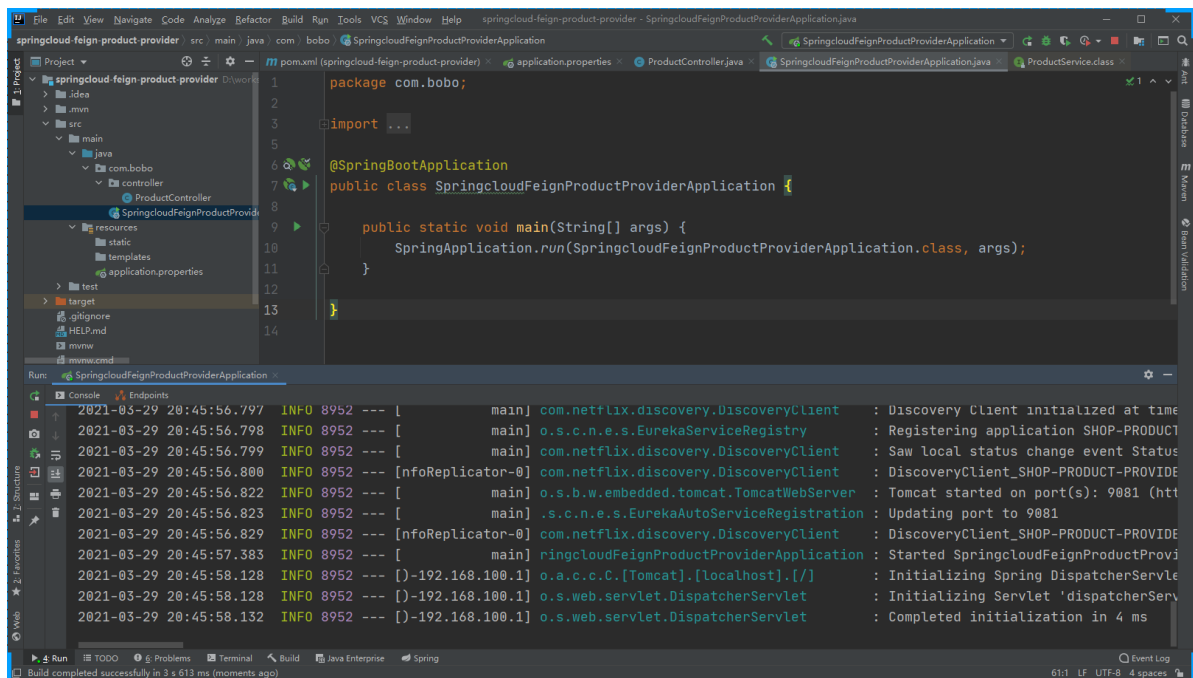
```

        * @return
        */
    @Override
    public List<Product> findAll() {
        List<Product> list = new ArrayList<>();
        list.add(new Product(1, "电视"));
        list.add(new Product(2, "洗衣机"));
        list.add(new Product(3, "冰箱"));
        list.add(new Product(4, "空调"));
        return list;
    }
}

```

3.4.3 启动服务

直接启动服务，去注册中心中查看是否注册成功



← → ↻ ⚠ 不安全 | 192.168.100.120:8761 ☆ ⓘ 更新

EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.

DS Replicas

192.168.100.121

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
EUREKA-PROVIDER	n/a (1)	(1)	UP (1) - LAPTOP-O17142BD:eureka-provider:9090
EUREKA-SERVER	n/a (2)	(2)	UP (2) - bobo02:eureka-server:8761, bobo01:eureka-server:8761
SHOP-PRODUCT-PROVIDER	n/a (1)	(1)	UP (1) - LAPTOP-O17142BD:shop-product-provider:9081

General Info

Name	Value
total-avail-memory	60mb
environment	test
num-of-cpus	1
current-memory-usage	44mb (73%)
server-up-time	06:31
registered-replicas	http://192.168.100.121:8761/eureka/
unavailable-replicas	http://192.168.100.121:8761/eureka/
available-replicas	

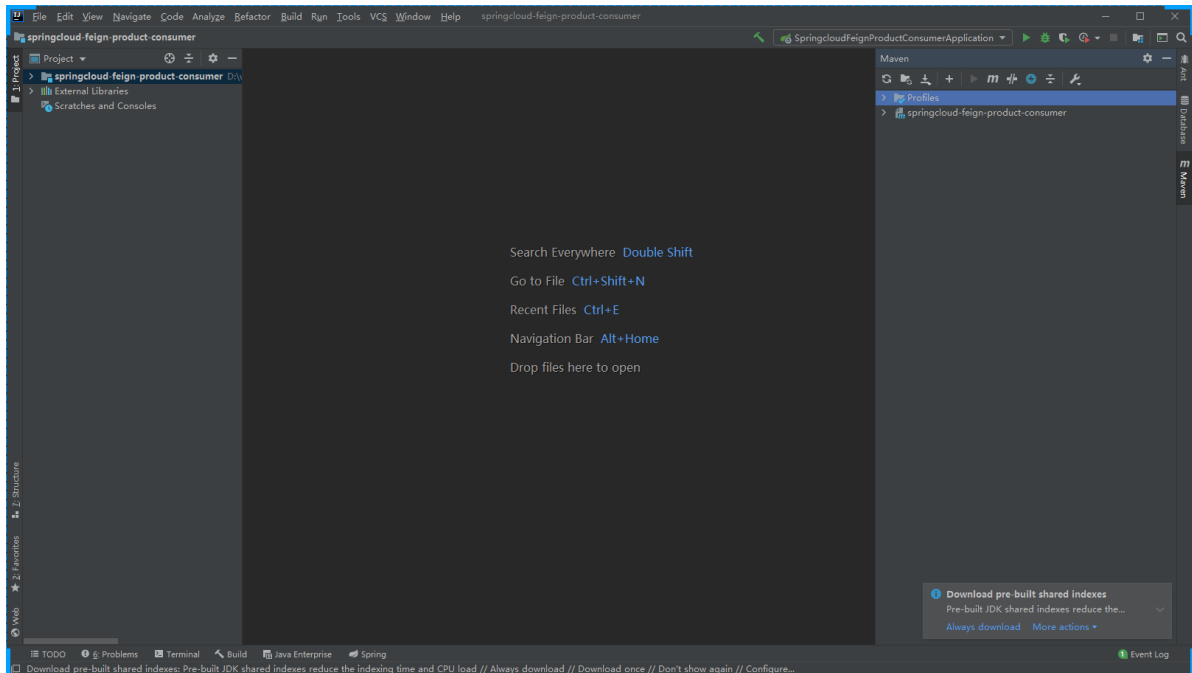
Instance Info

Name	Value
Address	192.168.100.120

3.5 编写Consumer服务

3.5.1 创建项目

创建一个SpringBoot项目即可



添加相关的依赖

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
```

```

        <scope>test</scope>
        <exclusions>
            <exclusion>
                <groupId>org.junit.vintage</groupId>
                <artifactId>junit-vintage-engine</artifactId>
            </exclusion>
        </exclusions>
    </dependency>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
    </dependency>
    <!-- 添加Feign的依赖 -->
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-feign</artifactId>
        <version>1.4.3.RELEASE</version>
    </dependency>

    <!-- Service服务的依赖 -->
    <dependency>
        <groupId>com.bobo</groupId>
        <artifactId>springcloud-feign-prodcut-service</artifactId>
        <version>0.0.1-SNAPSHOT</version>
    </dependency>
</dependencies>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>Hoxton.SR10</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>

```

3.5.2 配置信息

```

spring.application.name=shop-product-consumer
server.port=9080

# 设置服务注册中心的地址
eureka.client.service-
url.defaultzone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

```

3.5.3 创建接口

```
package com.bobo.service;

import org.springframework.cloud.openfeign.FeignClient;

/**
 * Feign的消费者的 Service服务
 * name中我们指定的 在Eureka中注册的服务提供者的名称
 */
@FeignClient(name = "shop-product-provider")
public interface ProductConsumerService extends ProductService {
}
```

3.5.4 创建控制器

控制器中就专门负责请求的处理

```
package com.bobo.controller;

import com.bobo.pojo.Product;
import com.bobo.service.ProductConsumerService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

/**
 * 服务消费者的控制器
 */
@RestController
public class ProductController {

    @Autowired
    private ProductConsumerService service;

    @RequestMapping(value = "/list", method = RequestMethod.GET)
    public List<Product> getAll() {
        System.out.println("list ..... ");
        return service.findAll();
    }
}
```

3.5.5 创建启动器

在启动器中我们需要放开Feign客户端的使用

```

/**
 * 我们需要放开Feign的客户端注解
 */
@EnableFeignClients
@SpringBootApplication
public class SpringcloudFeignProductConsumerApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringcloudFeignProductConsumerApplication.class,
args);
    }

}

```

3.5.6 启动测试

启动服务后，首先在Eureka注册中心中我们可以发现服务，然后访问测试即可

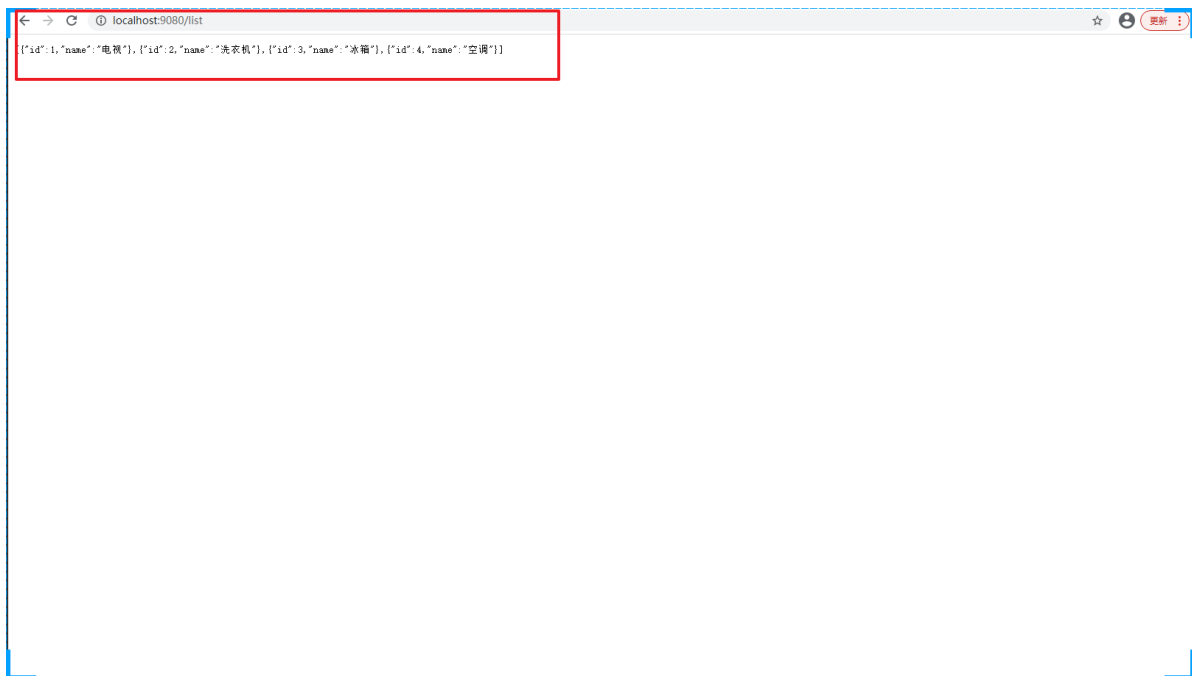
The screenshot shows the Eureka Registry web interface. At the top, there is a red warning message: "EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE." Below this, the "DS Replicas" section shows the address "192.168.100.121". The "Instances currently registered with Eureka" section contains a table with the following data:

Application	AMIs	Availability Zones	Status
EUREKA-PROVIDER	n/a (1)	(1)	UP (1) - LAPTOP-O17142BD:eureka-provider:9090
EUREKA-SERVER	n/a (2)	(2)	UP (2) - bobo02:eureka-server:8761, bobo01:eureka-server:8761
SHOP-PRODUCT-CONSUMER	n/a (1)	(1)	UP (1) - LAPTOP-O17142BD:shop-product-consumer:9080
SHOP-PRODUCT-PROVIDER	n/a (1)	(1)	UP (1) - LAPTOP-O17142BD:shop-product-provider:9081

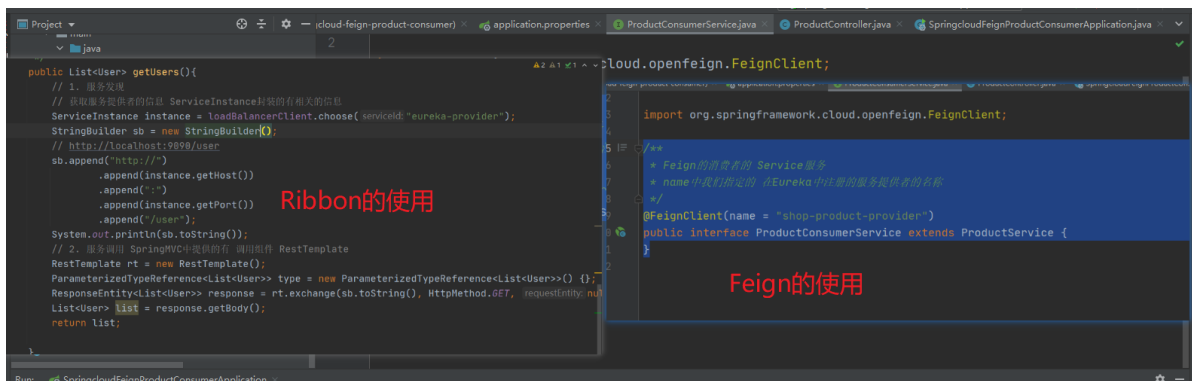
The "SHOP-PRODUCT-CONSUMER" row is highlighted with a red border. Below the table, the "General Info" section shows various system metrics:

Name	Value
total-avail-memory	60mb
environment	test
num-of-cpus	1
current-memory-usage	34mb (56%)
server-up-time	06:51
registered-replicas	http://192.168.100.121:8761/eureka/
unavailable-replicas	http://192.168.100.121:8761/eureka/
available-replicas	

The "Instance Info" section is partially visible at the bottom.



和Ribbon使用的对比



4. 参数处理

我们在实际的处理过程中会面对各种各样的请求情况，那么就会出现传递各种类型参数的情况，所以我们来分析下。

4.1 单个参数

4.1.1 Service服务

我们首先在Service服务中新增加一个接收单个参数的方法

```
package com.bobo.service;

import com.bobo.pojo.Product;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;

import java.util.List;
```

```

/**
 * 公共服务接口
 */
@RequestMapping("/product")
public interface ProductService {

    /**
     * 查询所有商品的方法
     * @return
     */
    @GetMapping("/findAll")
    public List<Product> findAll();

    /**
     * 根据ID查询商品信息
     * Feign本身也是基于Http请求的客户端
     * 那么在接收参数的时候我们需要通过@RequestParam注解来指明要接收的参数
     * @param id
     * @return
     */
    @GetMapping("/getProductById")
    public Product getProductById(@RequestParam("id") Integer id);

}

```

记得install打包

4.1.2 Provider服务

既然新增加了方法的声明，那么我们就需要在Provider中实现该功能

```

package com.bobo.controller;

import com.bobo.pojo.Product;
import com.bobo.service.ProductService;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;
import java.util.List;

@RestController
public class ProductController implements ProductService {

    /**
     * 具体实现Service中声明的接口
     * 同时要注意我们需要在该方法的头部添加@RequestMapping等注解
     * @return
     */
    @Override
    public List<Product> findAll() {
        List<Product> list = new ArrayList<>();
        list.add(new Product(1, "电视"));
        list.add(new Product(2, "洗衣机"));
        list.add(new Product(3, "冰箱"));
    }
}

```

```

        list.add(new Product(4,"空调"));
        return list;
    }

    /**
     * 单个参数的处理
     * @param id
     * @return
     */
    @Override
    public Product getProductById(Integer id) {
        System.out.println("服务端: " + id);
        return new Product(id,"微波炉");
    }
}

```

4.1.3 Consumer服务

在消费者中我们通过Feign直接调用即可

```

package com.bobo.controller;

import com.bobo.pojo.Product;
import com.bobo.service.ProductConsumerService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

/**
 * 服务消费者的控制器
 */
@RestController
public class ProductController {

    @Autowired
    private ProductConsumerService service;

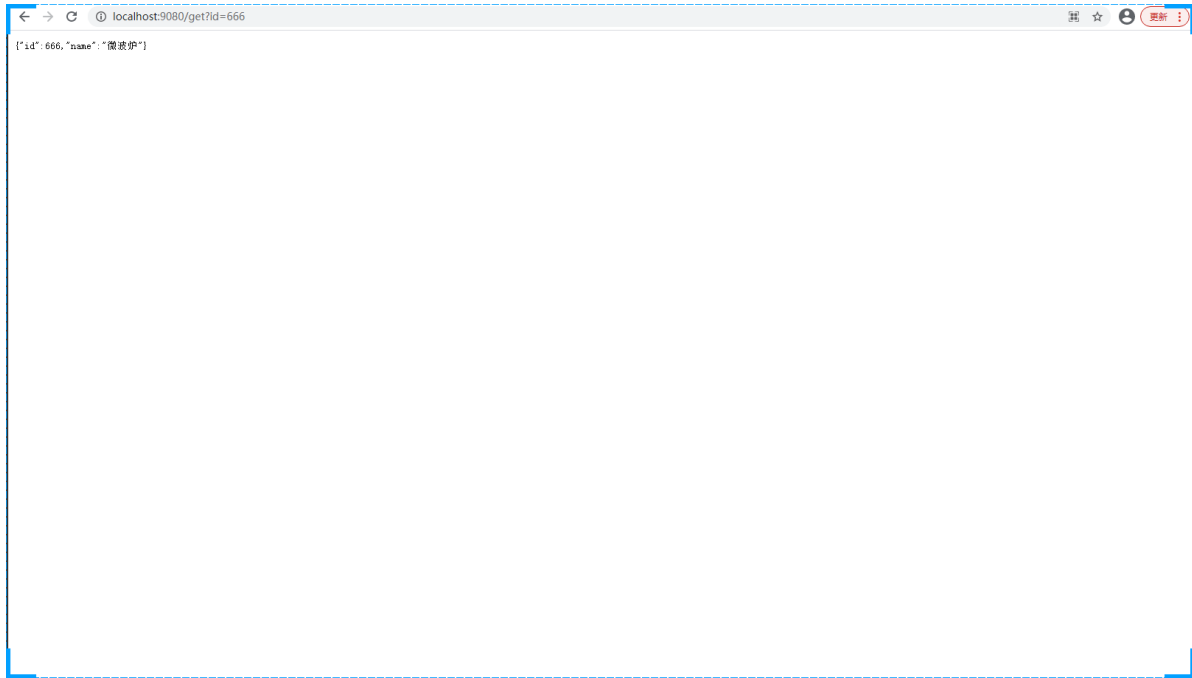
    @RequestMapping(value = "/list",method = RequestMethod.GET)
    public List<Product> getAll() {
        System.out.println("list ..... ");
        return service.findAll();
    }

    @GetMapping("/get")
    public Product getProductById(@RequestParam("id") Integer id){
        return service.getProductById(id);
    }
}

```

4.1.4 启动服务

启动服务测试



4.2 多个参数

多个参数传递的情况下，我们接收的时候可以通过多个参数来接收，也可以通过自定义对象来接收，我们要区分对应的GET方式和POST方式的请求

4.2.1 Service服务

```
package com.bobo.service;

import com.bobo.pojo.Product;
import org.springframework.web.bind.annotation.*;

import java.util.List;

/**
 * 公共服务接口
 */
@RequestMapping("/product")
public interface ProductService {

    /**
     * 查询所有商品的方法
     * @return
     */
    @GetMapping("/findAll")
    public List<Product> findAll();

    /**
     * 根据ID查询商品信息
     * Feign本身也是基于Http请求的客户端
     */
}
```

```

    *    那么在接收参数的时候我们需要通过@RequestParam注解来指明要接收的参数
    * @param id
    * @return
    */
@GetMapping("/getProductById")
public Product getProductById(@RequestParam("id") Integer id);

/**
 * GET方式
 * 获取多个参数
 * @return
 */
@GetMapping("/addProductGet")
public Product addProductGet(@RequestParam("id") Integer id,
                             @RequestParam("name") String name);

/**
 * Post方式提交
 * 获取多个参数: @RequestBody
 * @param product
 * @return
 */
@PostMapping("/addProductPost")
public Product addProductPost(@RequestBody Product product);
}

```

4.2.2 Provider服务

```

package com.bobo.controller;

import com.bobo.pojo.Product;
import com.bobo.service.ProductService;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;
import java.util.List;

@RestController
public class ProductController implements ProductService {

    /**
     * 具体实现Service中声明的接口
     * 同时要注意我们需要在该方法的头部添加@RequestMapping等注解
     * @return
     */
    @Override
    public List<Product> findAll() {
        List<Product> list = new ArrayList<>();
        list.add(new Product(1, "电视"));
        list.add(new Product(2, "洗衣机"));
        list.add(new Product(3, "冰箱"));
    }
}

```

```

        list.add(new Product(4,"空调"));
        return list;
    }

    /**
     * 单个参数的处理
     * @param id
     * @return
     */
    @Override
    public Product getProductById(Integer id) {
        System.out.println("服务端: " + id);
        return new Product(id,"微波炉");
    }

    @Override
    public Product addProductGet(Integer id, String name) {
        return new Product(id,name);
    }

    @Override
    public Product addProductPost(@RequestBody Product product) {
        return product;
    }
}

```

4.2.3 Consumer服务

```

package com.bobo.controller;

import com.bobo.pojo.Product;
import com.bobo.service.ProductConsumerService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

/**
 * 服务消费者的控制器
 */
@RestController
public class ProductController {

    @Autowired
    private ProductConsumerService service;

    @RequestMapping(value = "/list",method = RequestMethod.GET)
    public List<Product> getAll() {
        System.out.println("list ..... ");
        return service.findAll();
    }
}

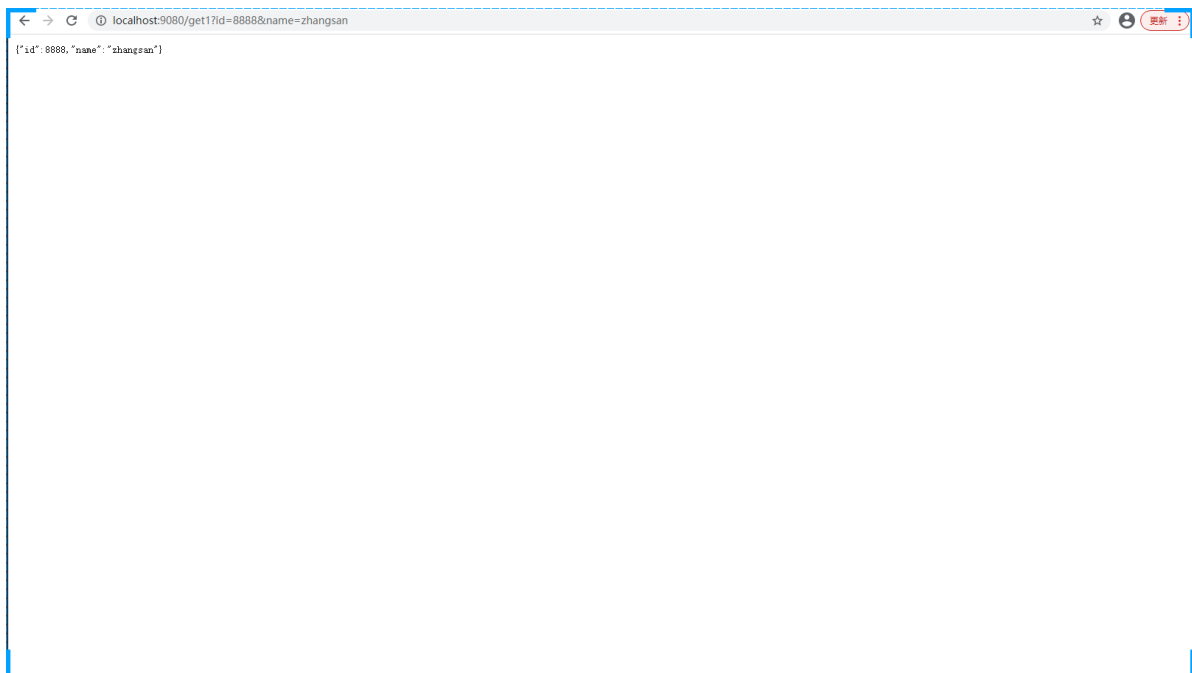
```

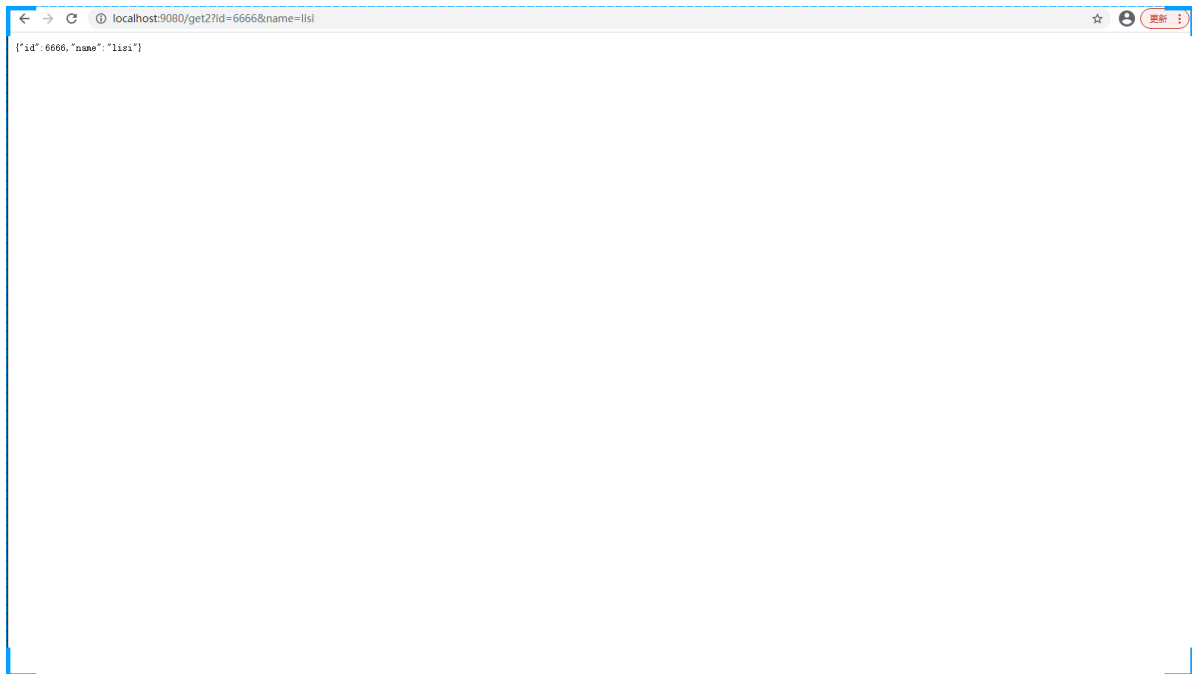
```
@GetMapping("/get")
public Product getProductById(@RequestParam("id") Integer id){
    return service.getProductById(id);
}

@RequestMapping("/get1")
public Product addProductGet(Product product){
    return service.addProductGet(product.getId(),product.getName());
}

@RequestMapping("/get2")
public Product addProductPost(Product product){
    return service.addProductPost(product);
}
}
```

4.2.4 启动测试





5. 压缩处理

在数据传输过程中压缩数据肯定是必须的，而gzip是我们比较常用的方式，而且我们在刚刚接触http协议的时候就介绍过gzip。接下来我们来看下在微服务环境下我们怎么通过gzip来压缩数据。

5.1 gzip 介绍

gzip是一种数据格式，采用用 deflate 算法压缩 data；gzip 是一种流行的文件压缩算法，应用十分广泛，尤其是在 Linux 平台。

gzip 能力：

当 Gzip 压缩到一个纯文本文件时，效果是非常明显的，大约可以减少 70%以上的文件大小。

gzip 作用：

网络数据经过压缩后实际上降低了网络传输的字节数，最明显的好处就是可以加快网页加载的速度。网页加载速度加快的好处不言而喻，除了节省流量，改善用户的浏览体验外，另一个潜在的好处是 Gzip 与搜索引擎的抓取工具有着更好的关系。例如 Google就可以通过直接读取 gzip 文件来比普通手工抓取 更快地检索网页。

5.2 HTTP压缩传输的规定

HTTP协议中关于压缩传输的规定

1. 客户端向服务器请求中带有：Accept-Encoding:gzip, deflate 字段，向服务器表示，客户端支持的压缩格式（gzip 或者 deflate），如果不发送该消息头，服务器是不会压缩的。
2. 服务端在收到请求之后，如果发现请求头中含有 Accept-Encoding 字段，并且支持该类型的压缩，就对响应报文压缩之后返回给客户端，并且携带 Content-Encoding:gzip 消息头，表示响应报文是根据该格式压缩过的。
3. 客户端接收到请求之后，先判断是否有 Content-Encoding 消息头，如果有，按该格式解压报文。否则按正常报文处理。

5.3 案例介绍

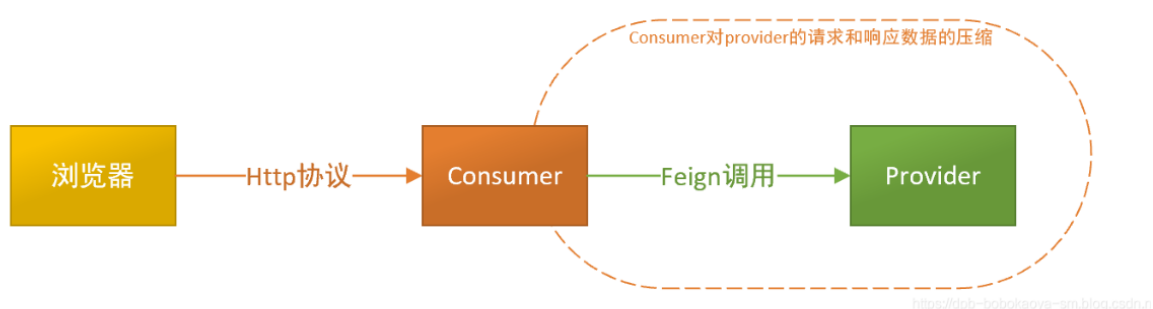
案例结构



<https://dpb-bobokaoya-sm.blog.csdn.net>

5.3.1.配置consumer到provider的压缩

配置consumer通过feign调用provider中的服务的压缩操作

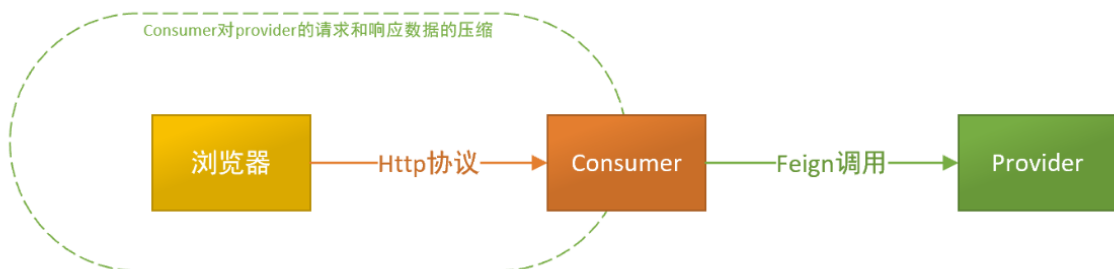


<https://dpb-bobokaoya-sm.blog.csdn.net>

```
#-----feign gzip
#配置请求 GZIP 压缩
feign.compression.request.enabled=true
#配置响应 GZIP 压缩
feign.compression.response.enabled=true
#配置压缩支持的 MIME TYPE
feign.compression.request.mime-types=text/xml,application/xml,application/json
#配置压缩数据大小的最小阈值，默认 2048
feign.compression.request.min-request-size=512
```

5.3.2.配置浏览器到consumer的压缩

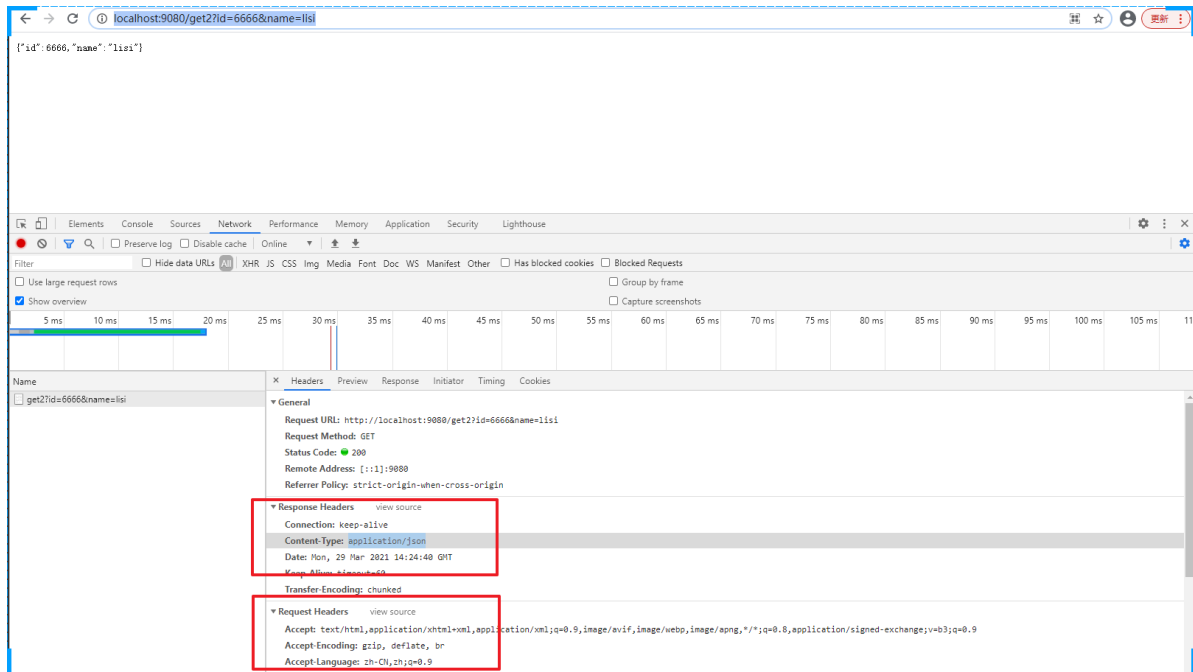
配置浏览器发送请求到consumer的数据压缩及响应的压缩。



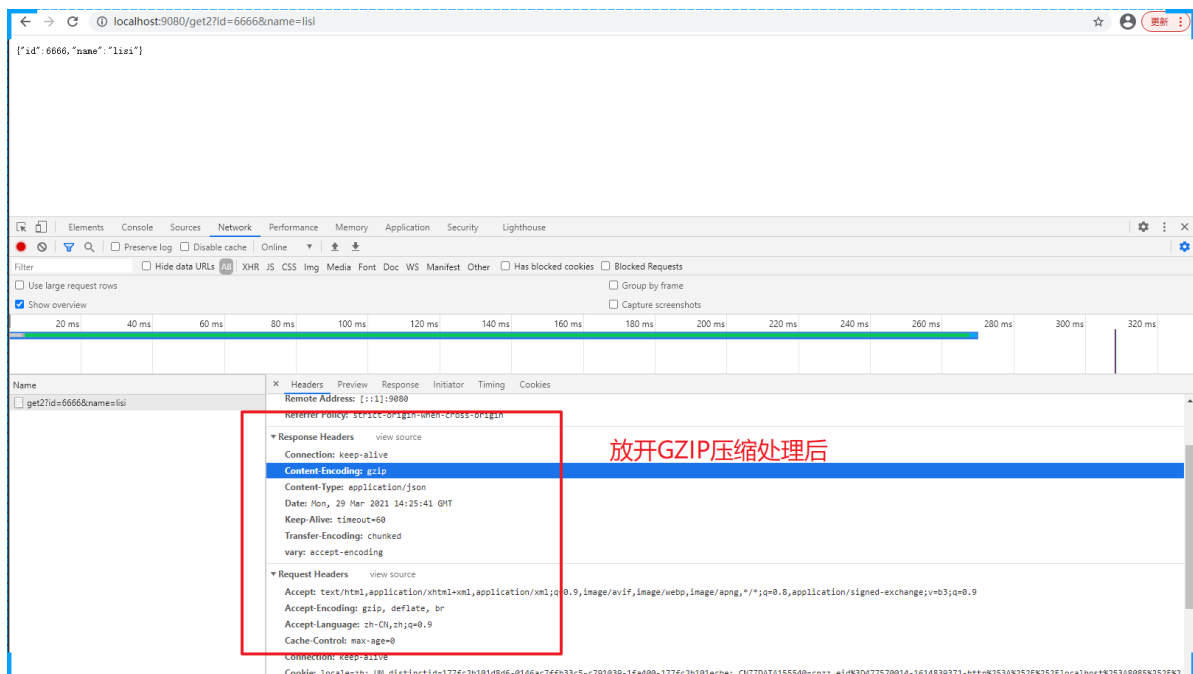
<https://dpb-bobokaoya-sm.blog.csdn.net>

```
#-----spring boot gzip
#是否启用压缩
server.compression.enabled=true
server.compression.mime-types=application/json,application/
xml,text/html,text/xml,text/plain
```

没有使用之前



放开GZIP压缩后



6. HttpClient连接池

为什么 http 连接池能提升性能？

6.1 http 的背景原理

- a. 两台服务器建立 http 连接的过程是很复杂的一个过程，涉及到多个数据包的交换，并且也很耗时间。
- b. Http 连接需要的 3 次握手 4 次分手开销很大，这一开销对于大量的比较小的 http 消息来说更大。

6.2 优化解决方案

- a. 如果我们直接采用 http 连接池，节约了大量的 3 次握手 4 次分手；这样能大大提升吞吐率。
- b. feign 的 http 客户端支持 3 种框架；HttpURLConnection、httpClient、okhttp；默认是 HttpURLConnection。
- c. 传统的 HttpURLConnection 是 JDK 自带的，并不支持连接池，如果要实现连接池的机制，还需要自己去管理连接对象。对于网络请求这种底层相对复杂的操作，如果有可用的其他方案，也没有必要自己去管理连接对象。
- d. HttpClient 相比传统 JDK 自带的 HttpURLConnection，它封装了访问 http 的请求头，参数，内容体，响应等等；它不仅使客户端发送 HTTP 请求变得容易，而且也方便了开发人员测试接口（基于 Http 协议的），即提高了开发的效率，也方便提高代码的健壮性；另外高并发大量的请求网络的时候，还是用“连接池”提升吞吐量。

6.2 案例实现

6.2.1 添加依赖

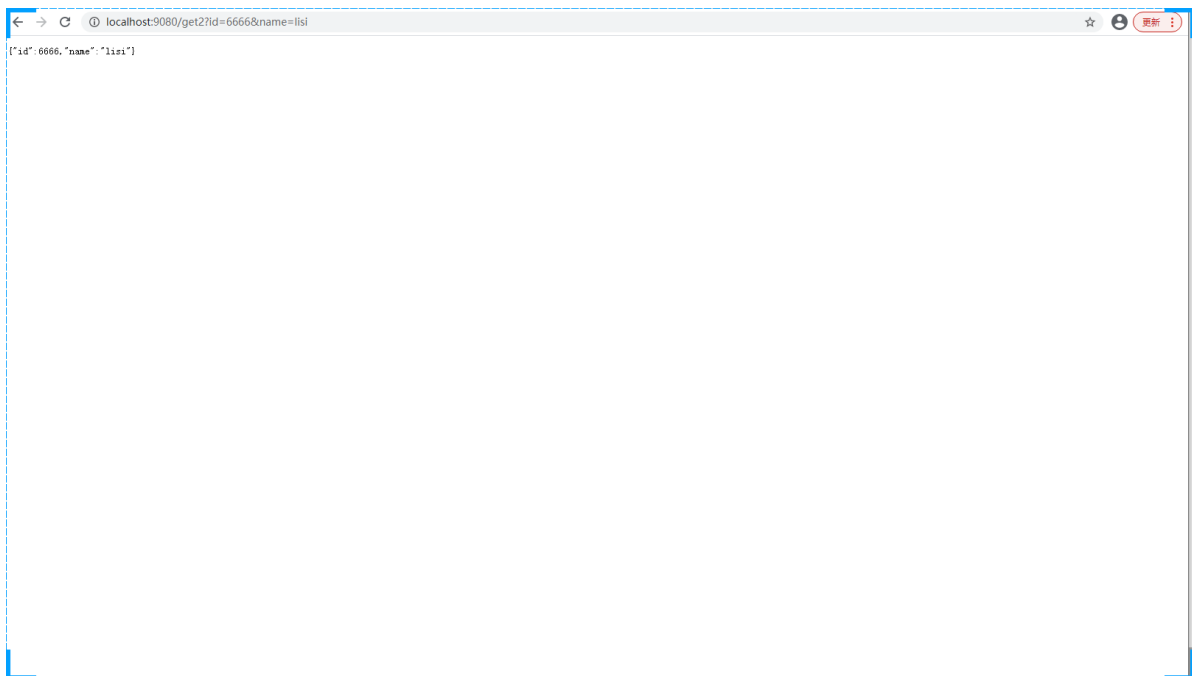
```
<!-- 添加HttpClient的依赖 -->
<dependency>
    <groupId>org.apache.httpcomponents</groupId>
    <artifactId>httpClient</artifactId>
</dependency>
<!-- 添加Feign对HttpClient的支持 -->
<dependency>
    <groupId>io.github.openfeign</groupId>
    <artifactId>feign-httpclient</artifactId>
</dependency>
```

6.2.2 修改配置

在属性文件中放开Feign对HttpClient的支持

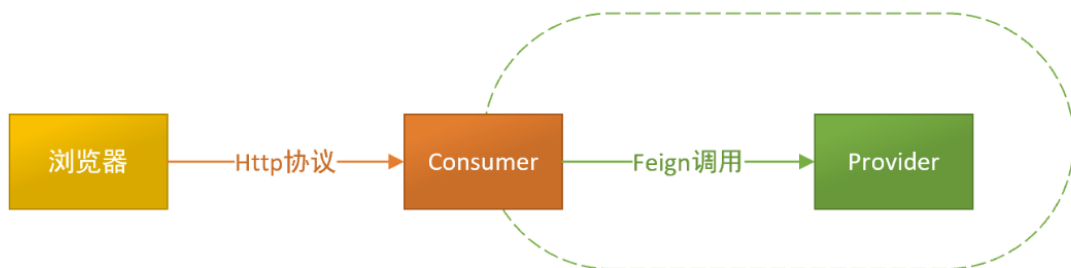
```
## 启用 httpClient
feign.httpclient.enabled=true
```

6.2.3 测试访问



7. 日志处理

在前面的案例中我们可以发现，浏览器到consumer中的请求情况我们可以通过浏览器来帮助我们查看，但是consumer对provider的调用情况我们是不清楚的，这时我们可以通过日志来分析



7.1 添加日志文件

我们通过logback来记录日志信息

```
<?xml version="1.0" encoding="UTF-8" ?>
<configuration>
    <!--定义日志文件的存储地址 勿在 LogBack 的配置中使用相对路径-->
    <property name="LOG_HOME" value="${catalina.base}/logs/" />
    <!-- 控制台输出 -->
    <appender name="Stdout" class="ch.qos.logback.core.ConsoleAppender">
        <!-- 日志输出编码 -->
        <layout class="ch.qos.logback.classic.PatternLayout">
            <!--格式化输出：%d表示日期，%thread表示线程名，%-5level：级别从左显示5个字符宽度%msg：日志消息，%n是换行符-->
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50}
- %msg%n
        </pattern>
```

```

        </layout>
    </appender>
    <!-- 按照每天生成日志文件 -->
    <appender name="RollingFile"
class="ch.qos.logback.core.rolling.RollingFileAppender">
        <rollingPolicy
class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
            <!-- 日志文件输出的文件名 -->
            <FileNamePattern>${LOG_HOME}/server.%d{yyyy-MM-
dd}.log</FileNamePattern>
            <MaxHistory>30</MaxHistory>
        </rollingPolicy>
        <layout class="ch.qos.logback.classic.PatternLayout">
            <!-- 格式化输出: %d表示日期, %thread表示线程名, %-5level: 级别从左显示5个字符宽
度%msg: 日志消息, %n是换行符 -->
            <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{50}
- %msg%n
            </pattern>
        </layout>
        <!-- 日志文件最大的大小 -->
        <triggeringPolicy
class="ch.qos.logback.core.rolling.SizeBasedTriggeringPolicy">
            <MaxFileSize>10MB</MaxFileSize>
        </triggeringPolicy>
    </appender>

    <!-- 日志输出级别 -->
    <root level="DEBUG">
        <appender-ref ref="Stdout" />
        <appender-ref ref="RollingFile" />
    </root>

    <!-- 日志异步到数据库 -->
    <!--
        <appender name="DB" class="ch.qos.logback.classic.db.DBAppender">
            日志异步到数据库
            <connectionSource
class="ch.qos.logback.core.db.DriverManagerConnectionSource">
                连接池
                <dataSource class="com.mchange.v2.c3p0.ComboPooledDataSource">
                    <driverClass>com.mysql.jdbc.Driver</driverClass>
                    <url>jdbc:mysql://127.0.0.1:3306/databaseName</url>
                    <user>root</user>
                    <password>root</password>
                </dataSource>
            </connectionSource>
        </appender> -->

</configuration>

```

注意日志级别设置为 DEBUG

7.2 设置Feign日志级别

这个我们需要在SpringBoot的配置类中向IoC容器中注入一个Logger.Level对象

```
package com.bobo;

import feign.Logger;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.openfeign.EnableFeignClients;
import org.springframework.context.annotation.Bean;

/**
 * 我们需要放开Feign的客户端注解
 */
@EnableFeignClients
@SpringBootApplication
public class SpringCloudFeignProductConsumerApplication {

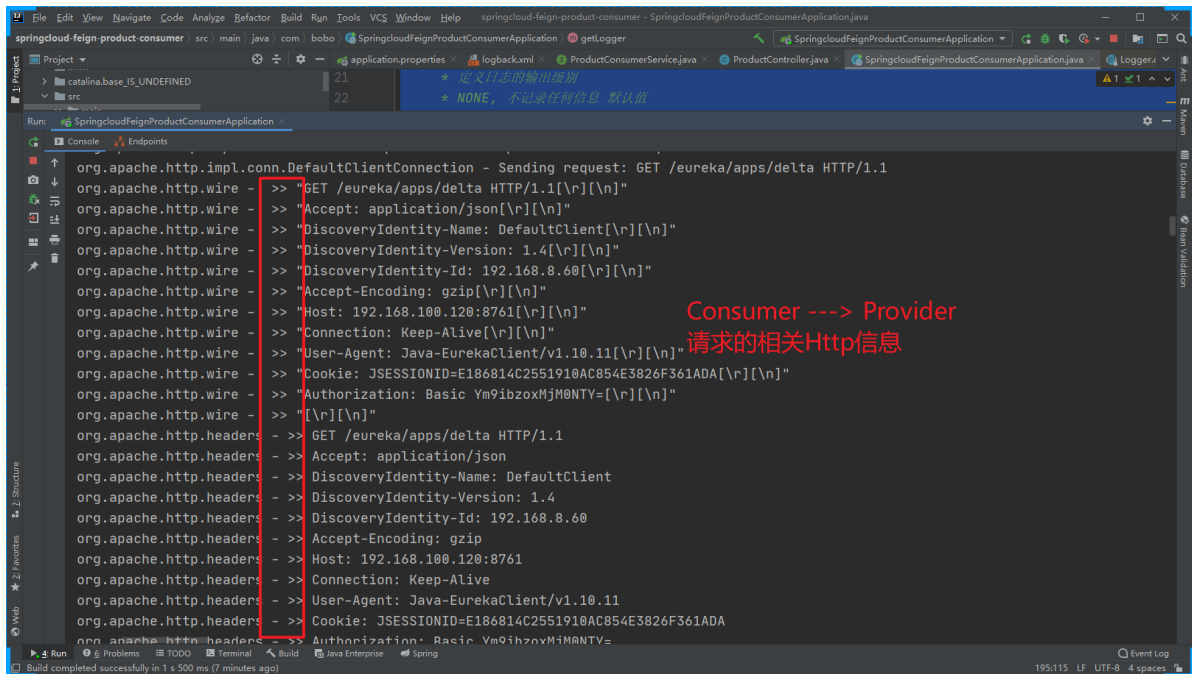
    public static void main(String[] args) {
        SpringApplication.run(SpringCloudFeignProductConsumerApplication.class,
args);
    }

    /**
     * 定义日志的输出级别
     * NONE，不记录任何信息 默认值
     * BASIC，记录基本信息(请求方法，请求URL，状态码和用时)
     * HEADERS,在BASIC的基础上再记录一些常用信息
     * FULL;记录请求和响应的所有的数据
     * @return
     */
    @Bean
    public Logger.Level getLogger(){
        return Logger.Level.FULL;
    }

}
```

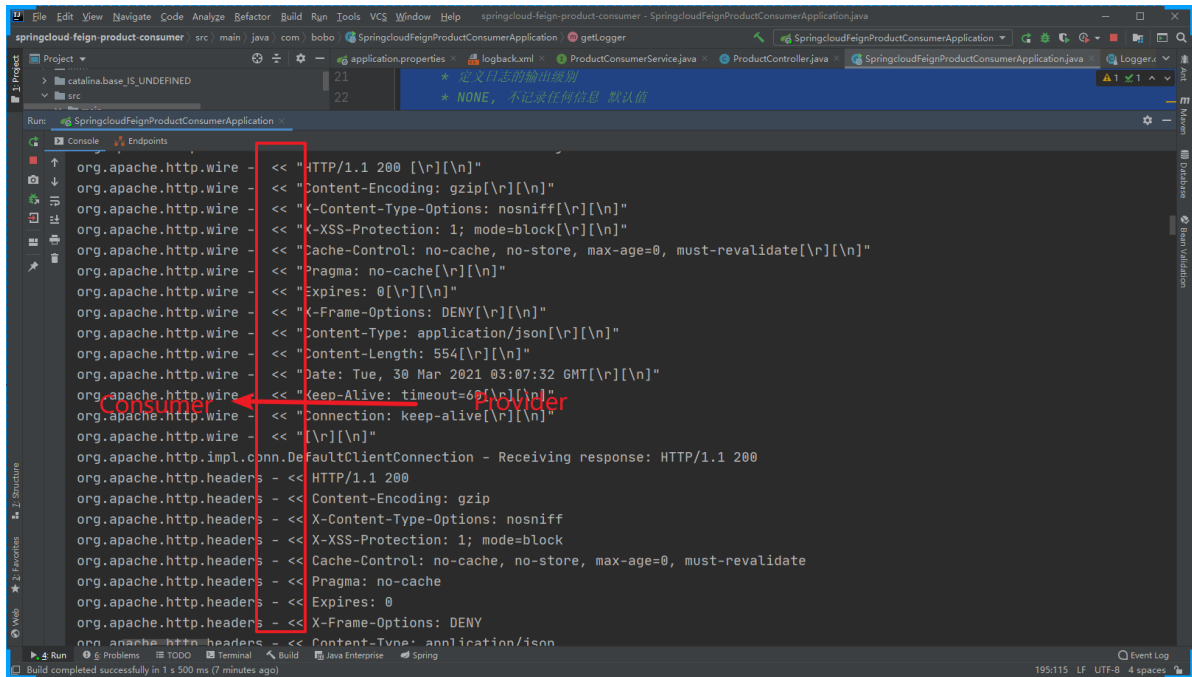
7.3 测试

访问请求后我们就可以在控制台输出中查看到对应的请求信息



```
org.apache.http.impl.conn.DefaultClientConnection - Sending request: GET /eureka/apps/delta HTTP/1.1
org.apache.http.wire - >> "GET /eureka/apps/delta HTTP/1.1[\r][\n]"
org.apache.http.wire - >> "Accept: application/json[\r][\n]"
org.apache.http.wire - >> "DiscoveryIdentity-Name: DefaultClient[\r][\n]"
org.apache.http.wire - >> "DiscoveryIdentity-Version: 1.4[\r][\n]"
org.apache.http.wire - >> "DiscoveryIdentity-Id: 192.168.8.60[\r][\n]"
org.apache.http.wire - >> "Accept-Encoding: gzip[\r][\n]"
org.apache.http.wire - >> "Host: 192.168.100.120:8761[\r][\n]"
org.apache.http.wire - >> "Connection: Keep-Alive[\r][\n]"
org.apache.http.wire - >> "User-Agent: Java-EurekaClient/v1.10.11[\r][\n]"
org.apache.http.wire - >> "Cookie: JSESSIONID=E186814C2551910AC854E3826F361ADA[\r][\n]"
org.apache.http.wire - >> "Authorization: Basic Ym91b2x0MjM0NTY=[\r][\n]"
org.apache.http.wire - >> "[\r][\n]"
org.apache.http.headers - >> GET /eureka/apps/delta HTTP/1.1
org.apache.http.headers - >> Accept: application/json
org.apache.http.headers - >> DiscoveryIdentity-Name: DefaultClient
org.apache.http.headers - >> DiscoveryIdentity-Version: 1.4
org.apache.http.headers - >> DiscoveryIdentity-Id: 192.168.8.60
org.apache.http.headers - >> Accept-Encoding: gzip
org.apache.http.headers - >> Host: 192.168.100.120:8761
org.apache.http.headers - >> Connection: Keep-Alive
org.apache.http.headers - >> User-Agent: Java-EurekaClient/v1.10.11
org.apache.http.headers - >> Cookie: JSESSIONID=E186814C2551910AC854E3826F361ADA
org.apache.http.headers - >> Authorization: Basic Ym91b2x0MjM0NTY=
```

Consumer ---> Provider
请求的相关Http信息



```
org.apache.http.wire - << "HTTP/1.1 200 [\r][\n]"
org.apache.http.wire - << "Content-Encoding: gzip[\r][\n]"
org.apache.http.wire - << "X-Content-Type-Options: nosniff[\r][\n]"
org.apache.http.wire - << "X-XSS-Protection: 1; mode=block[\r][\n]"
org.apache.http.wire - << "Cache-Control: no-cache, no-store, max-age=0, must-revalidate[\r][\n]"
org.apache.http.wire - << "Pragma: no-cache[\r][\n]"
org.apache.http.wire - << "Expires: 0[\r][\n]"
org.apache.http.wire - << "X-Frame-Options: DENY[\r][\n]"
org.apache.http.wire - << "Content-Type: application/json[\r][\n]"
org.apache.http.wire - << "Content-Length: 554[\r][\n]"
org.apache.http.wire - << "Date: Tue, 30 Mar 2021 03:07:32 GMT[\r][\n]"
org.apache.http.wire - << "Keep-Alive: timeout=60[\r][\n]"
org.apache.http.wire - << "Connection: keep-alive[\r][\n]"
org.apache.http.wire - << "[\r][\n]"
org.apache.http.impl.conn.DefaultClientConnection - Receiving response: HTTP/1.1 200
org.apache.http.headers - << HTTP/1.1 200
org.apache.http.headers - << Content-Encoding: gzip
org.apache.http.headers - << X-Content-Type-Options: nosniff
org.apache.http.headers - << X-XSS-Protection: 1; mode=block
org.apache.http.headers - << Cache-Control: no-cache, no-store, max-age=0, must-revalidate
org.apache.http.headers - << Pragma: no-cache
org.apache.http.headers - << Expires: 0
org.apache.http.headers - << X-Frame-Options: DENY
org.apache.http.headers - << Content-Type: application/json
```

Consumer ← Provider

8.超时时间设置

Feign调用服务的时候默认的时长是1秒钟，也就是如果超过1秒钟没有连接上，或者超过1秒钟没有响应，那么就会相应的报错，但是实际的情况因为业务不同处理的时间会不一样，所以这时我们需要手动的方式来调整超时时间。

8.1 全局配置

Feign的负载均衡底层使用的是Ribbon，我们可以在属性文件中配置Ribbon的连接和读写信息

```
## 全局配置 超时时间
## 请求连接时间 默认是1秒钟
ribbon.ConnectionTimeout=8000
## 请求处理的时间
ribbon.ReadTimeout=8000
```

8.2 局部配置

```
#局部配置
# 对所有操作请求都进行重试
shop-product-provider.ribbon.OkToRetryOnAllOperations=true
# 对当前实例的重试次数
shop-product-provider.ribbon.MaxAutoRetries=2
# 切换实例的重试次数
shop-product-provider.ribbon.MaxAutoRetriesNextServer=0
# 请求连接的超时时间
shop-product-provider.ribbon.ConnectTimeout=3000
# 请求处理的超时时间
shop-product-provider.ribbon.ReadTimeout=3000
```

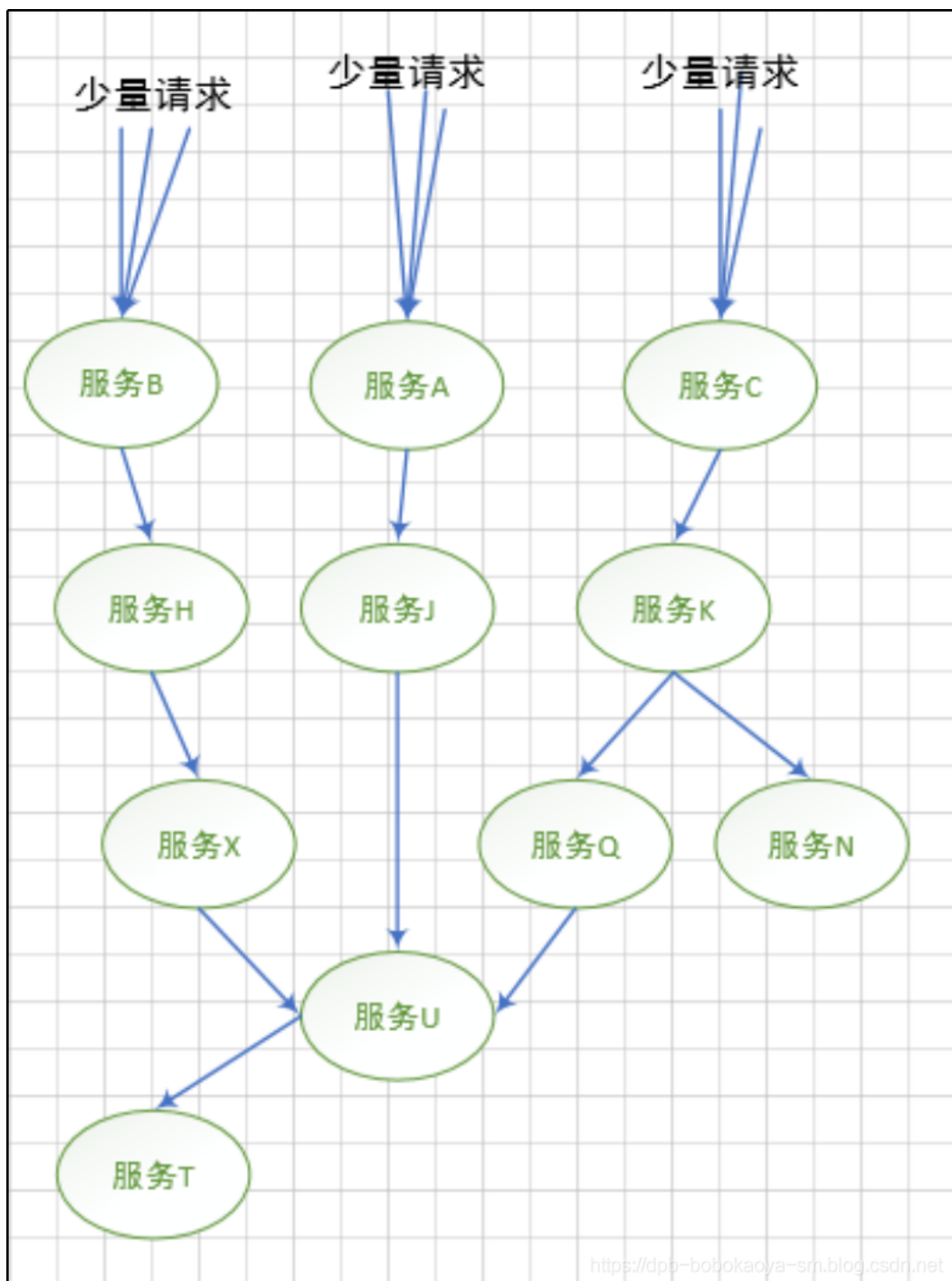
五、Hystrix

在微服务环境中，因为一个节点的故障而造成的其他节点的不可用的情况是比较常见的，这也就是我们常说的灾难性雪崩现象，而Hystrix给我们提供了解决这种情况的方案。

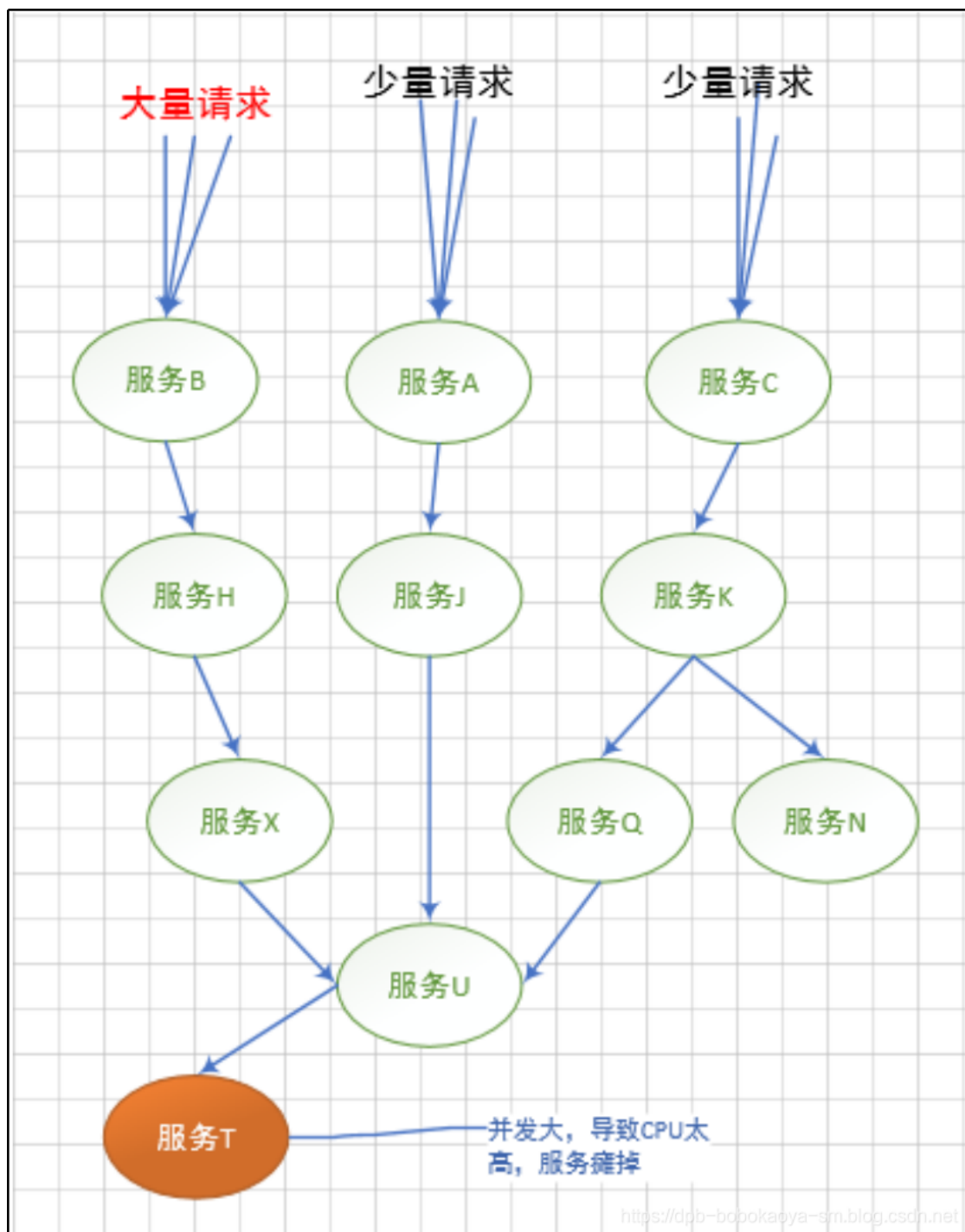
1. Hystrix介绍

1.1 什么是灾难性的雪崩效应

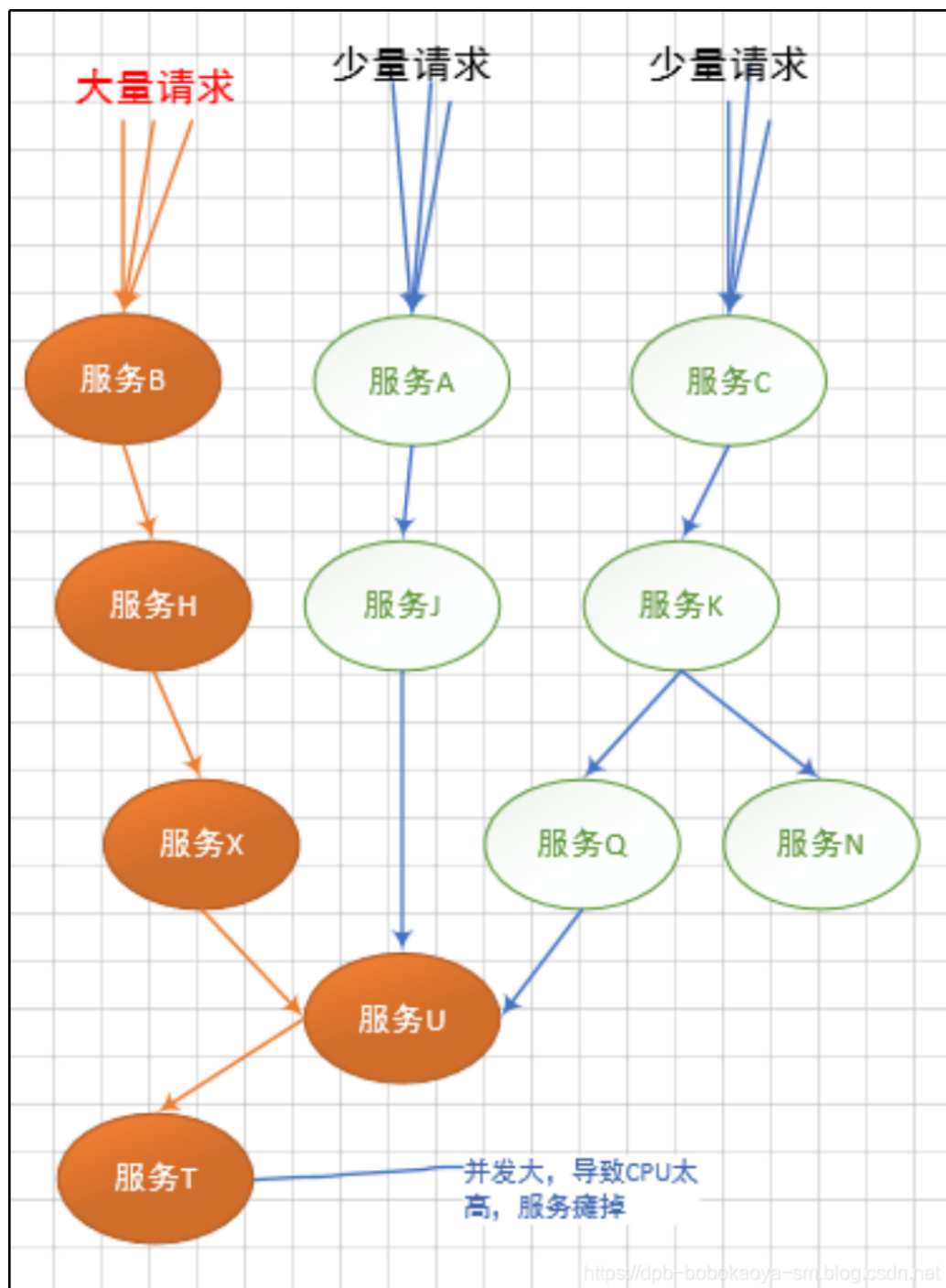
什么是灾难性的雪崩效应?我们通过结构图来说明，如下



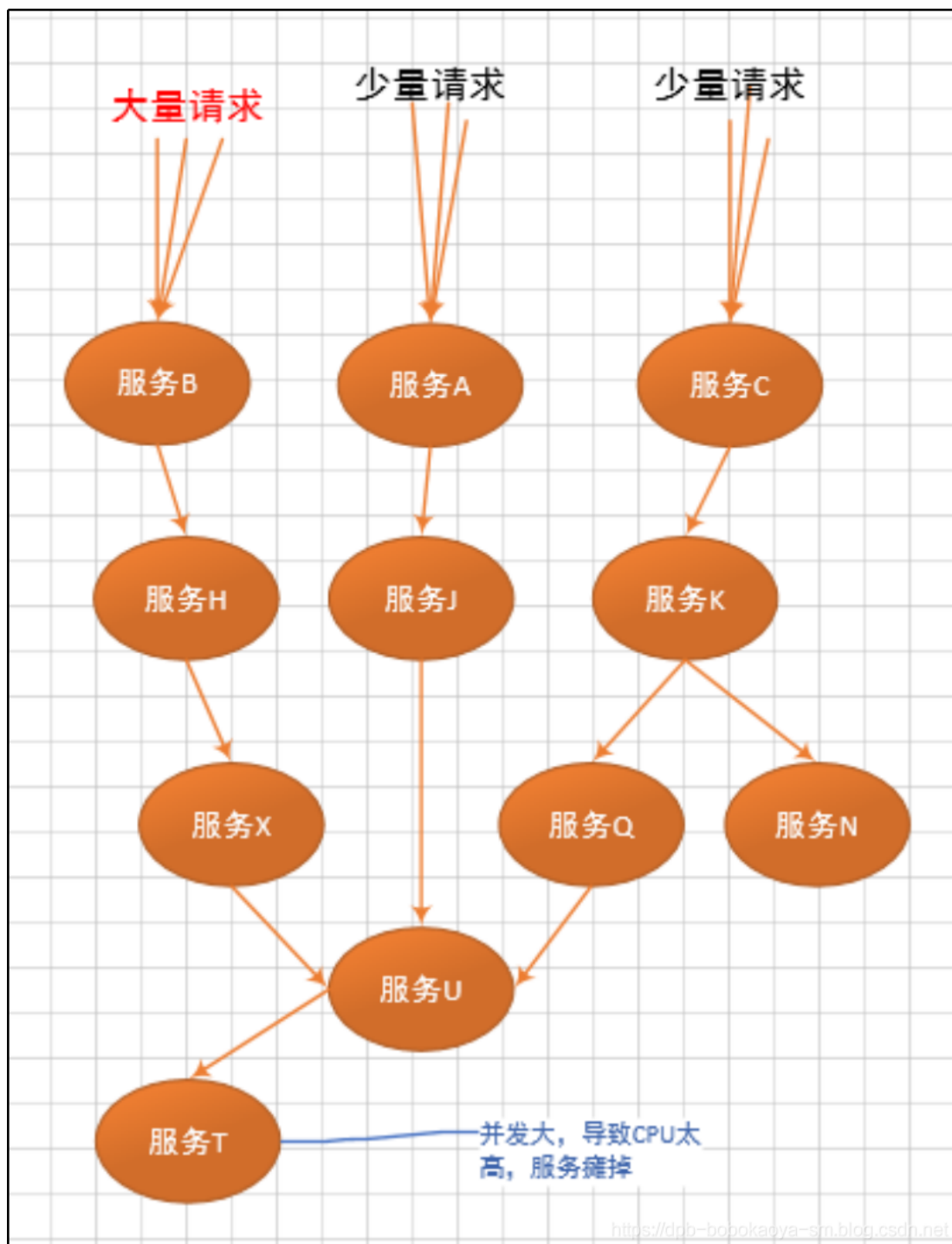
正常情况下各个节点相互配置，完成用户请求的处理工作



当某种请求增多，造成"服务T"故障的情况时，会延伸的造成"服务U"不可用，及继续扩展，如下



最终造成下面这种所有服务不可用的情况



这就是我们讲的灾难性雪崩

造成雪崩的原因可以归纳为以下三个:

1. 服务提供者不可用(硬件故障, 程序Bug, 缓存击穿, 用户大量请求)
2. 重试加大流量(用户重试, 代码逻辑重试)
3. 服务调用者不可用(同步等待造成的资源耗尽)

最终的结果就是一个服务不可用, 导致一系列服务的不可用, 而往往这种后果是无法预料的。

1.2 如何解决灾难性雪崩效应

我们可以通过以下5种方式来解决雪崩效应

1.降级

超时降级、资源不足时(线程或信号量)降级，降级后可以配合降级接口返回托底数据。实现一个 fallback 方法, 当请求后端服务出现异常的时候, 可以使用 fallback 方法返回的值。

2.缓存

Hystrix 为了降低访问服务的频率，支持将一个请求与返回结果做缓存处理。如果再次请求的 URL 没有变化，那么 Hystrix 不会请求服务，而是直接从缓存中将结果返回。这样可以大大降低访问服务的压力。

3.请求合并

在微服务架构中，我们将一个项目拆分成很多个独立的模块，这些独立的模块通过远程调用来互相配合工作，但是，在高并发情况下，通信次数的增加会导致总的通信时间增加，同时，线程池的资源也是有限的，高并发环境会导致有大量的线程处于等待状态，进而导致响应延迟，为了解决这些问题，我们需要来了解 Hystrix 的请求合并。

4.熔断

当失败率(如因网络故障/超时造成的失败率高)达到阈值自动触发降级，熔断器触发的快速失败会进行快速恢复。

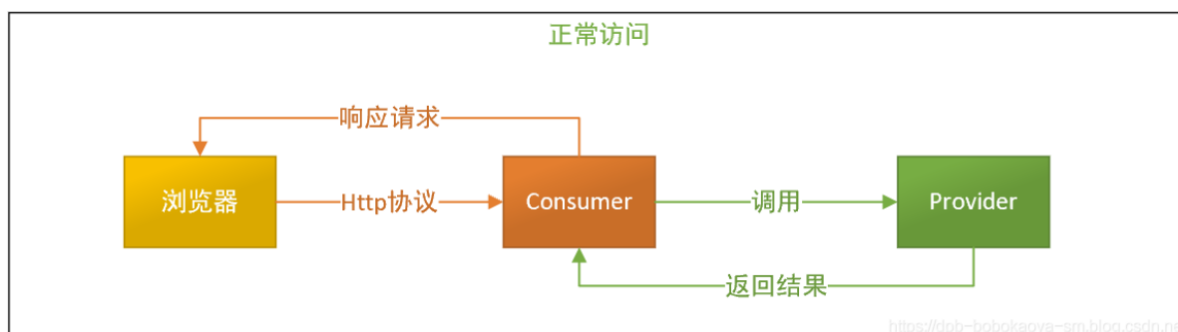
5.隔离（线程池隔离和信号量隔离）

限制调用分布式服务的资源使用，某一个调用的服务出现问题不会影响其他服务调用。

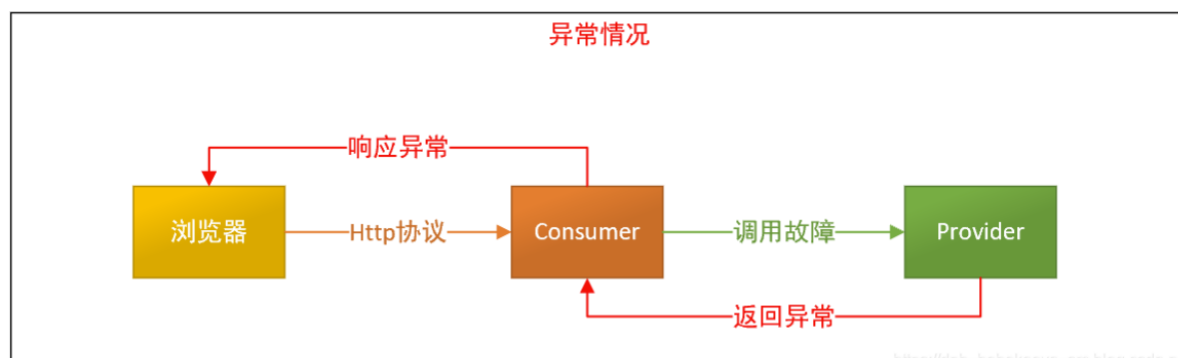
2. 降级

2.1 场景介绍

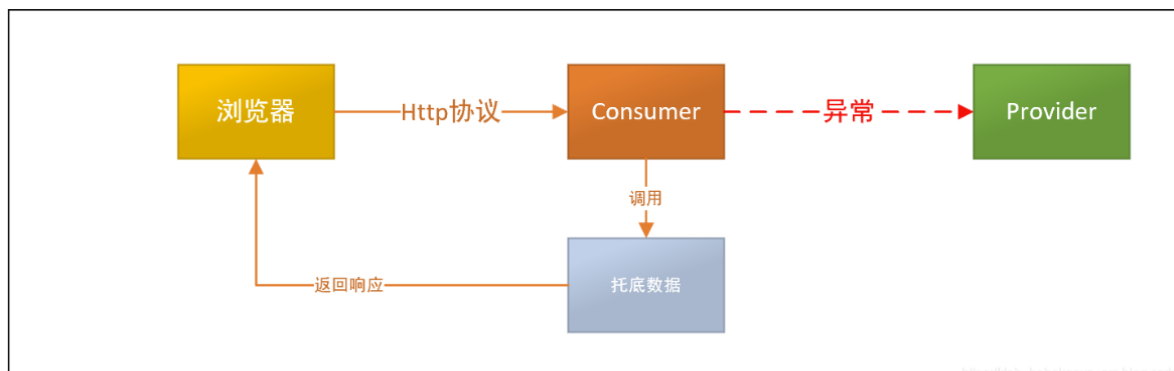
先来看下正常服务调用的情况



当consumer调用provider服务出现问题的情况下:



此时我们对consumer的服务调用做降级处理



2.2 实现案例

创建一个基于Ribbon的Consumer服务，并添加对应的依赖

```
<dependency>
    <groupId>org.apache.httpcomponents</groupId>
    <artifactId>httpClient</artifactId>
</dependency>
<!-- 添加Hystrix的依赖 -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-hystrix</artifactId>
    <version>1.3.2.RELEASE</version>
</dependency>
```

2.3 配置文件

```
spring.application.name=eureka-consumer-hystrix
server.port=9091

# 设置服务注册中心地址 执行Eureka服务端 如果有多个注册地址 那么用逗号连接
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/
```

2.4 修改启动类

我们需要在启动类中添加 开启熔断

```
package com.bobo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.circuitbreaker.EnableCircuitBreaker;
```

```

@EnableCircuitBreaker // 开启Hystrix的熔断
@SpringBootApplication
public class SpringCloudEurekaConsumerApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringCloudEurekaConsumerApplication.class, args);
    }

}

```

2.5 业务层修改

业务层代码中的方法是通过Ribbon来获取负载均衡的服务器地址的，通过RestTemplate来调用服务，在方法的头部添加@HystrixCommand注解，通过fallbackMethod属性指定当调用Provider方法异常的时候fallback方法请求返回托底数据

```

package com.bobo.service;

import com.bobo.pojo.User;
import com.netflix.hystrix.contrib.javanica.annotation.HystrixCommand;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.client.loadbalancer.LoadBalancerClient;
import org.springframework.core.ParameterizedTypeReference;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

import java.util.ArrayList;
import java.util.List;

@Service
public class UserService {

    /**
     * Ribbon 实现的负载均衡
     * LoadBalancerClient 通过服务名称可以获取对应服务的相关信息
     * ip 端口 等
     */
    @Autowired
    private LoadBalancerClient loadBalancerClient;

    /**
     * 远程调用 服务提供者获取用户信息的方法
     * 1.发现服务
     * 2.调用服务
     * @return
     */
    @HystrixCommand(fallbackMethod = "fallBack")
    public List<User> getUsers(){
        // 1. 服务发现
        // 获取服务提供者的信息 ServiceInstance封装的有相关的信息
    }
}

```

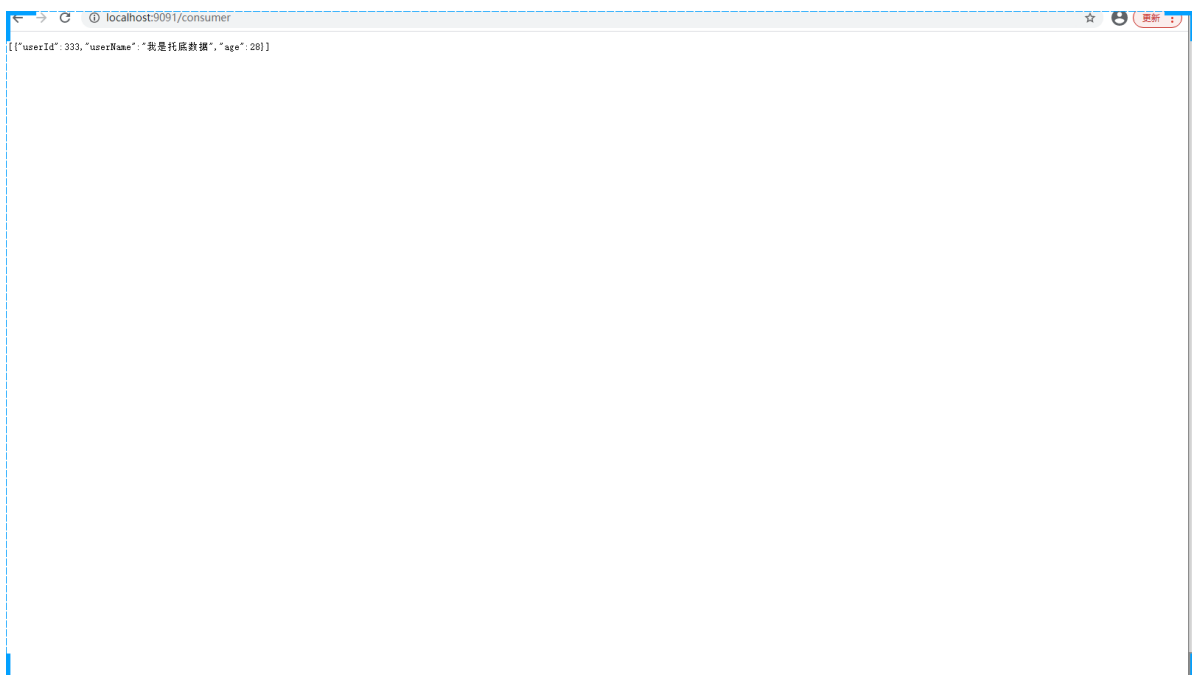
```

        ServiceInstance instance = loadBalancerClient.choose("eureka-provider");
        StringBuilder sb = new StringBuilder();
        // http://localhost:9090/user
        sb.append("http://")
            .append(instance.getHost())
            .append(":")
            .append(instance.getPort())
            .append("/user");
        System.out.println(sb.toString());
        // 2. 服务调用 SpringMVC中提供的有 调用组件 RestTemplate
        RestTemplate rt = new RestTemplate();
        ParameterizedTypeReference<List<User>> type = new
        ParameterizedTypeReference<List<User>>() {};
        ResponseEntity<List<User>> response = rt.exchange(sb.toString(),
        HttpMethod.GET, null, type);
        List<User> list = response.getBody();
        return list;
    }

    /**
     * 托底方法
     * @return
     */
    public List<User> fallBack(){
        List<User> list = new ArrayList<>();
        list.add(new User(333,"我是托底数据",28));
        return list;
    }
}

```

2.6 请求测试



3.缓存

Hystrix 为了降低访问服务的频率，支持将一个请求与返回结果做缓存处理。如果再次请求的 URL 没有变化，那么 Hystrix 不会请求服务，而是直接从缓存中将结果返回。这样可以大大降低访问服务的压力。

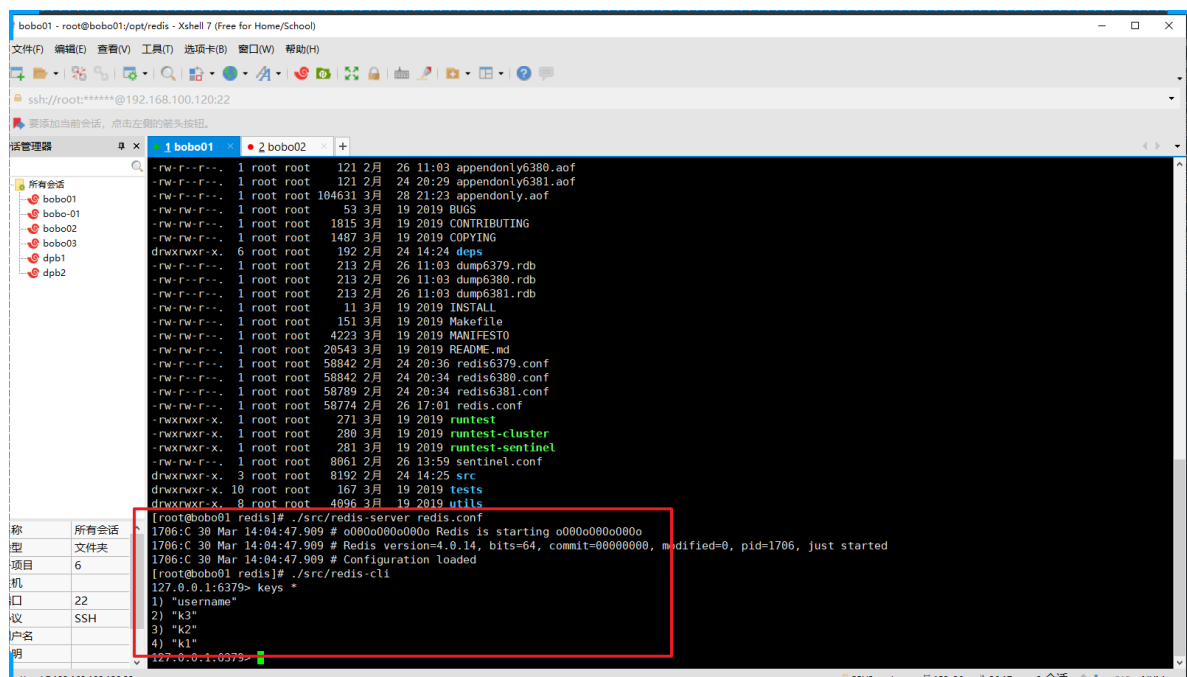
Hystrix 自带缓存。有两个缺点：

1. 是一个本地缓存。在集群情况下缓存是不能同步的。
2. 不支持第三方缓存容器。Redis, memcache 不支持的。

所以我们使用Spring的cache。

3.1 启动Redis服务

使用Redis作为我们的缓存服务器



3.2 添加相关的依赖

因为需要用到SpringDataRedis的支持，需要添加对应的依赖

```
<!-- 添加Hystrix的依赖 -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-hystrix</artifactId>
    <version>1.3.2.RELEASE</version>
</dependency>

<!-- 添加SpringDataRedis的依赖 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

3.3 修改属性文件

我们需要在属性文件中添加Redis的配置信息

```
spring.application.name=eureka-consumer-hystrix
server.port=9091

# 设置服务注册中心地址 执行Eureka服务端 如果有多个注册地址 那么用逗号连接
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

# Redis
spring.redis.database=0
#Redis 服务器地址
spring.redis.host=192.168.100.120
#Redis 服务器连接端口
spring.redis.port=6379
#Redis 服务器连接密码（默认为空）
spring.redis.password=
#连接池最大连接数（负值表示没有限制）
spring.redis.pool.max-active=100
#连接池最大阻塞等待时间（负值表示没有限制）
spring.redis.pool.max-wait=3000
#连接池最大空闲连接数
spring.redis.pool.max-idle=200
#连接池最小空闲连接数
spring.redis.pool.min-idle=50
#连接超时时间（毫秒）
spring.redis.pool.timeout=600
```

3.4 修改启动类

我们需要在启动类中开启缓存的使用

```
@EnableCaching // 开启缓存
@EnableCircuitBreaker // 开启Hystrix的熔断
@SpringBootApplication
public class SpringcloudEurekaConsumerApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringcloudEurekaConsumerApplication.class, args);
    }

}
```

3.5 业务处理

```
package com.bobo.service;

import com.bobo.pojo.User;
import com.netflix.hystrix.contrib.javanica.annotation.HystrixCommand;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.cache.annotation.CacheConfig;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.client.loadbalancer.LoadBalancerClient;
import org.springframework.core.ParameterizedTypeReference;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

import java.util.ArrayList;
import java.util.List;

@Service
// cacheNames 当前类中的方法在Redis中添加的Key的前缀
@CacheConfig(cacheNames = {"com.bobo.cache"})
public class UserService {

    /**
     * Ribbon 实现的负载均衡
     * LoadBalancerClient 通过服务名称可以获取对应服务的相关信息
     * ip 端口 等
     */
    @Autowired
    private LoadBalancerClient loadBalancerClient;

    /**
     * 远程调用 服务提供者获取用户信息的方法
     * 1.发现服务
     * 2.调用服务
     * @return
     */
    @HystrixCommand(fallbackMethod = "fallBack")
    public List<User> getUsers(){
        // 1. 服务发现
        // 获取服务提供者的信息 ServiceInstance封装的有相关的信息
        ServiceInstance instance = loadBalancerClient.choose("eureka-provider");
        StringBuilder sb = new StringBuilder();
        // http://localhost:9090/user
        sb.append("http://")
            .append(instance.getHost())
            .append(":")
            .append(instance.getPort())
            .append("/user");
        System.out.println(sb.toString());
        // 2. 服务调用 SpringMVC中提供的有 调用组件 RestTemplate
        RestTemplate rt = new RestTemplate();
        ParameterizedTypeReference<List<User>> type = new
        ParameterizedTypeReference<List<User>>() {};
```

```

        ResponseEntity<List<User>> response = rt.exchange(sb.toString(),
        HttpMethod.GET, null, type);
        List<User> list = response.getBody();
        return list;
    }

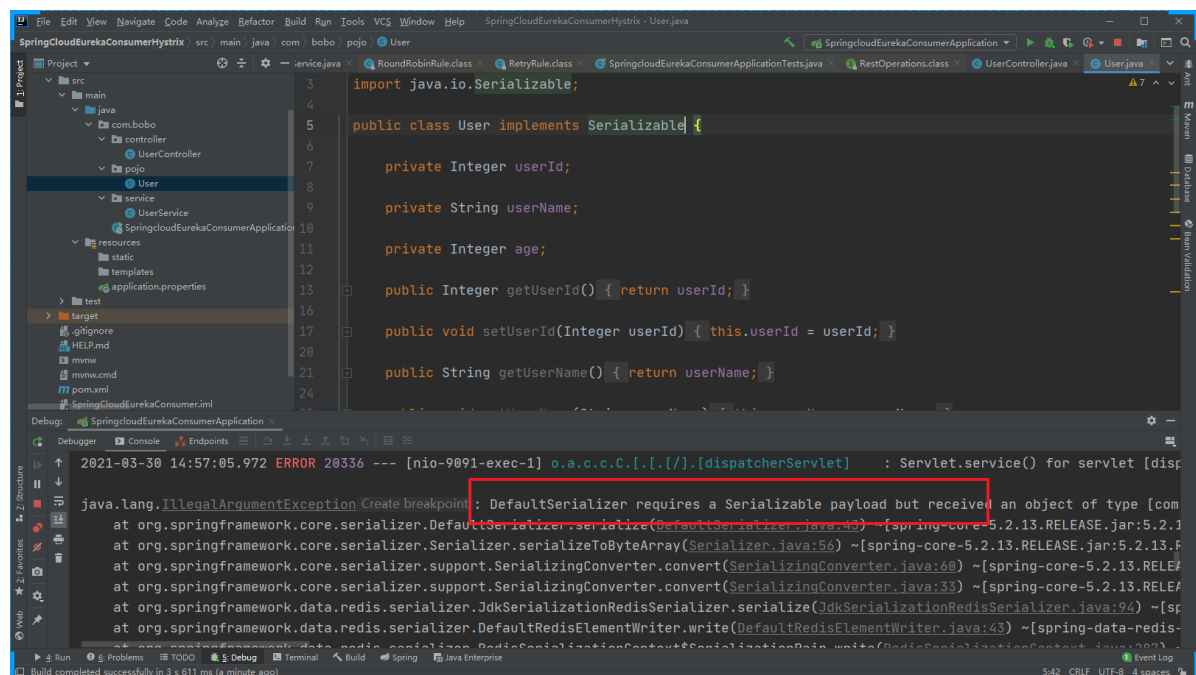
    /**
     * 托底方法
     * @return
     */
    public List<User> fallback(){
        List<User> list = new ArrayList<>();
        list.add(new User(333,"我是托底数据",28));
        return list;
    }

    @Cacheable(key="'user'+#id")
    public User getUserById(Integer id){
        system.out.println("*****查询操作*****"+ id);
        return new User(id,"缓存测试数据",22);
    }
}

```

3.6 启动测试

因为我们使用到了缓存，所以会对POJO对象做持久化处理，所以需要实现序列化接口，否则会抛如下的异常



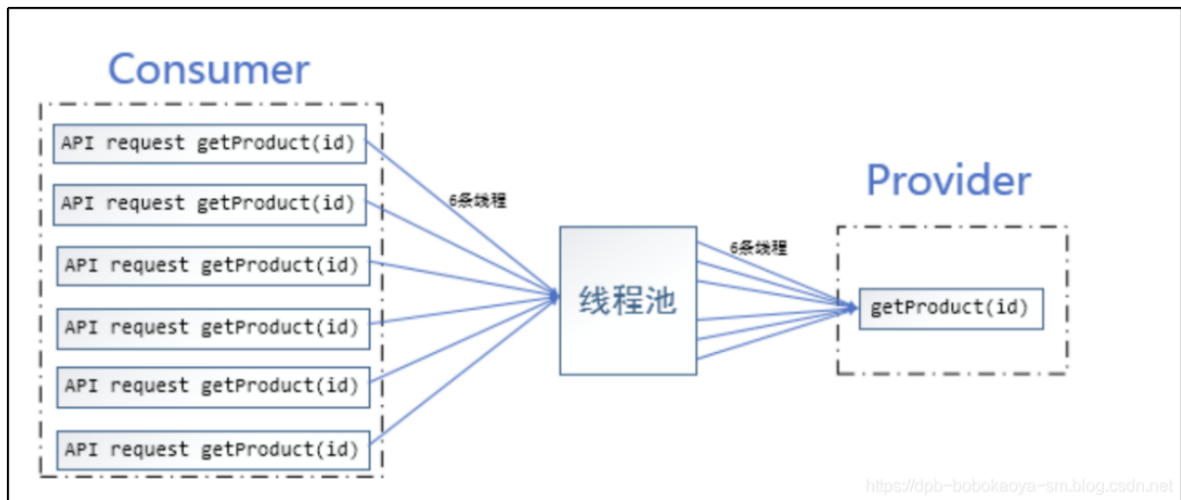
Redis中有对应的数据

```
bobo01 - root@bobo01:/opt/redis - Xshell 7 (Free for Home/School)
文件(F) 编辑(E) 查看(V) 工具(T) 选项卡(B) 窗口(W) 帮助(H)
ssh://root:*****@192.168.100.120:22
要添加当前会话，点击左侧的箭头按钮。
会话管理器
1 bobo01
2 bobo02
所有会话
bobo01
bobo01
bobo02
bobo03
djb1
djb2
名称 所有会话
类型 文件夹
项目 6
主机
端口 22
协议 SSH
用户名
密码
1706:C 30 Mar 14:04:47.989 # 000000000000 Redis is starting o000o000o000
1706:C 30 Mar 14:04:47.989 # Redis version=4.0.14, bits=64, commit=00000000, modified=0, pid=1706, just started
1706:C 30 Mar 14:04:47.989 # Configuration loaded
[root@bobo01 redis]# ./src/redis-server redis.conf
127.0.0.1:6379> keys *
1) "username"
2) "k3"
3) "k2"
4) "k1"
127.0.0.1:6379> keys *
1) "username"
2) "k3"
3) "k2"
4) "com.bobo.cache:user666"
5) "k1"
127.0.0.1:6379>
```

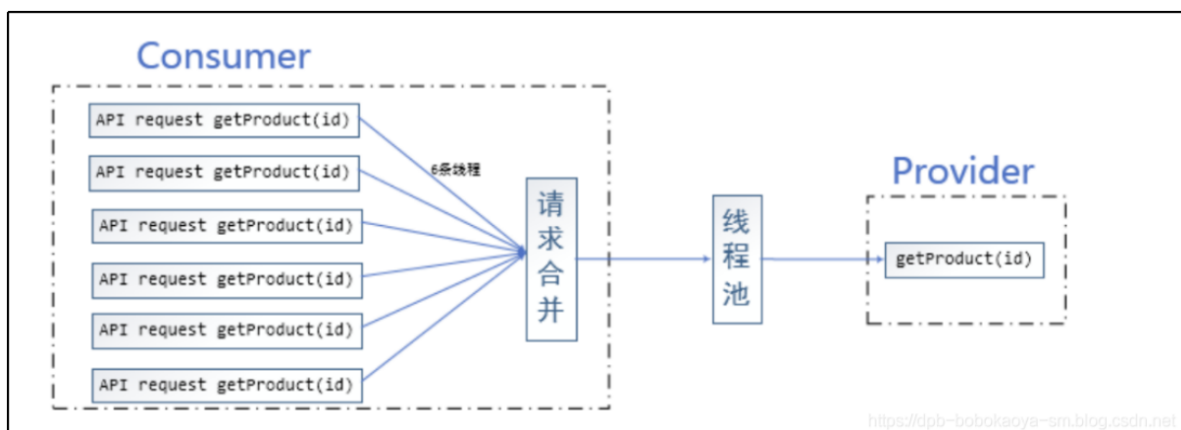
4. 请求合并

4.1 没有合并请求的场景

没有合并的场景中，对于provider的调用会非常的频繁，容易造成处理不过来的情况



4.2 合并请求的场景



4.3 什么情况下使用请求合并

在微服务架构中，我们将一个项目拆分成很多个独立的模块，这些独立的模块通过远程调用来互相配合工作，但是，在高并发情况下，通信次数的增加会导致总的通信时间增加，同时，线程池的资源也是有限的，高并发环境会导致有大量的线程处于等待状态，进而导致响应延迟，为了解决这些问题，我们需要了解 Hystrix 的请求合并。

4.4 请求合并的缺点

设置请求合并之后，本来一个请求可能 5ms 就搞定了，但是现在必须再等 10ms 看看还有没有其他请求一起的，这样一个请求的耗时就从 5ms 增加到 15ms 了，不过，如果我们要发起的命令本身就是一个高延迟的命令，那么这个时候就可以使用请求合并了，因为这个时候时间窗的时间消耗就显得微不足道了，另外高并发也是请求合并的一个非常重要的场景。

4.5 案例实现

业务处理代码

```
package com.bobo.service;

import com.bobo.pojo.User;
import com.netflix.hystrix.contrib.javanica.annotation.HystrixCollapser;
import com.netflix.hystrix.contrib.javanica.annotation.HystrixCommand;
import com.netflix.hystrix.contrib.javanica.annotation.HystrixProperty;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.cache.annotation.CacheConfig;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.client.loadbalancer.LoadBalancerClient;
import org.springframework.core.ParameterizedTypeReference;
import org.springframework.http.HttpMethod;
import org.springframework.http.ResponseEntity;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Future;

@Service
// cacheNames 当前类中的方法在Redis中添加的Key的前缀
@CacheConfig(cacheNames = {"com.bobo.cache"})
public class UserService {

    /**
     * Ribbon 实现的负载均衡
     * LoadBalancerClient 通过服务名称可以获取对应服务的相关信息
     * ip 端口 等
     */
    @Autowired
```

```

private LoadBalancerClient loadBalancerClient;

/**
 * 远程调用 服务提供者获取用户信息的方法
 * 1.发现服务
 * 2.调用服务
 * @return
 */
@HystrixCommand(fallbackMethod = "fallBack")
public List<User> getUsers(){
    // 1. 服务发现
    // 获取服务提供者的信息 ServiceInstance封装的有相关的信息
    ServiceInstance instance = loadBalancerClient.choose("eureka-provider");
    StringBuilder sb = new StringBuilder();
    // http://localhost:9090/user
    sb.append("http://")
        .append(instance.getHost())
        .append(":")
        .append(instance.getPort())
        .append("/user");
    System.out.println(sb.toString());
    // 2. 服务调用 SpringMVC中提供的有 调用组件 RestTemplate
    RestTemplate rt = new RestTemplate();
    ParameterizedTypeReference<List<User>> type = new
    ParameterizedTypeReference<List<User>>() {};
    ResponseEntity<List<User>> response = rt.exchange(sb.toString(),
    HttpMethod.GET, null, type);
    List<User> list = response.getBody();
    return list;
}

/**
 * 托底方法
 * @return
 */
public List<User> fallBack(){
    List<User> list = new ArrayList<>();
    list.add(new User(333,"我是托底数据",28));
    return list;
}

@Cacheable(key="'user'+#id")
public User getUserById(Integer id){
    System.out.println("*****查询操作*****"+ id);
    return new User(id,"缓存测试数据",22);
}

/**
 * Consumer中的Controller要调用的方法
 * 这个方法的返回值必须是 Future 类型
 * 利用Hystrix 合并请求
 * @param id
 * @return
 */
@HystrixCollapser(
    batchMethod = "batchUser"
    ,scope = com.netflix.hystrix.HystrixCollapser.Scope.GLOBAL

```

```

        ,collapserProperties = {
            // 请求时间间隔在20ms以内的请求会被合并，默认值是10ms
            @HystrixProperty(name = "timerDelayInMilliseconds",value =
"20")

            // 设置触发批处理执行之前 在批处理中允许的最大请求数
            ,@HystrixProperty(name = "maxRequestsInBatch",value = "200")
        }
    }

    public Future<User> getUserId(Integer id){
        System.out.println("*****id*****");
        return null;
    }

    @HystrixCommand
    public List<User> batchUser(List<Integer> ids){
        for (Integer id : ids) {
            System.out.println(id);
        }
        List<User> list = new ArrayList<>();
        list.add(new User(1,"张三1",18));
        list.add(new User(2,"张三2",18));
        list.add(new User(3,"张三3",18));
        list.add(new User(4,"张三4",18));
        return list;
    }
}

```

控制器处理

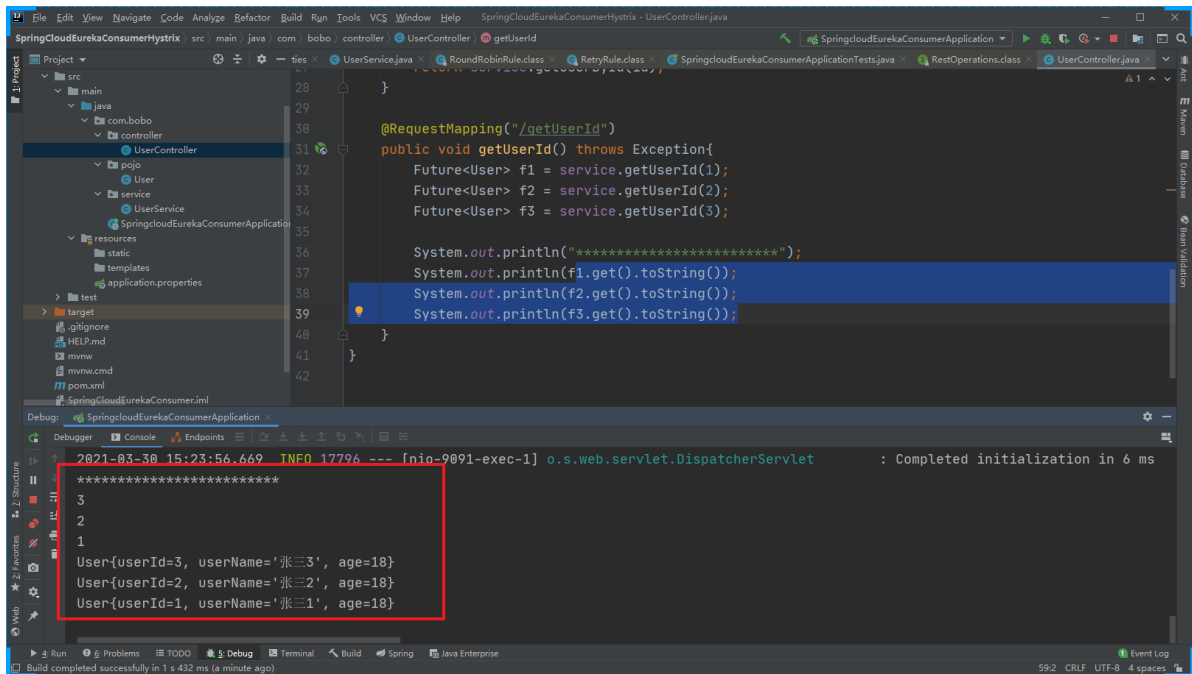
```

@RequestMapping("/getUserId")
public void getUserId() throws Exception{
    Future<User> f1 = service.getUserId(1);
    Future<User> f2 = service.getUserId(1);
    Future<User> f3 = service.getUserId(1);

    System.out.println("*****");
    System.out.println(f1.get().toString());
    System.out.println(f2.get().toString());
    System.out.println(f3.get().toString());
}

```

访问测试

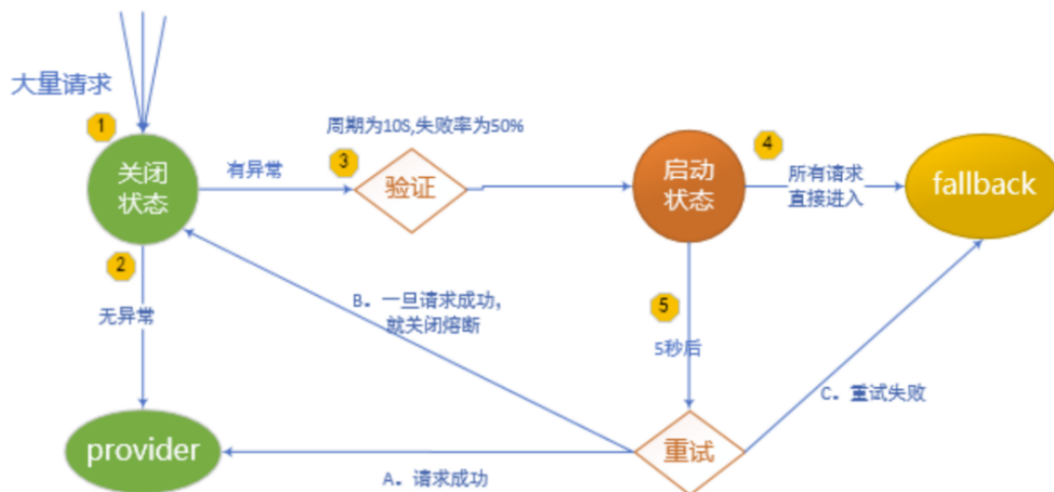


参数	作用/默认值	备注
@HystrixCollapser		
batchMethod	合并请求的方法	
scope	请求方式/REQUEST	请求方式分为REQUEST和GLOBAL。REQUEST范围只对一个request请求内的多次服务请求进行合并。GLOBAL是多单应用中的所有线程的请求中的多次服务请求合并。
timerDelayInMilliseconds	请求时间间隔在10ms之内的请求会被合并为一个请求/10ms	建议尽量设置的小一点，如果并发量不大的话，其实也没有必要使用HystrixCollapser来处理。
maxRequestsInBatch	设置触发批处理执行之前,在批处理中允许的最大请求数/Integer.MAX_VALUE	

5.熔断

熔断其实是在降级的基础上引入了重试的机制。当某个时间内失败的次数达到了多少次就会触发熔断机制，具体的流程如下

熔断机制相当于电路的跳闸功能。
例如：我们可以配置熔断策略为当请求错误比例在10s内>50%时，该服务将进入熔断状态，后续请求都会进入fallback。



<https://dph-bobokeoya-sm.blog.csdn.net>

案例核心代码

```

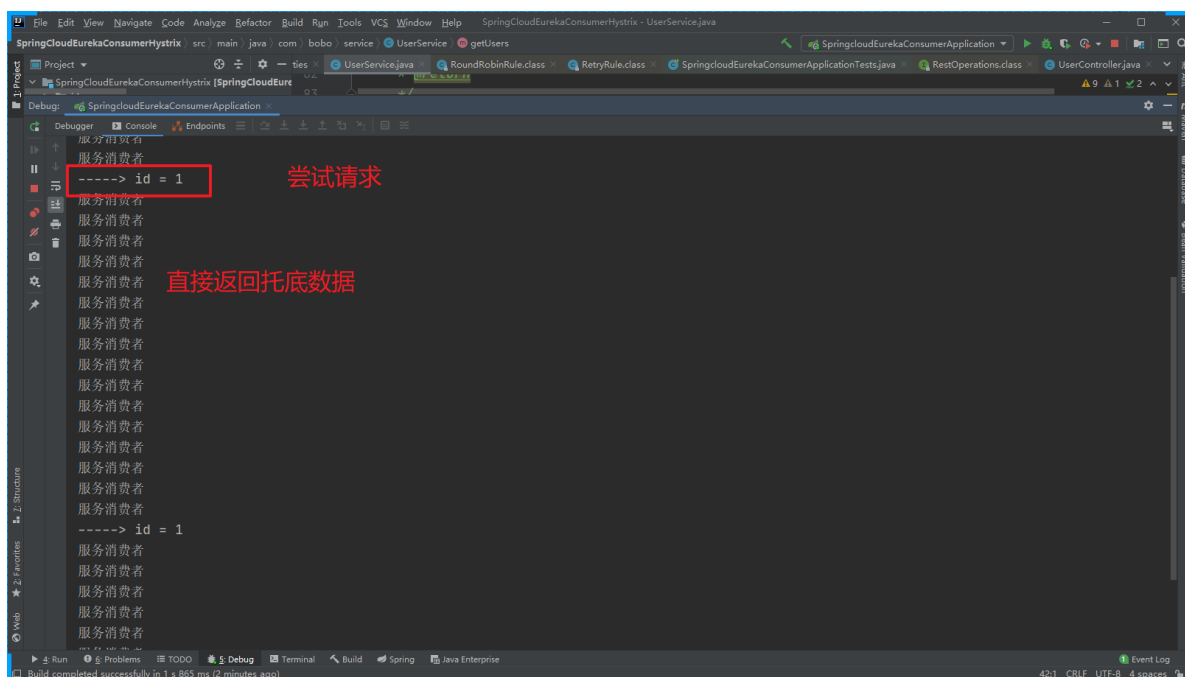
@HystrixCommand(fallbackMethod = "fallback",
    commandProperties = {
        //默认 20 个;10s 内请求数大于 20 个时就启动熔断器，当请求符合熔断条件
        //时将触发 getFallback()。
        @HystrixProperty(name=
HystrixPropertiesManager.CIRCUIT_BREAKER_REQUEST_VOLUME_THRESHOLD,
            value="10"),
        //请求错误率大于 50%时就熔断，然后 for 循环发起请求，当请求符合熔断条件
        //时将触发 getFallback()。
        @HystrixProperty(name=HystrixPropertiesManager.CIRCUIT_BREAKER_ERROR_THRESHOLD_
PERCENTAGE,
            value="50"),
        //默认 5 秒;熔断多少秒后去尝试请求
        @HystrixProperty(name=HystrixPropertiesManager.CIRCUIT_BREAKER_SLEEP_WINDOW_IN_
MILLISECONDS,
            value="5000"),
    })
public List<User> getUsers(){
    // 1. 服务发现
    // 获取服务提供者的信息 ServiceInstance封装的有相关的信息
    ServiceInstance instance = loadBalancerClient.choose("eureka-provider");
    StringBuilder sb = new StringBuilder();
    // http://localhost:9090/user
    sb.append("http://")
        .append(instance.getHost())
        .append(":")
        .append(instance.getPort())
        .append("/user");
    System.out.println("---->" + sb.toString());
    // 2. 服务调用 SpringMVC中提供的有 调用组件 RestTemplate
    RestTemplate rt = new RestTemplate();
  
```

```

        ParameterizedTypeReference<List<User>> type = new
        ParameterizedTypeReference<List<User>>() {};
        ResponseEntity<List<User>> response = rt.exchange(sb.toString(),
        HttpMethod.GET, null, type);
        List<User> list = response.getBody();
        return list;
    }

```

测试



熔断参数			
参数	作用	默认值	备注
circuitBreaker.enabled	是否开启熔断	TRUE	
circuitBreaker.requestVolumeThreshold	一个统计窗口内熔断触发的最小个数/10s	20	10s内请求数大于20个时就启动熔断器，当请求符合熔断条件时将触发getFallback()。
circuitBreaker.sleepWindowInMilliseconds	熔断多少秒后去尝试请求	5000ms	
circuitBreaker.errorThresholdPercentage	失败率达到多少百分比后熔断	默认配置下采样周期为10s，失败率为50%	请求错误率大于50%时就熔断，然后for循环发起请求，当请求符合熔断条件时将触发getFallback()。
circuitBreaker.forceOpen	是否强制开启熔断	FALSE	置为true时，所有请求都将被拒绝，直接到fallback
circuitBreaker.forceClosed	是否强制关闭熔断	FALSE	置为true时，将忽略错误

6. 隔离

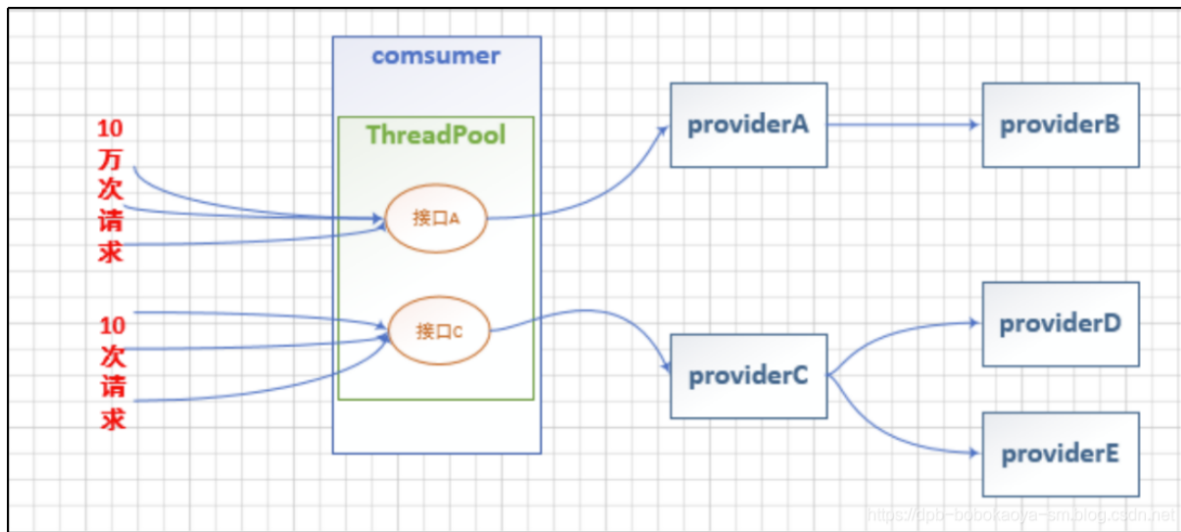
在应对服务雪崩效应时，除了前面介绍的降级，缓存，请求合并及熔断外还有一种方式就是隔离，隔离又分为线程池隔离和信号量隔离。接下来我们分别来介绍。

6.1 线程池隔离

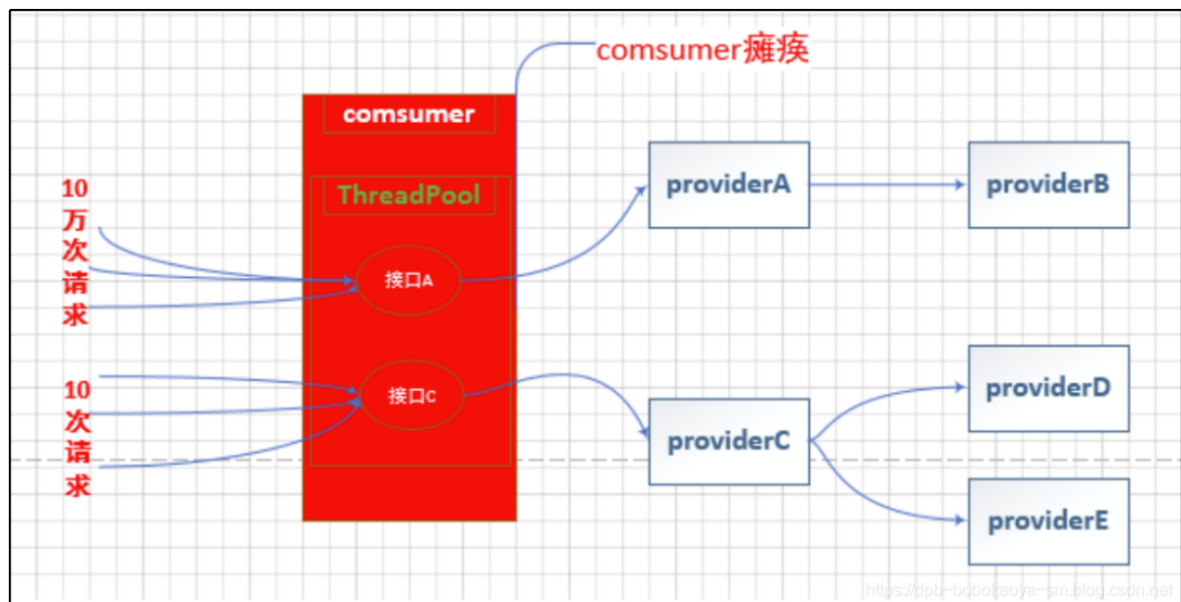
6.1.1 概念介绍

我们通过以下几个图片来解释线程池隔离到底是怎么回事

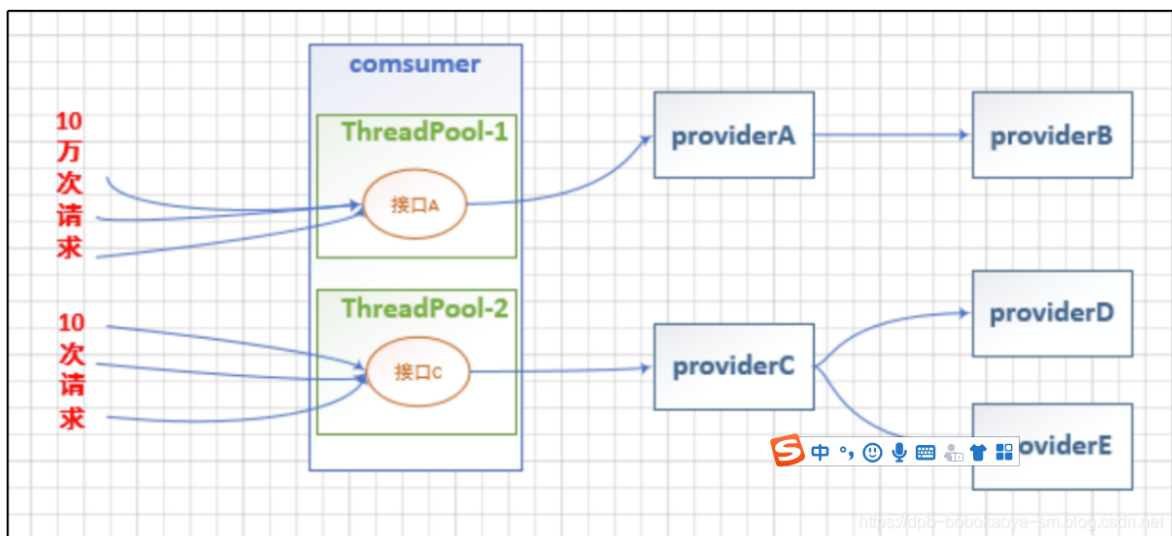
在没有使用线程池隔离时



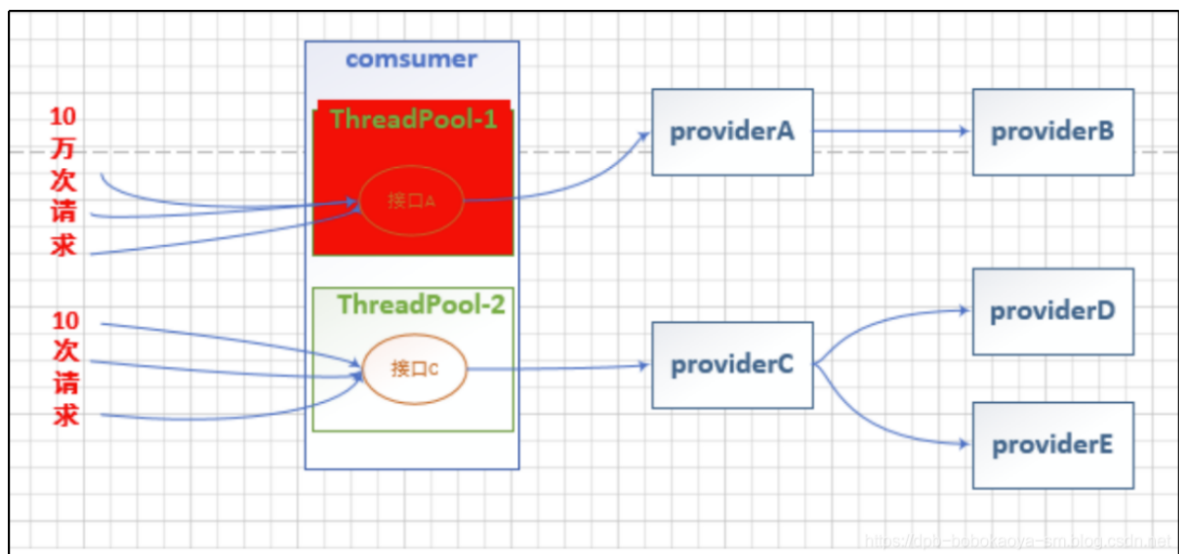
当接口A压力增大，接口C同时也会受到影响



使用线程池的场景



当服务接口A访问量增大时，因为接口C在不同的线程池中所以不会受到影响



通过上面的图片来看，线程池隔离的作用还是蛮明显的。但线程池隔离的使用也不是在任何场景下都适用的，线程池隔离的优缺点如下：

优点

1. 使用线程池隔离可以完全隔离依赖的服务(例如图中的A，B，C服务)，请求线程可以快速放回
2. 当线程池出现问题时，线程池隔离是独立的不会影响其他服务和接口
3. 当失败的服务再次变得可用时，线程池将清理并可立即恢复，而不需要一个长时间的恢复
4. 独立的线程池提高了并发性

缺点

线程池隔离的主要缺点是它们增加计算开销(CPU).每个命令的执行涉及到排队，调度和上下文切换都是在一个单独的线程上运行的。

6.1.2 案例实现

```
@HystrixCommand(
    groupKey = "eureka-provider"
    ,threadPoolKey = "getUsers"
```

```

        ,threadPoolProperties = {
            @HystrixProperty(name = "coreSize",value = "30") // 线程池大小
            ,@HystrixProperty(name = "maxQueueSize",value = "100") // 最
            大队列长度
            ,@HystrixProperty(name = "keepAliveTimeMinutes",value = "2")
            // 线程存活时间
            ,@HystrixProperty(name = "queueSizeRejectionThreshold",value
            = "15") // 拒绝请求
        },fallbackMethod = "fallBack"
    )
    public List<User> getUsersThreadPool(Integer id){
        System.out.println("-----》 " + Thread.currentThread().getName());
        // 1. 服务发现
        // 获取服务提供者的信息 ServiceInstance封装的有相关的信息
        ServiceInstance instance = loadBalancerClient.choose("eureka-provider");
        StringBuilder sb = new StringBuilder();
        // http://localhost:9090/user
        sb.append("http://")
            .append(instance.getHost())
            .append(":")
            .append(instance.getPort())
            .append("/user");
        System.out.println("---->" + sb.toString());
        // 2. 服务调用 SpringMVC中提供的有 调用组件 RestTemplate
        RestTemplate rt = new RestTemplate();
        ParameterizedTypeReference<List<User>> type = new
        ParameterizedTypeReference<List<User>>() {};
        ResponseEntity<List<User>> response = rt.exchange(sb.toString(),
        HttpMethod.GET, null, type);
        List<User> list = response.getBody();
        return list;
    }
}

```

测试

两个不同的请求，对应在不同的线程池中

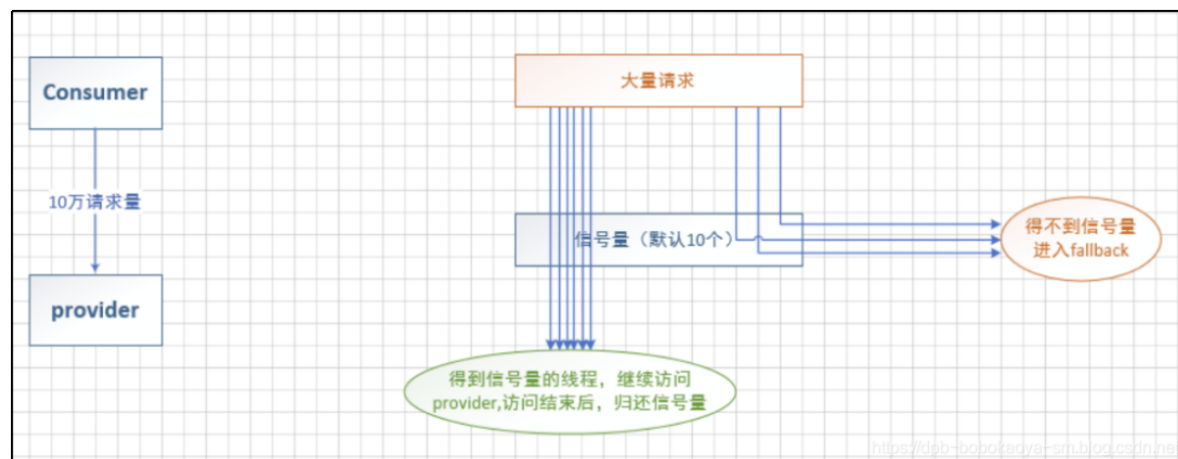
```

Run: SpringCloudEurekaConsumerHystrix - UserService.java
Project: SpringCloudEurekaConsumerHystrix [SpringCloudEurekaConsumerHystrix]
src/main/java/com/bobo/service/UserService.java
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
```

线程池隔离参数			
参数	作用	默认值	备注
groupKey	服务名（相同服务用一个名称，如商品、用户等等）	getClass().getSimpleName();	在consumer里面为每个provider服务，设置group标识，一个group使用一个线程池
commandKey	接口（服务下面的接口，如购买商品）	当前执行方法名	consumer的接口名称
threadPoolKey	线程池的名称：配置全局唯一标识线程池的名称，相同线程池名称的线程池是同一个。	默认是分组名groupKey	配置全局唯一标识线程池的名称，相同线程池名称的线程池是同一个。
coreSize	线程池大小：这是最大的并发执行数量。	10	设置标准：requests per second at peak when healthy × 99th percentile latency in seconds + some breathing room 每秒最大支撑的请求数 (99%平均响应时间 + 一个缓冲值) 比如：每秒能处理1000个请求，99%的请求响应时间是60ms，那么公式是：1000*(0.060+0.012)
maxQueueSize	最大队列长度：设置BlockingQueue的最大长度	-1	默认值：-1 如果使用正数，队列将从同步队列（SynchronousQueue）改为阻塞队列（LinkedBlockingQueue）
queueSizeRejectionThreshold	拒绝请求：设置拒绝请求的临界值	5	此属性不适用于maxQueueSize = -1时 设置该值的原因是maxQueueSize值运行时不能改变，我们可以通过修改这个变量动态修改允许排队的长度
keepAliveTimeMinutes	线程存活时间：设置存活时间，单位分钟。	1分钟	控制一个线程从实用完成到被释放的时间。 blog.csdn.net

6.2 信号量隔离

信号量隔离其实就是我们定义的队列并发时最多支持多大的访问，其他的访问通过托底数据来响应，如下结构图



```

@HystrixCommand(
    fallbackMethod = "fallBack"
    ,commandProperties = {

        @HystrixProperty(name=HystrixPropertiesManager.EXECUTION_ISOLATION_STRATEGY
            ,value = "SEMAPHORE") // 信号量隔离

        ,@HystrixProperty(name=HystrixPropertiesManager.EXECUTION_ISOLATION_SEMAPHORE_MAX_CONCURRENT_REQUESTS
            ,value="100" // 信号量最大并发度
        )
    }
)

```

```

    }
    public List<User> getUsersSignal(Integer id){
        System.out.println("-----》" + Thread.currentThread().getName());
        // 1. 服务发现
        // 获取服务提供者的信息 ServiceInstance封装的有相关的信息
        ServiceInstance instance = loadBalancerClient.choose("eureka-provider");
        StringBuilder sb = new StringBuilder();
        // http://localhost:9090/user
        sb.append("http://")
            .append(instance.getHost())
            .append(":")
            .append(instance.getPort())
            .append("/user");
        System.out.println("---->" + sb.toString());
        // 2. 服务调用 SpringMVC中提供的有 调用组件 RestTemplate
        RestTemplate rt = new RestTemplate();
        ParameterizedTypeReference<List<User>> type = new
        ParameterizedTypeReference<List<User>>() {};
        ResponseEntity<List<User>> response = rt.exchange(sb.toString(),
        HttpMethod.GET, null, type);
        List<User> list = response.getBody();
        return list;
    }
}

```

信号量隔离参数			
参数	作用	默认值	备注
execution.isolation.strategy	隔离策略配置项	THREAD	只有2种THREAD和SEMAPHORE
execution.isolation.thread.timeoutInMilliseconds	超时时间	1000ms	1.在THREAD模式下，达到超时时间，自动中断 2.在SEMAPHORE模式下，会等待执行完成后，再去判断是否超时
execution.isolation.thread.interruptOnTimeout	是否打开超时线程中断	TRUE	THREAD模式有效
execution.isolation.semaphore.maxConcurrentRequests	信号量最大并发度	10	SEMAPHORE模式有效
fallback.isolation.semaphore.maxConcurrentRequests	fallback最大并发度	10	SEMAPHORE模式有效

6.3 两者的区别

线程池隔离和信号量隔离的区别

线程池和信号量的区别？		
	线程池隔离	信号量隔离
线程	请求线程和调用provider线程不是同一条线程	请求线程和调用provider线程是同一条线程
开销	排队、调度、上下文开销等	无线程切换，开销低
异步	支持	不支持
并发支持	支持（最大线程池大小）	支持（最大信号量上限）
传递Header	无法传递http Header	可以传递http Header
支持超时	能支持超时	不支持超时
1.什么情况下，用线程池隔离？		
请求并发量大，并且耗时长（请求耗时长一般是计算量大，或读数据库）：采用线程隔离策略，这样的话，可以保证大量的容器(tomcat)线程可用，不会由于服务原因，一直处于阻塞或等待状态，快速失败返回。		
2.什么情况下，用信号量隔离？		
请求并发量大，并且耗时短（请求耗时短可能是计算量小，或读缓存）：采用信号量隔离策略，因为这类服务的返回通常会非常的快，不会占用容器线程太长时间，而且也减少了线程切换的一些开销，提高了缓存服务的效率。		

7. Feign的使用

我们前面介绍的案例都是基于Ribbon来时的，那么我们看下在Feign中我们是如何来实现服务的降级的

我们需要放开Feign对Hystrix的支持

```
# Feign默认是不开启 Hystrix的，默认false
feign.hystrix.enabled=true
```

添加对应的依赖

```
<!-- 添加Hystrix的依赖 -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
```

开启Hystrix的支持

```
package com.bobo;

import feign.Logger;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.circuitbreaker.EnableCircuitBreaker;
import org.springframework.cloud.openfeign.EnableFeignClients;
```

```

import org.springframework.context.annotation.Bean;

/**
 * 我们需要放开Feign的客户端注解
 */
@EnableCircuitBreaker
@EnableFeignClients
@SpringBootApplication
public class SpringcloudFeignProductConsumerApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringcloudFeignProductConsumerApplication.class,
args);
    }

    /**
     * 定义日志的输出级别
     * NONE，不记录任何信息 默认值
     * BASIC，记录基本信息(请求方法，请求URL，状态码和用时)
     * HEADERS，在BASIC的基础上再记录一些常用信息
     * FULL；记录请求和响应的所有的数据
     * @return
     */
    @Bean
    public Logger.Level getLogger(){
        return Logger.Level.FULL;
    }

}

```

业务处理

```

package com.bobo.hystrix;

import com.bobo.pojo.Product;
import com.bobo.service.ProductConsumerService;
import org.springframework.stereotype.Component;

import java.awt.*;
import java.util.ArrayList;
import java.util.List;

@Component
public class ProductServiceFallback implements ProductConsumerService {

    /**
     * 这个其实就是返回托底数据的方法
     * @return
     */
    @Override
    public List<Product> findAll() {
        List<Product> list = new ArrayList<>();
        list.add(new Product(-1,"我是托底数据"));
    }
}

```

```

        return list;
    }

    @Override
    public Product getProductById(Integer id) {
        return null;
    }

    @Override
    public Product addProductGet(Integer id, String name) {
        return null;
    }

    @Override
    public Product addProductPost(Product product) {
        return null;
    }
}

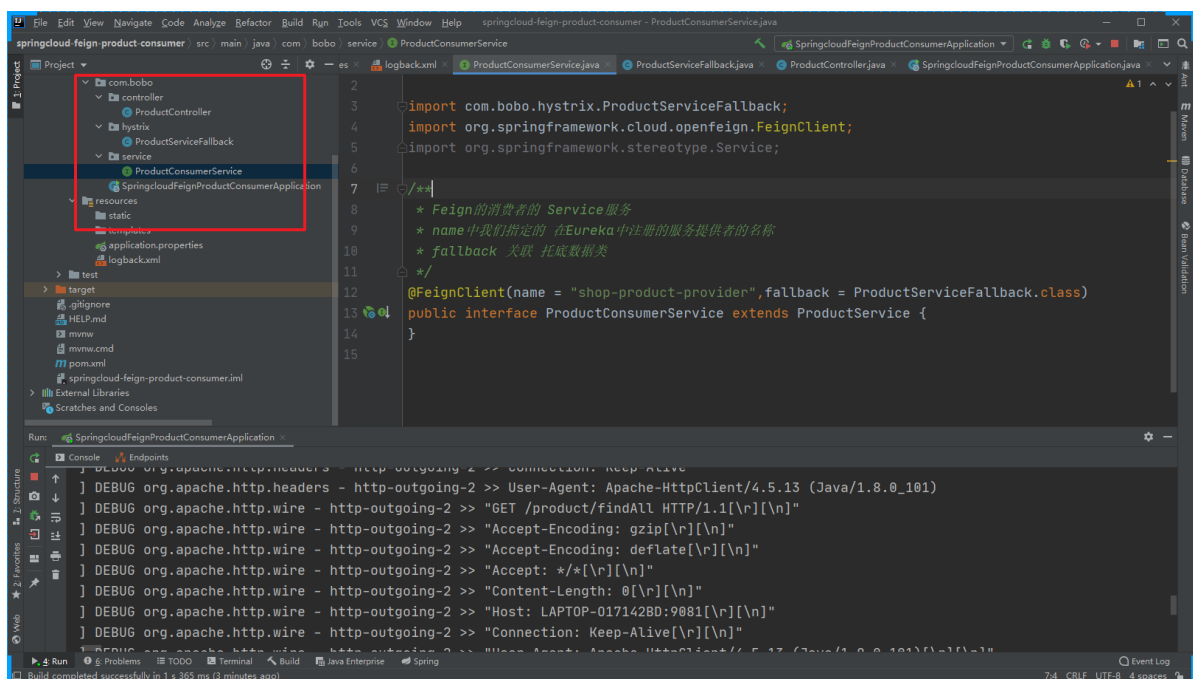
```

在接口中关联托底类

```

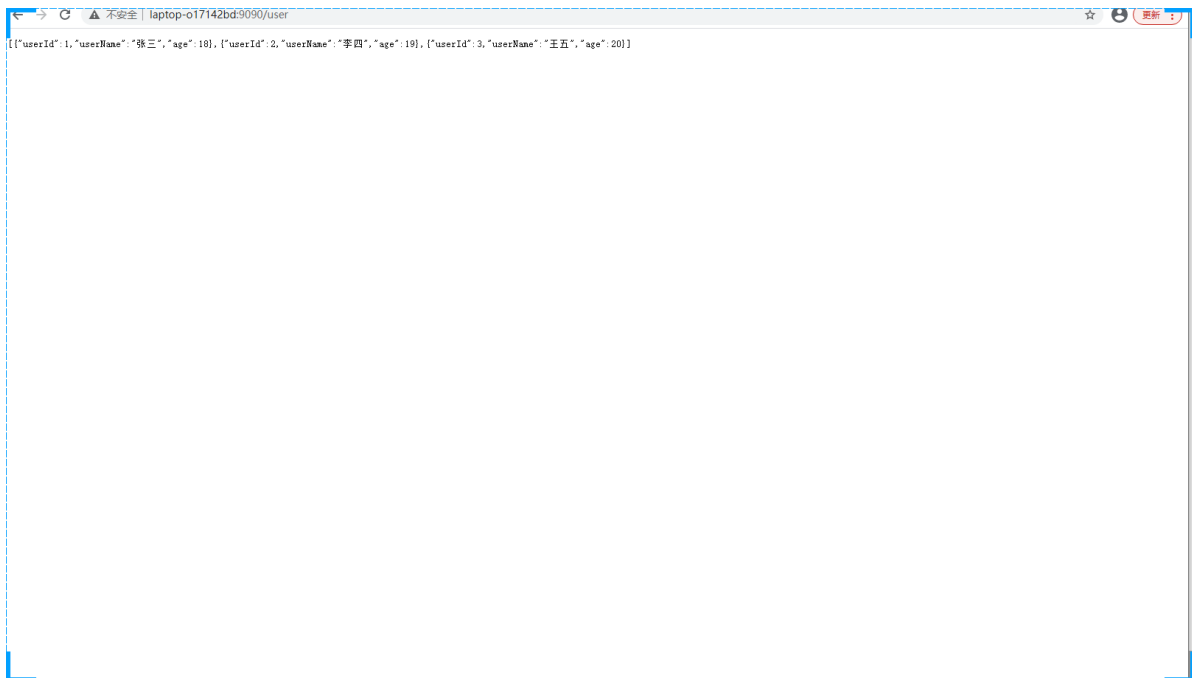
/**
 * Feign的消费者的 Service服务
 * name中我们指定的 在Eureka中注册的服务提供者的名称
 * fallback 关联 托底数据类
 */
@FeignClient(name = "shop-product-provider", fallback =
ProductServiceFallback.class)
public interface ProductConsumerService extends ProductService {
}

```

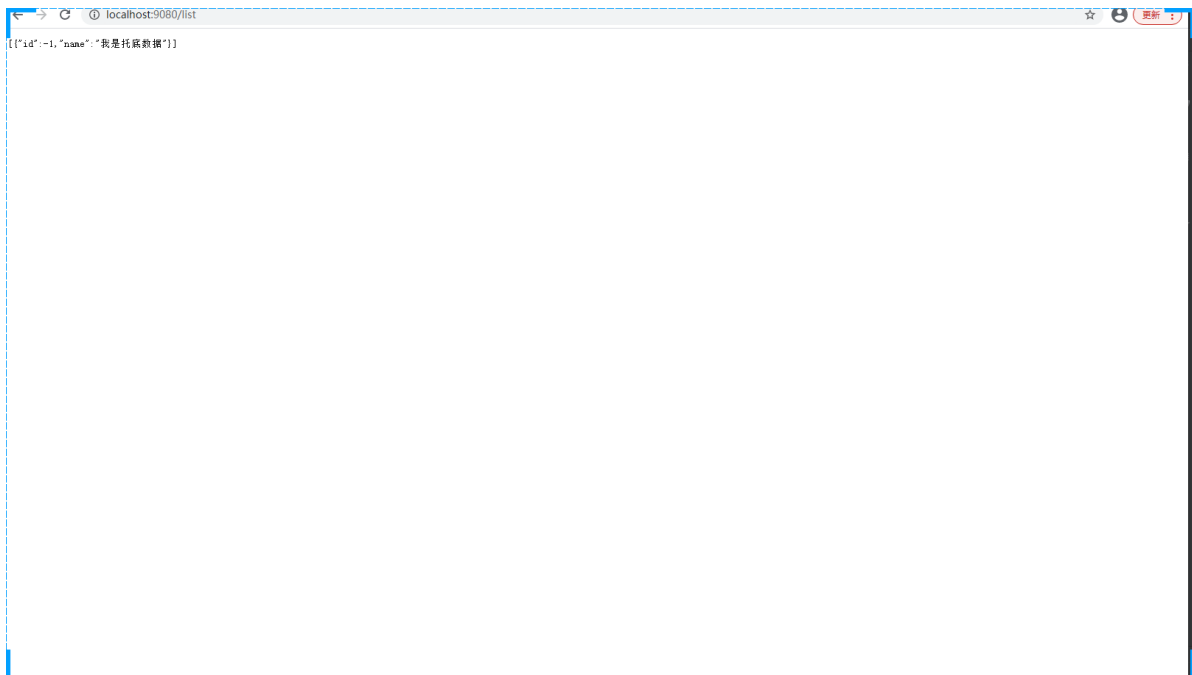


测试:

provider正常

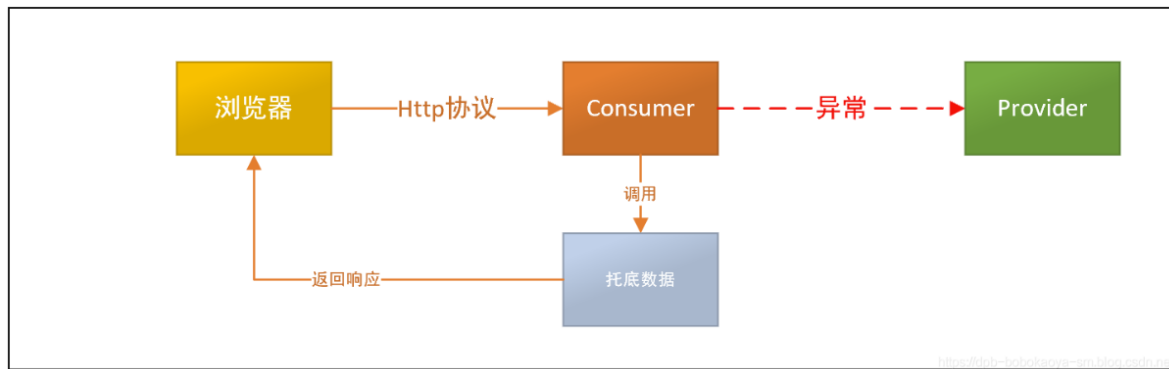


provider异常，返回托底数据



8.Feign异常日志

我们上面案例中provider出现异常，consumer返回了托底数据，但是我们并不清楚provider发生了什么异常问题，这时我们需要记录Feign的降级信息



添加一个FallbackFactory类

```
package com.bobo.hystrix;

import com.bobo.pojo.Product;
import com.bobo.service.ProductConsumerService;
import feign.hystrix.FallbackFactory;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.logging.Logger;

@Component
public class ProductServiceFallbackFactory implements
    FallbackFactory<ProductConsumerService> {

    private Logger logger =
        Logger.getLogger(ProductServiceFallbackFactory.class.getName());

    @Override
    public ProductConsumerService create(Throwable throwable) {
        System.out.println("***** Feign fallback
Exception:" + throwable.toString());
        return new ProductServiceFallback();
    }
}
```

然后在接口中关联

```
package com.bobo.service;

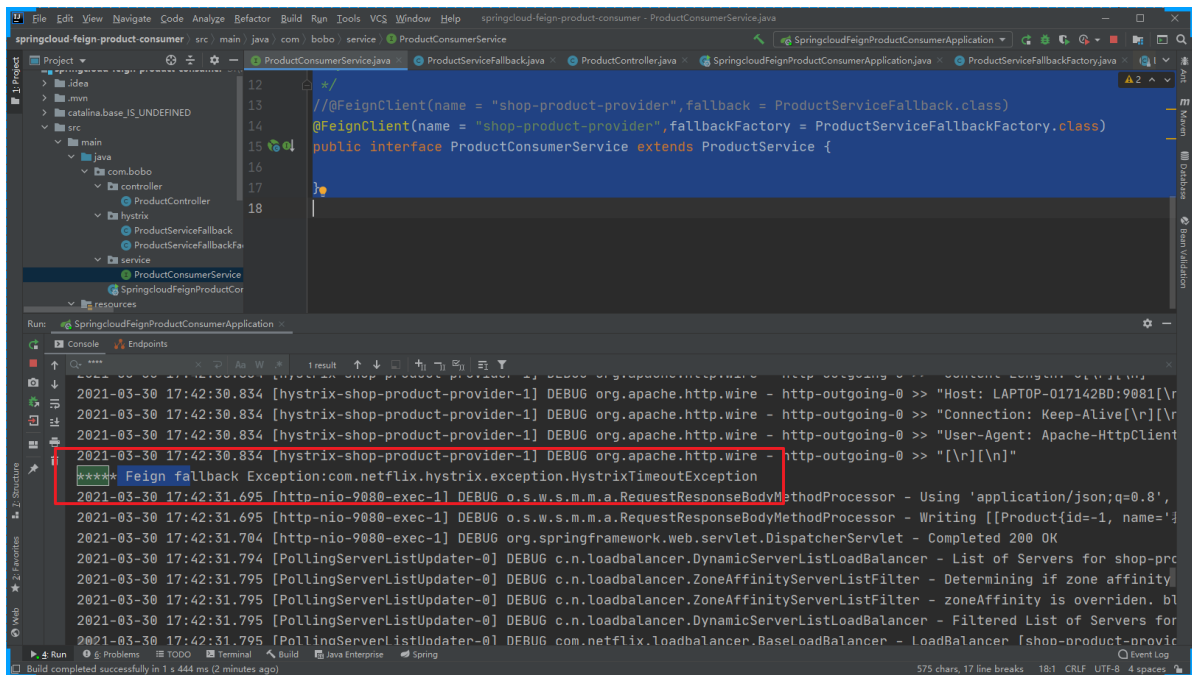
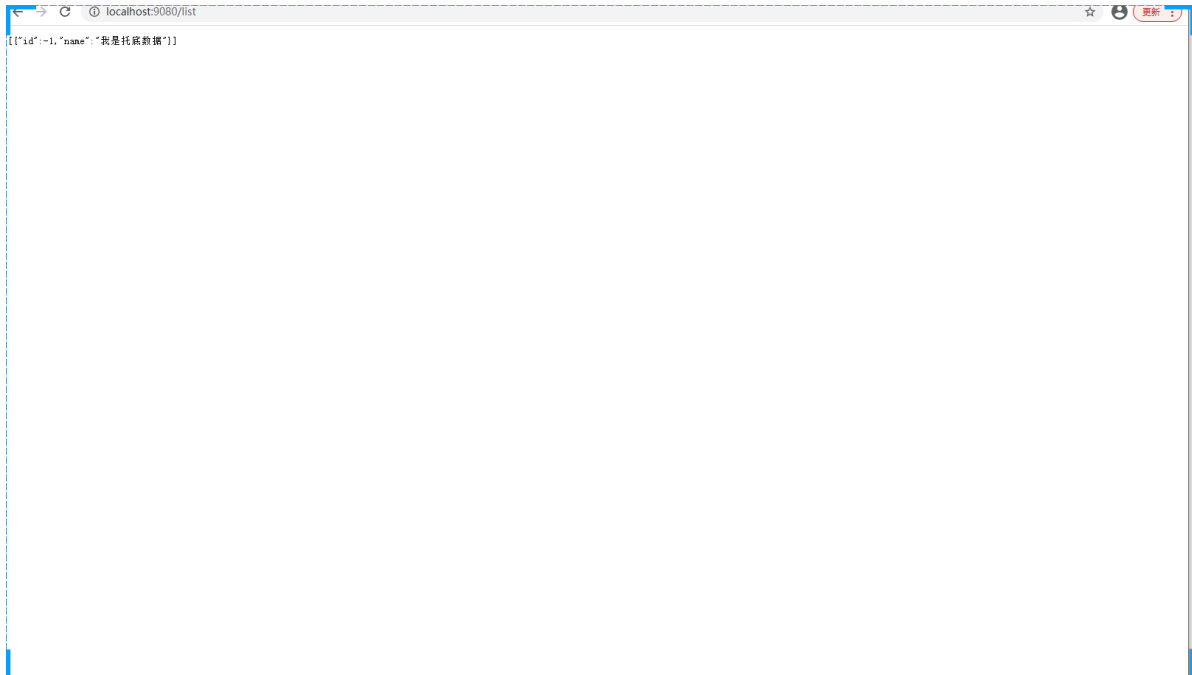
import com.bobo.hystrix.ProductServiceFallback;
import com.bobo.hystrix.ProductServiceFallbackFactory;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.stereotype.Service;

/**
 * Feign的消费者的 Service服务
 * name中我们指定的 在Eureka中注册的服务提供者的名称
 * fallback 关联 托底数据类
 */
```

```
//@FeignClient(name = "shop-product-provider",fallback =
ProductServiceFallback.class)
@FeignClient(name = "shop-product-provider",fallbackFactory =
ProductServiceFallbackFactory.class)
public interface ProductConsumerService extends ProductService {

}
```

然后在返回托底数据的情况我们可以看到日志信息



六、Zuul

1.什么是Zuul

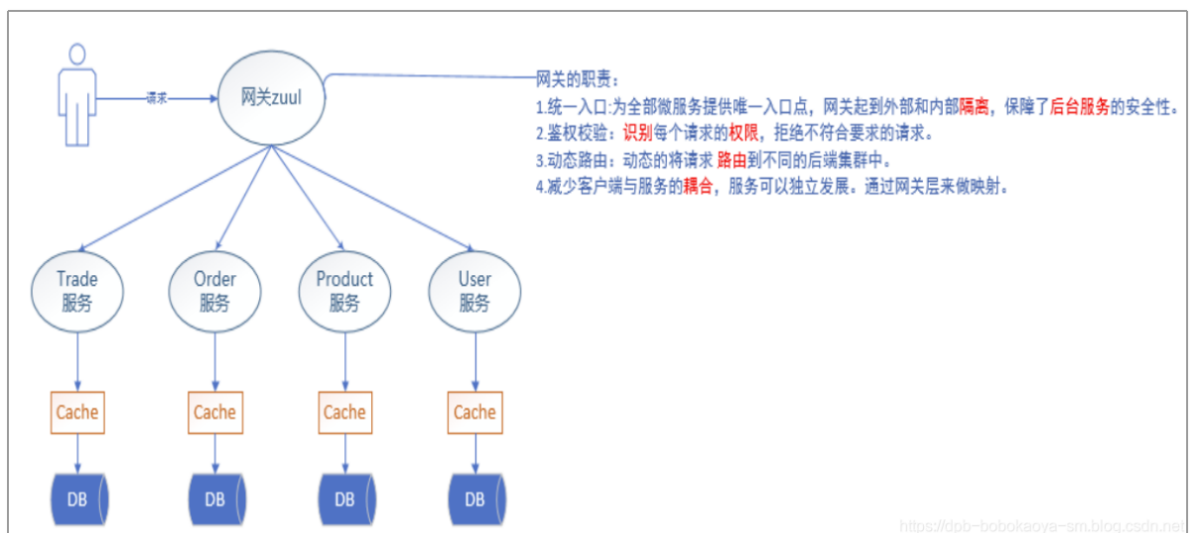
zuul 是netflix开源的一个API Gateway 服务器, 本质上是一个web servlet应用。

Zuul 在云平台上提供动态路由, 监控, 弹性, 安全等边缘服务的框架。Zuul 相当于是设备和Netflix 流应用的 Web 网站后端所有请求的前门。

zuul的例子可以参考netflix在github上的 simple webapp, 可以按照netflix 在github wiki 上文档说明来进行使用。



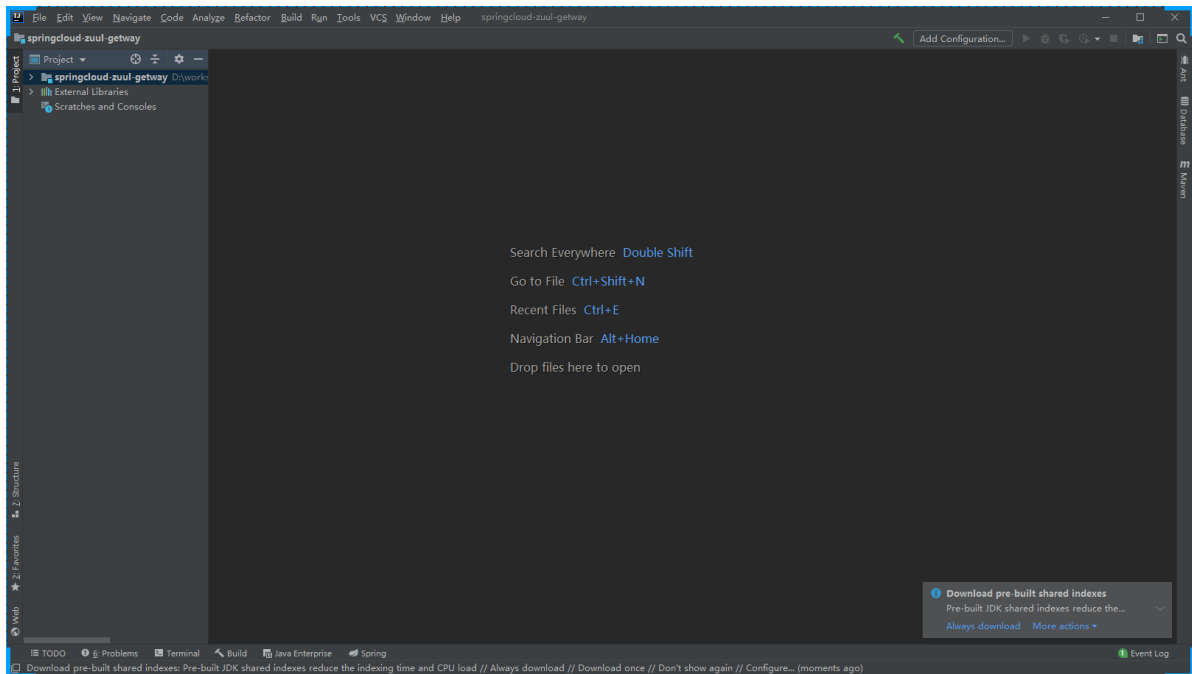
2.解决了什么问题



3.入门案例

3.1 创建项目

创建网关服务项目, 也是一个普通的SpringBoot项目, 添加相关的依赖



添加相关的依赖

```
<properties>
  <java.version>1.8</java.version>
  <spring-cloud.version>Hoxton.SR10</spring-cloud.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-zuul</artifactId>
  </dependency>

  <!-- 表示这是一个SpringCloud Eureka 的Client -->
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
    <exclusions>
      <exclusion>
        <groupId>org.junit.vintage</groupId>
        <artifactId>junit-vintage-engine</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
</dependencies>
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
```

```
        <artifactId>spring-cloud-dependencies</artifactId>
        <version>${spring-cloud.version}</version>
        <type>pom</type>
        <scope>import</scope>
    </dependency>
</dependencies>
</dependencyManagement>
```

3.2 配置文件

在属性文件中我们同样的需要添加Eureka的相关信息

```
spring.application.name=Zuul-getway
server.port=9020

# 设置服务注册中心的地址
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/
```

3.3 启动器

我们需要在启动类中添加Zuul的注解

```
package com.bobo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.zuul.EnableZuulProxy;

@EnableZuulProxy
@SpringBootApplication
public class SpringcloudZuulGetwayApplication {

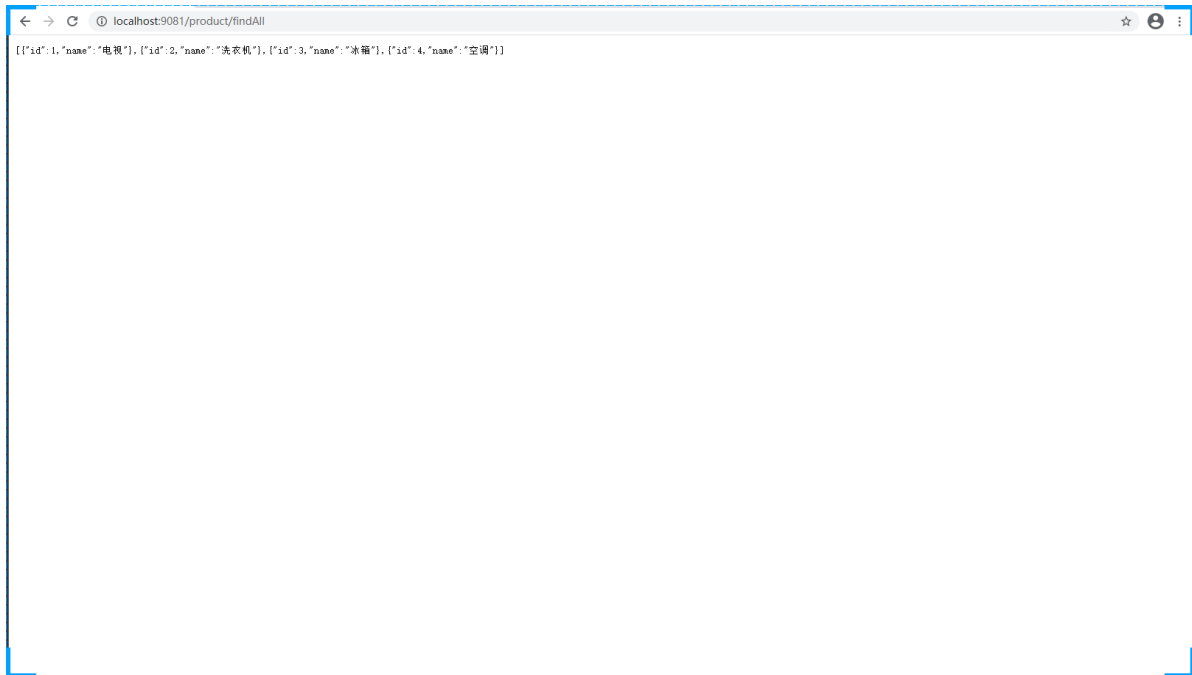
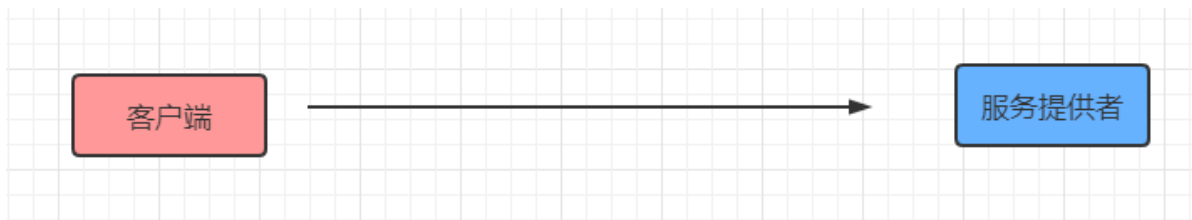
    public static void main(String[] args) {
        SpringApplication.run(SpringcloudZuulGetwayApplication.class, args);
    }

}
```

3.4 测试效果

首先直接访问服务提供者

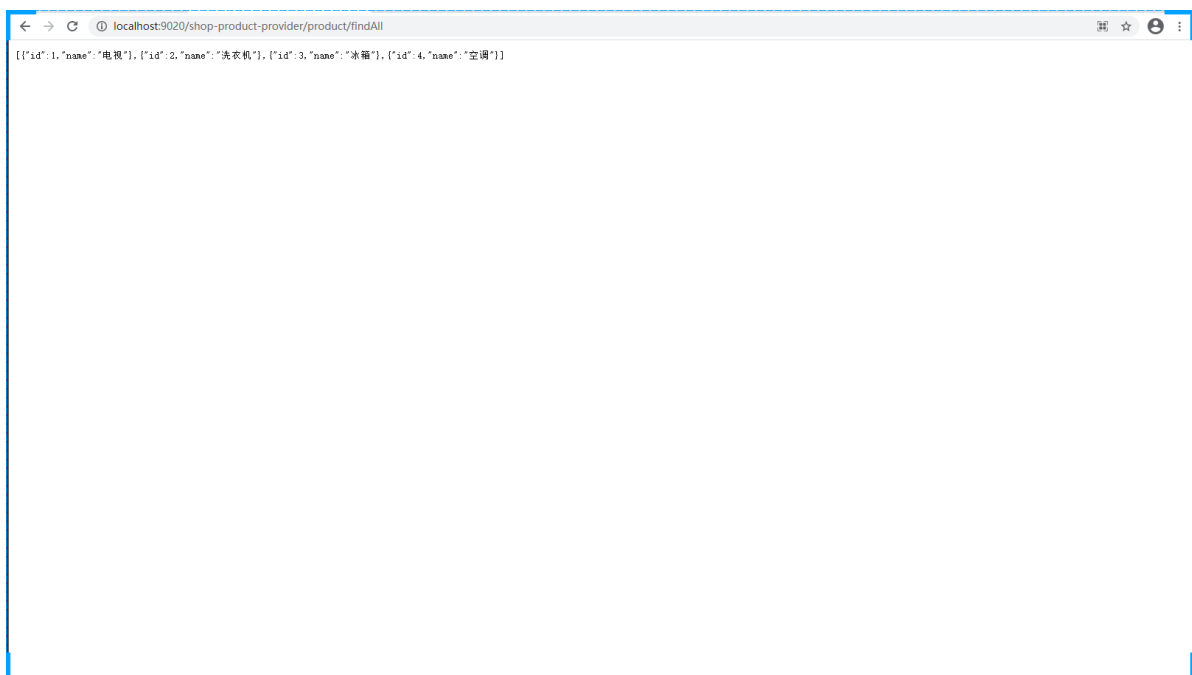
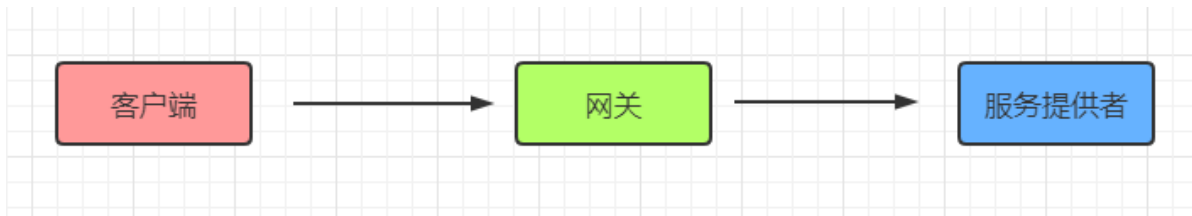
访问地址: <http://localhost:9081/product/findAll>



通过Zuul来间接访问服务

<http://localhost:9020/shop-product-provider/product/findAll>

http://网关地址:网关端口/要访问的服务的名称/访问的服务中的接口地址



4.路由规则

4.1 指定路由

根据客户端提交的请求地址直接路由到服务地址

```
spring.application.name=Zuul-getway
server.port=9020

# 设置服务注册中心的地址
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

zuul.routes.shop-product-provider.path=/bobo/**
zuul.routes.shop-product-provider.url=http://127.0.0.1:9081/
```

说明:

zuul.routes.shop-product-provider.path

zuul.routes.固定的

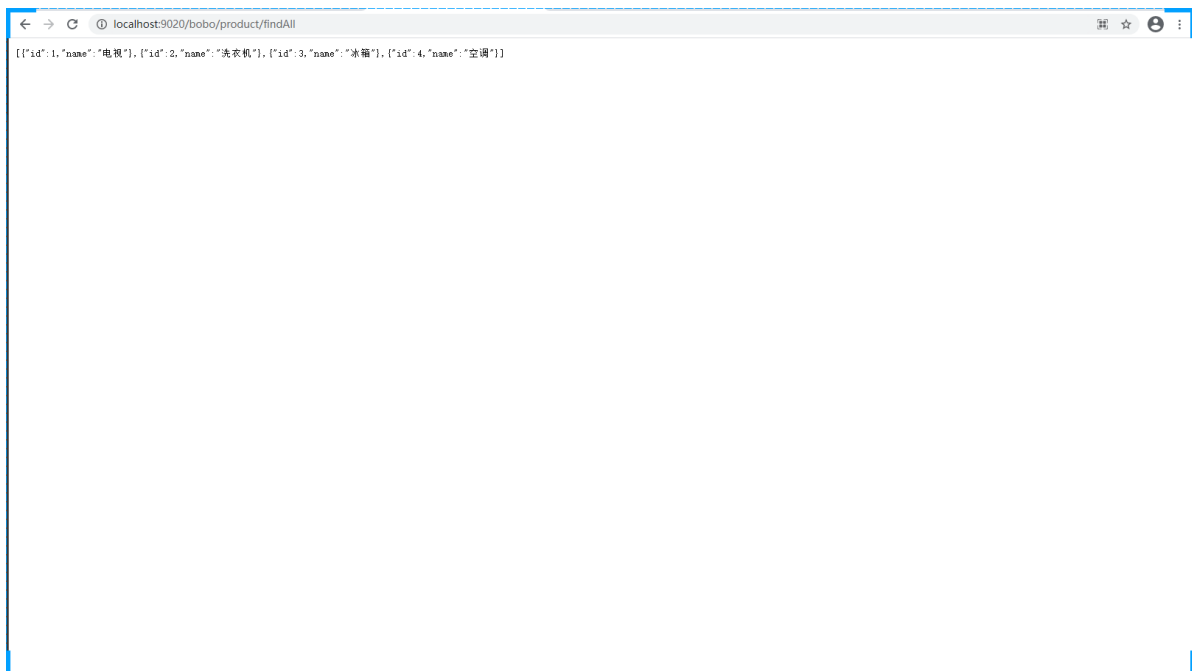
shop-product-provider 自定义的

path:固定的

/bobo/** 表示请求的url地址

zuul.routes.shop-product-provider.url=<http://127.0.0.1:9081/>

表示当用户的请求是 http://xxx/bobo/** 者路由到 <http://127.0.0.1:9081/>服务处理



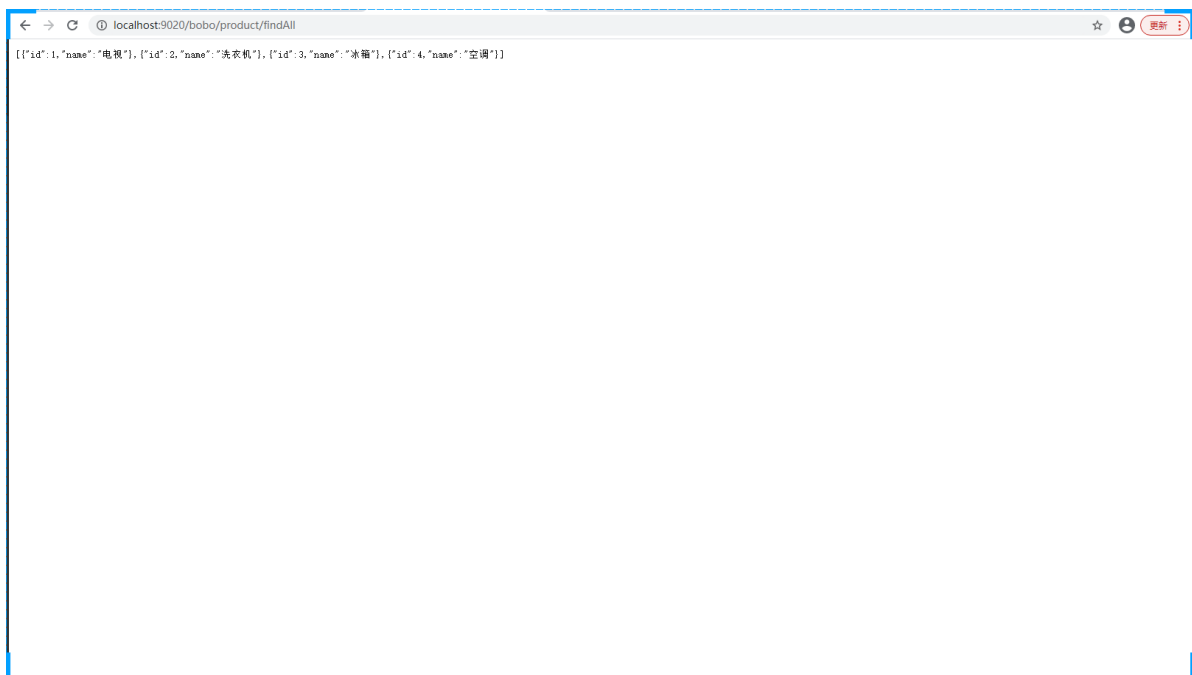
通配符	含义	举例	解释
?	匹配任意单个字符	/e-book-product/?	匹配:/e-book-product/a ./e-book-product/b /e-book-product/c
*	匹配任意数量的字符 (单级目录)	/e-book-product/*	匹配:/e-book-product/abc, /e-book-product/bbb, 不匹配:/e-book-product/a/b/c
**	匹配任意数量的字符 (多级目录)	/e-book-product/**	匹配:/e-book-product/abc, /e-book-product/bbb, 也匹配:/e-book-product/a/b/c

4.2 指定路由服务名称

上面的指定路由是根据客户端提交的请求地址路由到具体的服务中，这种方式其实是不太灵活的，我们可以路由到Eureka中注册的服务名称

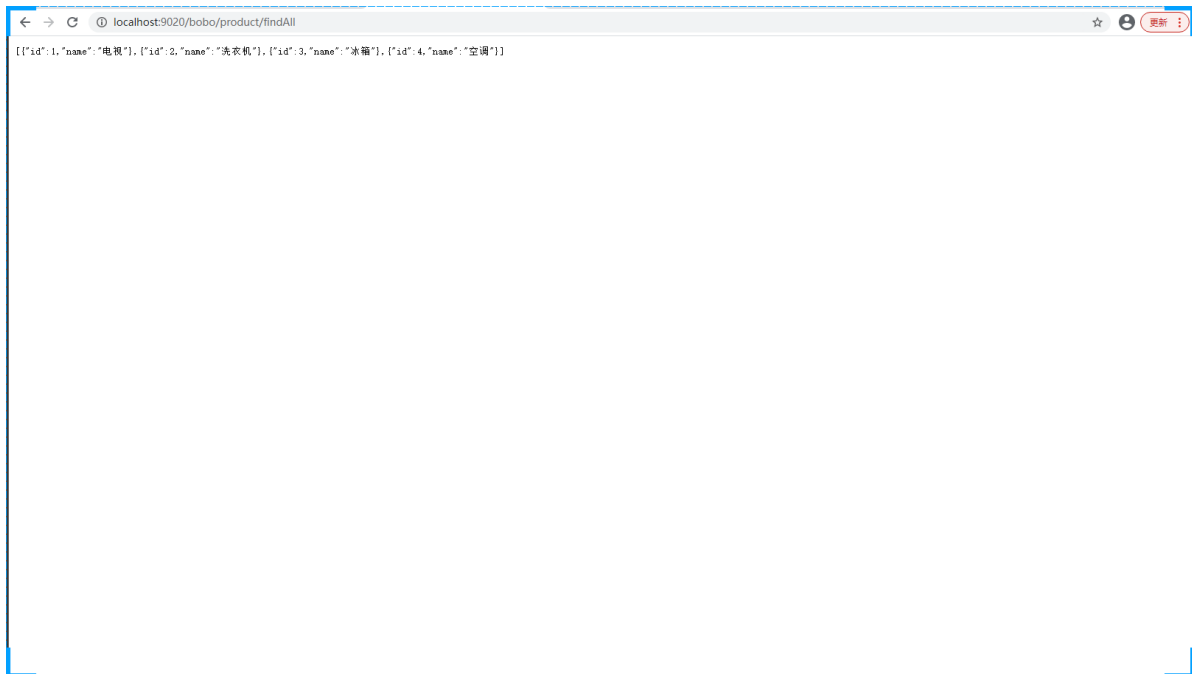
第一种方式

```
# 服务名称 路由
zuul.routes.shop-product-provider.path=/bobo/**
zuul.routes.shop-product-provider.service-id=shop-product-provider
```



第二种方式

```
# 服务名称 路由 第二种方式 shop-product-provider必须是我们要请求的服务的名称
zuul.routes.shop-product-provider.path=/bobo/**
```



4.3 路由的排除方式

排除路由，也就是该路由器会忽略掉某几个服务，即使客户端发送了请求也访问不了

```
zuul.ignored-services=eureka-provider
```

如果有多个服务要排除。这时可以通过","连接的方式来排除多个服务

```
zuul.ignored-services=eureka-provider1,eureka-provider2
```

如果有很多个要排除的服务，一个个来写太麻烦了，这时我们可以先排除掉所有的服务，然后再添加我们要放开的服务即可

```
# 先排除所有的服务
zuul.ignored-services=*
# 然后再放开特定的服务
zuul.routes.shop-product-provider.path=/bobo/**
```

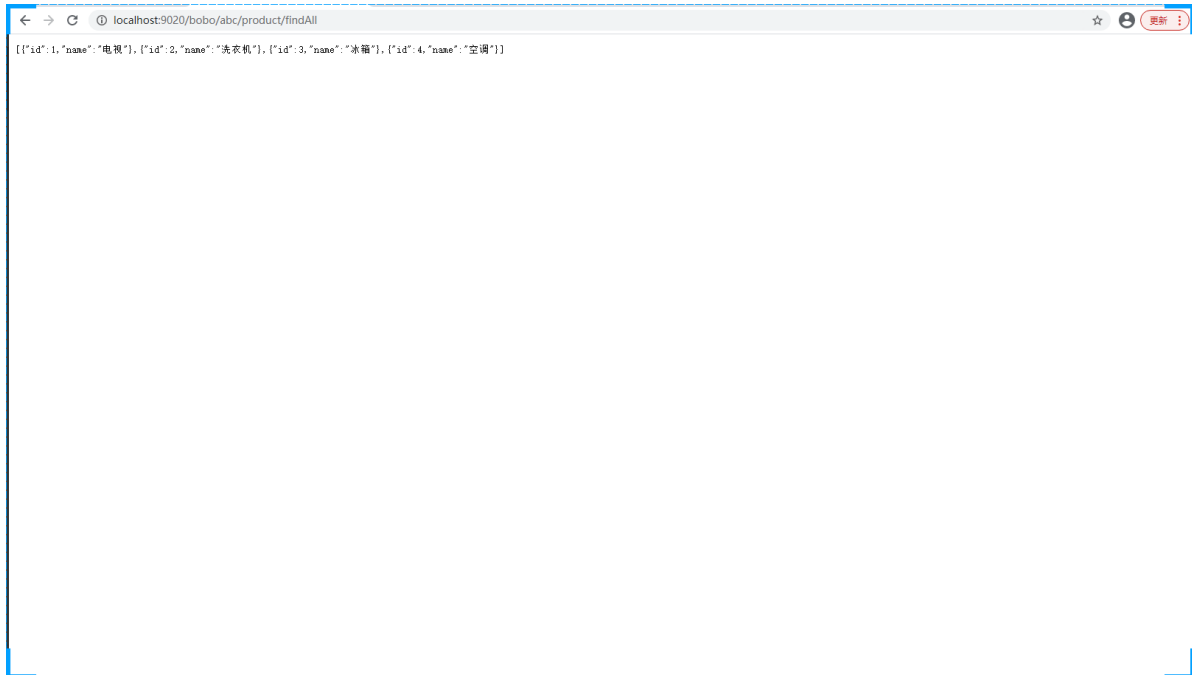
如果我们要针对某个服务中特殊接口来处理，我们也可以通过关键字处理

```
# 先排除所有的服务包含 findAll关键字的请求
zuul.ignored-services=/**/findAll/**
# 然后再放开特定的服务 同时也会排除findAll服务
zuul.routes.shop-product-provider.path=/bobo/**
```

4.4 指定路由前缀

也就是给提交的请求添加一个前缀

```
# http://127.0.0.1:9080/bobo/abc/product/findAll
zuul.prefix=/bobo
zuul.routes.shop-product-provider.path=/abc/**
```



5. 自定义网关过滤器

Zuul的核心是一系列的Filters，其作用可以类比Servlet框架的Filter。我们实际开发中会有很多的需求涉及到自定义的过滤器。所以我们来看看如何实现

5.1 创建自定义过滤器

```
package com.bobo.zuul;

import com.netflix.zuul.ZuulFilter;
import com.netflix.zuul.context.RequestContext;
import com.netflix.zuul.exception.ZuulException;
import org.springframework.stereotype.Component;

import javax.servlet.http.HttpServletRequest;

/**
 * 自定义的网关过滤器
 */
@Component
public class LoggerFilter extends ZuulFilter {
```

```

/**
 * 指定过滤器的类型
 *
 * @return
 */
@Override
public String filterType() {
    return "pre";
}

/**
 * 过滤器的执行顺序
 *     数值越小，优先级越高
 * @return
 */
@Override
public int filterOrder() {
    return 0;
}

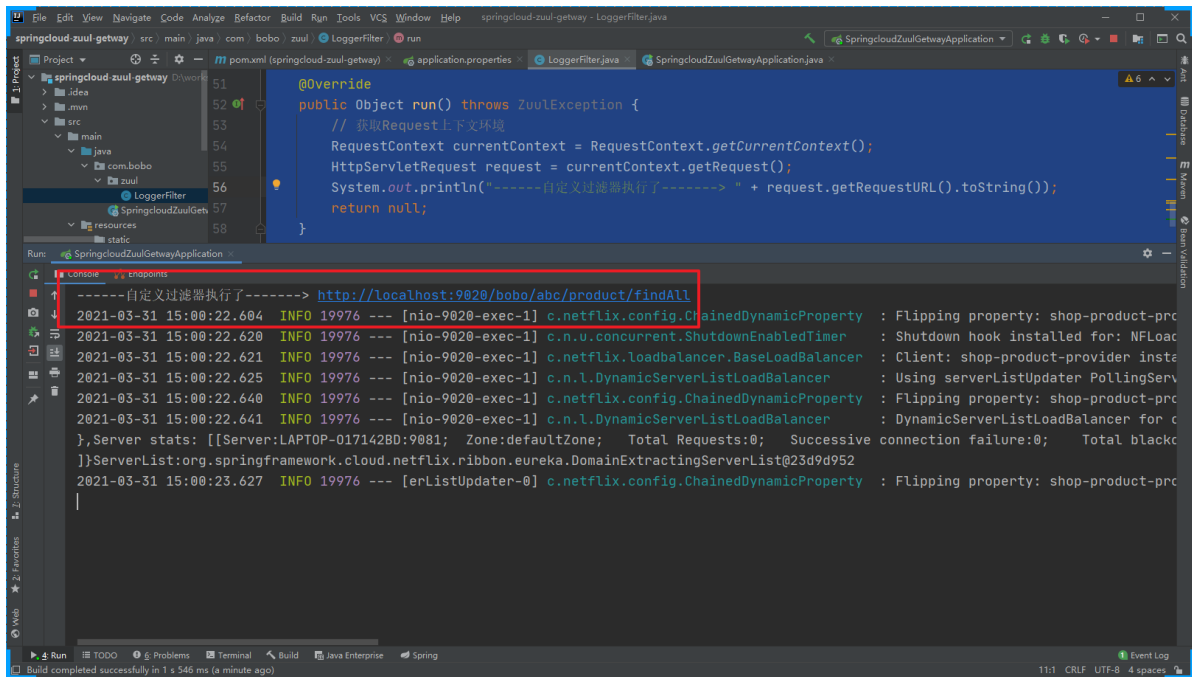
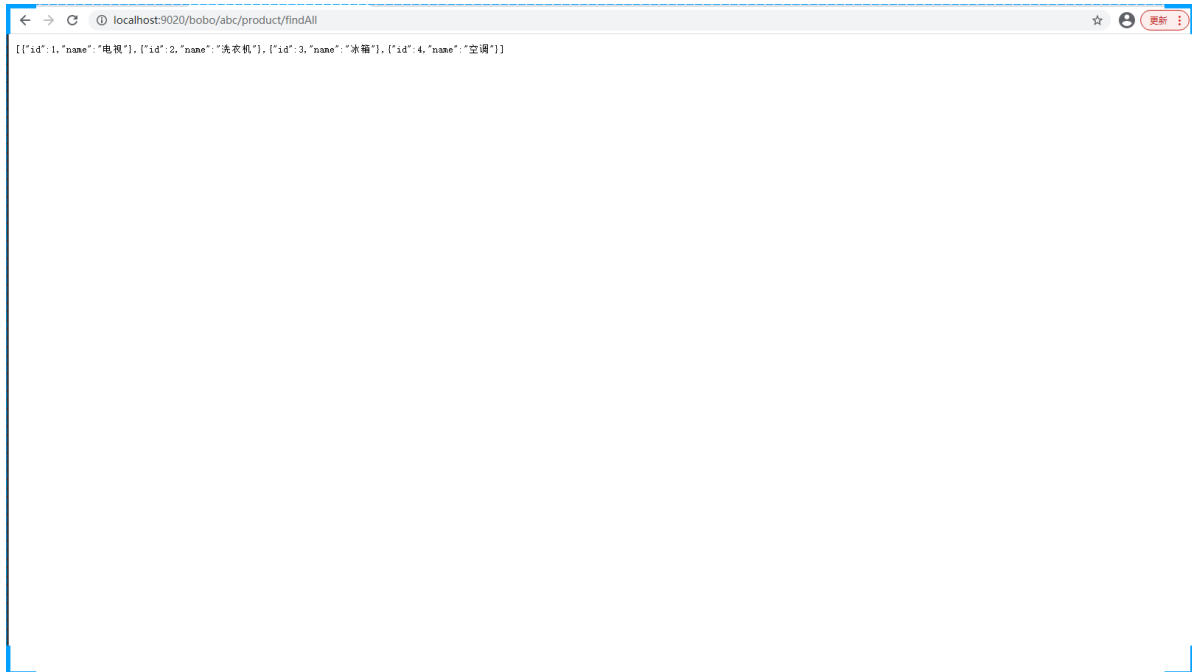
/**
 * 是否开启过滤器 默认值 false
 * @return
 */
@Override
public boolean shouldFilter() {
    return true;
}

/**
 * 过滤器要执行的逻辑代码
 * @return
 * @throws ZuulException
 */
@Override
public Object run() throws ZuulException {
    // 获取Request上下文环境
    RequestContext currentContext = RequestContext.getCurrentContext();
    HttpServletRequest request = currentContext.getRequest();
    System.out.println("-----自定义过滤器执行了-----> " +
request.getRequestURL().toString());
    return null;
}
}

```

5.2 演示效果

启动一个provider服务后，启动我们的网关服务，然后查询访问，再查看控制台输出



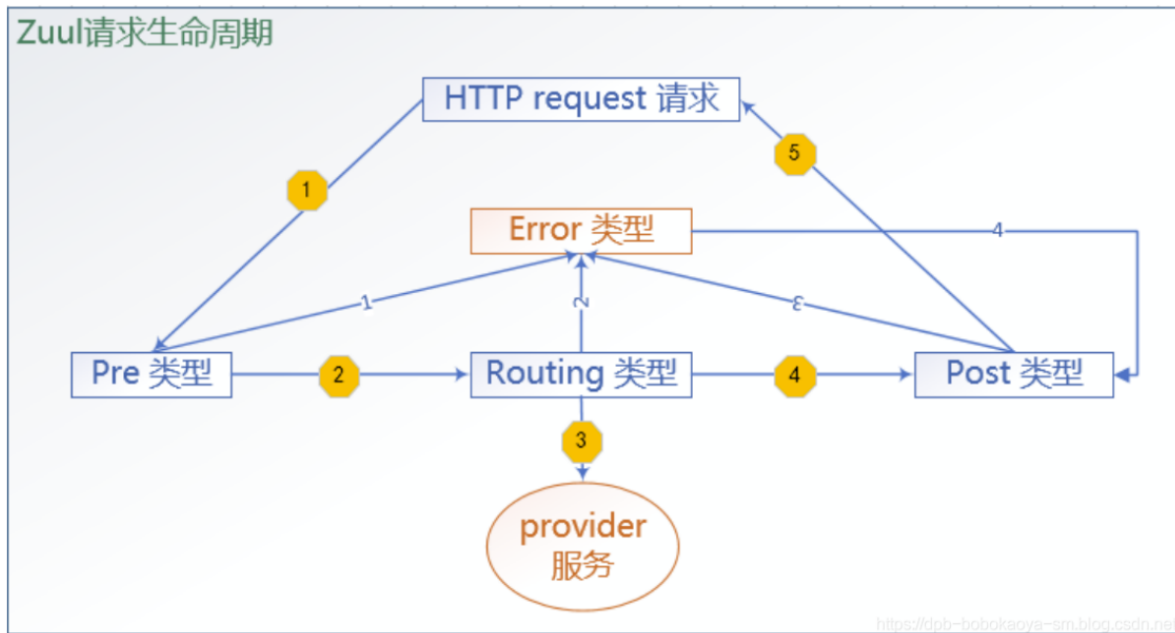
5.3 相关方法介绍

自定义过滤器中的相关方法介绍

过滤器有哪些类型？它有何用	
filterType	过滤器的类型用，它字符串来表示，在Zuul中默认定义了四种不同生命周期的过滤器类型，具体如下：
	pre 可以在请求被路由之前调用。一般用于身份权限验证、记录调用日志等等。
	routing 在路由执行之后被调用。
	post 在routing和error过滤器之后被调用。可用于信息收集、统计信息例如性能指标，对response的结构做些特殊处理。
filterOrder	用int值来定义过滤器的执行顺序，数值越小优先级越高。
shouldFilter	返回一个boolean类型来判断该过滤器是否要执行。
run	逻辑处理。一般2个用途：
	第一，请求前拦截，对请求进行验证判断，如果请求无效就直接断路；如果有效可再加工处理 第二，请求结果后处理，即对结果做一些加工处理。

<https://job-bobokaoya-sm.blog.csdn.net>

5.4 请求生命周期

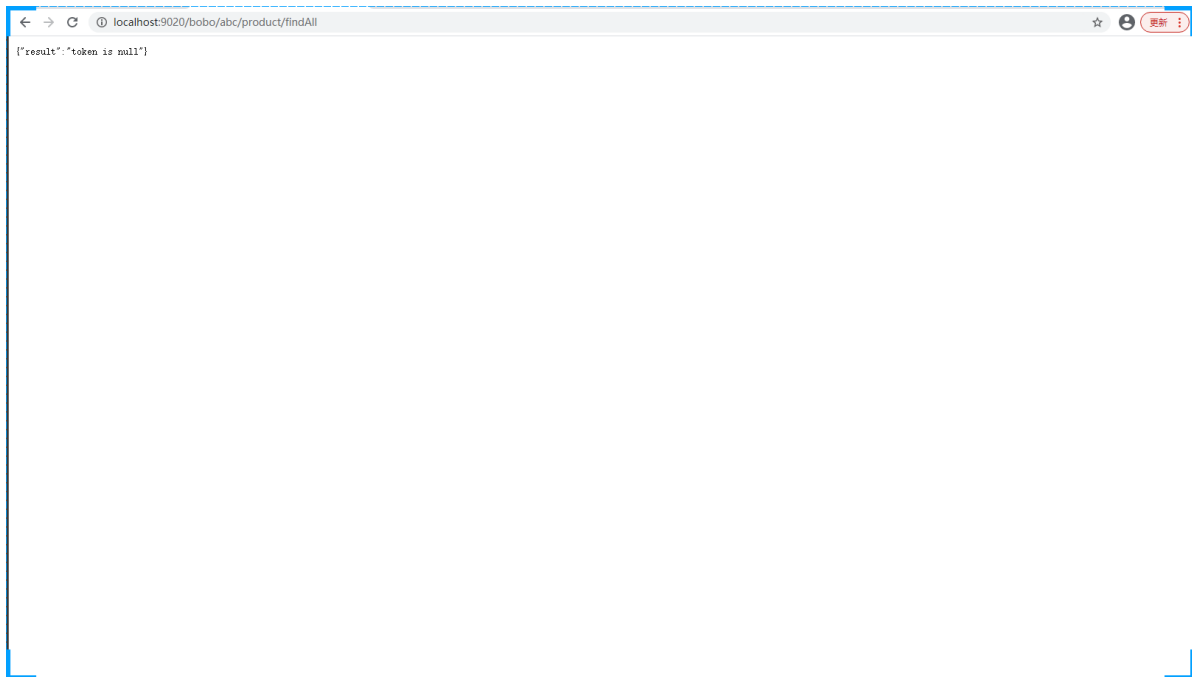
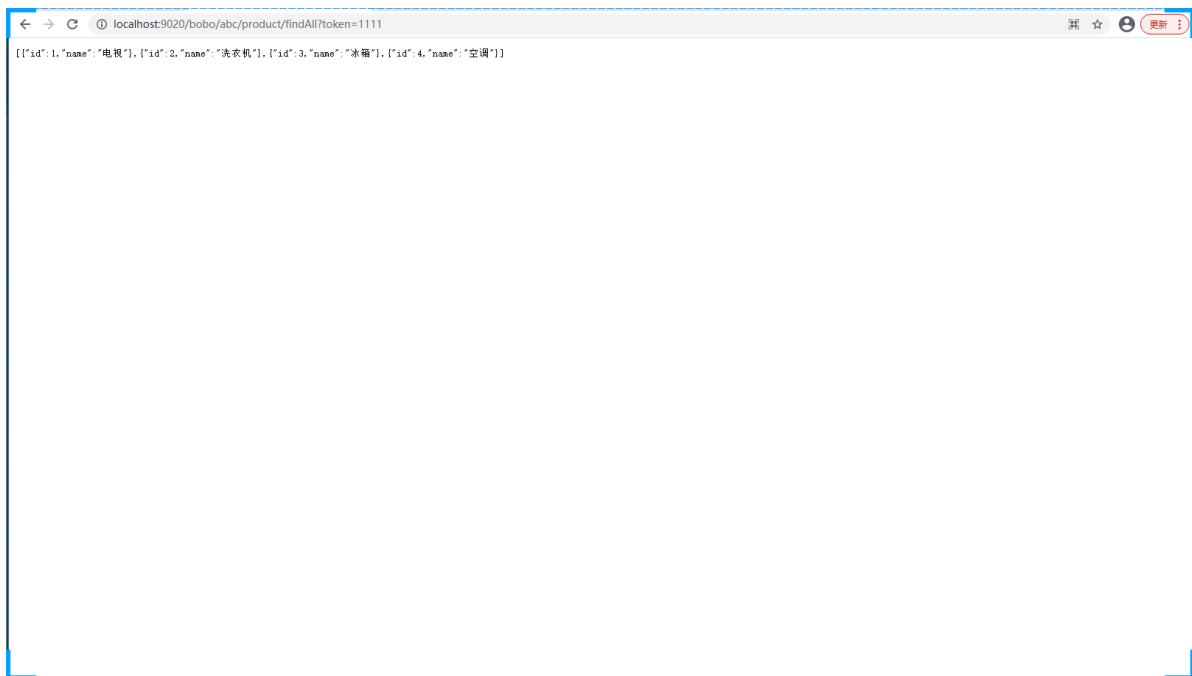


5.5 案例分析

模拟一个身份认证的过滤器

```
@Override
public Object run() throws ZuulException {
    // 获取Request上下文环境
    RequestContext currentContext = RequestContext.getCurrentContext();
    HttpServletRequest request = currentContext.getRequest();
    System.out.println("-----自定义过滤器执行了-----> " +
request.getRequestURL().toString());
    // 做一个身份认证
    String token = request.getParameter("token");
    if(token == null){
        // 没有传递身份认证信息 拦截请求
        // 表示 请求结束 不再继续向下请求
        currentContext.setSendZuulResponse(false);
        // 添加一个响应码
        currentContext.setResponseStatusCode(401);
        // 响应内容
        currentContext.setResponseBody("{\"result\":\"token is null\"}");
        // 响应类型
        currentContext.getResponse().setContentType("text/json;charset=utf-8");
    }
    return null;
}
```

效果



七、Config

1. 为什么需要使用配置中心

1.1 服务配置的现状

服务配置的现状		
在微服务系统中，每个微服务不仅仅只有代码，他还需要连接其他资源，例如数据库的配置或功能性的开关等等。但是随着微服务系统的不断迭代，整个微服务系统可能会成为一个网状结构，这个时候就要考虑整个微服务系统的扩展性、伸缩性、耦合性等等。其中一个很重要的环节就是配置管理的问题。		
常见的配置类型	服务配置	数据库配置
		MQ队列配置
		redis缓存配置
	各类开关	功能开关
		业务开关
		服务开关
	业务配置	模块A
		模块B
		模块C

1.2 常用的配置管理解决方案的缺点

常用的配置管理解决方案的缺点
1.硬编码【缺点:需要修改代码、繁琐、风险大】
2.写在properties里面【缺点：在集群环境下，需要替换和重启】
3.写在xml配置文件中,一般和应用一起打包【缺点:需要重新打包和重启】

1.3 为什么要使用 spring cloud config 配置中心?

为什么要使用spring cloud config 配置中心？
由于常用的配置管理有很大的缺点，故spring cloud config采用了集中式的配置中心来管理每个服务的配置信息。spring cloud config配置中心，在微服务分布式系统中，采用服务端和客户端来提供可扩展的配置服务。配置中心负责管理所有的服务的各种环境配置文件。配置服务中心默认采用Git的方式存储配置文件，因此我们很容易部署修改，有助于对环境配置进行版本管理。

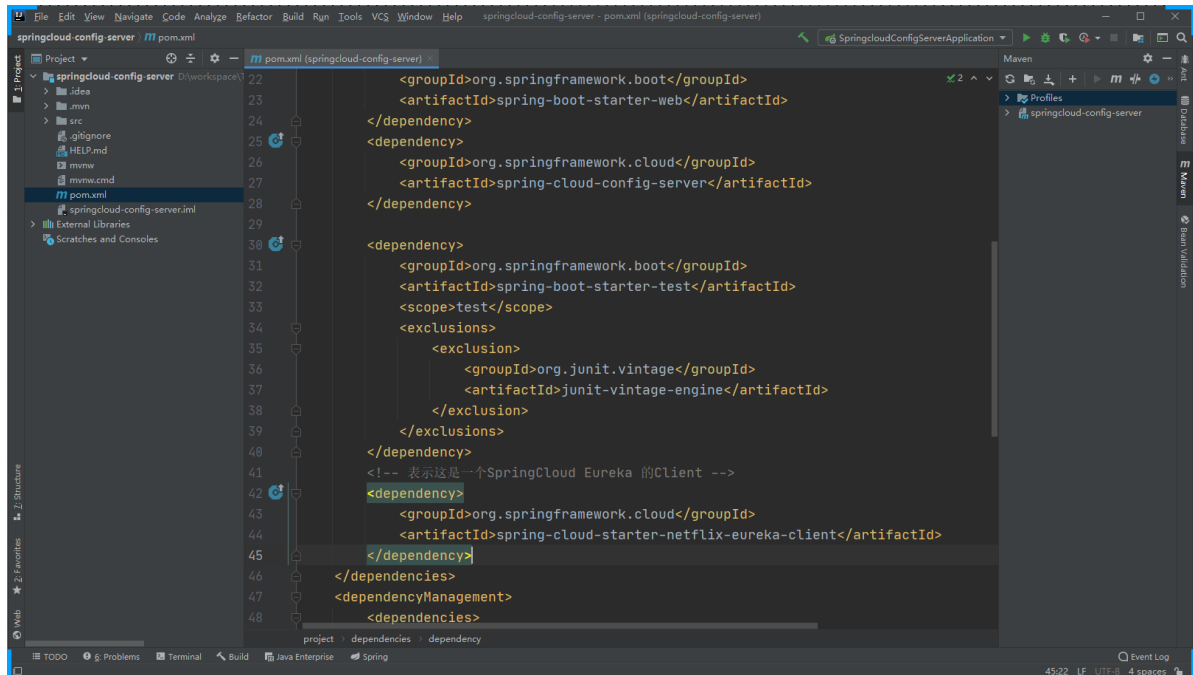
1.4 spring cloud config配置中心,它解决了什么问题?

spring cloud config 配置中心，它解决了什么问题？
Spring Cloud Config它解决了微服务配置的中心化、版本控制、平台独立、语言独立等问题。其特性如下： 1.提供服务端和客户端支持(spring cloud config server和spring cloud config client) 2.集中式管理分布式环境下的应用配置 3.基于Spring环境，无缝与Spring应用集成 4.可用于任何语言开发的程序 5.默认实现基于git仓库，可以进行版本管理

2. ConfigServer

2.1 创建项目

我们先创建一个Config服务端程序



添加相关的依赖

```
<properties>
    <java.version>1.8</java.version>
    <spring-cloud.version>Hoxton.SR10</spring-cloud.version>
</properties>
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-config-server</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
        <exclusions>
            <exclusion>
                <groupId>org.junit.vintage</groupId>
                <artifactId>junit-vintage-engine</artifactId>
            </exclusion>
        </exclusions>
    </dependency>
    <!-- 表示这是一个SpringCloud Eureka 的Client -->
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
    </dependency>
</dependencies>
```

```

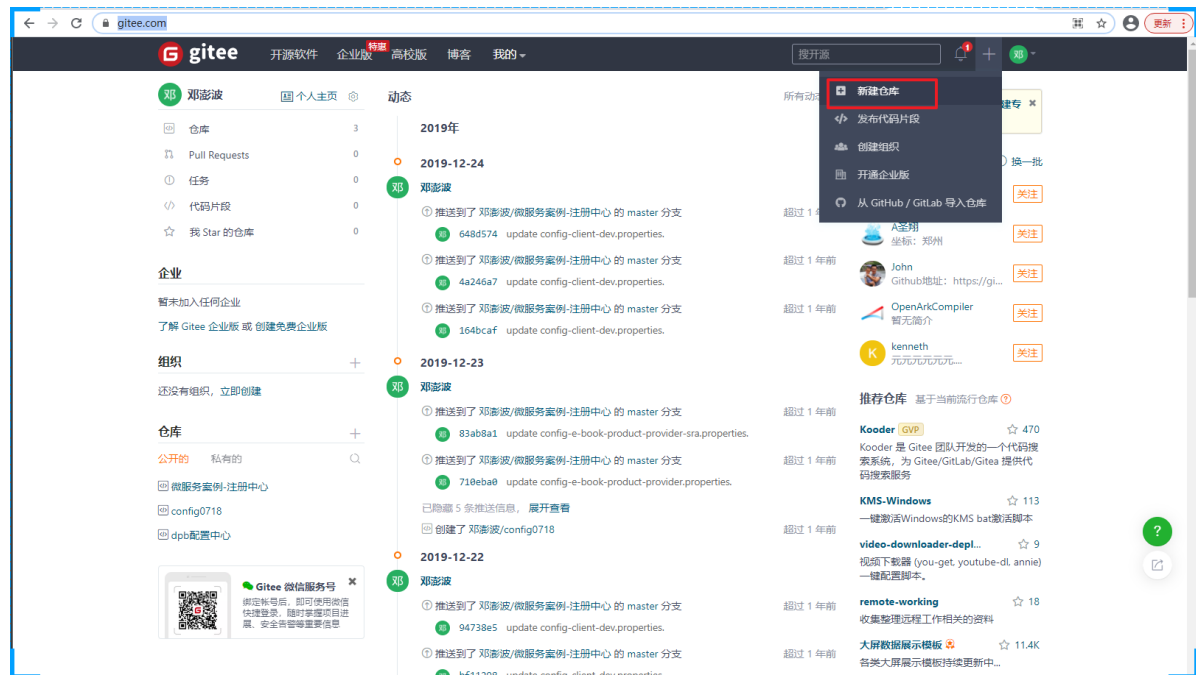
</dependencies>
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

```

2.2 版本控制

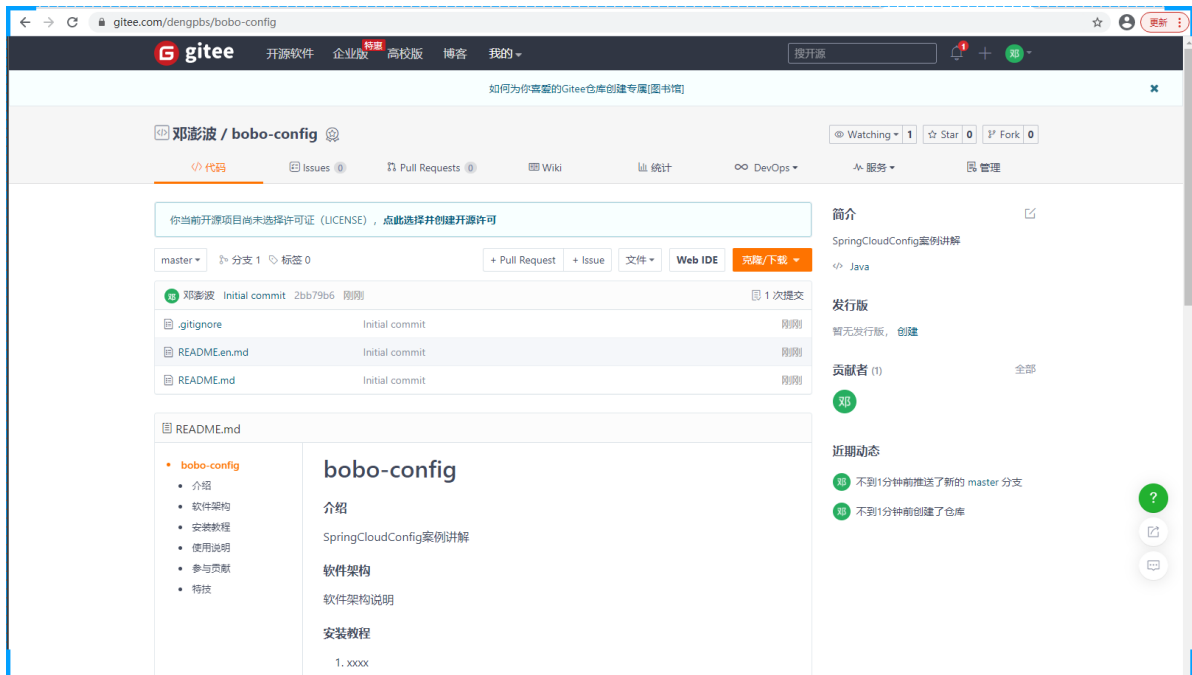
配置服务端的配置文件我们需要通过Git来进行版本控制，我们可以通过Github或者码云来处理，本课程以码云为案例

网站地址：<https://gitee.com/> 注册一个账号





创建完成



2.3 修改属性文件

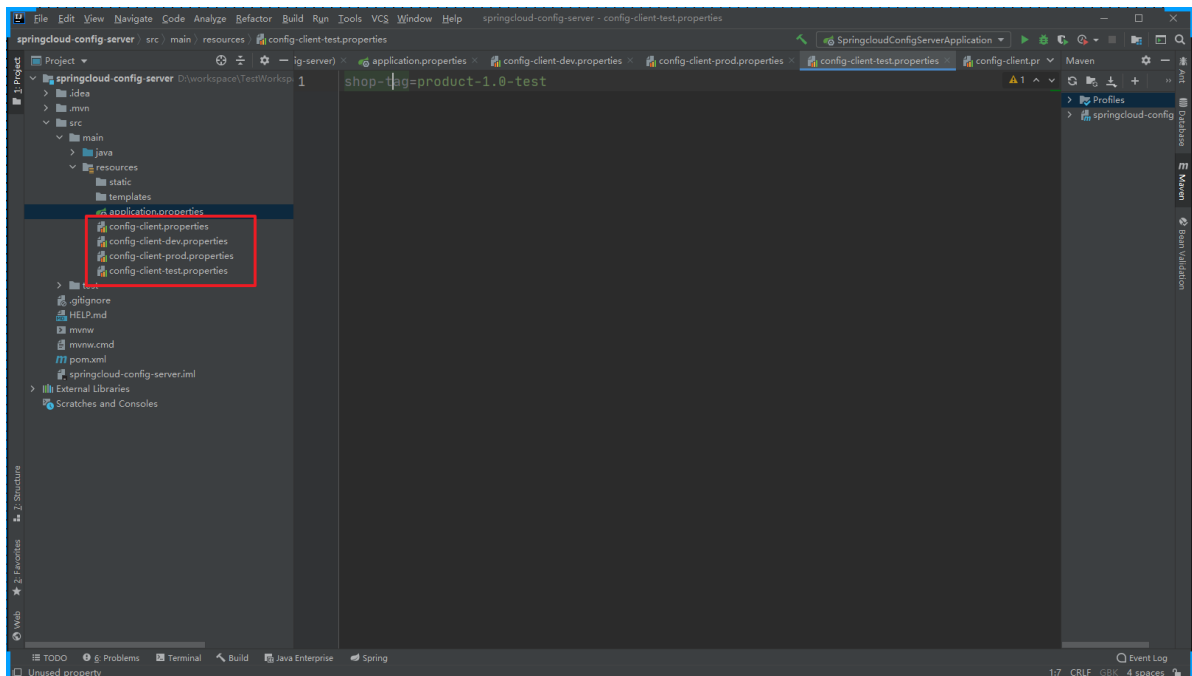
```
spring.application.name=config-server
server.port=9082

# 设置服务注册中心的地址
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

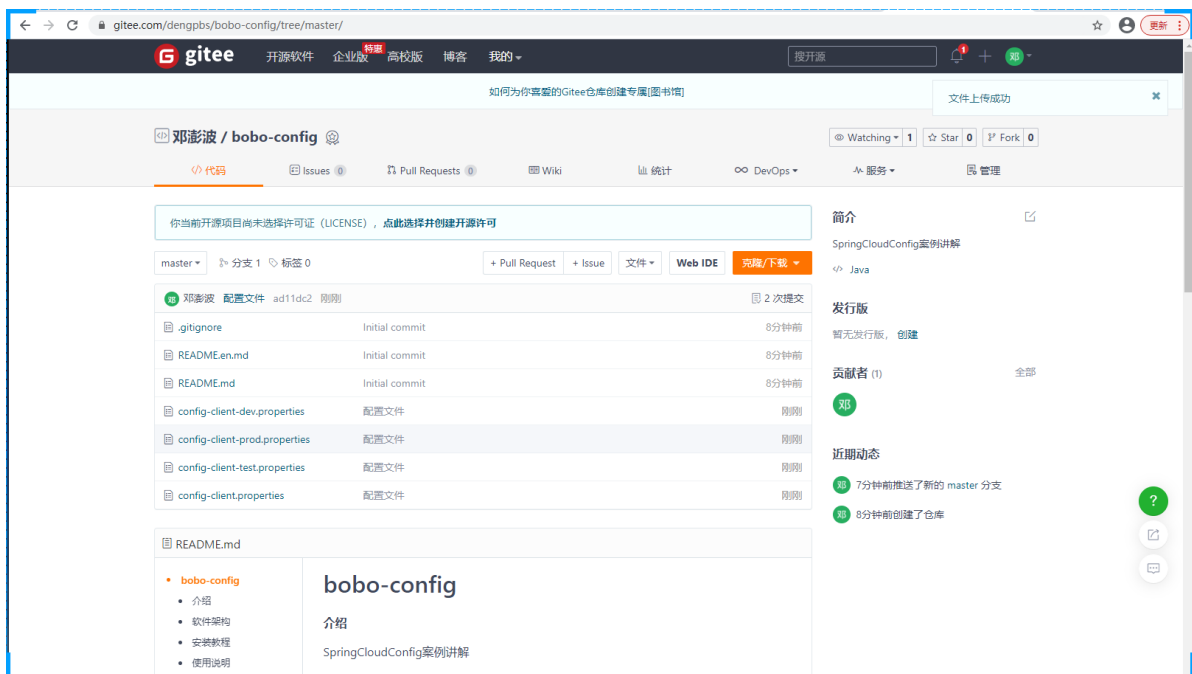
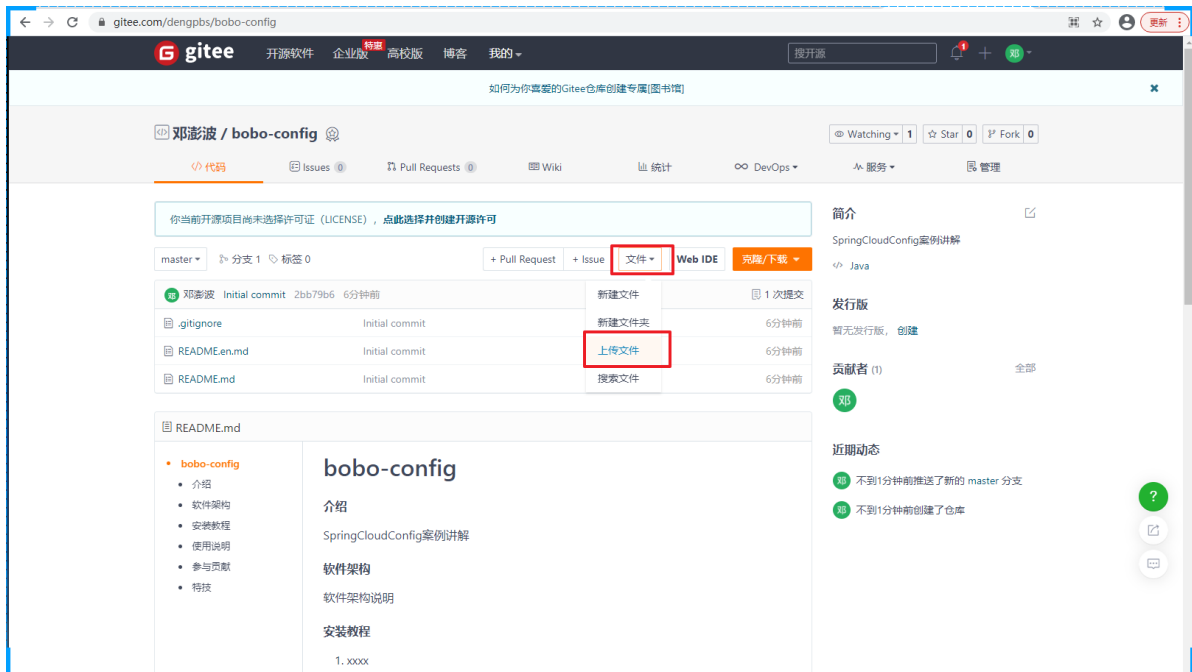
# Git配置
spring.cloud.config.server.git.uri=https://gitee.com/dengpbs/bobo-config
#spring.cloud.config.server.git.username=
#spring.cloud.config.server.git.password=
```

2.4 配置文件

我们在项目中分别创建4个配置文件，每个文件中都有一个shop-tag只是值不一致



将创建的这四个配置文件上传到我们刚刚创建的仓库中



上传成功后我们可以在ConfigServer中删除掉原来的那四个文件

2.5 修改启动类

修改启动类并启动

```
package com.bobo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;

@EnableConfigServer
@SpringBootApplication
public class SpringcloudConfigServerApplication {
```

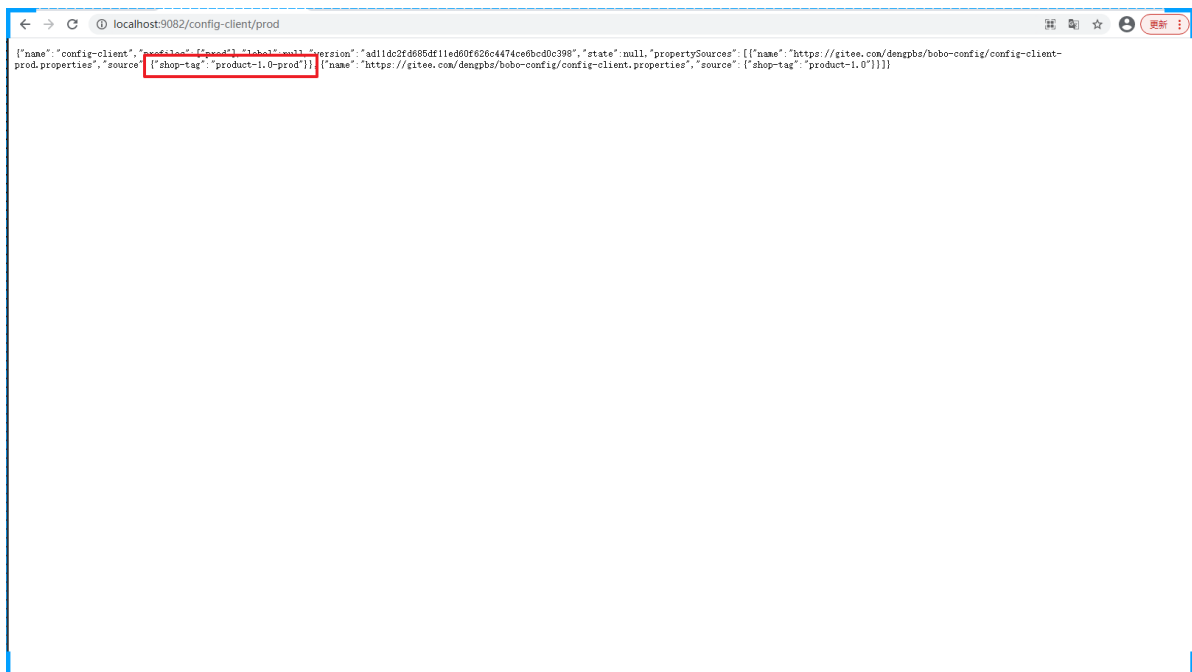
```
public static void main(String[] args) {  
    SpringApplication.run(SpringcloudConfigServerApplication.class, args);  
}  
  
}
```

访问测试

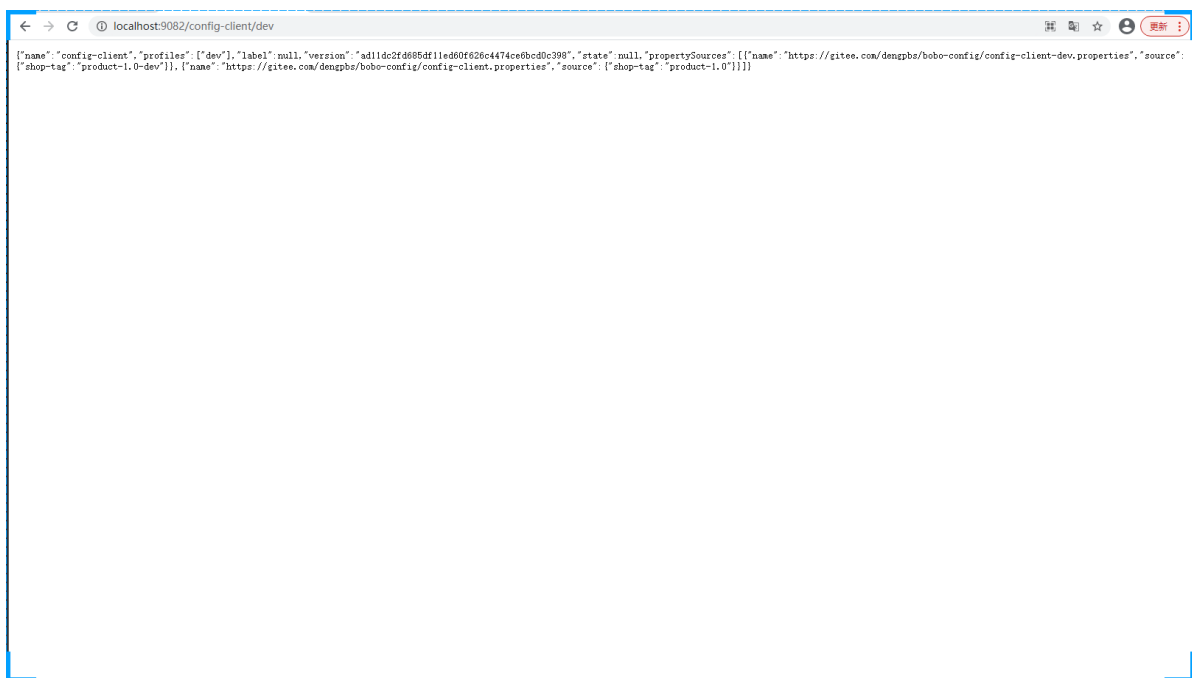
<http://localhost:9082/config-client/test>



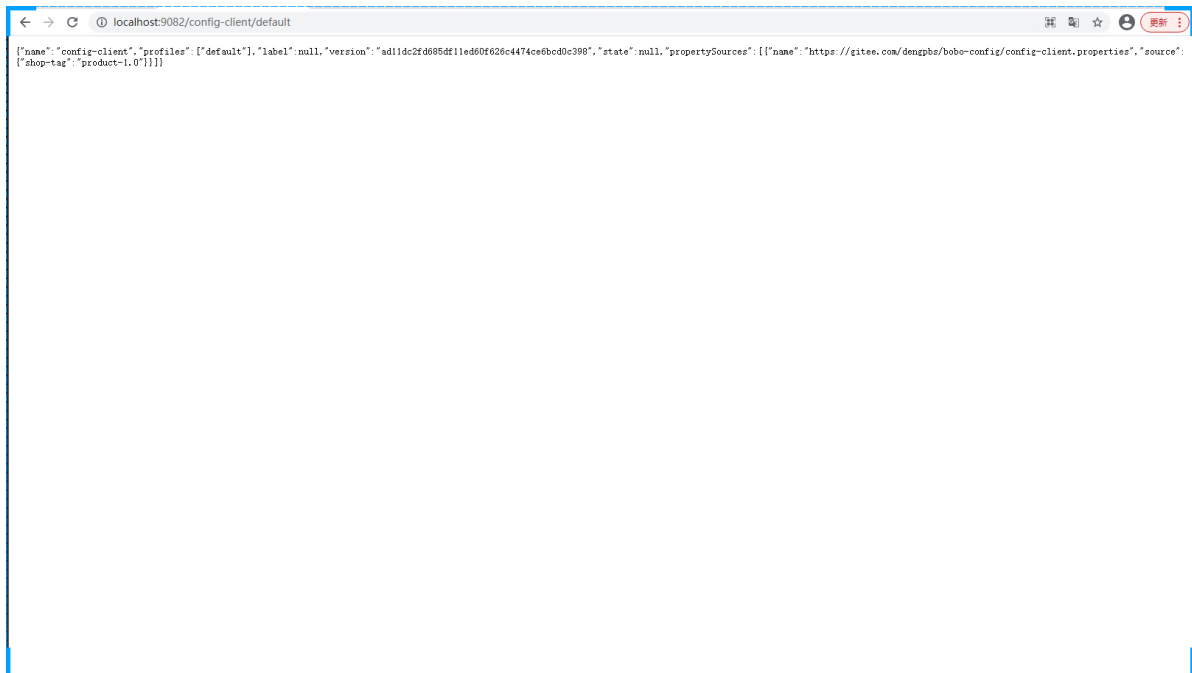
<http://localhost:9082/config-client/prod>



<http://localhost:9082/config-client/dev>



<http://localhost:9082/config-client/default>



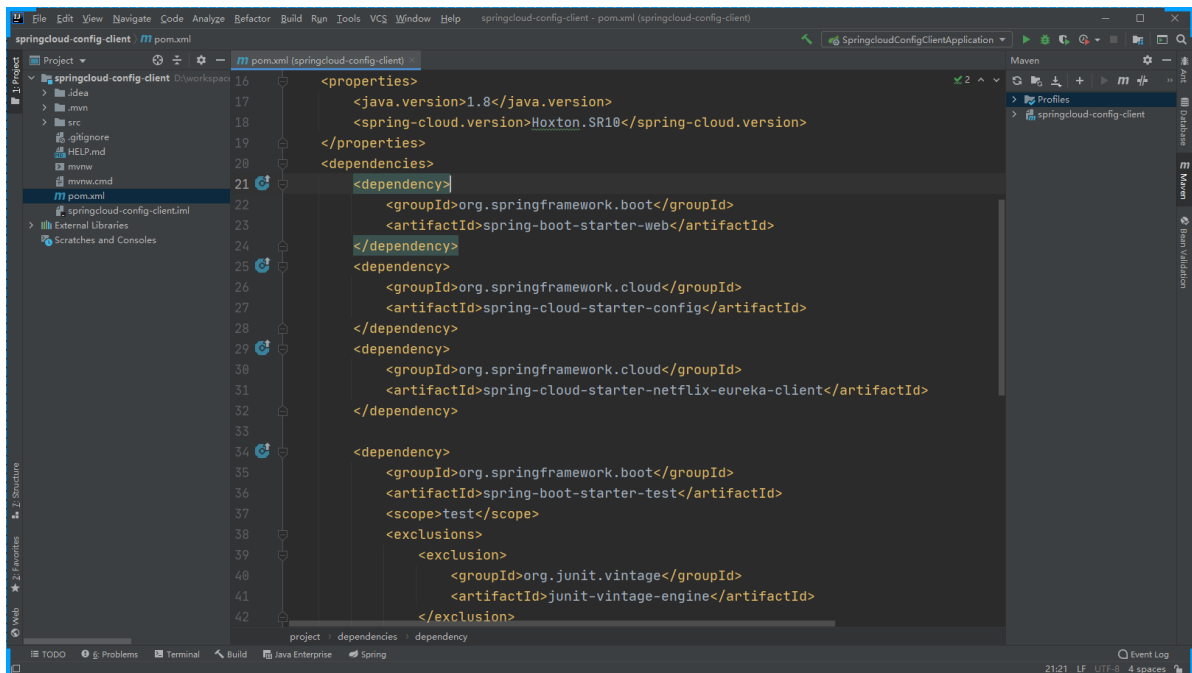
2.6 命名规则

通过访问我们可以发现，配置文件的名称不是随便定义的，而是有一定的命名规则在这里的

配置文件命名规则	
//{application}/{profile}/{label}	application:表示目标服务的名称
//{application}-{profile}.properties	profile:表示获取指定环境下配置，例如开发环境、测试环境、生产环境 默认值default，实际开发中可以是 dev、test、prodn等
//{label}/{application}-{profile}.properties	label: git标签，默认值master
//{application}-{profile}.yml	
//{label}/{application}-{profile}.yml	
举例	
主干	分支
https://gitee.com/oldlu_wk/config 文件如下： config-client.properties config-client-dev.properties config-client-test.properties config-client-prod.properties 访问URL： http://127.0.0.1:9050/config-client/dev http://127.0.0.1:9050/config-client/test http://127.0.0.1:9050/config-client/prod http://127.0.0.1:9050/config-client/default	https://gitee.com/oldlu_wk/config/tree/branch1.0/ 文件如下： config-client.properties config-client-dev.properties config-client-test.properties config-client-prod.properties 访问URL： http://127.0.0.1:9050/config-client/dev/branch1.0/ http://127.0.0.1:9050/config-client/test/branch1.0/ http://127.0.0.1:9050/config-client/prod/branch1.0/ http://127.0.0.1:9050/config-client/default/branch1.0/

3. ConfigClient

3.1 创建项目



添加相关依赖

```

<properties>
  <java.version>1.8</java.version>
  <spring-cloud.version>Hoxton.SR10</spring-cloud.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
  </dependency>

```

```

<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
  <exclusions>
    <exclusion>
      <groupId>org.junit.vintage</groupId>
      <artifactId>junit-vintage-engine</artifactId>
    </exclusion>
  </exclusions>
</dependency>
</dependencies>
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

```

3.1 修改配置文件

注意：在配置中心的客户端服务中，配置文件的名称必须是bootstrap.properties或者bootstrap.yml文件

官方解释

Spring Cloud 构建于 Spring Boot 之上，在 Spring Boot 中有两种上下文，一种是 bootstrap，另外一种 application，bootstrap 是应用程序的父上下文，也就是说 bootstrap 加载优先于 application。bootstrap 主要用于从额外的资源来加载配置信息，还可以在本地外部配置文件中解密属性。这两个上下文共用一个环境，它是任何 Spring 应用程序的外部属性的来源。bootstrap 里面的属性会优先加载，它们默认也不能被本地相同配置覆盖。

```

spring.application.name=config-client
server.port=9083

# 设置服务注册中心的地址
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

# 开启配置中心 默认是false
spring.cloud.config.discovery.enabled=true
# 对应的eureka中的配置中心 serviceId 默认是configserver

```

```
spring.cloud.config.discovery.service-id=config-server
# 指定环境
spring.cloud.config.profile=dev

# git 标签
spring.cloud.config.label=master
```

3.3 创建控制器

我们在控制器中获取配置中心中的信息

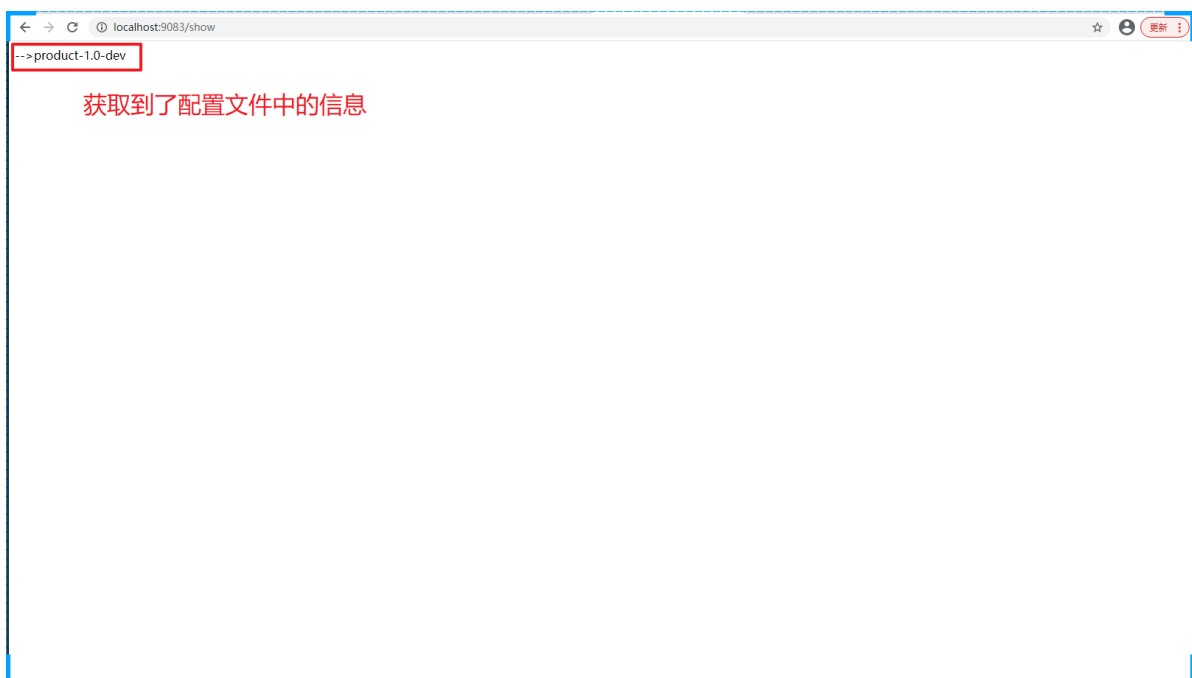
```
@RestController
public class ShowController {

    @Value("${shop-tag}")
    private String msg;

    @RequestMapping("/show")
    public String showMsg(){
        return "-->" + msg;
    }
}
```

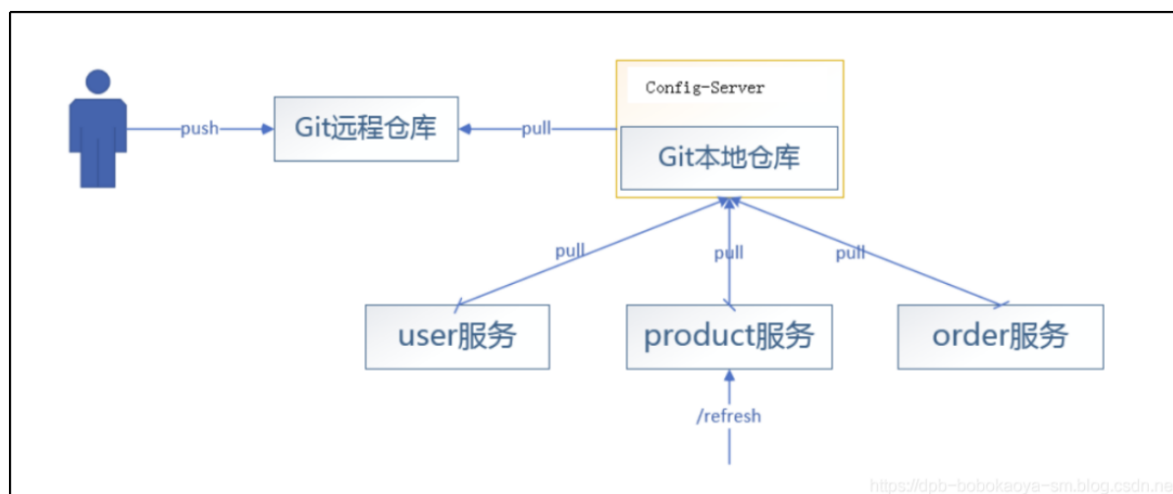
3.4 启动服务

启动服务访问测试



4. 动态刷新

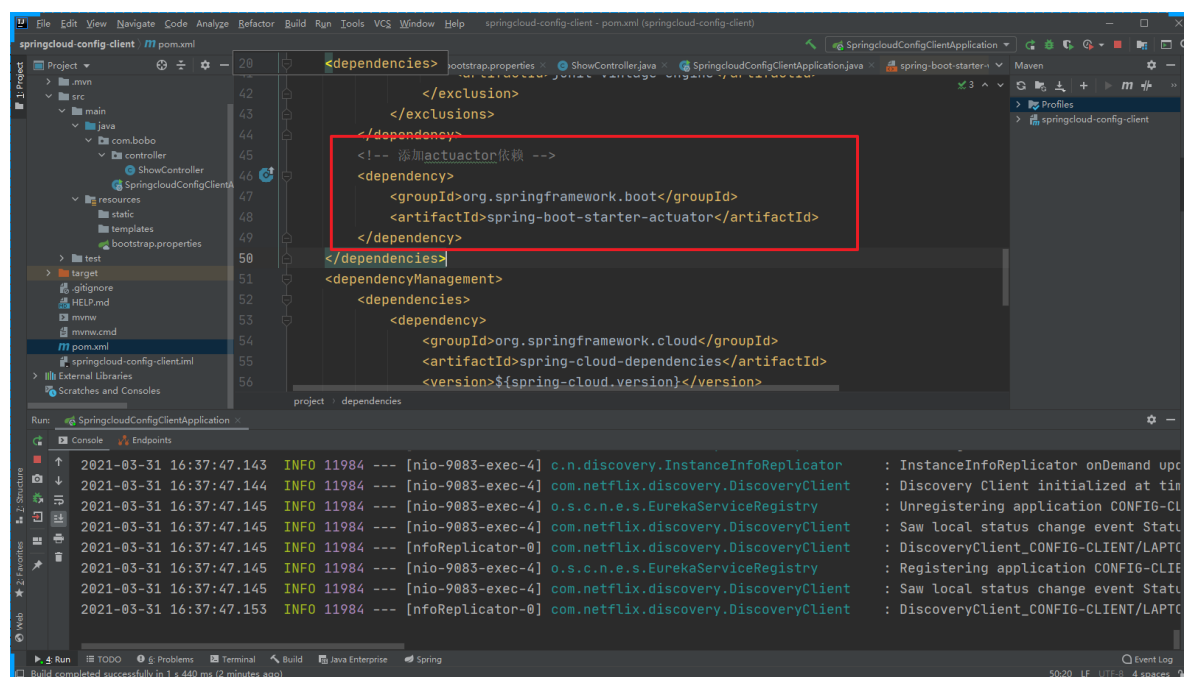
4.1 原理结构



4.2 动态刷新

通过上图我们能看到配置服务其实是从本地的Git仓库中获取的信息，Git本地库通过pull命令同步远程库中的内容。当配置中心客户端重新启动的时候会显示的执行pull命令来拉取最新的配置信息，这种方式是需要重启客户端服务的，显然不是太友好，这时我们可以通过Actuator来实现。

添加依赖



修改属性文件

```
spring.application.name=config-client
server.port=9083

# 设置服务注册中心的地址
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

# 开启配置中心 默认是false
spring.cloud.config.discovery.enabled=true
```

```
# 对应的eureka中的配置中心 serviceId 默认是configserver
spring.cloud.config.discovery.service-id=config-server

# 指定环境
spring.cloud.config.profile=dev

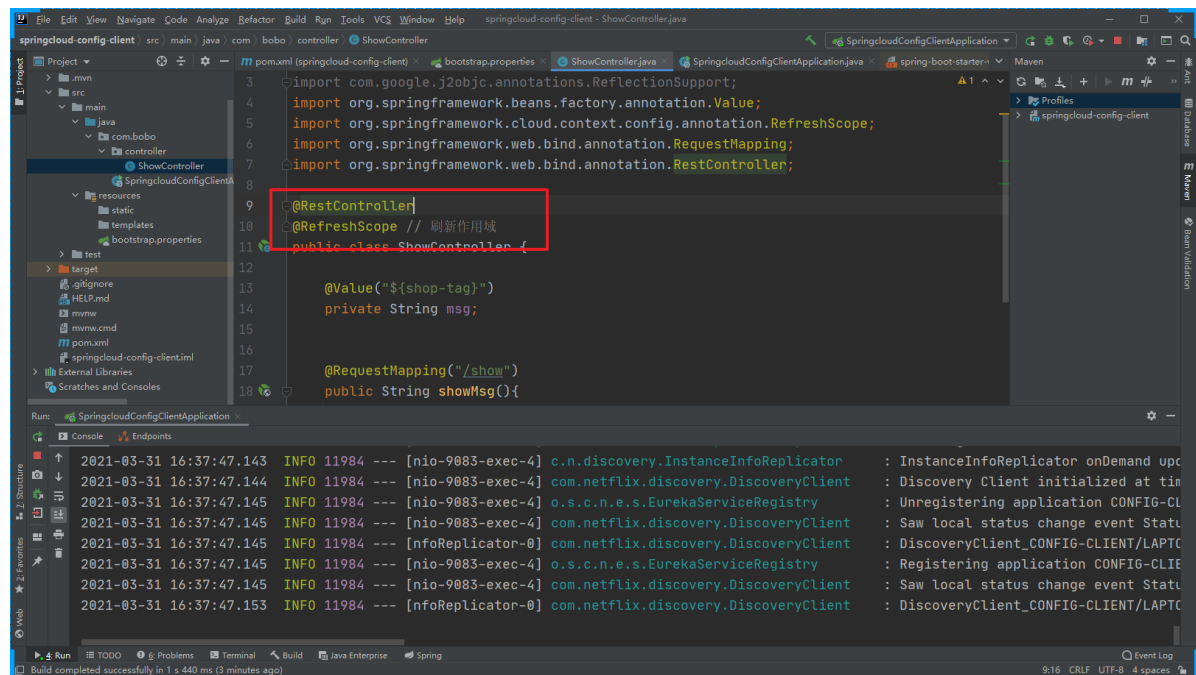
# git 标签
spring.cloud.config.label=master

# SpringBoot 默认开启了权限拦截, 会导致 /refresh 出现401拒绝访问
management.security.enabled=false

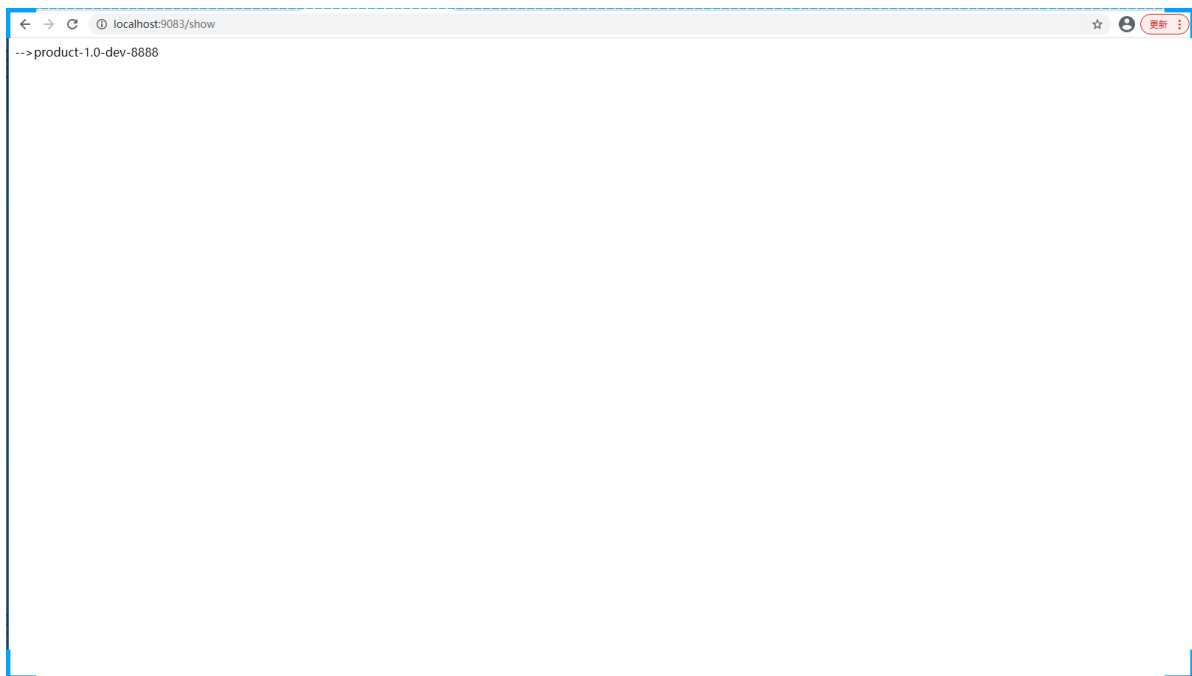
# 在Actuator中默认只放开了 info和health 如果要放开所有*
management.endpoints.web.exposure.include=*

# 放开shutdown 接口
management.endpoints.enabled-by-default=true
```

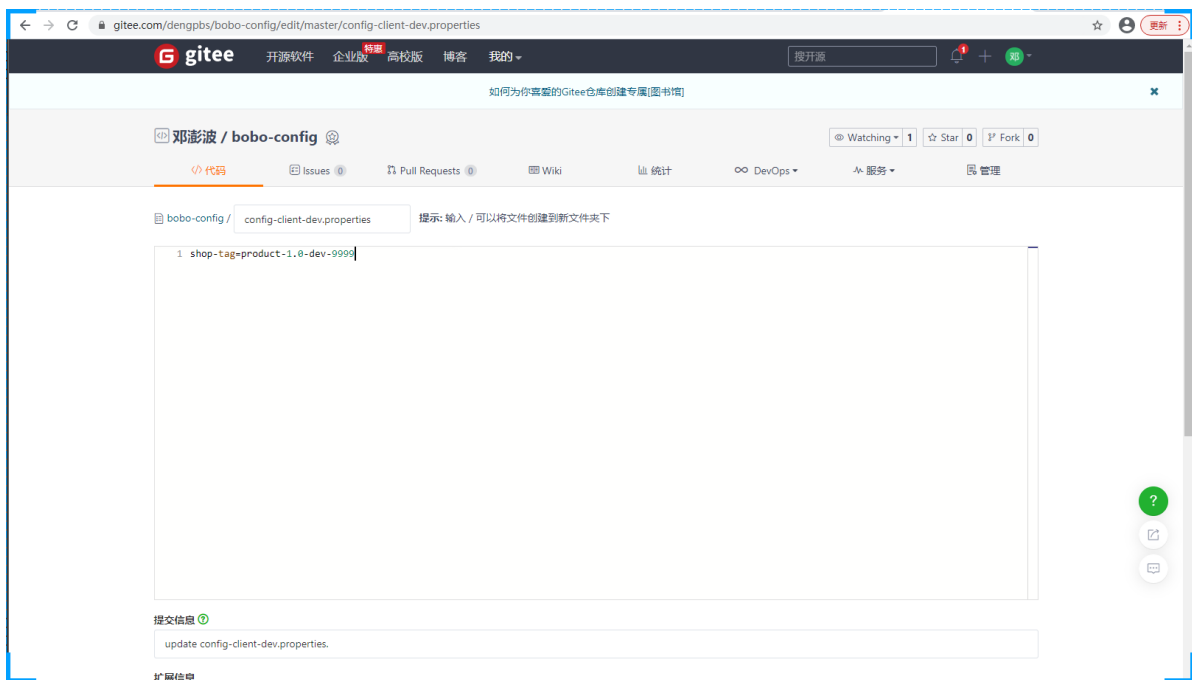
修改Bean对象作用域



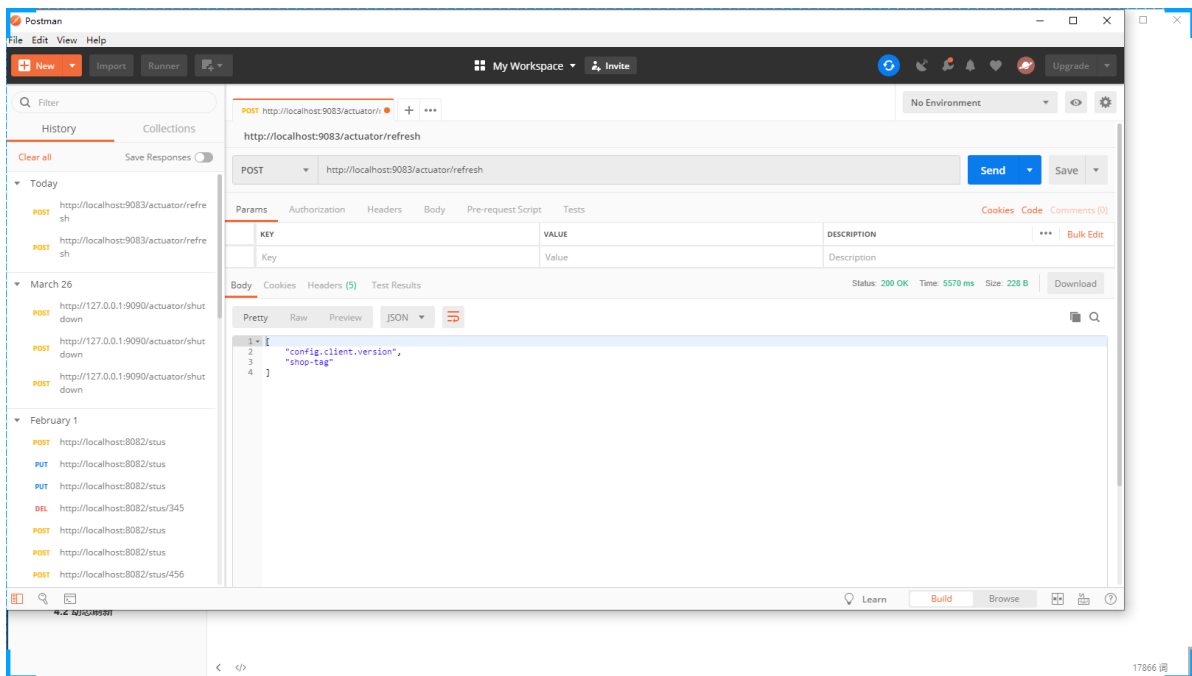
测试, 因为refresh接口只支持post方式提交, 我们通过postman来测试



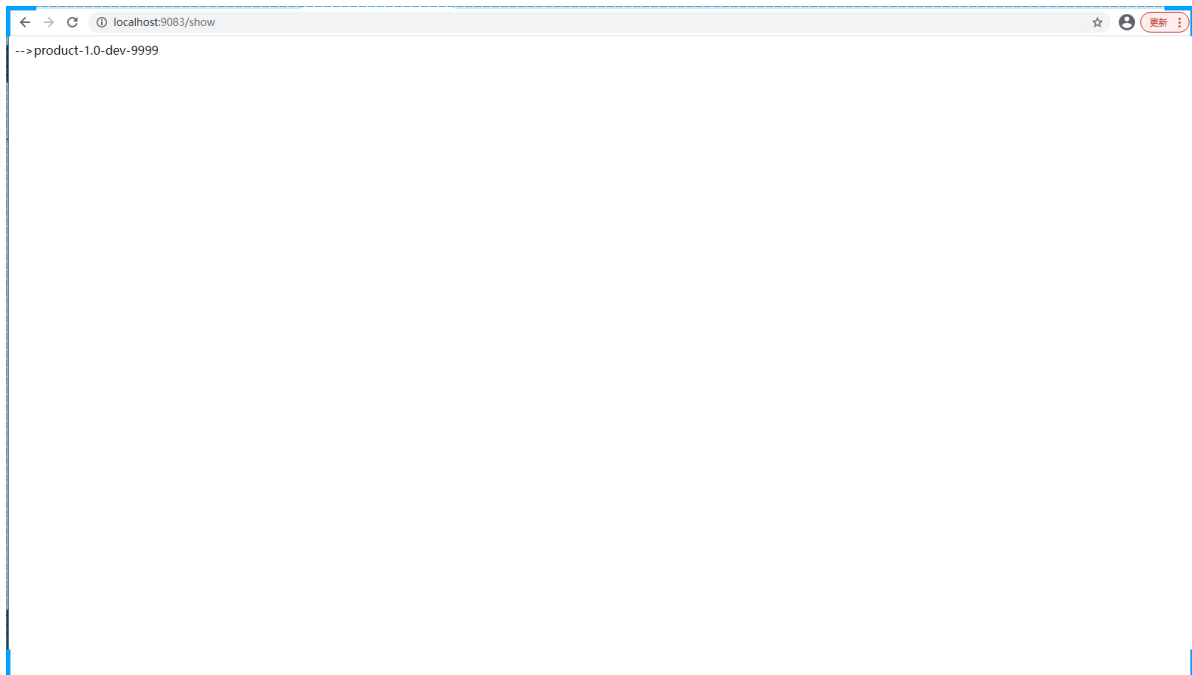
然后修改码云中的数据



然后通过postman来发送post请求



然后访问请求，发现数据更新了



八、Bus

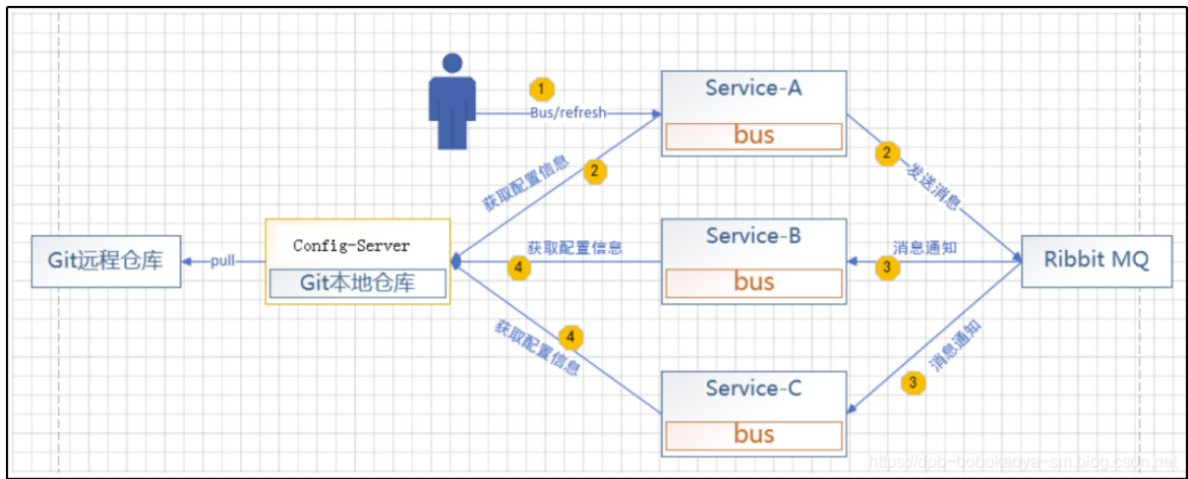
1.什么是消息总线bus

SpringCloud Bus集成了市面上常用的消息中间件(rabbit mq,kafka等)，连接微服务系统中的所有的节点，当有数据变更的时候，可以通过消息代理广播通知微服务及时**变更**数据，例如微服务的配置更新。

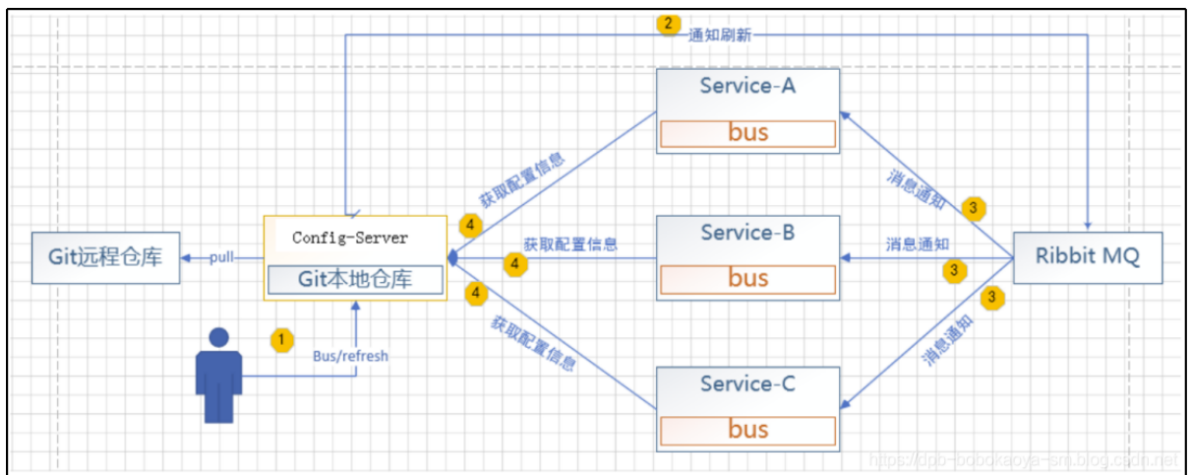
2.bus解决了什么问题？

解决了微服务数据变更，及时同步的问题。

刷新客户端服务

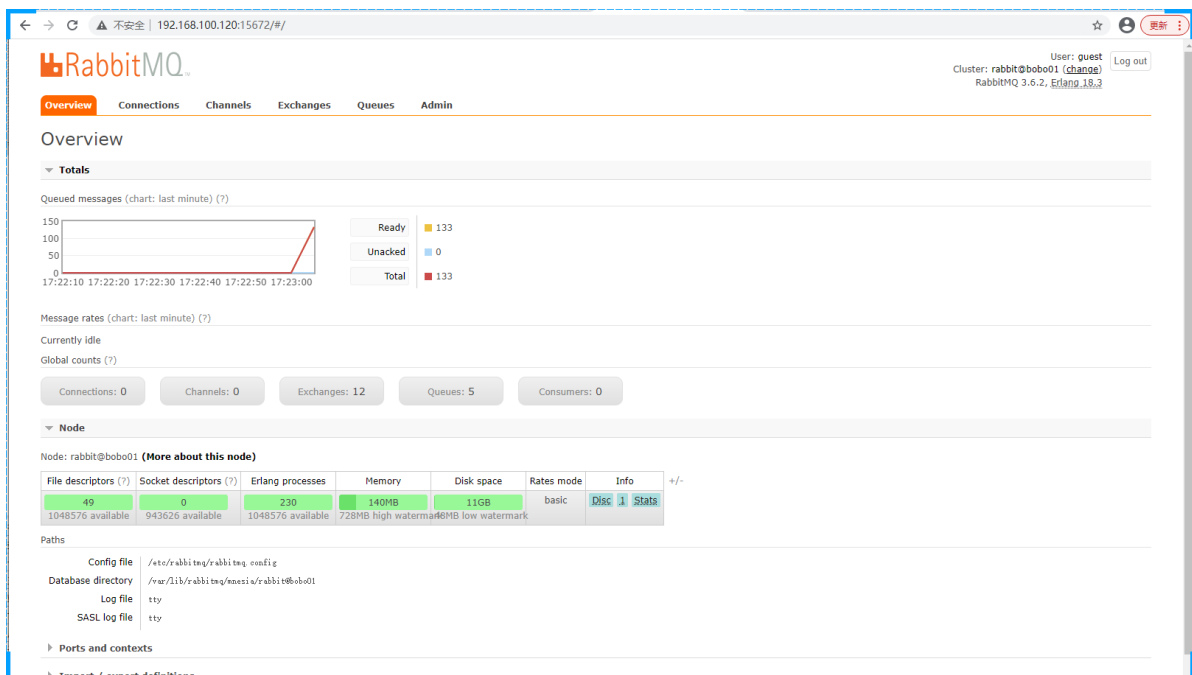


刷新服务端服务



3. 启动RabbitMQ

开启消息中间件服务



4. 客户端刷新

4.1 创建客户端

我们需要创建两个客户端，添加依赖环境

```
<!-- 添加Bus的依赖 -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

4.2 属性文件

我们需要在属性文件中添加 RabbitMQ的相关信息

```
spring.application.name=config-client
server.port=9083

# 设置服务注册中心的地址
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

# 开启配置中心 默认是false
spring.cloud.config.discovery.enabled=true
# 对应的eureka中的配置中心 serviceId 默认是configserver
spring.cloud.config.discovery.service-id=config-server
# 指定环境
spring.cloud.config.profile=dev

# git 标签
spring.cloud.config.label=master

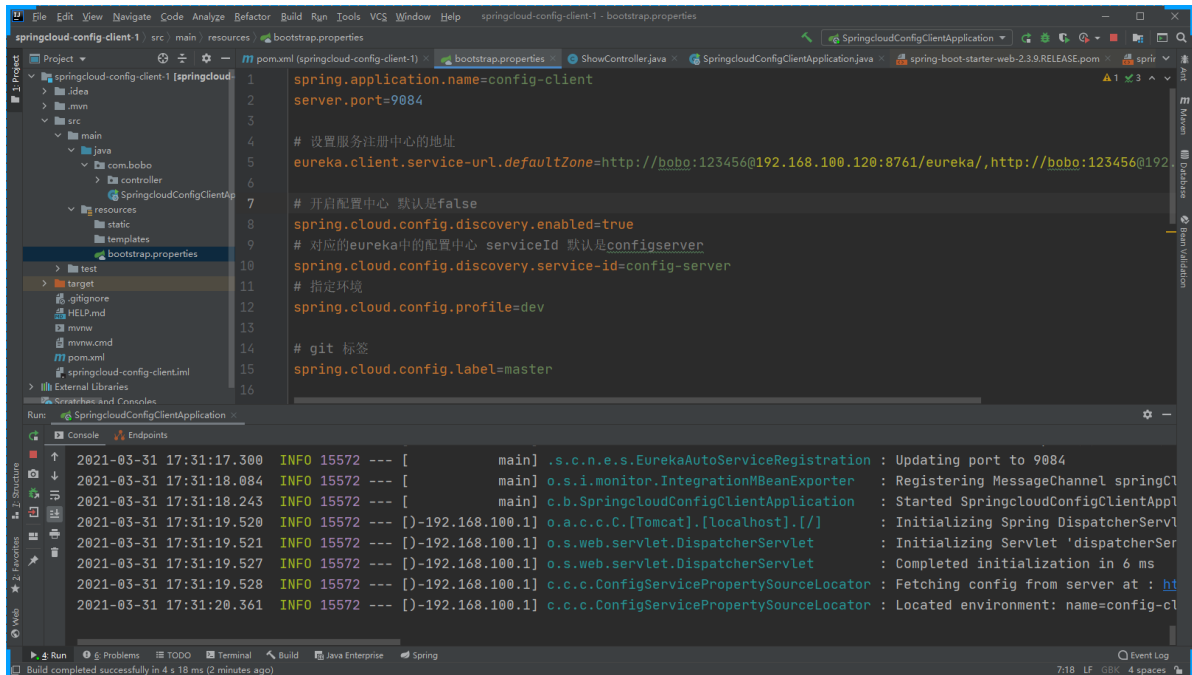
# SpringBoot 默认开启了权限拦截，会导致 /refresh 出现401拒绝访问
management.security.enabled=false

# 在Actuator中默认只放开了 info和health 如果要放开所有*
management.endpoints.web.exposure.include=*
# 放开shutdown 接口
management.endpoints.enabled-by-default=true

# 添加RabbitMQ的配置信息
spring.rabbitmq.host=192.168.100.120
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
spring.rabbitmq.virtual-host=
```

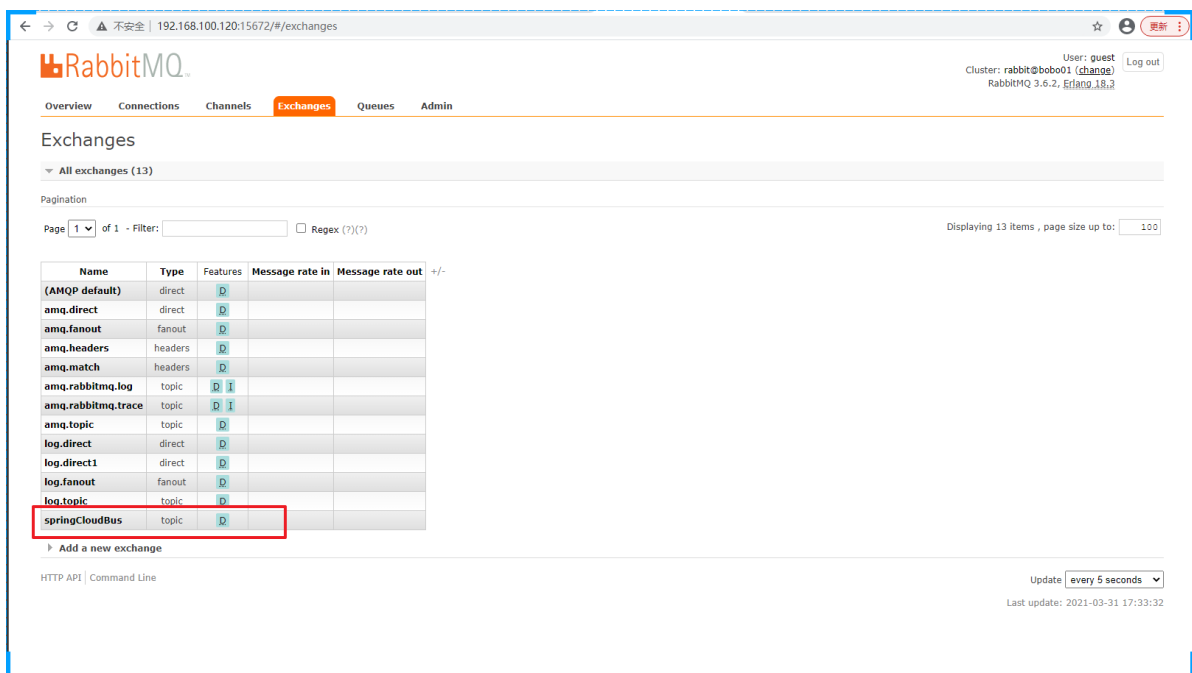
4.3 再创建一个客户端

我们可以复制前面创建好的客户端，修改下端口即可



4.4 测试

分别启动两个客户端服务



← → ↻ 不安全 | 192.168.100.120:15672/#/connections ☆ 更新

RabbitMQ

User: guest
Cluster: rabbit@bobo01 (change)
RabbitMQ 3.6.2, Erlang 18.3

Log out

Overview **Connections** Channels Exchanges Queues Admin

Connections

▼ All connections (2)

Pagination

Page 1 of 1 · Filter: ☐ Regexp (?) (?)

Displaying 2 items , page size up to: 100

Overview			Details			Network		+/-
Name	User name	State	SSL / TLS	Protocol	Channels	From client	To client	
192.168.100.1:57342 rabbitConnectionFactory#53a3fc8b:0	guest	running	o	AMQP 0-9-1	2	2B/s	0B/s	
192.168.100.1:57356 rabbitConnectionFactory#7a6b214c:0	guest	running	o	AMQP 0-9-1	2	0B/s	0B/s	

HTTP API | Command Line

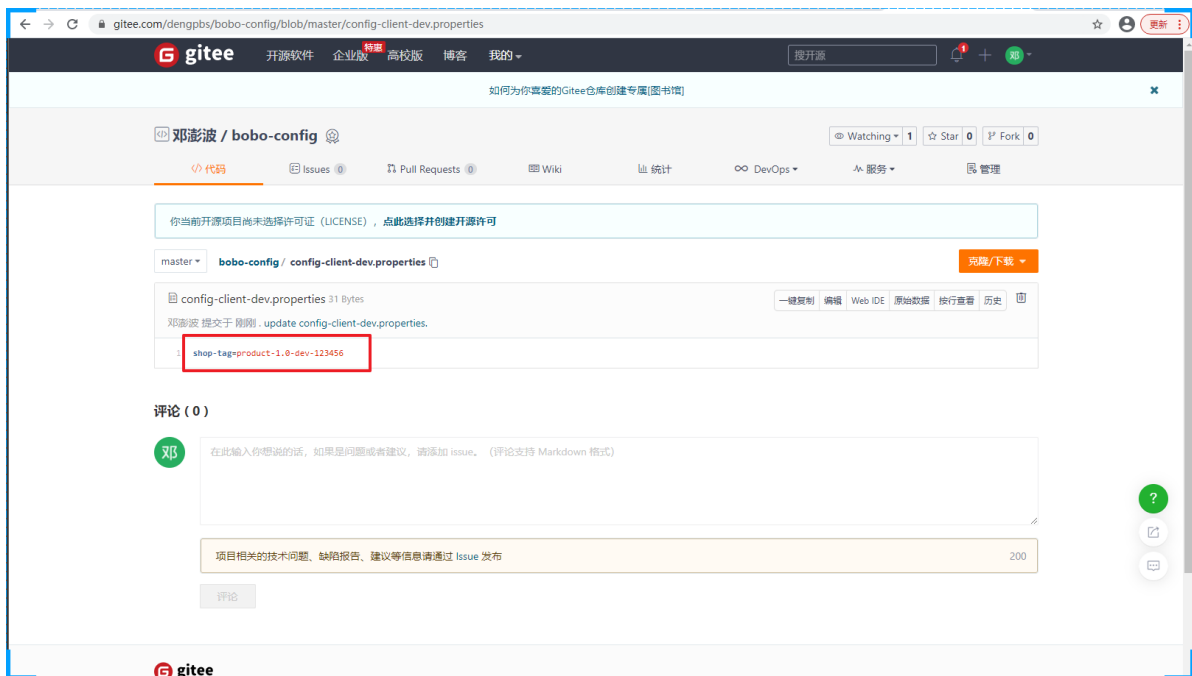
Update every 5 seconds
Last update: 2021-03-31 17:33:55

更新Git远程服务中的数据，然后在刷新

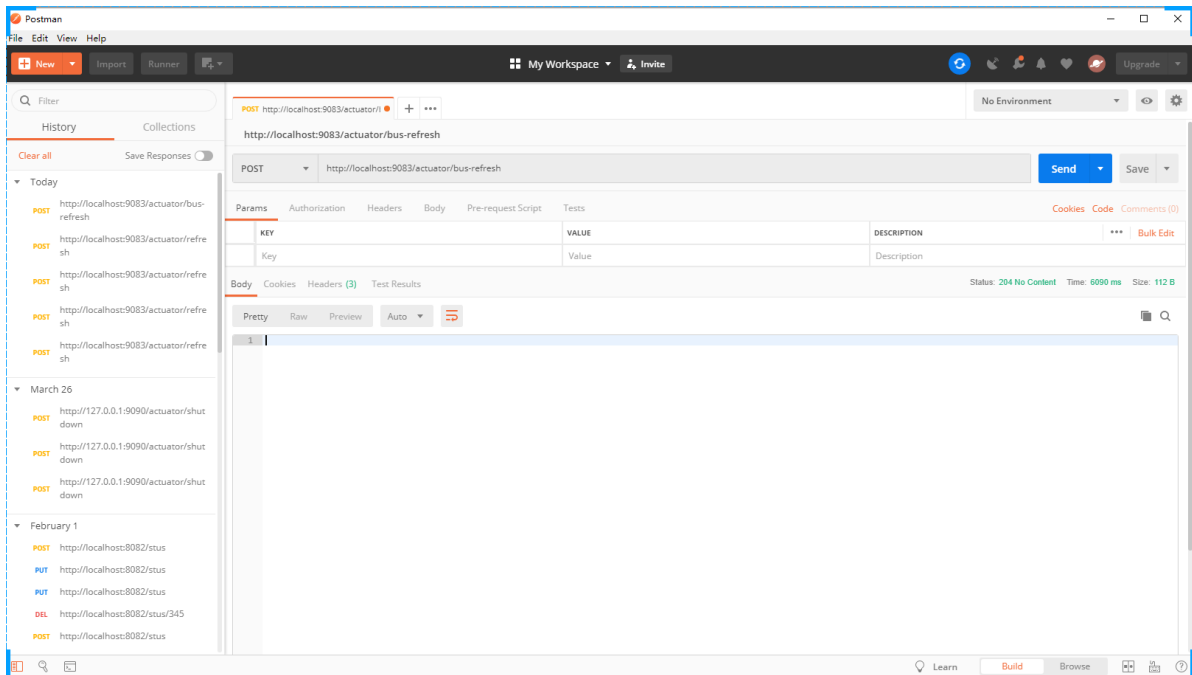
← → ↻ localhost:9084/show ☆ 更新

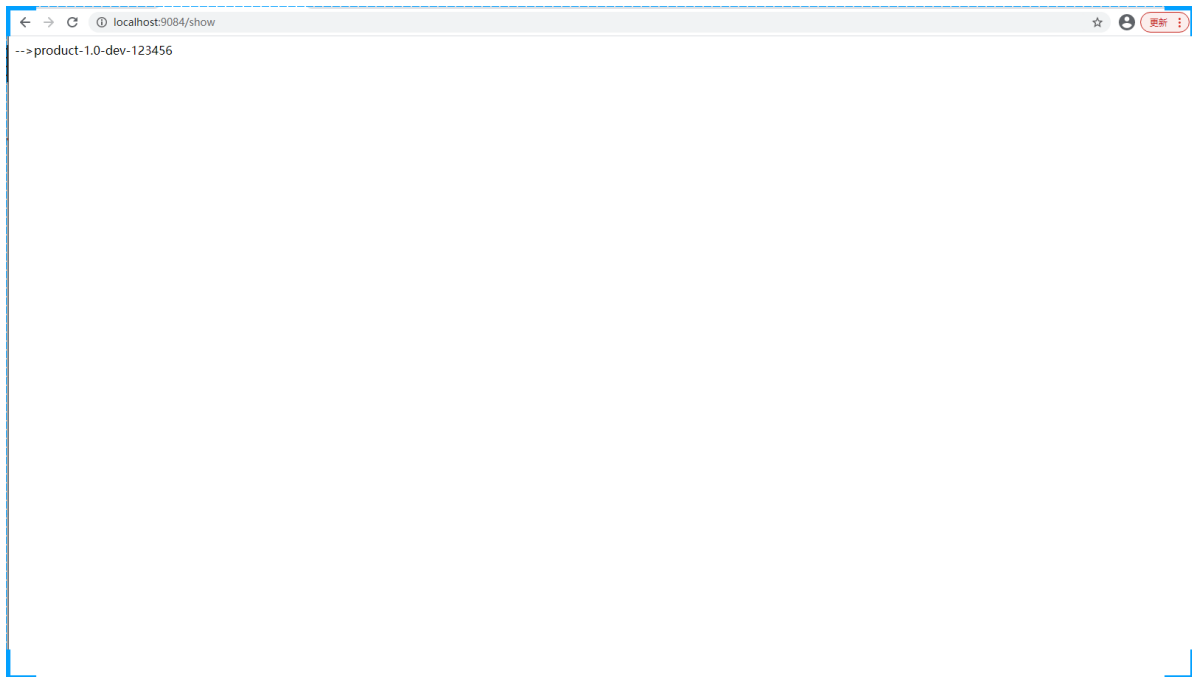
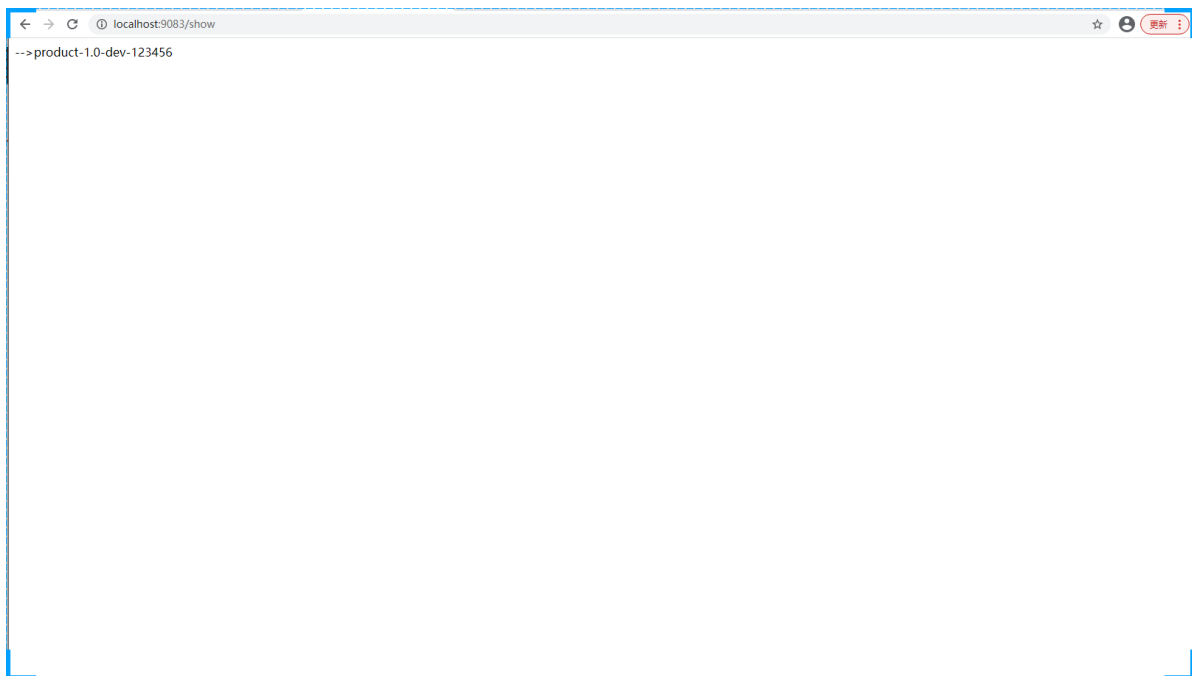
-->product-1.0-dev-9999

更新后



然后通过PostMan来刷新





那么在这我们就看到了我们期望的效果，刷新一个客户端，其他的客户端也同步获取到了最新的数据

5. 服务端刷新

其实相关的案例我们已经有了，只需要在之前的ConfigServer中配置RabbitMQ的相关信息即可

```
<!-- 添加actuator依赖 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

然后添加相关的配置信息

```
# SpringBoot 默认开启了权限拦截，会导致 /refresh 出现401拒绝访问
management.security.enabled=false
```

```
# 在Actuator中默认只放开了 info和health 如果要放开所有*
management.endpoints.web.exposure.include=*
```

```
# 放开shutdown 接口
management.endpoints.enabled-by-default=true
```

```
# 添加RabbitMQ的配置信息
```

```
spring.rabbitmq.host=192.168.100.120
```

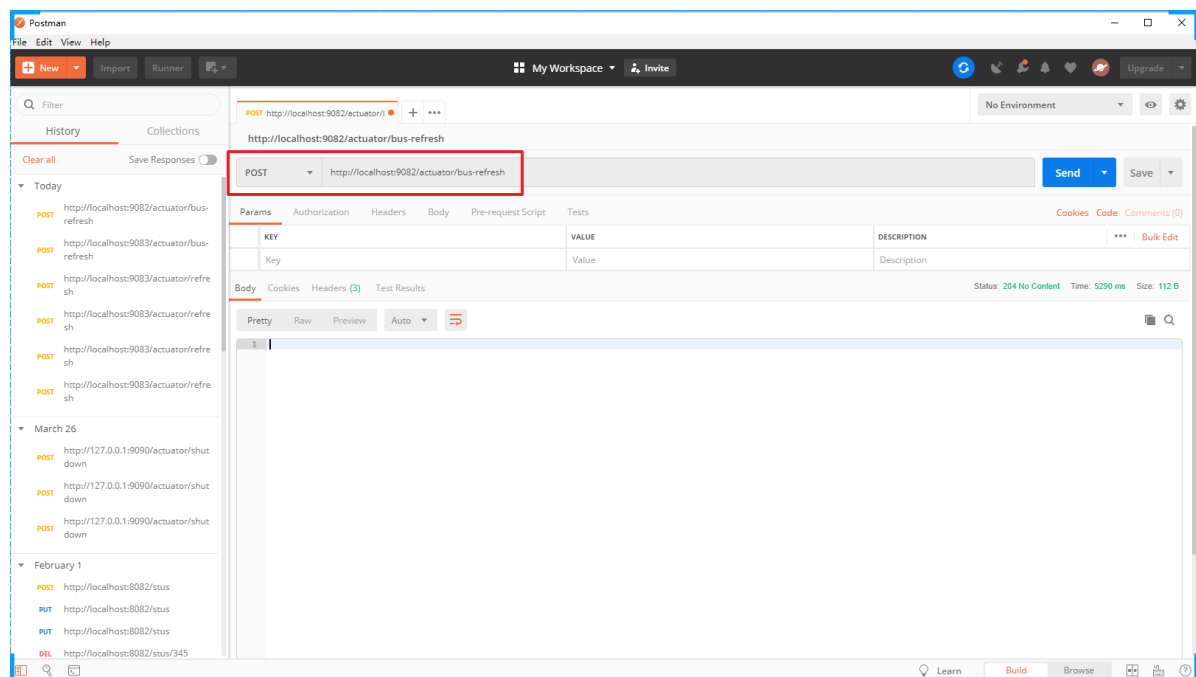
```
spring.rabbitmq.port=5672
```

```
spring.rabbitmq.username=guest
```

```
spring.rabbitmq.password=guest
```

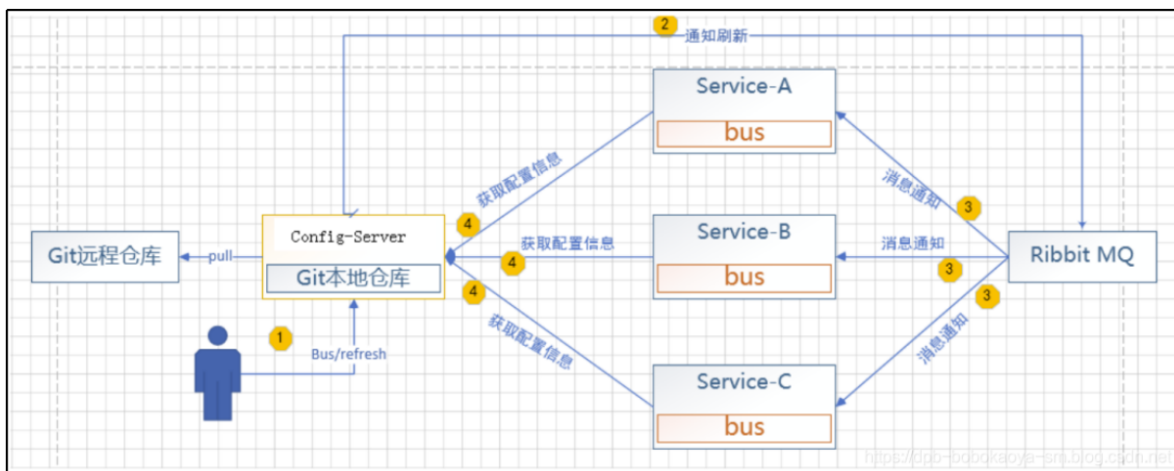
```
spring.rabbitmq.virtual-host=
```

我们就可以按照客户端刷新的步骤来测试了



6. 局部刷新

如果配置文件更新后，只是需要作用于部分的服务，而不需要所有的服务都刷新。



6.1 刷新指定服务

<http://config-server/actuator/bus-refresh?destination=需要刷新的服务名称:端口号>

6.2 刷新指定集群服务

http://config-server/actuator/bus-refresh?destination=需要刷新的服务名称:**

九、Stream

在实际开发过程中，服务与服务之间通信经常会使用到消息中间件，而以往使用了哪个中间件比如 RabbitMQ，那么该中间件和系统的耦合性就会非常高，如果我们要替换为 Kafka 那么变动会比较大，这时我们可以使用 Spring Cloud Stream 来整合我们的消息中间件，来降低系统和中间件的耦合性。

1.什么是SpringCloudStream

官方定义 Spring Cloud Stream 是一个构建消息驱动微服务的框架。

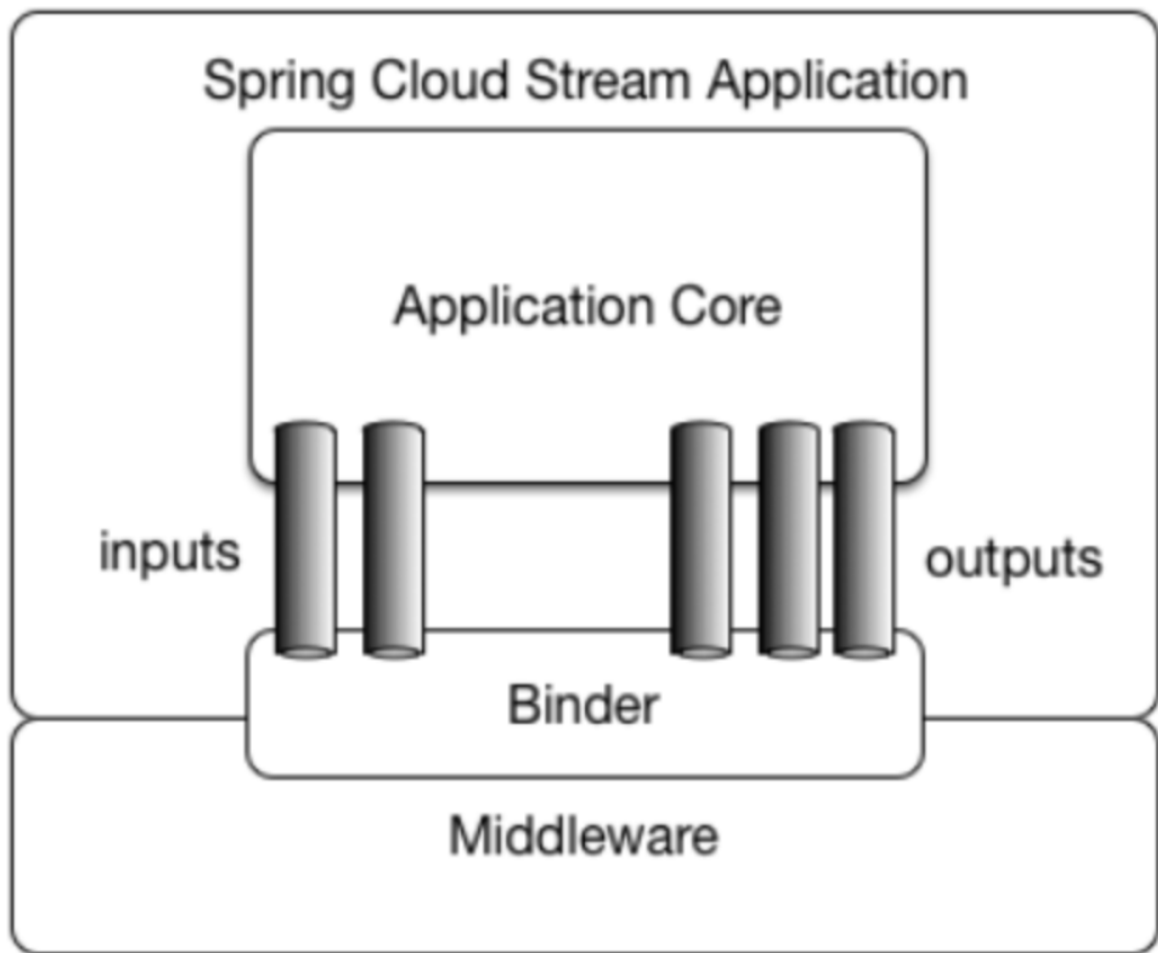
应用程序通过 inputs 或者 outputs 来与 Spring Cloud Stream 中 binder 交互，通过我们配置来 binding，而 Spring Cloud Stream 的 binder 负责与消息中间件交互。所以，我们只需要搞清楚如何与 Spring Cloud Stream 交互就可以方便使用消息驱动的方式。

通过使用 Spring Integration 来连接消息代理中间件以实现消息事件驱动。Spring Cloud Stream 为一些供应商的消息中间件产品提供了个性化的自动化配置实现，引用了发布-订阅、消费组、分区的三个核心概念。目前仅支持 RabbitMQ、Kafka。

2.Stream 解决了什么问题？

Stream 解决了开发人员无感知的使用消息中间件的问题，因为 Stream 对消息中间件的进一步封装，可以做到代码层面对中间件的无感知，甚至于动态的切换中间件 (rabbitmq 切换为 kafka)，使得微服务开发的高度解耦，服务可以关注更多自己的业务流程

官网结构图



组成	说明
Middleware	中间件，目前只支持RabbitMQ和Kafka
Binder	Binder是应用与消息中间件之间的封装，目前实行了Kafka和RabbitMQ的Binder，通过Binder可以很方便的连接中间件，可以动态的改变消息类型(对应于Kafka的topic，RabbitMQ的exchange)，这些都可以通过配置文件来实现
@Input	注解标识输入通道，通过该输入通道接收到的消息进入应用程序
@Output	注解标识输出通道，发布的消息将通过该通道离开应用程序
@StreamListener	监听队列，用于消费者的队列的 消息接收
@EnableBinding	指信道channel和exchange绑定在一起

3. 创建生产者

3.1 创建项目

创建一个SpringBoot项目，添加相关的依赖

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-stream-rabbit</artifactId>
</dependency>
```

3.2 属性文件

添加Eureka客户端信息和RabbitMQ的配置信息

```
spring.application.name=config-server
server.port=9085

# 设置服务注册中心的地址
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

# 添加RabbitMQ的配置信息
spring.rabbitmq.host=192.168.100.120
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
spring.rabbitmq.virtual-host=
```

3.3 创建消息发送接口

```
package com.bobo.provider;

import org.springframework.cloud.stream.annotation.Output;
import org.springframework.messaging.SubscribableChannel;

/**
 * 发送消息的接口
 */
public interface ISendService {

    /**
     * 指定输出的交换器名称
     * @return
     */
    @Output("bobo-exchange")
    SubscribableChannel send();
}
```

3.4 启动类

在启动类中绑定我们刚刚创建的发送消息的接口类型

```
@SpringBootApplication
// 绑定我们刚刚创建的发送消息的接口类型
@EnableBinding(value = {ISendService.class})
public class SpringcloudStreamProviderApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringcloudStreamProviderApplication.class, args);
    }

}
```

3.5 测试类

```
package com.bobo;

import com.bobo.provider.ISendService;
import org.junit.jupiter.api.Test;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.messaging.Message;
import org.springframework.messaging.support.MessageBuilder;

@SpringBootTest
class SpringcloudStreamProviderApplicationTests {

    @Autowired
    private ISendService sendService;

    @Test
    void contextLoads() {
        String msg = "Hello stream ....";
        Message<String> message = MessageBuilder.withPayload(msg).build();
        sendService.send().send(message);
    }

}
```

4. 创建消费者

4.1 创建项目

创建SpringBoot项目添加相关的依赖

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-stream-rabbit</artifactId>
</dependency>
```

4.2 属性文件

```
spring.application.name=config-server
server.port=9086

# 设置服务注册中心的地址
eureka.client.service-
url.defaultZone=http://bobo:123456@192.168.100.120:8761/eureka/,http://bobo:1234
56@192.168.100.121:8761/eureka/

# 添加RabbitMQ的配置信息
spring.rabbitmq.host=192.168.100.120
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
spring.rabbitmq.virtual-host=
```

4.3 创建接收消息的接口

```
package com.bobo.consumer;

import org.springframework.cloud.stream.annotation.Input;
import org.springframework.messaging.SubscribableChannel;

/**
 * 接收服务的接口
 */
public interface IReceiverService {

    /**
     * 指定接收消息的交换器
     * @return
     */
    @Input("bobo-exchange")
    SubscribableChannel receiver();
}
```

4.4 创建处理消息的处理类

```
package com.bobo.consumer;

import org.springframework.cloud.stream.annotation.EnableBinding;
import org.springframework.cloud.stream.annotation.StreamListener;
import org.springframework.stereotype.Service;

@Service
@EnableBinding(IReceiversService.class)
public class ReciversService {

    @StreamListener("bobo-exchange")
    public void onReceiver(byte[] msg){
        System.out.println("消费者: " + new String(msg));
    }
}
```

4.5 启动类

```
package com.bobo;

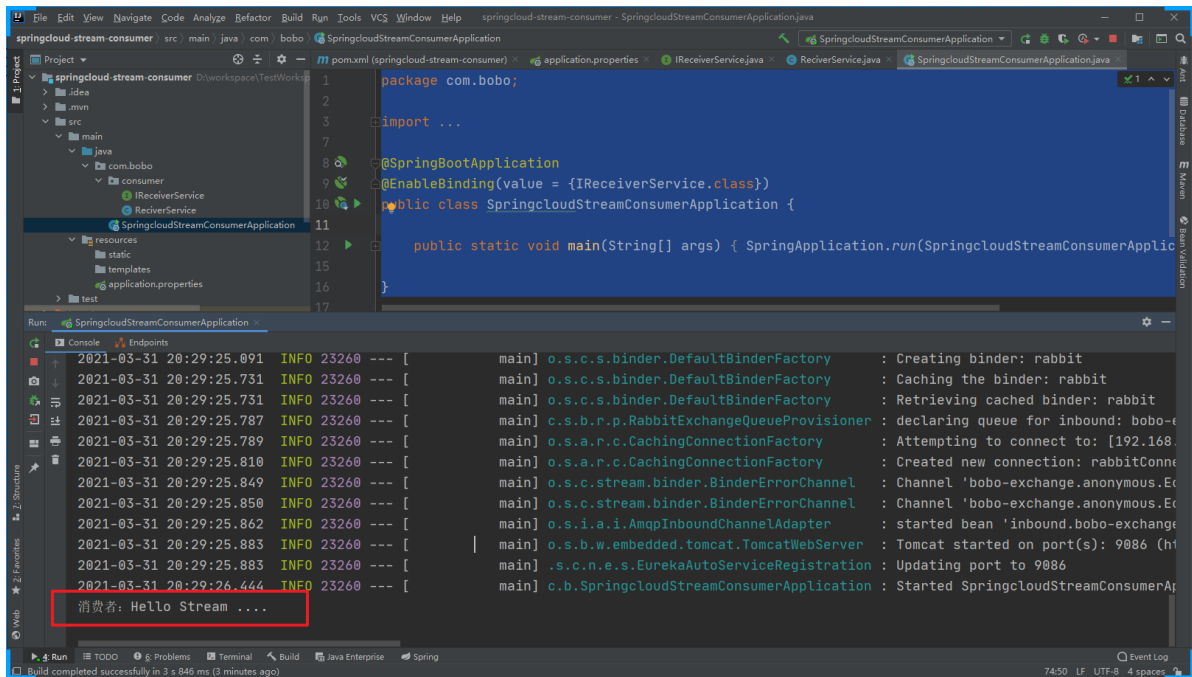
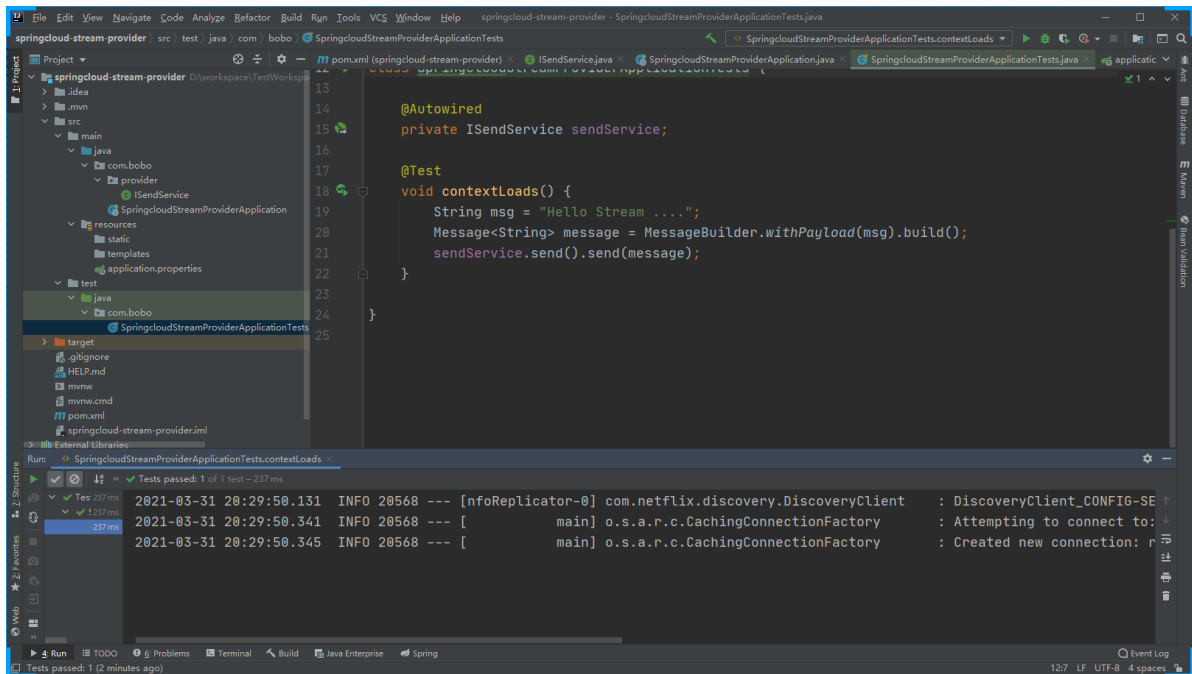
import com.bobo.consumer.IReceiversService;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.stream.annotation.EnableBinding;

@SpringBootApplication
@EnableBinding(value = {IReceiversService.class})
public class SpringcloudStreamConsumerApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringcloudStreamConsumerApplication.class, args);
    }
}
```

5. 测试

先启动消费者，然后启动消息提供者来串联



← → 不安全 192.168.100.120:15672/#/exchanges

RabbitMQ

User: guest
Cluster: rabbit@bobo01 (change)
RabbitMQ 3.6.2, Erlang 18.3

Log out

OverviewConnectionsChannelsExchangesQueuesAdmin

Exchanges

All exchanges (14)

Page 1 of 1 · Filter: ☐ Regexp (?) (?)

Displaying 14 items , page size up to: 100

Name	Type	Features	Message rate in	Message rate out	+/-
(AMQP default)	direct	D			
amq.direct	direct	D			
amq.fanout	fanout	D			
amq.headers	headers	D			
amq.match	headers	D			
amq.rabbitmq.log	topic	D I			
amq.rabbitmq.trace	topic	D I			
amq.topic	topic	D			
bobo-exchange	topic	D	0.00/s	0.00/s	
log.direct	direct	D			
log.direct1	direct	D			
log.fanout	fanout	D			
log.topic	topic	D			
springCloudBus	topic	D	0.00/s	0.00/s	

⤵ Add a new exchange

HTTP API | Command Line

Update every 5 seconds

Last update: 2021-03-31 20:32:29